

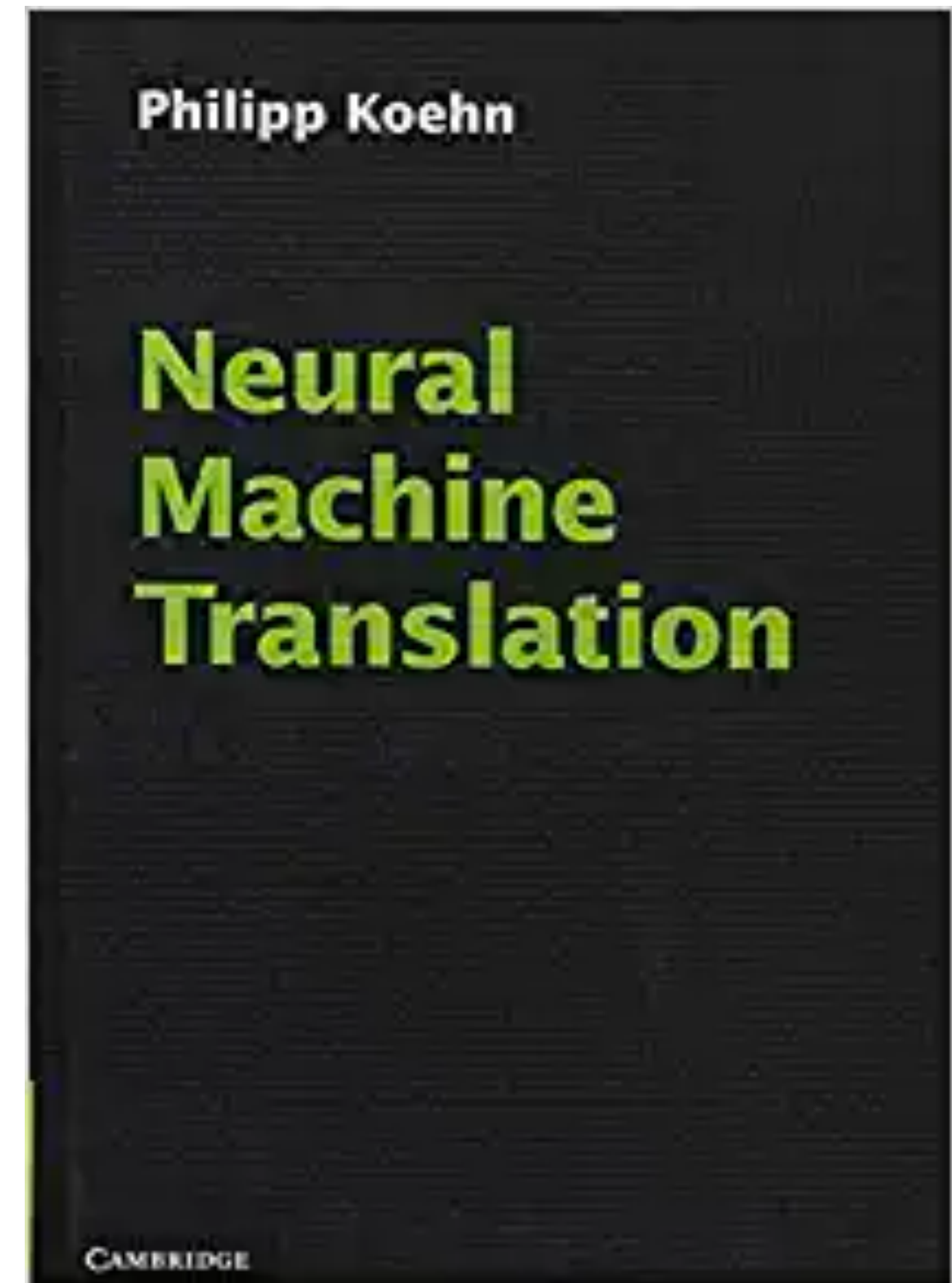
Encoder-Decoder (aka Seq2Seq) and Attention

Wei Xu

(many slides from Greg Durrett)

This Lecture

- ▶ Machine Translation
 - ▶ Sequence-to-Sequence Model
 - ▶ Attention Mechanism
-
- ▶ Reading — Eisenstein 18.3-18.5



MT Basics

MT Basics



< 2/8

特朗普偕家人在白宫阳台观看百年一遇日全食

>

People's Daily, August 30, 2017

MT Basics



Translate

English French Spanish Chinese - detected ▼



特朗普偕家人在白宫阳台观看百年一遇日全食✕

< 2/8

特朗普偕家人在白宫阳台观看百年一遇日全食

People's Daily, August 30, 2017

MT Basics



Translate

English French Spanish Chinese - detected

特朗普偕家人在白宫阳台观看百年一遇日全食

< 2/8

特朗普偕家人在白宫阳台观看百年一遇日全食

People's Daily, August 30, 2017

Trump Pope family watch a hundred years a year in the White House balcony

MT Basics



Translate

English French Spanish Chinese - detected ▼

特朗普偕家人在白宫阳台观看百年一遇日全食 ✕

People's Daily, August 30, 2017

Trump Pope family watch a hundred years a year in the White House balcony

MT Basics



Translate

English

French

Spanish

Chinese - detected



特朗普偕家人在白宫阳台观看百年一遇日全食

< 2/8

特朗普偕家人在白宫阳台观看百年一遇日全食

People's Daily, August 30, 2017

MT Basics



People's Daily, August 30, 2017

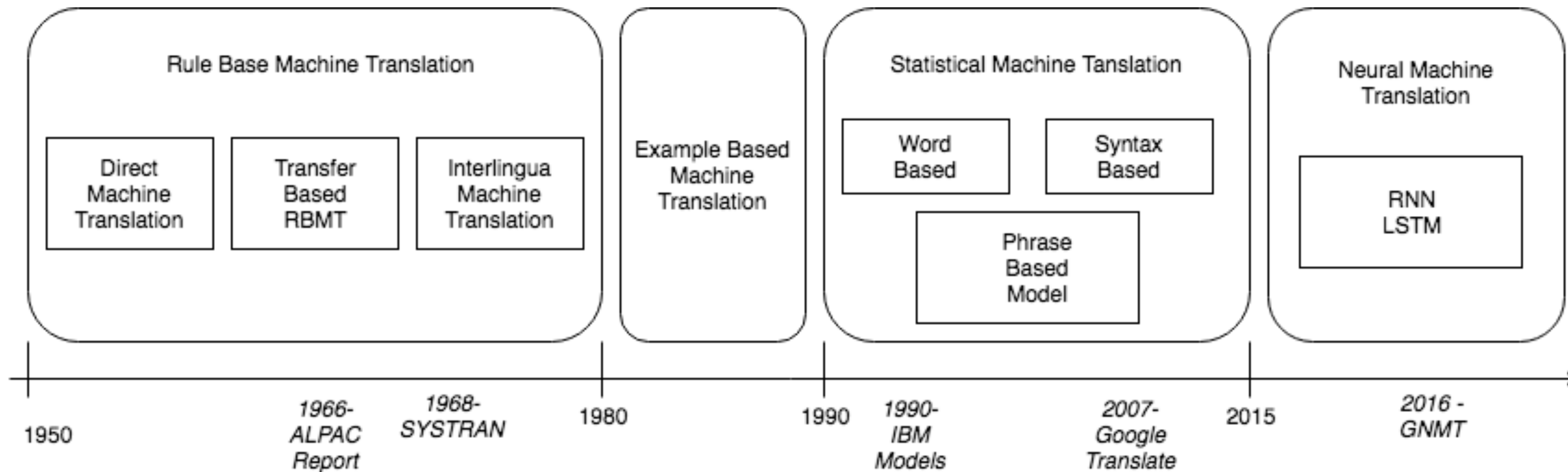
Translate

English French Spanish Chinese - detected

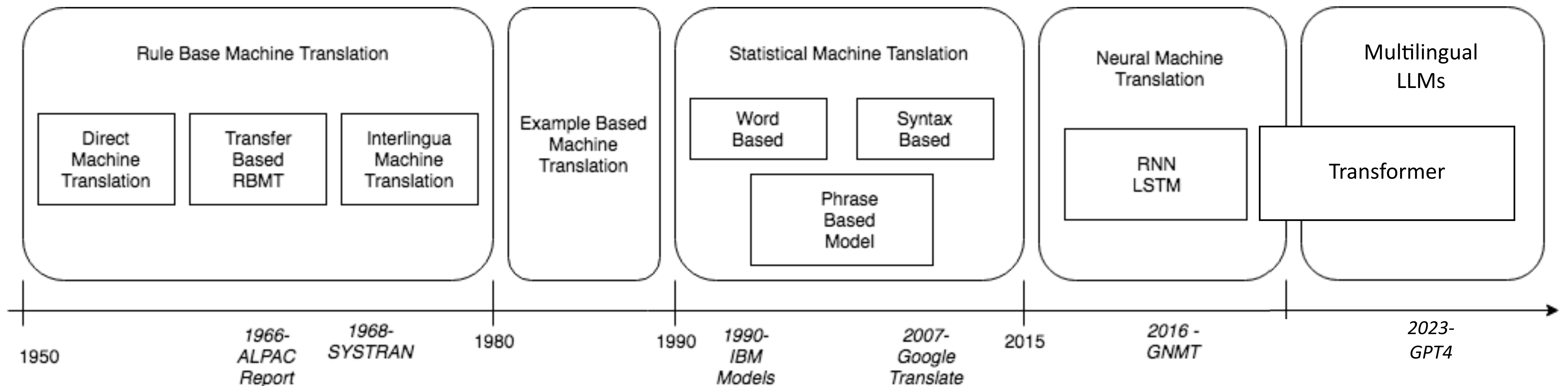
特朗普偕家人在白宫阳台观看百年一遇日全食

Trump and his family watch the once-in-a-century total solar eclipse from the White House balcony

History of MT



History of MT



Parallel Training Corpus

	facing with the swelling flow of through traffic zooming past their doors .		recanta de inconvenientes que más y más gente tiene que soportar por el tráfico que pasa por delante_de sus casas , que aumenta a_diario .
5	#77501757 Weekend traffic bans and traffic jams are a curse to road transport .	#74765580	Las prohibiciones de conducir los fines de semana y los embotellamientos asolan el transporte por carretera .
6	#79500725 Some people also want to recoup the cost of traffic jams from those who get stuck in them , according to the ' polluter pays ' principle .	#76764676	Algunos son partidarios de que incluso los costes ocasionados por los atascos se carguen a el ciudadano que se encuentra atrapado en ellos , de conformidad con el principio de que " quien contamina paga " .
7	#79500765 I think this is an excellent principle and I would like to see it applied in full , but not to traffic jams .	#76764713	Me parece un principio acertado y estoy dispuesta a aplicarlo íntegramente , pero no sobre los atascos , ya_que éstos son un claro indicio de el fracaso de la política gubernamental en_materia_de infraestructuras .
8	#79500768 Traffic jams are indicative of failed government policy on the infrastructure front , which is why the government itself , certainly in the Netherlands , must be regarded as the polluter .	#76764747	Por eso es preciso subrayar que en estos casos quien contamina es el propio Gobierno , a el menos en los Países_Bajos .
9	#81309716 This would increase traffic jams , weaken road safety and increase costs .	#78586130	Esto aumentaría los atascos , mermaría la seguridad vial e incrementaría los costes .
10	#81997391 In the previous legislature , Parliament gave its opinion on the Commission ' s proposals on the simplification of vertical directives on sugar , honey , fruit juices , milk and jams .	#79281114	En efecto , durante la precedente legislatura , el Parlamento se manifestó sobre las propuestas de la Comisión relativas a la simplificación de directivas verticales sobre el azúcar , la miel , los zumos de frutas , la leche y las confituras .
11	#81998167 For jams , I personally reintroduced an amendment that was not accepted by the Committee on the Environment , Public Health and Consumer Policy , but which I hold to .	#79281936	Para las confituras , yo personalmente volví a introducir una enmienda que no fue aceptada por la Comisión_de_Medio_Ambiente , Salud_Pública y Política_de_el_Consumidor , pero que es importante para mí .
12	#81998209 It concerns not accepting the general use of a chemical flavouring in jams and marmalades , that is vanillin .	#79281966	Se trata de no aceptar la utilización generalizada de un aroma químico en las confituras y " marmalades " , a saber , la vainillina .
13	#82800065 This is highlighted particularly in towns where it is necessary to find ways of solving environmental problems and the difficulties caused by traffic jams .	#80085988	Esto se pone_de_relieve aún más en las ciudades , en las que hay que encontrar medios para eliminar los inconvenientes derivados de los problemas medioambientales y de la congestión de el tráfico .

Phrase-Based MT

- ▶ Key idea: translation works better the bigger chunks you use

Phrase-Based MT

- ▶ Key idea: translation works better the bigger chunks you use
- ▶ Remember phrases from training data, translate piece-by-piece and stitch those pieces together to translate

Phrase-Based MT

- ▶ Key idea: translation works better the bigger chunks you use
- ▶ Remember phrases from training data, translate piece-by-piece and stitch those pieces together to translate
 - ▶ How to identify phrases? Word alignment over source-target bitext

Phrase-Based MT

- ▶ Key idea: translation works better the bigger chunks you use
- ▶ Remember phrases from training data, translate piece-by-piece and stitch those pieces together to translate
 - ▶ How to identify phrases? Word alignment over source-target bitext
 - ▶ How to stitch together? Language model over target language

Phrase-Based MT

- ▶ Key idea: translation works better the bigger chunks you use
- ▶ Remember phrases from training data, translate piece-by-piece and stitch those pieces together to translate
 - ▶ How to identify phrases? Word alignment over source-target bitext
 - ▶ How to stitch together? Language model over target language
- ▶ Decoder takes phrases and a language model and searches over possible translations

Phrase-Based MT

- ▶ Key idea: translation works better the bigger chunks you use
- ▶ Remember phrases from training data, translate piece-by-piece and stitch those pieces together to translate
 - ▶ How to identify phrases? Word alignment over source-target bitext
 - ▶ How to stitch together? Language model over target language
 - ▶ Decoder takes phrases and a language model and searches over possible translations
- ▶ NOT like standard discriminative models (take a bunch of translation pairs, learn a ton of parameters in an end-to-end way)

Evaluating MT

- ▶ Fluency: does it sound good in the target language?
- ▶ Fidelity/adequacy: does it capture the meaning of the original?

Evaluating MT

- ▶ Fluency: does it sound good in the target language?
- ▶ Fidelity/adequacy: does it capture the meaning of the original?
- ▶ BLEU score: geometric mean of 1-, 2-, 3-, and 4-gram precision vs. a reference, multiplied by brevity penalty

Evaluating MT

- ▶ Fluency: does it sound good in the target language?
- ▶ Fidelity/adequacy: does it capture the meaning of the original?
- ▶ BLEU score: geometric mean of 1-, 2-, 3-, and 4-gram precision vs. a reference, multiplied by brevity penalty

		1-gram	2-gram	3-gram
hypothesis 1	<u>I</u> am exhausted	3/3	1/2	0/1
hypothesis 2	Tired is <u>I</u>	1/3	0/2	0/1
hypothesis 3	<u>I</u> I I	1/3	0/2	0/1
reference 1	<u>I</u> am tired			
reference 2	<u>I</u> am ready to sleep now and so <u>exhausted</u>			

Evaluating MT

- ▶ Fluency: does it sound good in the target language?
- ▶ Fidelity/adequacy: does it capture the meaning of the original?
- ▶ BLEU score: geometric mean of 1-, 2-, 3-, and 4-gram precision vs. a reference, multiplied by brevity penalty

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

hypothesis 1

I am exhausted

hypothesis 2

Tired is I

hypothesis 3

I I I

reference 1

I am tired

reference 2

I am ready to sleep now and so exhausted

1-gram	2-gram	3-gram
3/3	1/2	0/1
1/3	0/2	0/1
1/3	0/2	0/1

Evaluating MT

- ▶ Fluency: does it sound good in the target language?
- ▶ Fidelity/adequacy: does it capture the meaning of the original?
- ▶ BLEU score: geometric mean of 1-, 2-, 3-, and 4-gram precision vs. a reference, multiplied by brevity penalty

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right).$$

Evaluating MT

- ▶ Fluency: does it sound good in the target language?
- ▶ Fidelity/adequacy: does it capture the meaning of the original?
- ▶ BLEU score: geometric mean of 1-, 2-, 3-, and 4-gram precision vs. a reference, multiplied by brevity penalty

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) . \quad \text{▶ Typically } n = 4, w_i = 1/4$$

Evaluating MT

- ▶ Fluency: does it sound good in the target language?
- ▶ Fidelity/adequacy: does it capture the meaning of the original?
- ▶ BLEU score: geometric mean of 1-, 2-, 3-, and 4-gram precision vs. a reference, multiplied by brevity penalty

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) . \quad \text{▶ Typically } n = 4, w_i = 1/4$$

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases} .$$

Evaluating MT

- ▶ Fluency: does it sound good in the target language?
- ▶ Fidelity/adequacy: does it capture the meaning of the original?
- ▶ BLEU score: geometric mean of 1-, 2-, 3-, and 4-gram precision vs. a reference, multiplied by brevity penalty

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) . \quad \text{▶ Typically } n = 4, w_i = 1/4$$

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases} . \quad \begin{array}{l} \text{▶ } r = \text{length of reference} \\ c = \text{length of prediction} \end{array}$$

Evaluating MT

- ▶ Fluency: does it sound good in the target language?
- ▶ Fidelity/adequacy: does it capture the meaning of the original?
- ▶ BLEU score: geometric mean of 1-, 2-, 3-, and 4-gram precision vs. a reference, multiplied by brevity penalty

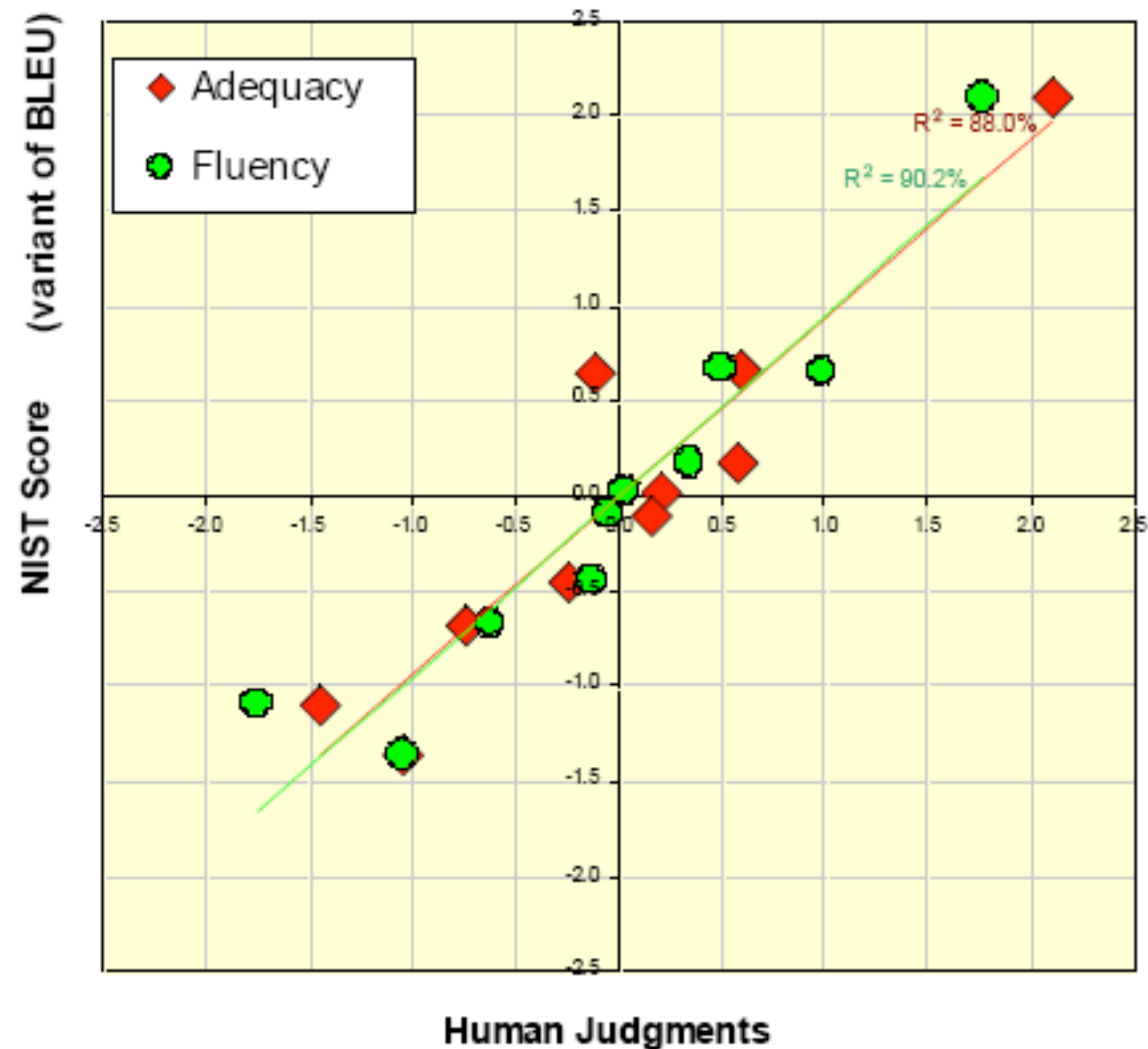
$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) . \quad \text{▶ Typically } n = 4, w_i = 1/4$$

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases} . \quad \begin{array}{l} \text{▶ } r = \text{length of reference} \\ c = \text{length of prediction} \end{array}$$

- ▶ Does this capture fluency and adequacy?

BLEU Score

- ▶ Better methods with human-in-the-loop
- ▶ HTER: human-assisted translation error rate
- ▶ If you're building real MT systems, you do user studies. In academia, you mostly use BLEU

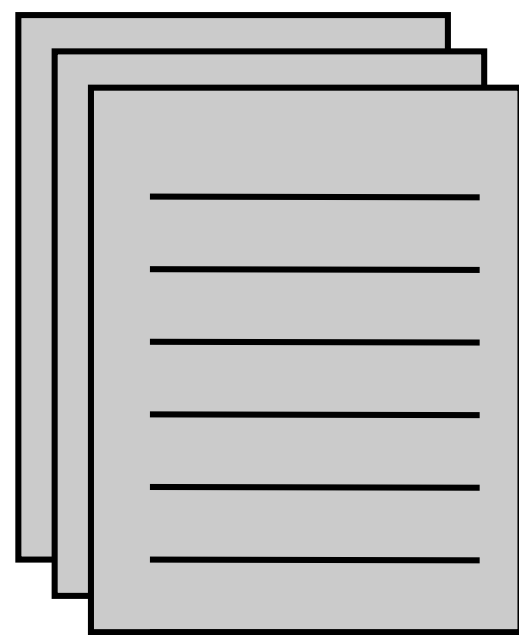


Phrase-Based MT

- ▶ Goal: translate from Foreign language to English

cat		chat		0.9
the cat		le chat		0.8
dog		chien		0.8
house		maison		0.6
my house		ma maison		0.9
language		langue		0.9
...				

Phrase table $P(f|e)$



Unlabeled English data

Language
model $P(e)$

$$P(e|f) \propto P(f|e)P(e)$$

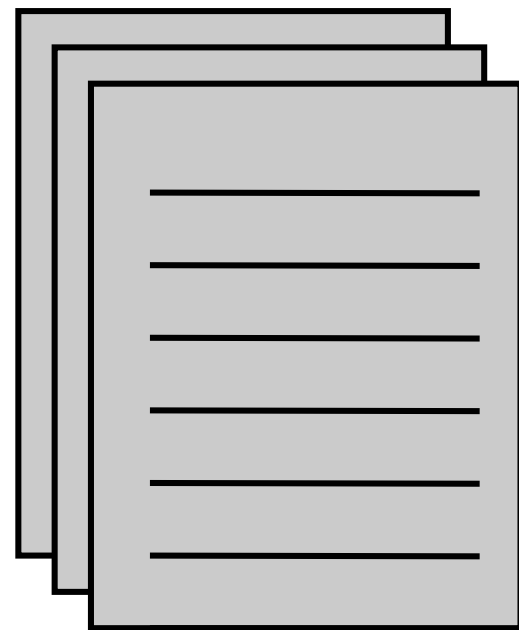
Noisy channel model:
combine scores from
translation model +
language model to
translate foreign to
English

“Translate faithfully but make fluent English”

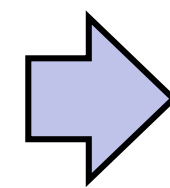
Phrase-Based MT

cat ||| chat ||| 0.9
the cat ||| le chat ||| 0.8
dog ||| chien ||| 0.8
house ||| maison ||| 0.6
my house ||| ma maison ||| 0.9
language ||| langue ||| 0.9
...

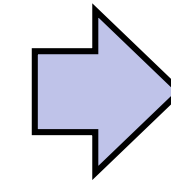
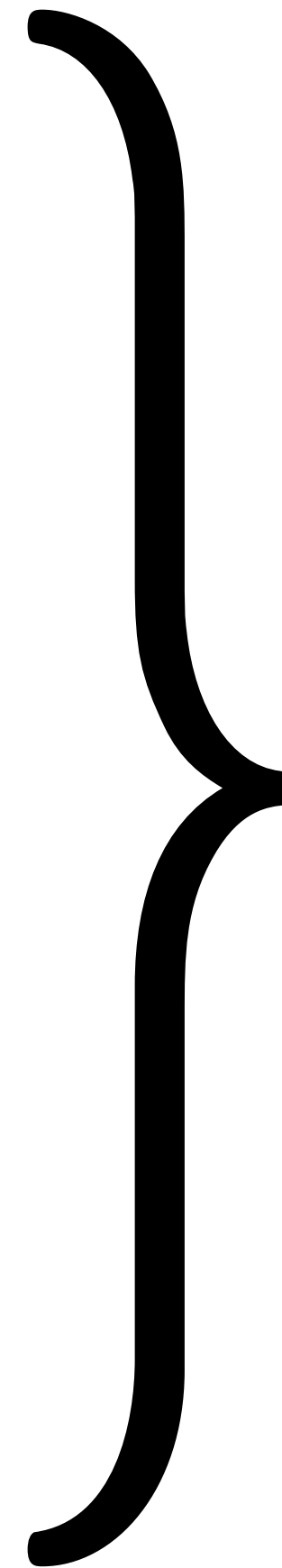
Phrase table $P(f|e)$



Unlabeled English data



Language
model $P(e)$



$$P(e|f) \propto P(f|e)P(e)$$

Noisy channel model:
combine scores from
translation model +
language model to
translate foreign to
English

“Translate faithfully but make fluent English”

Language Modeling

N-gram Language Models

I visited San _____ put a distribution over the next word

N-gram Language Models

I visited San _____ put a distribution over the next word

- ▶ Simple generative model: distribution of next word is a multinomial distribution conditioned on previous $n-1$ words

N-gram Language Models

I visited San _____ put a distribution over the next word

- ▶ Simple generative model: distribution of next word is a multinomial distribution conditioned on previous $n-1$ words

$$P(x|\text{visited San}) = \frac{\text{count}(\text{visited San}, x)}{\text{count}(\text{visited San})}$$

N-gram Language Models

I visited San _____ put a distribution over the next word

- ▶ Simple generative model: distribution of next word is a multinomial distribution conditioned on previous n-1 words

$$P(x|\text{visited San}) = \frac{\text{count}(\text{visited San}, x)}{\text{count}(\text{visited San})}$$

Maximum likelihood estimate of this probability from a corpus

N-gram Language Models

I visited San _____ put a distribution over the next word

- ▶ Simple generative model: distribution of next word is a multinomial distribution conditioned on previous n-1 words

$$P(x|\text{visited San}) = \frac{\text{count}(\text{visited San}, x)}{\text{count}(\text{visited San})}$$

Maximum likelihood estimate of this probability from a corpus

- ▶ Just relies on counts, even in 2008 could scale up to 1.3M word types, 4B n-grams (all 5-grams occurring >40 times on the Web)

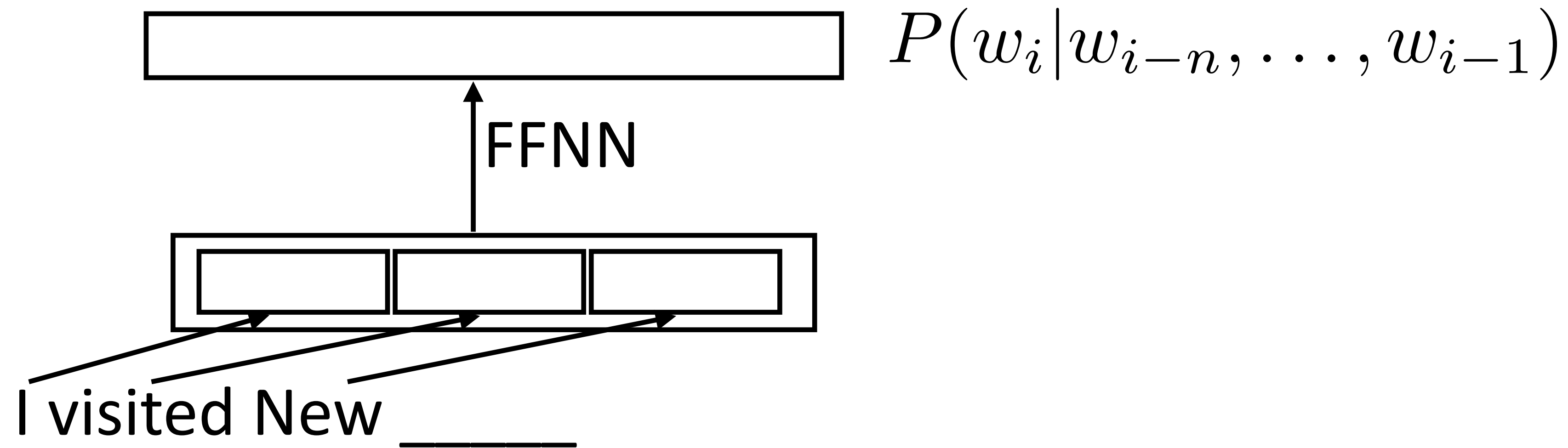
Neural Language Models

- ▶ Early work: feedforward neural networks looking at context

Mnih and Hinton (2003)

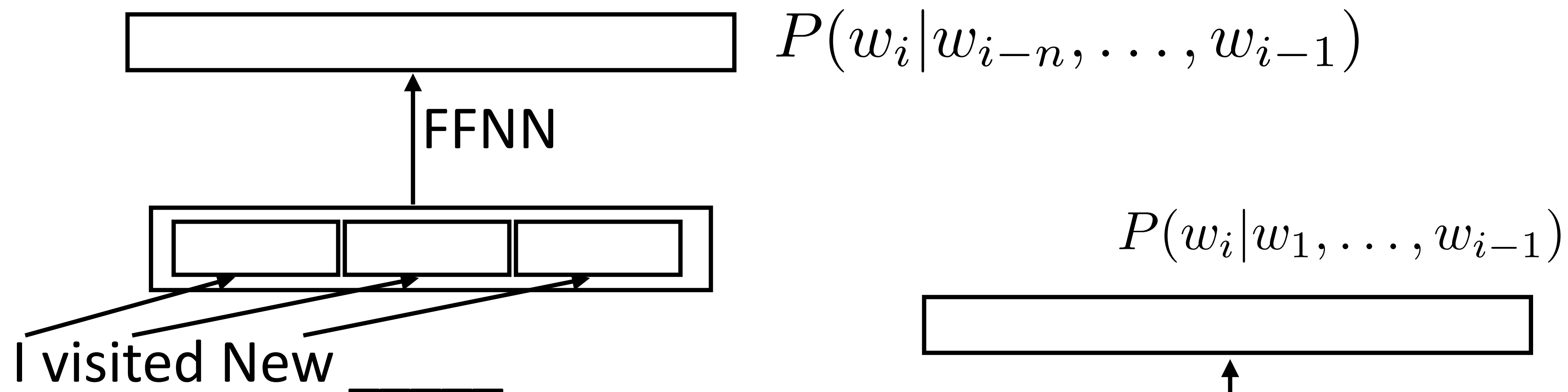
Neural Language Models

- ▶ Early work: feedforward neural networks looking at context

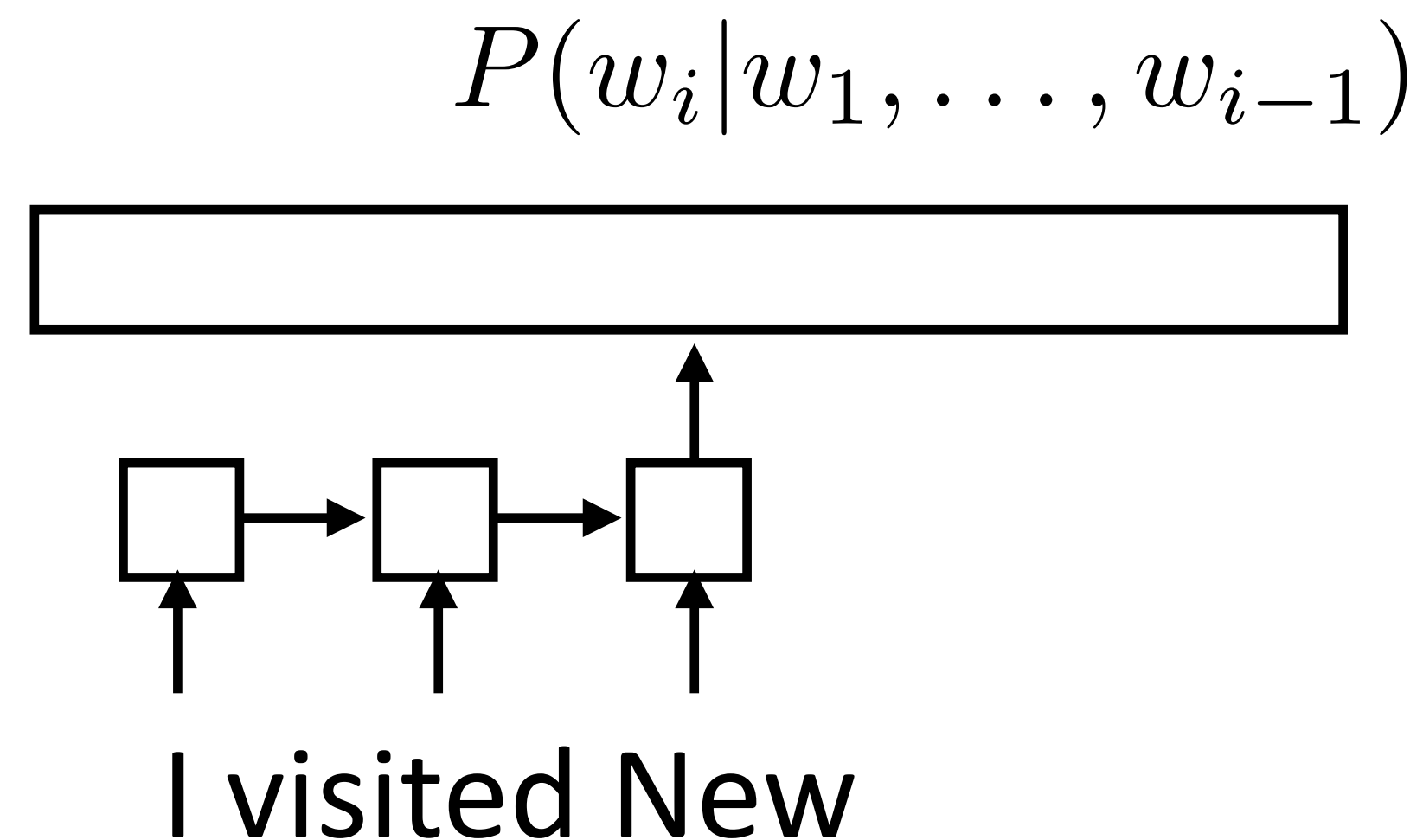


Neural Language Models

- ▶ Early work: feedforward neural networks looking at context

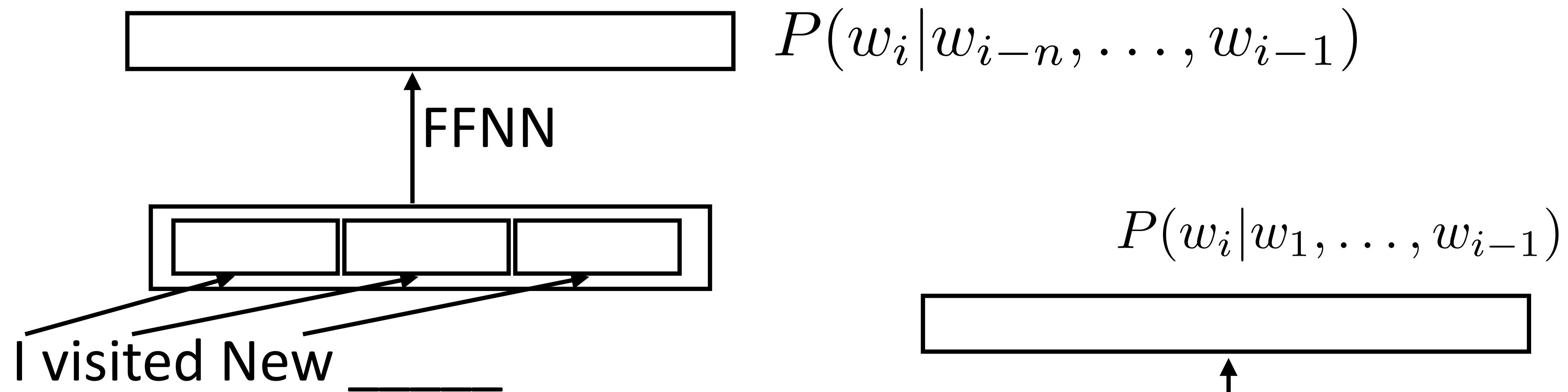


- ▶ Variable length context with RNNs:

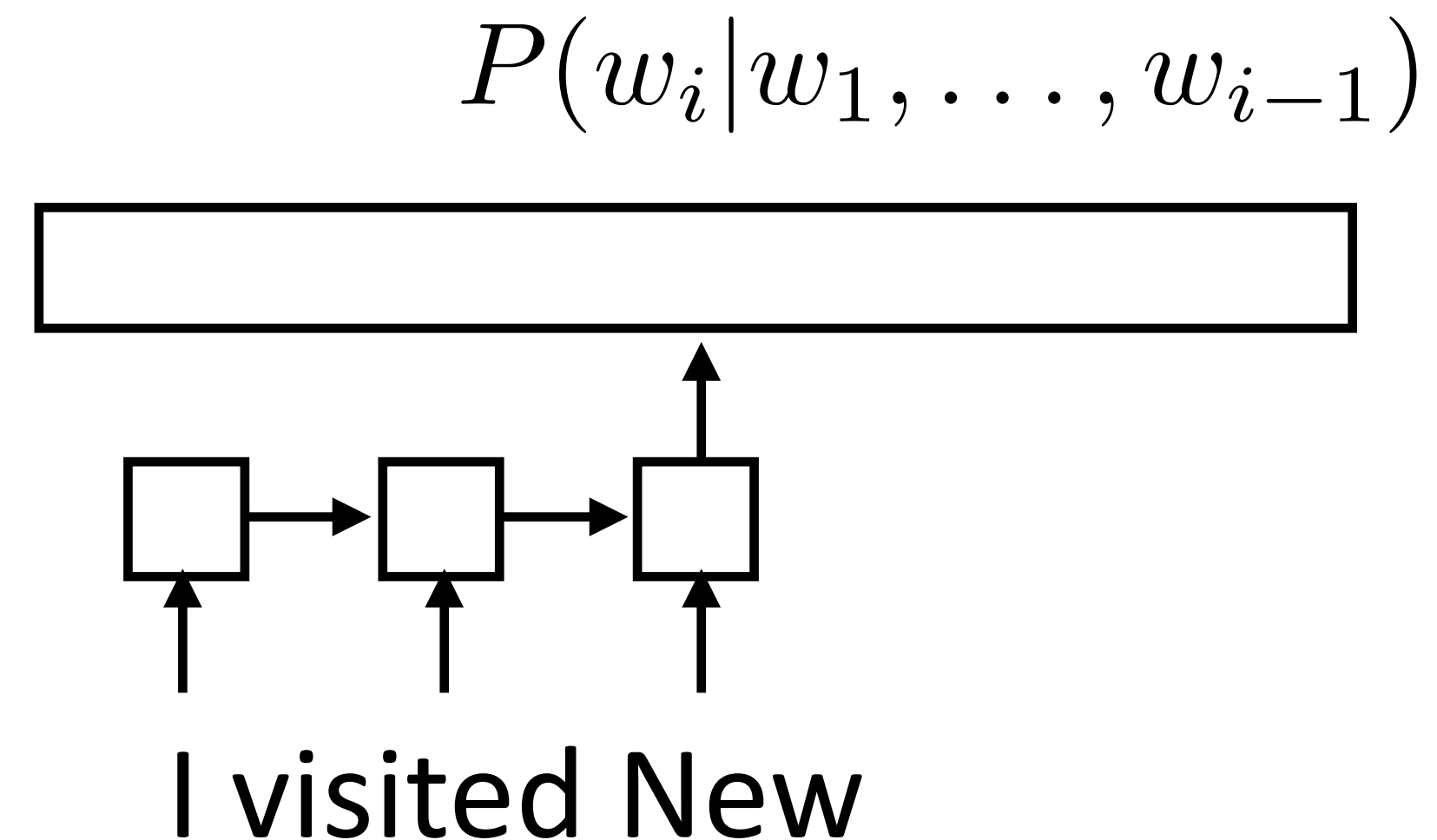


Neural Language Models

- ▶ Early work: feedforward neural networks looking at context

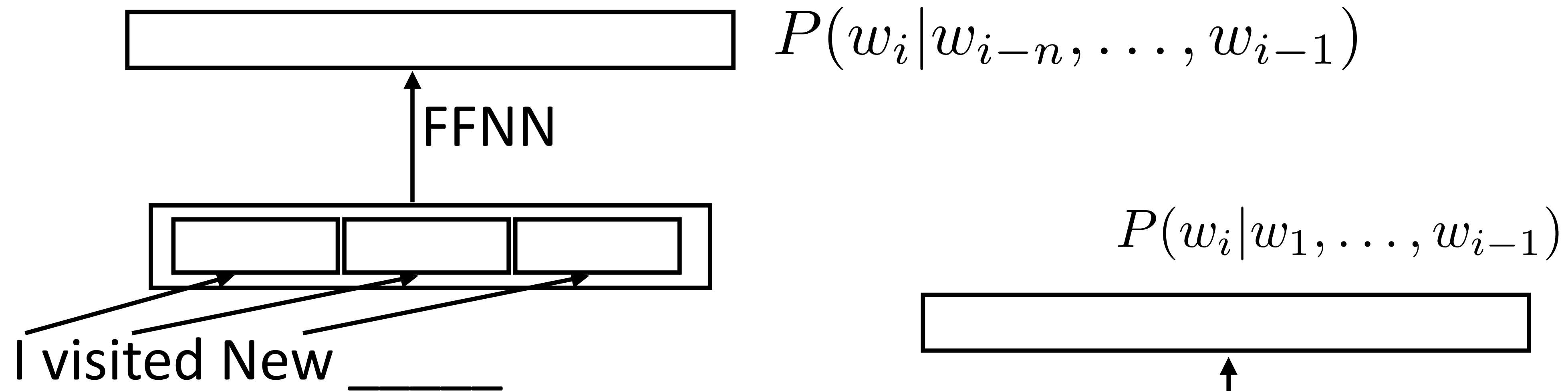


- ▶ Variable length context with RNNs:
 - ▶ Works like a decoder with no encoder

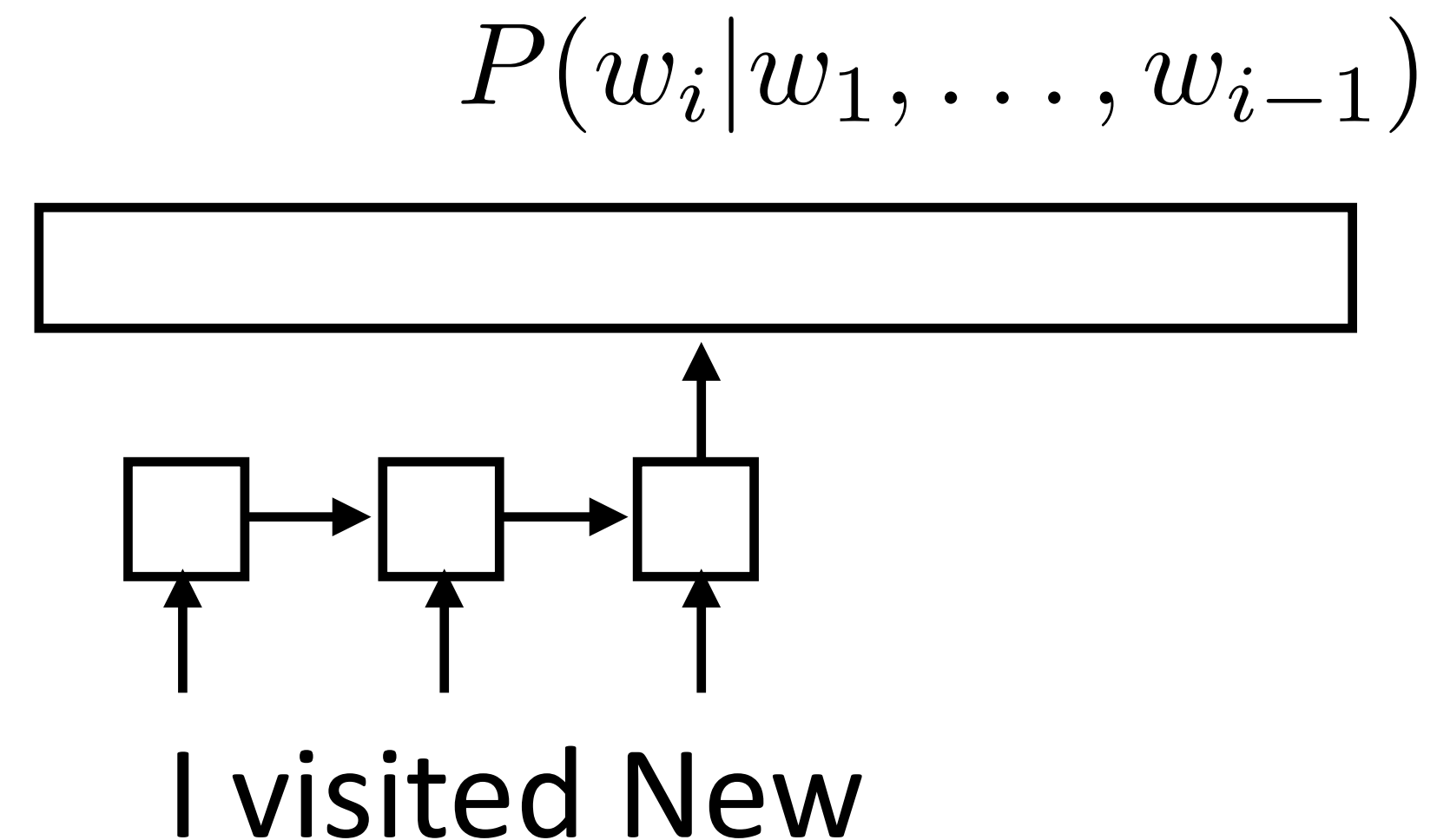


Neural Language Models

- ▶ Early work: feedforward neural networks looking at context



- ▶ Variable length context with RNNs:
 - ▶ Works like a decoder with no encoder



- ▶ Slow to train over lots of data!

Mnih and Hinton (2003)

Evaluation

Evaluation

- ▶ (One sentence) negative log likelihood: $\sum_{i=1}^n \log p(x_i | x_1, \dots, x_{i-1})$

Evaluation

- ▶ (One sentence) negative log likelihood: $\sum_{i=1}^n \log p(x_i | x_1, \dots, x_{i-1})$
- ▶ Perplexity: $2^{-\frac{1}{n} \sum_{i=1}^n \log_2 p(x_i | x_1, \dots, x_{i-1})}$

Evaluation

- ▶ (One sentence) negative log likelihood: $\sum_{i=1}^n \log p(x_i | x_1, \dots, x_{i-1})$
- ▶ Perplexity: $2^{-\frac{1}{n} \sum_{i=1}^n \log_2 p(x_i | x_1, \dots, x_{i-1})}$
 - ▶ NLL (base 2) averaged over the sentence, exponentiated

Evaluation

- ▶ (One sentence) negative log likelihood: $\sum_{i=1}^n \log p(x_i | x_1, \dots, x_{i-1})$
- ▶ Perplexity: $2^{-\frac{1}{n} \sum_{i=1}^n \log_2 p(x_i | x_1, \dots, x_{i-1})}$
 - ▶ NLL (base 2) averaged over the sentence, exponentiated
 - ▶ NLL = -2 -> on average, correct thing has prob 1/4 -> PPL = 4. PPL is sort of like branching factor

Results

Merity et al. (2017), Melis et al. (2017)

Results

- ▶ Evaluate on Penn Treebank: small dataset (1M words) compared to what's used in MT, but common benchmark

Merity et al. (2017), Melis et al. (2017)

Results

- ▶ Evaluate on Penn Treebank: small dataset (1M words) compared to what's used in MT, but common benchmark
- ▶ Kneser-Ney 5-gram model with cache: PPL = 125.7

Merity et al. (2017), Melis et al. (2017)

Results

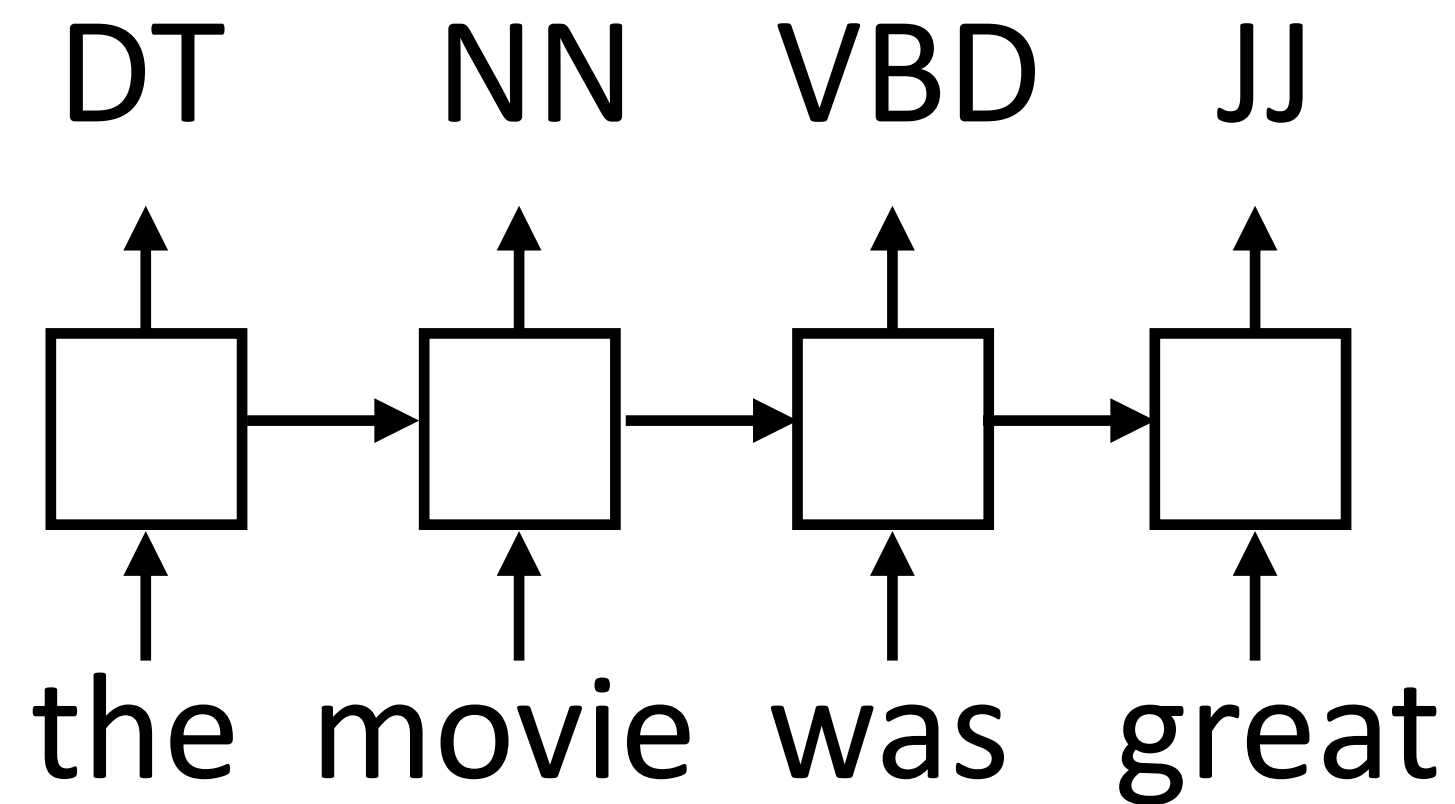
- ▶ Evaluate on Penn Treebank: small dataset (1M words) compared to what's used in MT, but common benchmark
- ▶ Kneser-Ney 5-gram model with cache: PPL = 125.7
- ▶ LSTM: PPL $\sim$ 60-80 (depending on how much you optimize it)

Merity et al. (2017), Melis et al. (2017)

Encoder-Decoder Models

Recap: RNN

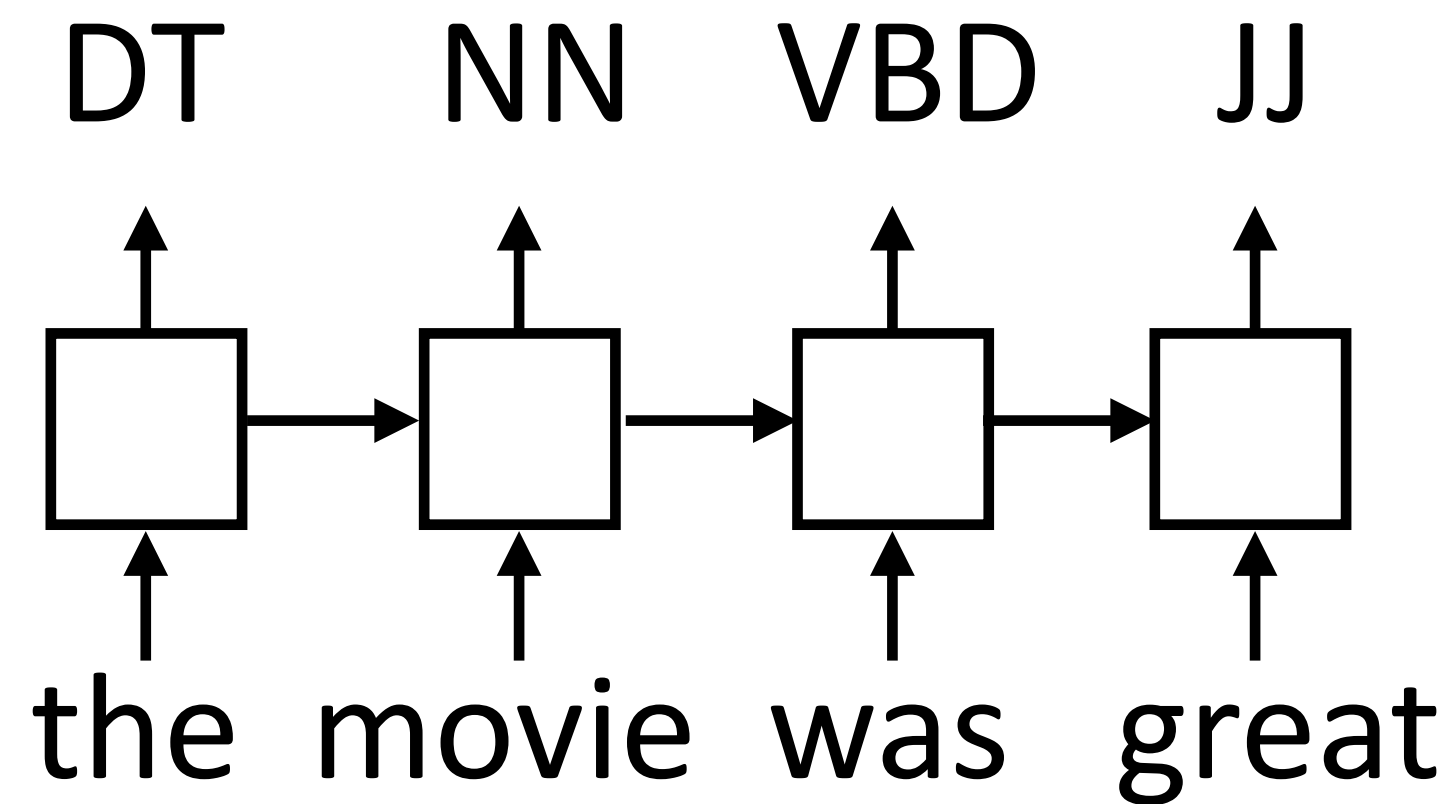
- ▶ Transducer: make some prediction for each element in a sequence



output $\mathbf{y}$ = score for each tag, then softmax

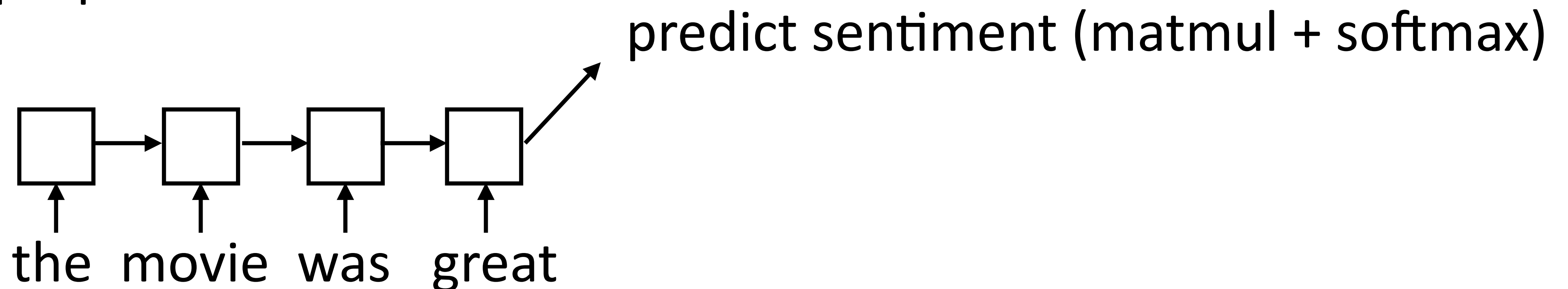
Recap: RNN

- ▶ Transducer: make some prediction for each element in a sequence



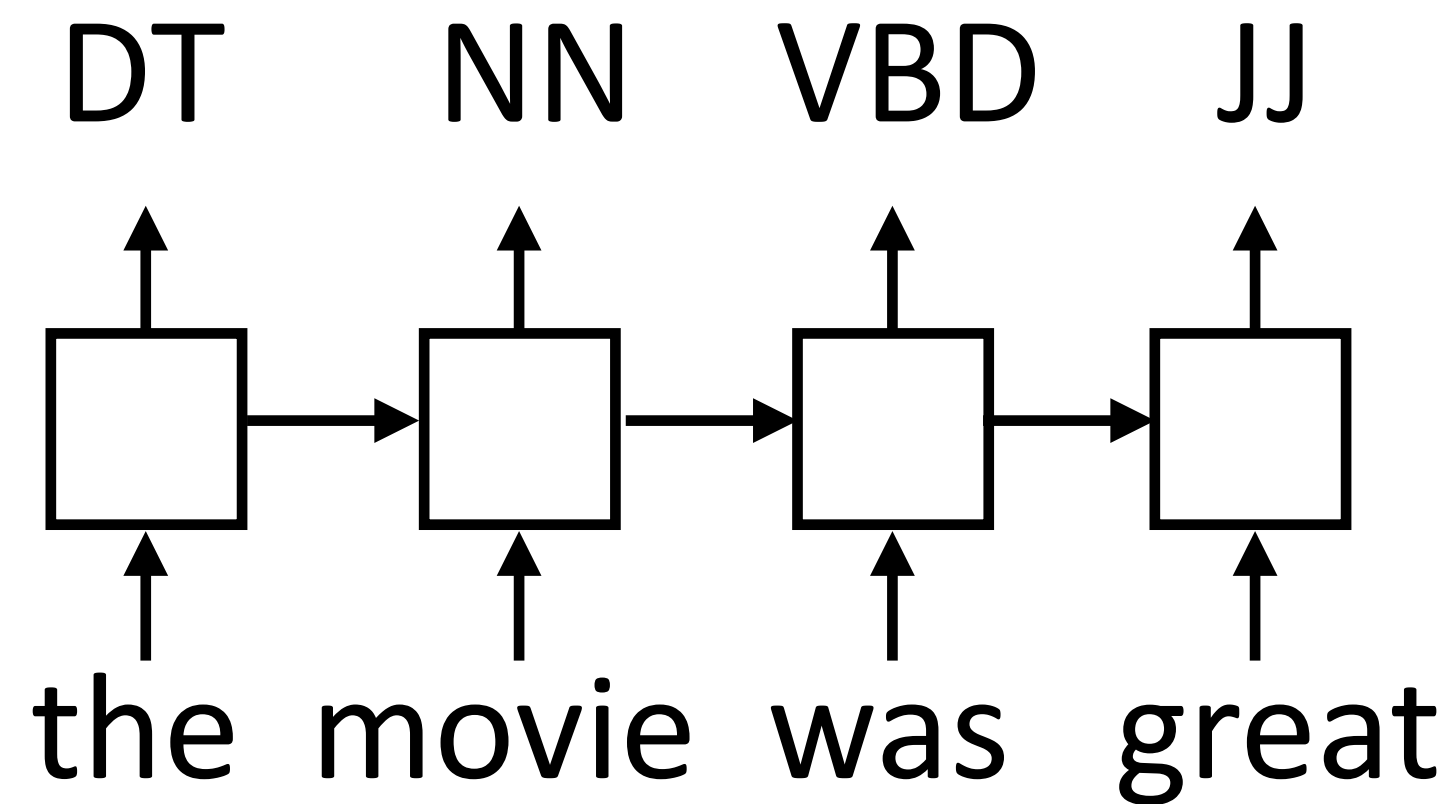
output $\mathbf{y}$ = score for each tag, then softmax

- ▶ Encoder: encode a sequence into a fixed-sized vector and use that for some purpose



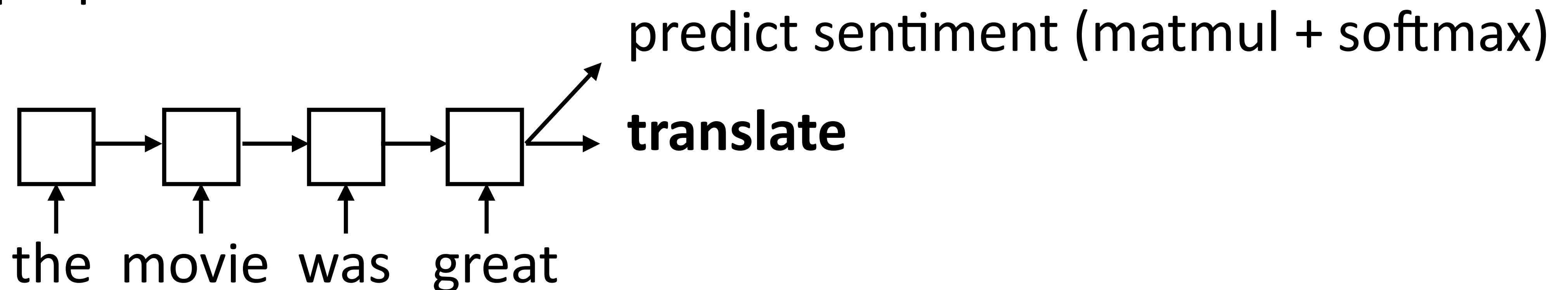
Recap: RNN

- ▶ Transducer: make some prediction for each element in a sequence



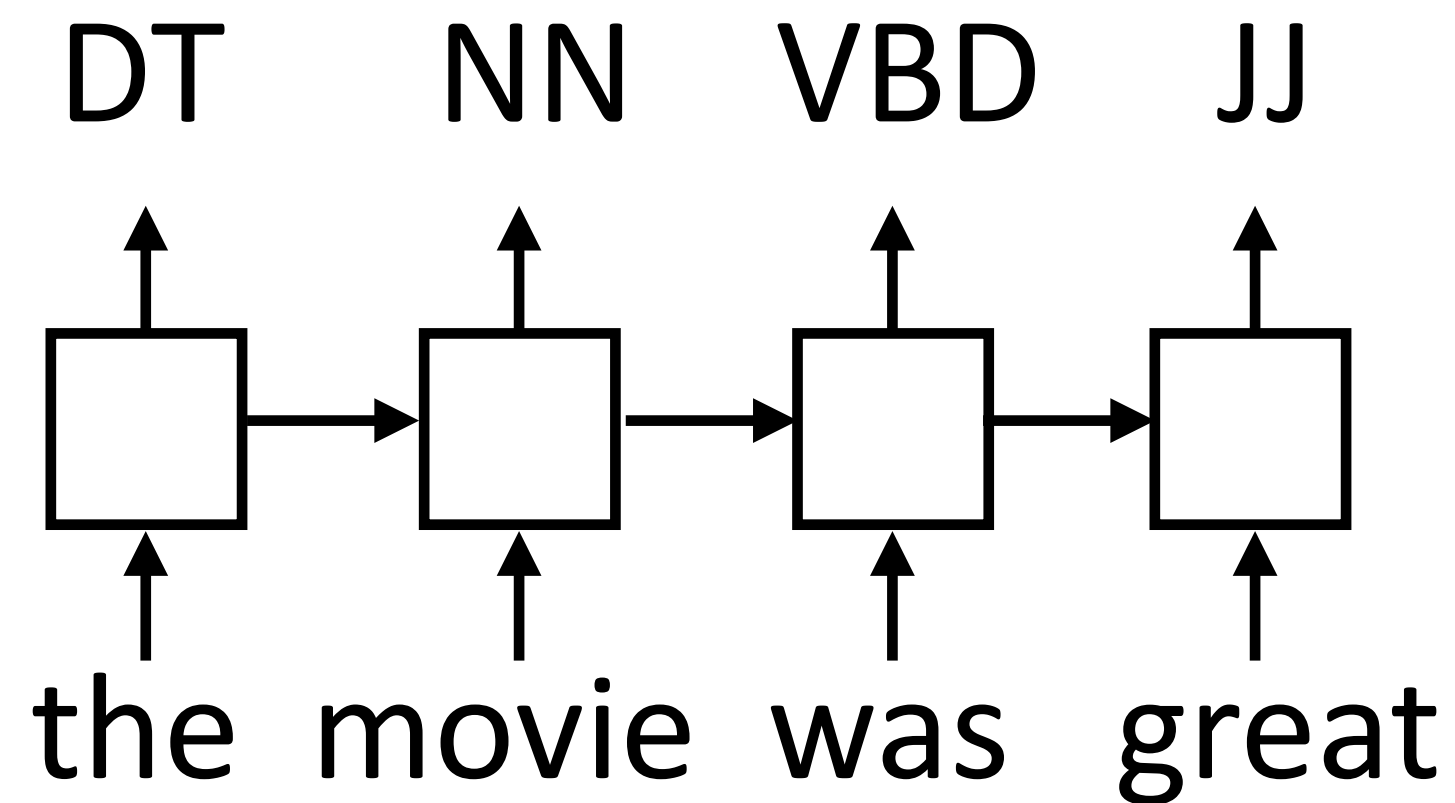
output $\mathbf{y}$ = score for each tag, then softmax

- ▶ Encoder: encode a sequence into a fixed-sized vector and use that for some purpose



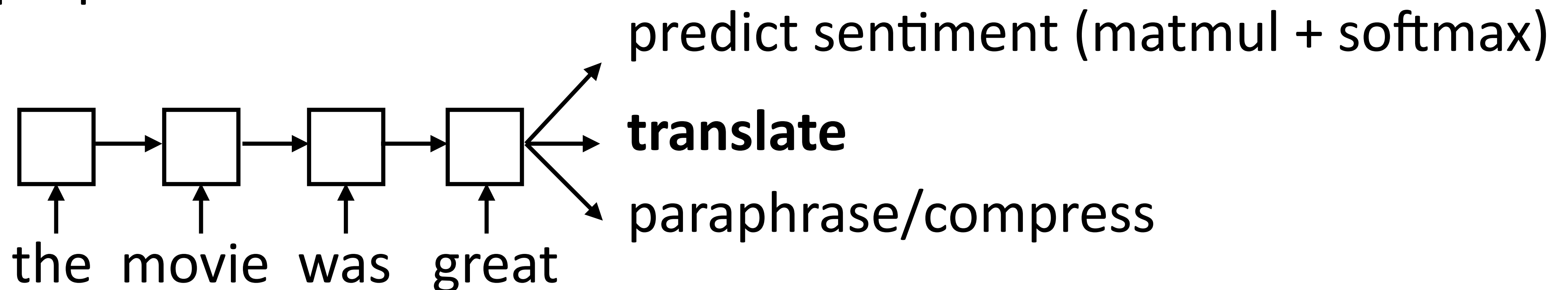
Recap: RNN

- ▶ Transducer: make some prediction for each element in a sequence



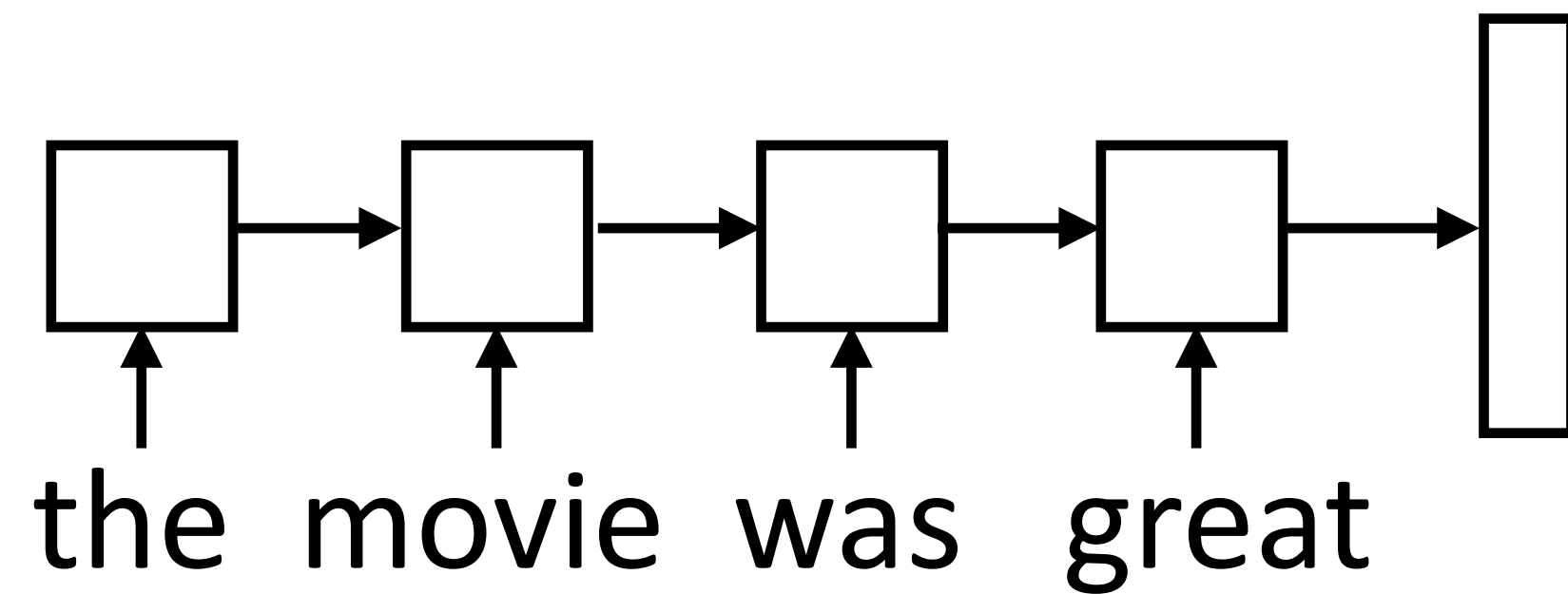
output $\mathbf{y}$ = score for each tag, then softmax

- ▶ Encoder: encode a sequence into a fixed-sized vector and use that for some purpose



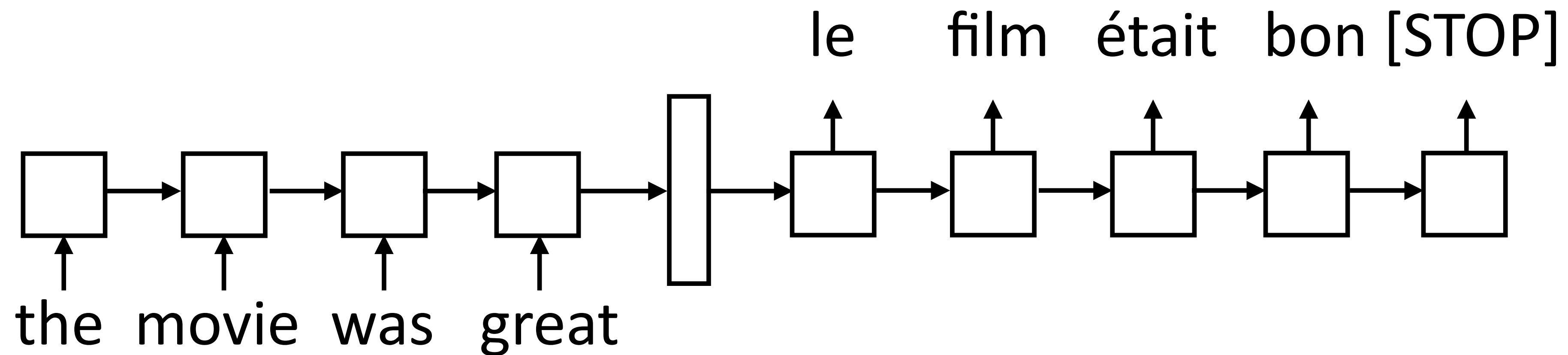
Encoder-Decoder

- ▶ Encode a sequence into a fixed-sized vector



Encoder-Decoder

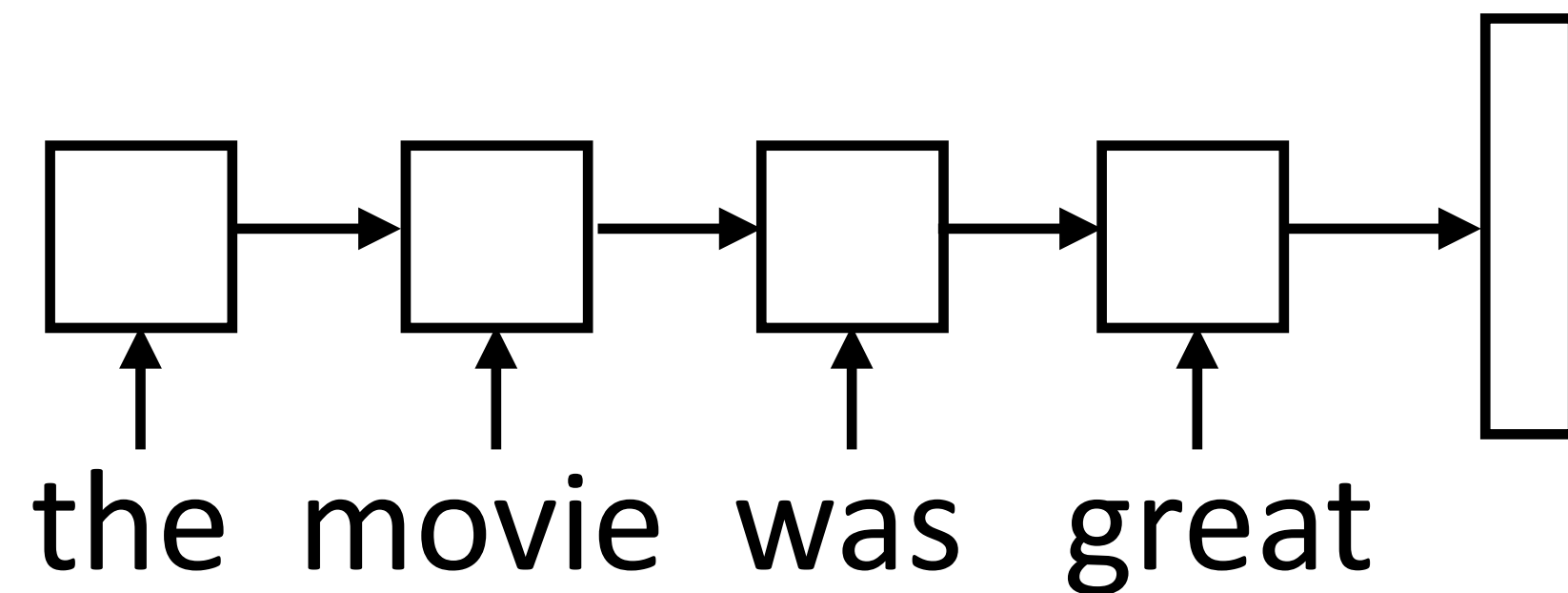
- ▶ Encode a sequence into a fixed-sized vector



- ▶ Now use that vector to produce a series of tokens as output from a separate LSTM *decoder*

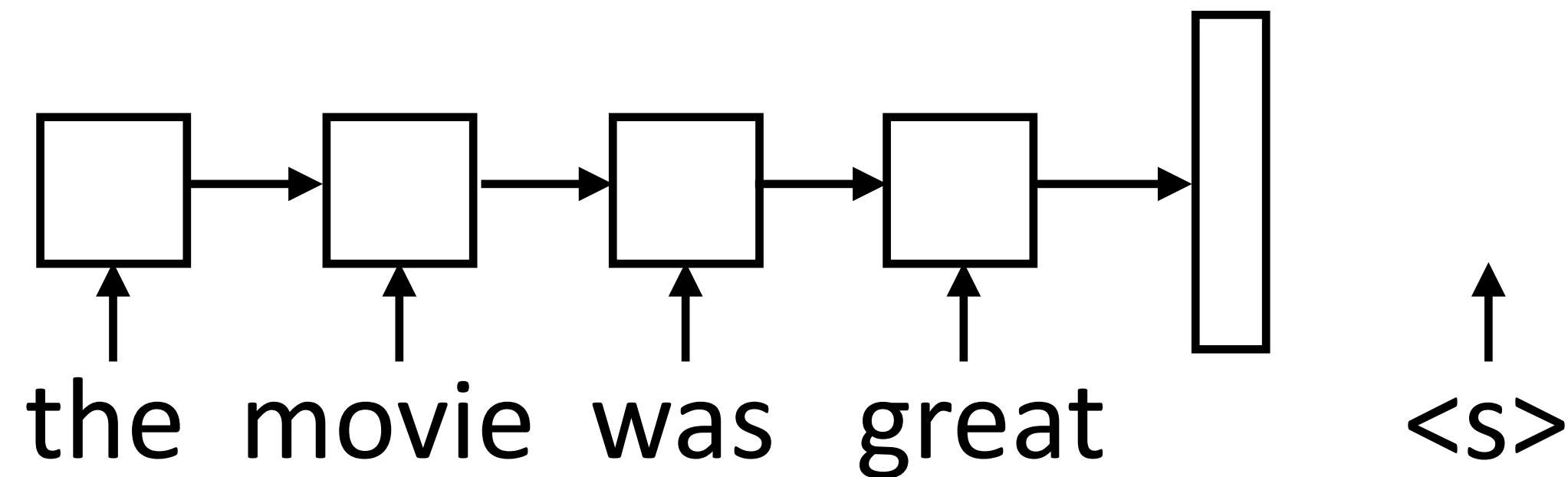
Model

- ▶ Generate next word conditioned on previous word as well as hidden state



Model

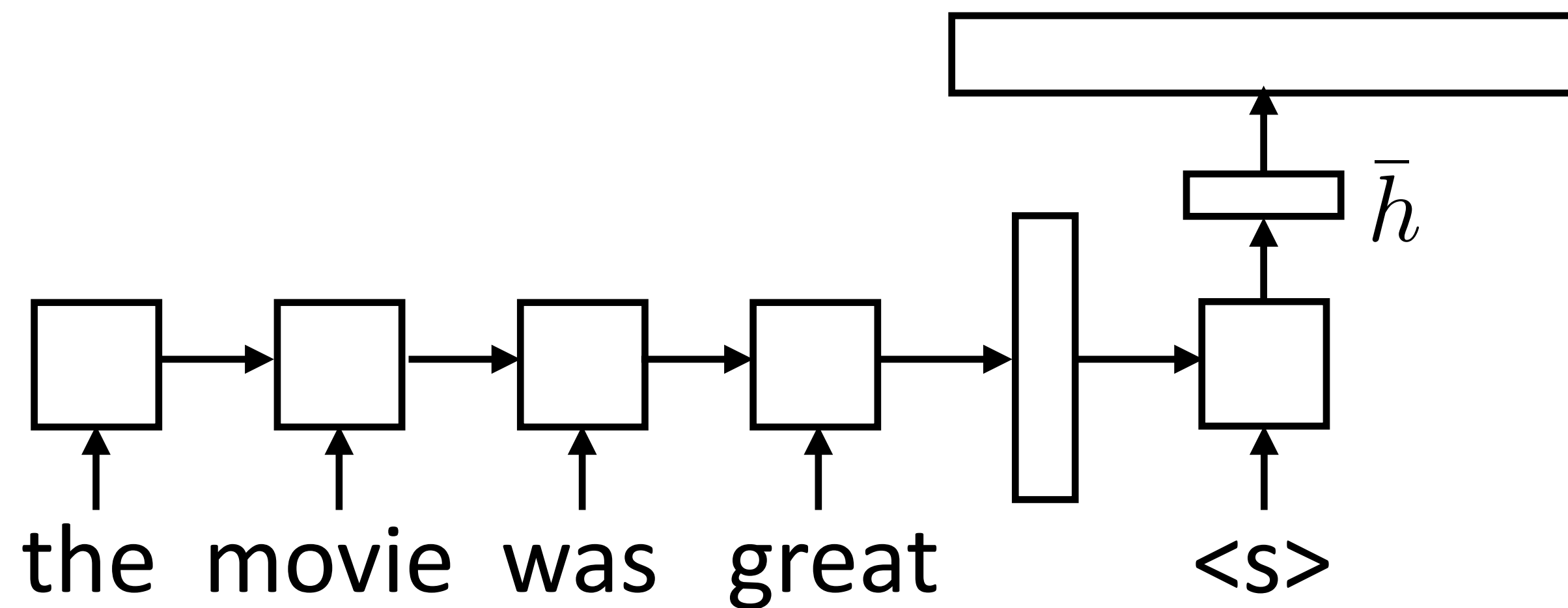
- ▶ Generate next word conditioned on previous word as well as hidden state



Model

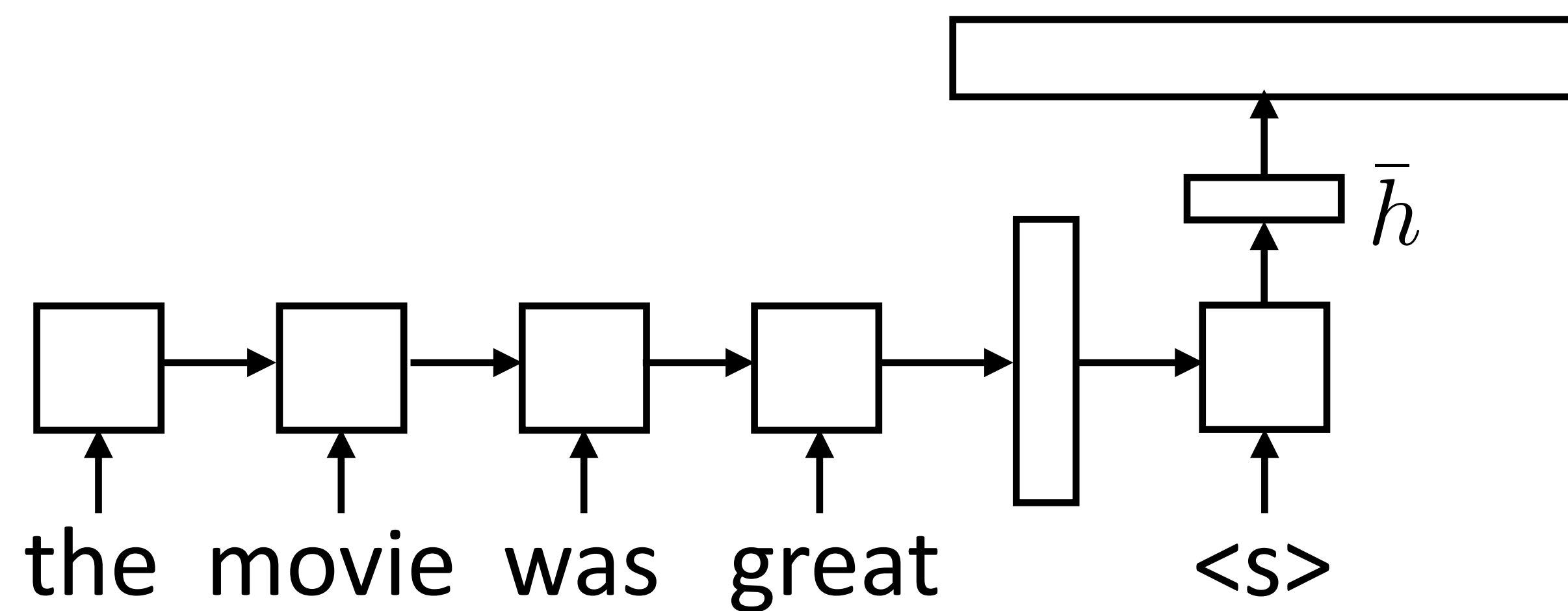
- ▶ Generate next word conditioned on previous word as well as hidden state
- ▶ W size is $|\text{vocab}| \times |\text{hidden state}|$, softmax over entire vocabulary

$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h})$$



Model

- ▶ Generate next word conditioned on previous word as well as hidden state
- ▶ W size is $|\text{vocab}| \times |\text{hidden state}|$, softmax over entire vocabulary

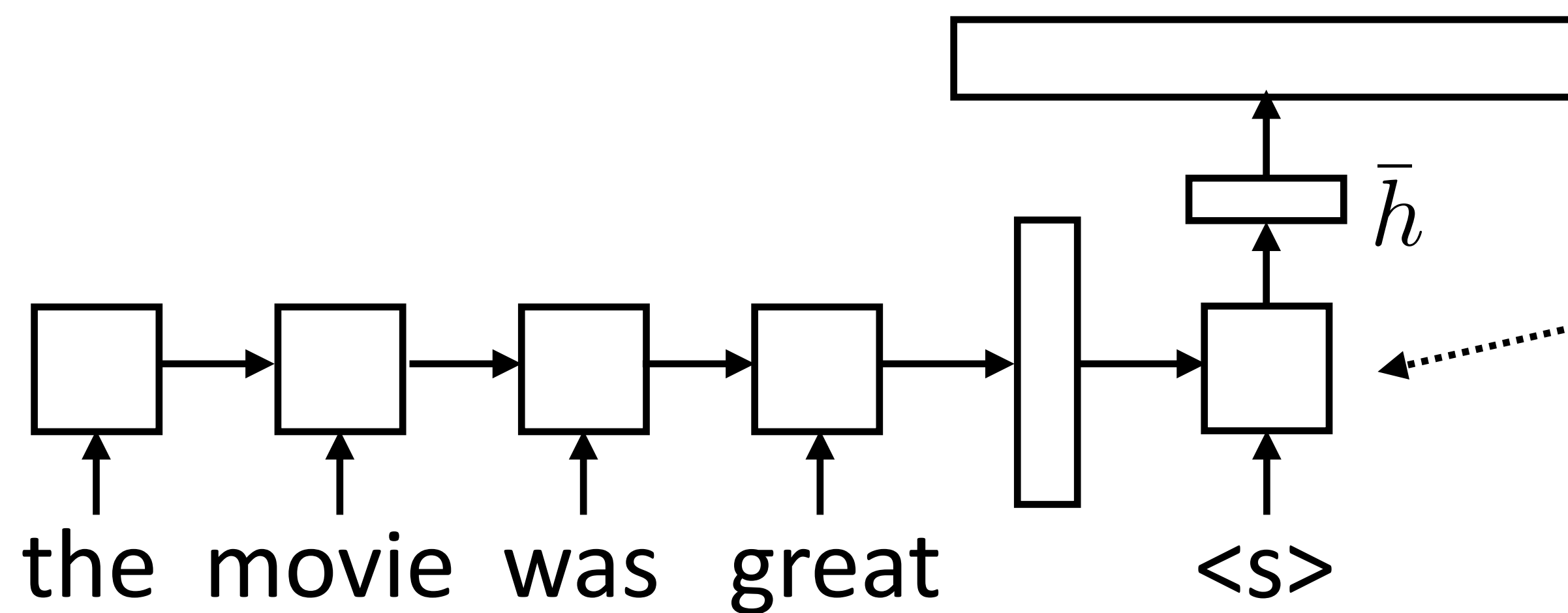


$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h})$$

$$P(\mathbf{y} | \mathbf{x}) = \prod_{i=1}^n P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$$

Model

- ▶ Generate next word conditioned on previous word as well as hidden state
- ▶ W size is $|\text{vocab}| \times |\text{hidden state}|$, softmax over entire vocabulary



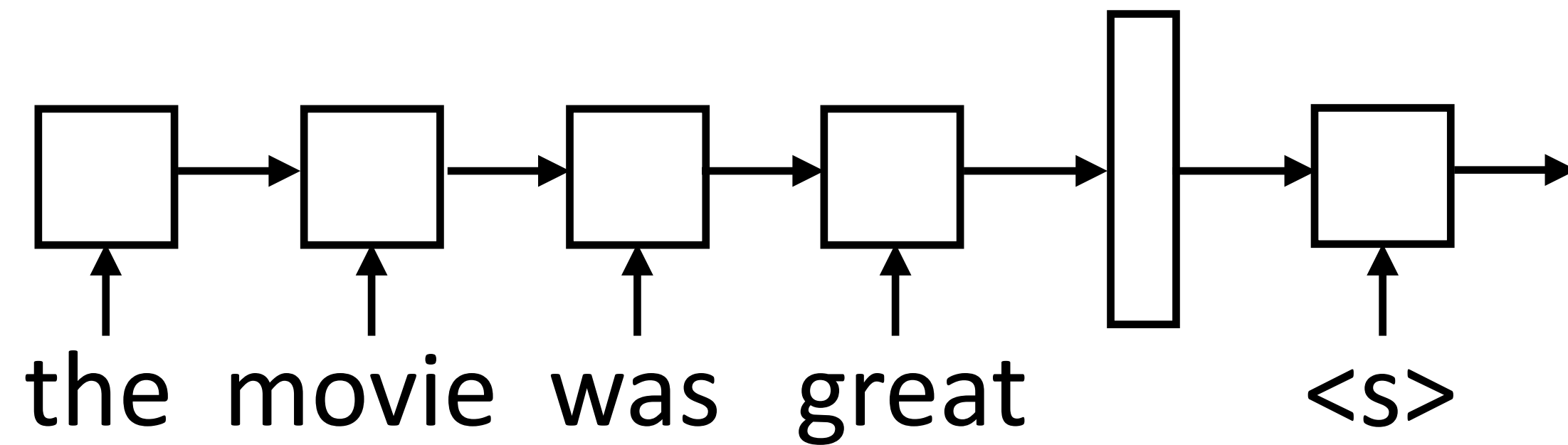
$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h})$$

$$P(\mathbf{y} | \mathbf{x}) = \prod_{i=1}^n P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$$

Decoder has separate parameters from encoder, so this can learn to be a language model (produce a plausible next word given current one)

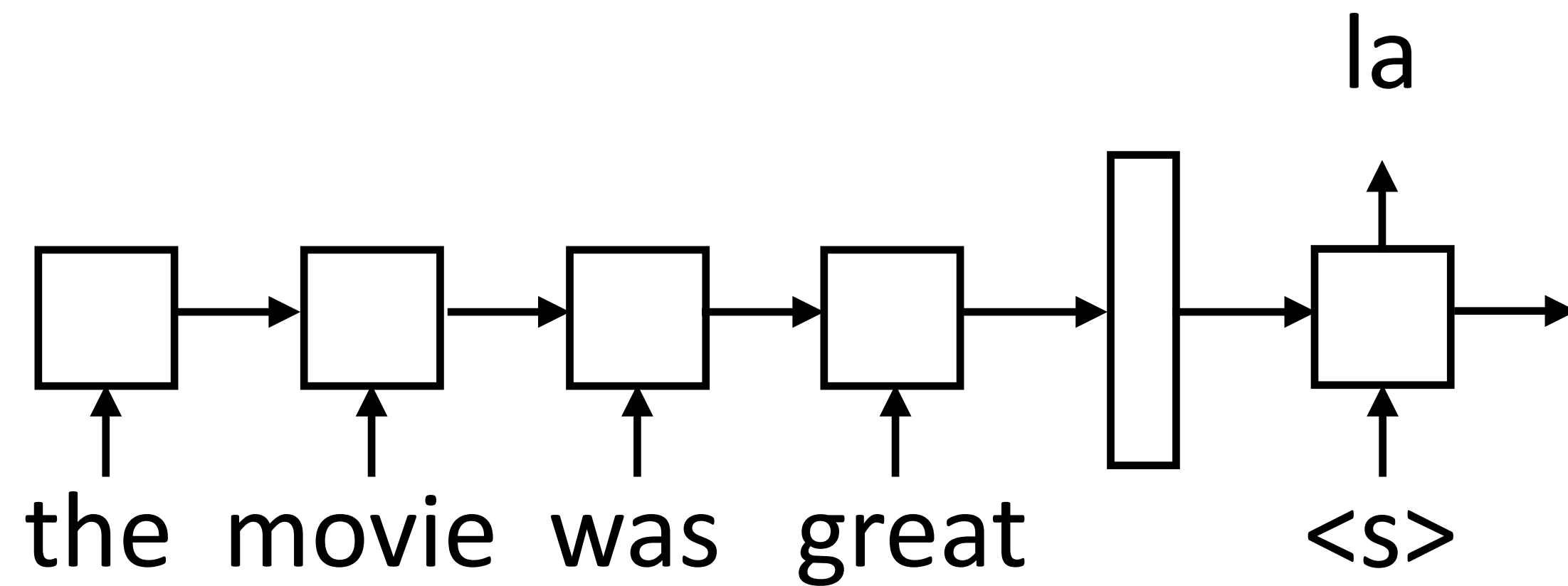
Inference

- ▶ Generate next word conditioned on previous word as well as hidden state



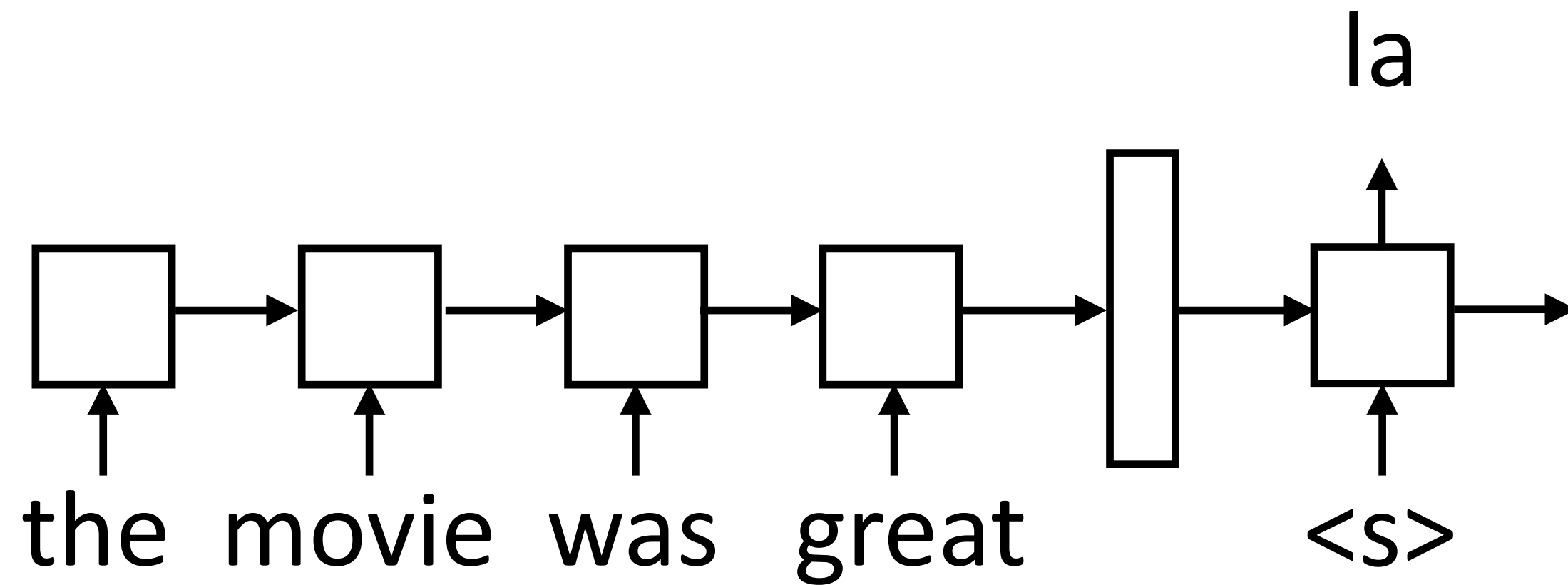
Inference

- ▶ Generate next word conditioned on previous word as well as hidden state



Inference

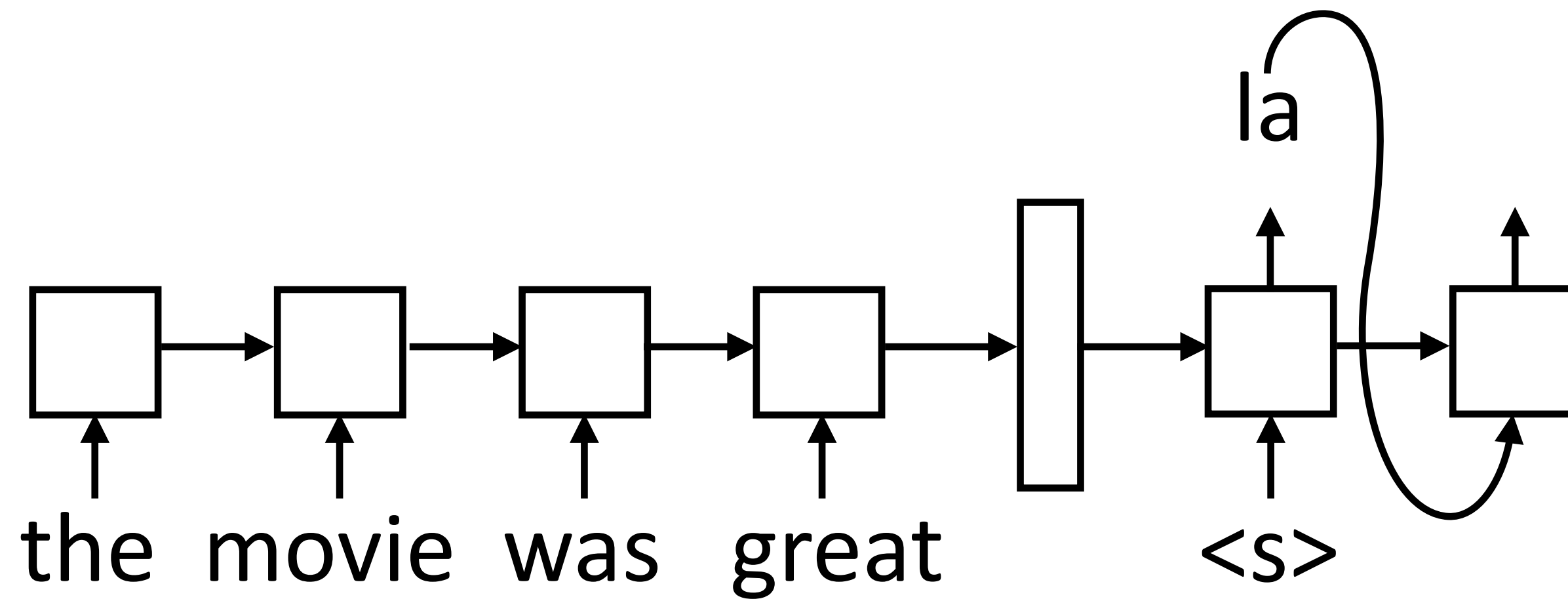
- ▶ Generate next word conditioned on previous word as well as hidden state



- ▶ During inference: need to compute the argmax over the word predictions and then feed that to the next RNN state

Inference

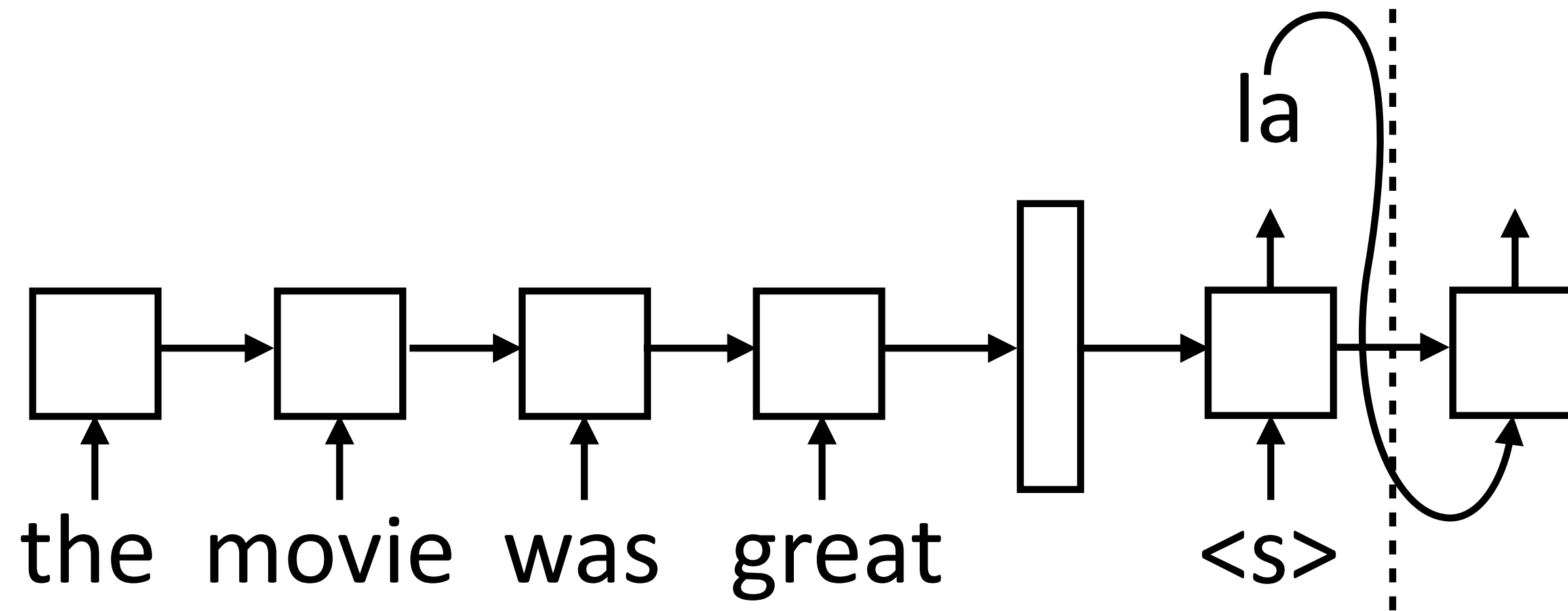
- ▶ Generate next word conditioned on previous word as well as hidden state



- ▶ During inference: need to compute the argmax over the word predictions and then feed that to the next RNN state

Inference

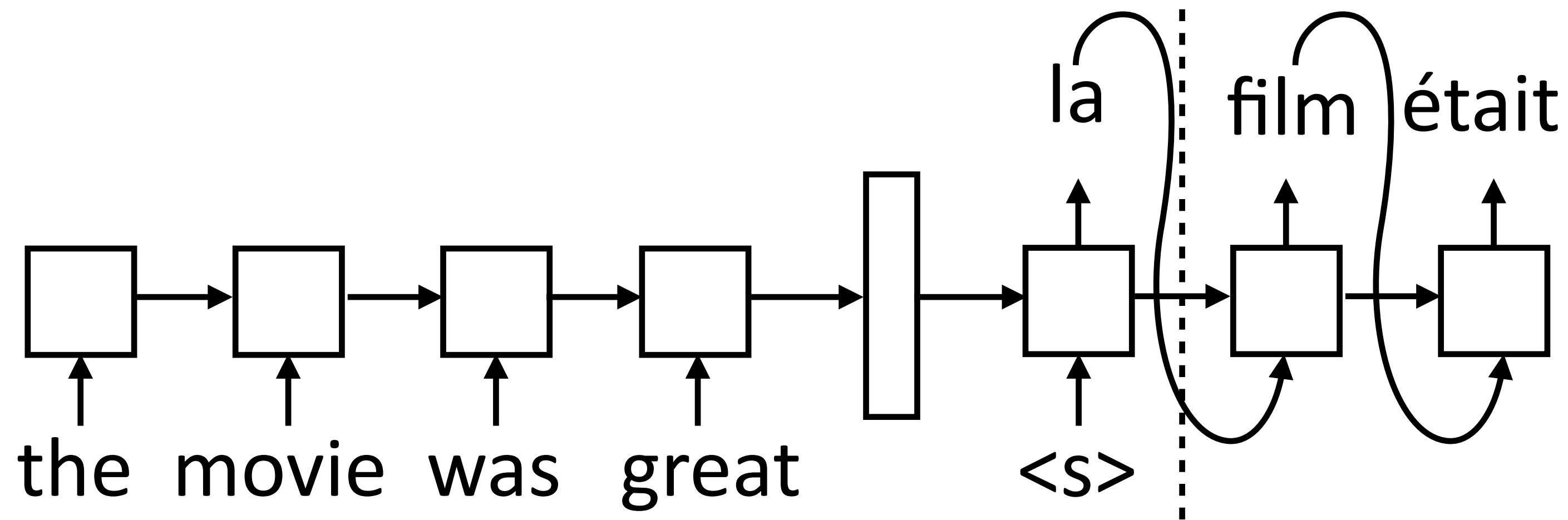
- ▶ Generate next word conditioned on previous word as well as hidden state



- ▶ During inference: need to compute the argmax over the word predictions and then feed that to the next RNN state

Inference

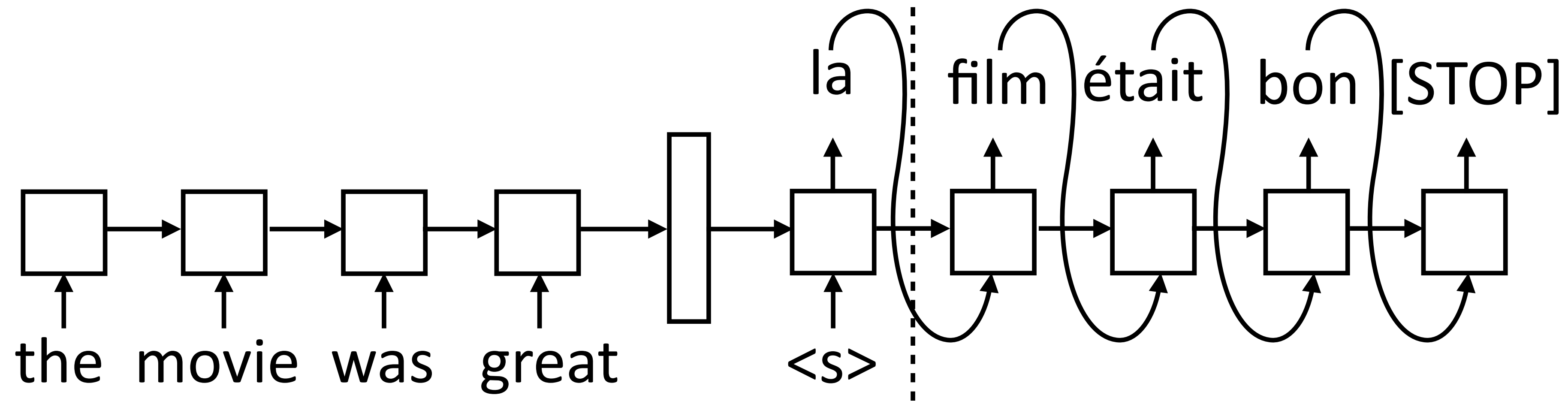
- ▶ Generate next word conditioned on previous word as well as hidden state



- ▶ During inference: need to compute the argmax over the word predictions and then feed that to the next RNN state

Inference

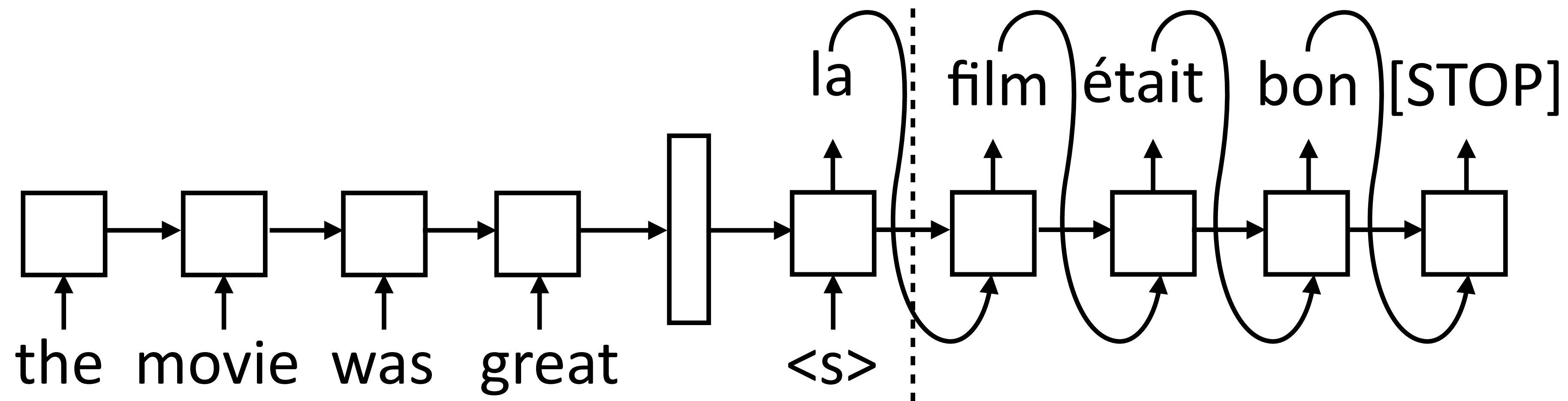
- ▶ Generate next word conditioned on previous word as well as hidden state



- ▶ During inference: need to compute the argmax over the word predictions and then feed that to the next RNN state

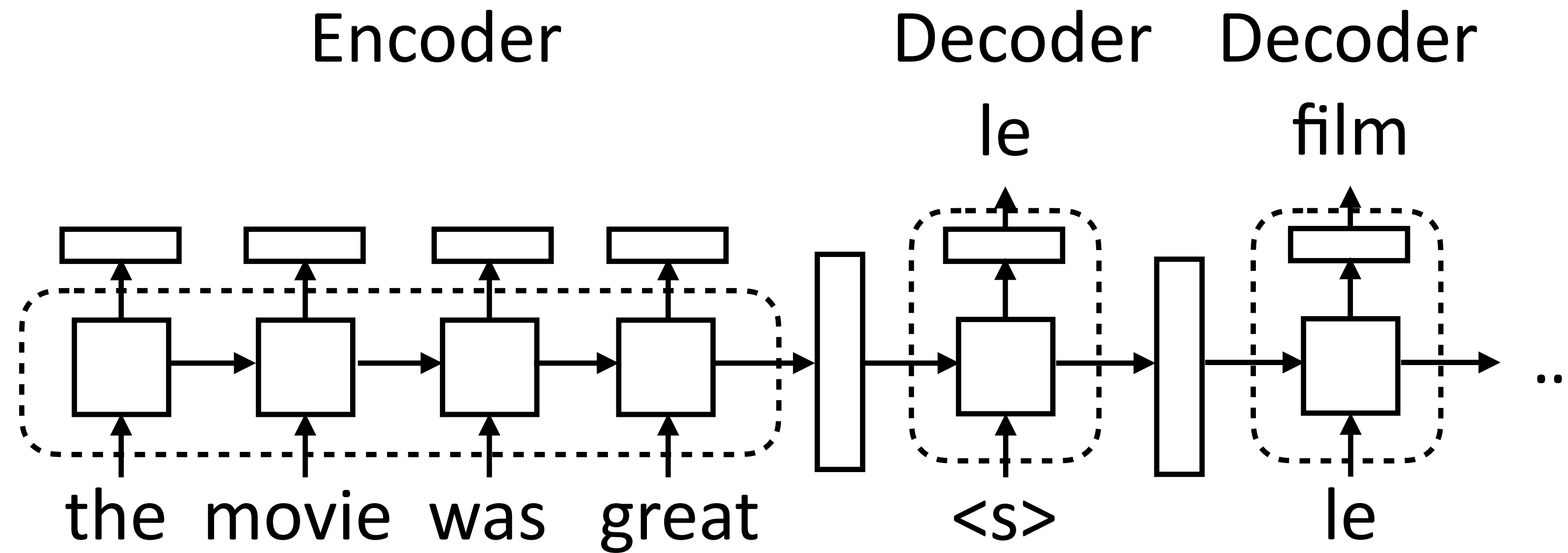
Inference

- ▶ Generate next word conditioned on previous word as well as hidden state

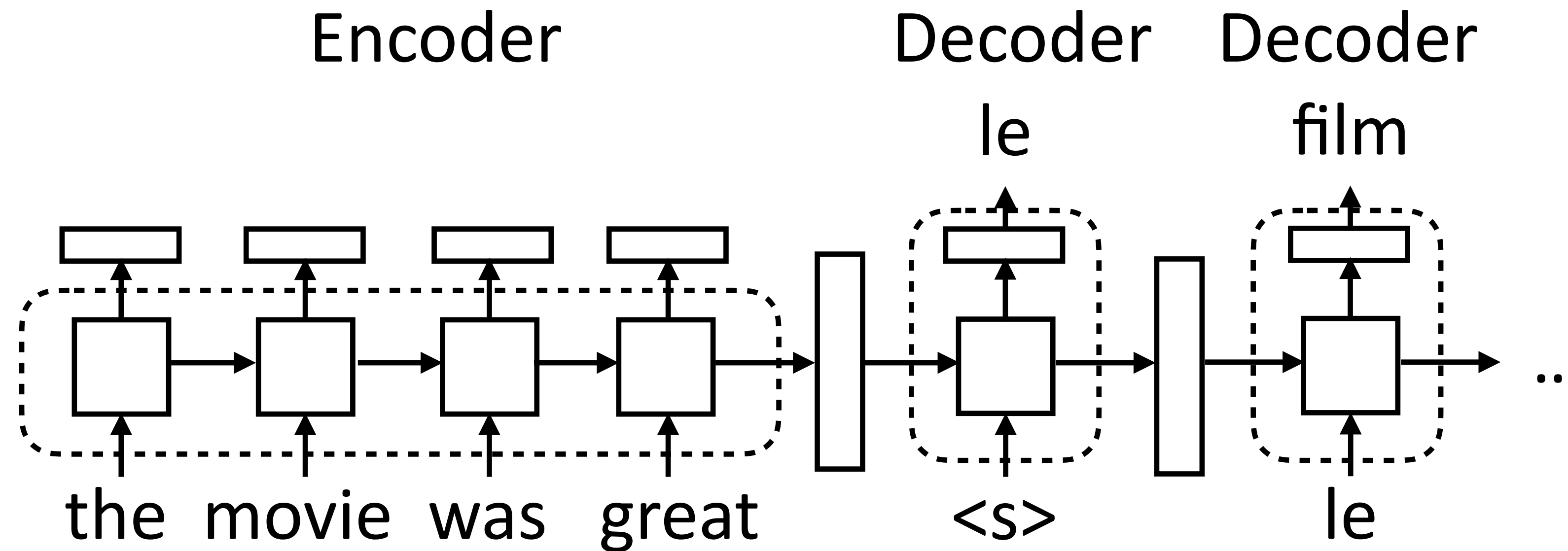


- ▶ During inference: need to compute the argmax over the word predictions and then feed that to the next RNN state
- ▶ Decoder is advanced one state at a time until [STOP] is reached

Implementing seq2seq Models

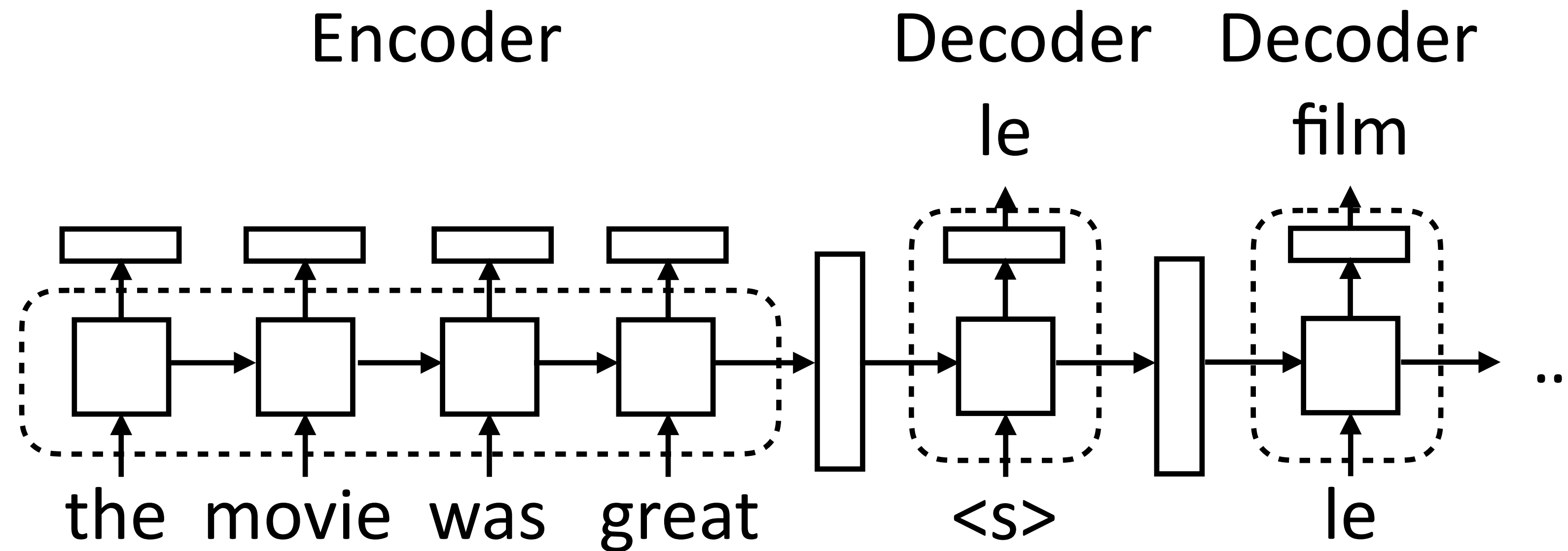


Implementing seq2seq Models



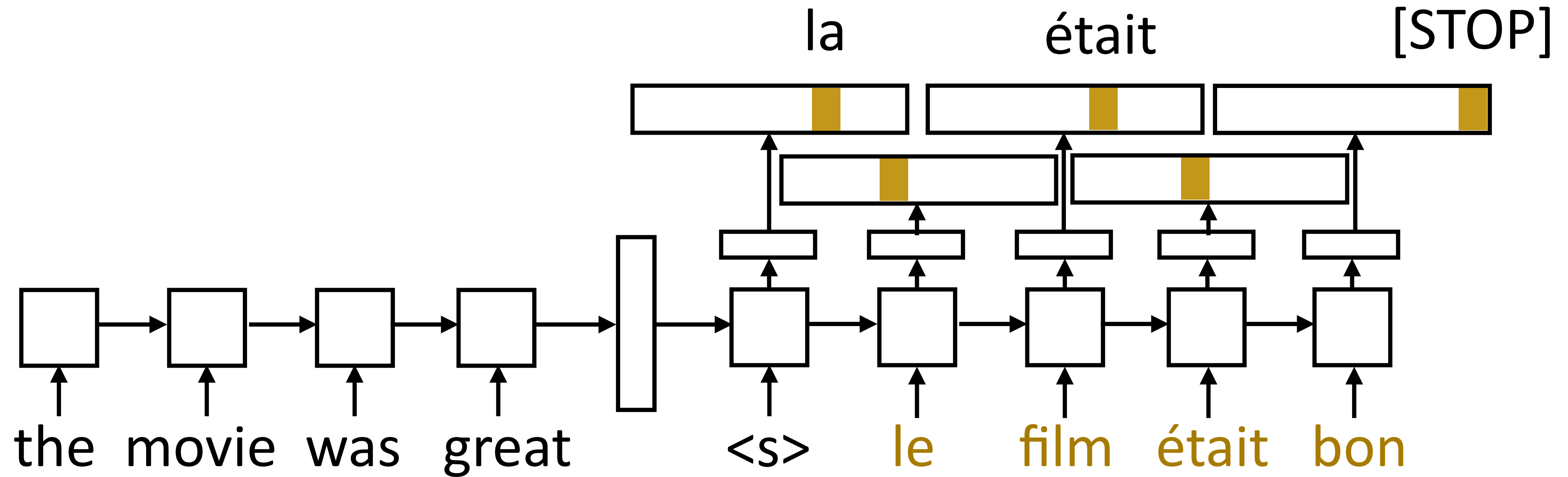
- ▶ Encoder: consumes sequence of tokens, produces a vector. Analogous to encoders for classification/tagging tasks

Implementing seq2seq Models



- ▶ Encoder: consumes sequence of tokens, produces a vector. Analogous to encoders for classification/tagging tasks
- ▶ Decoder: separate module, single cell. Takes two inputs: hidden state (vector h or tuple (h, c)) and previous token. Outputs token + new state

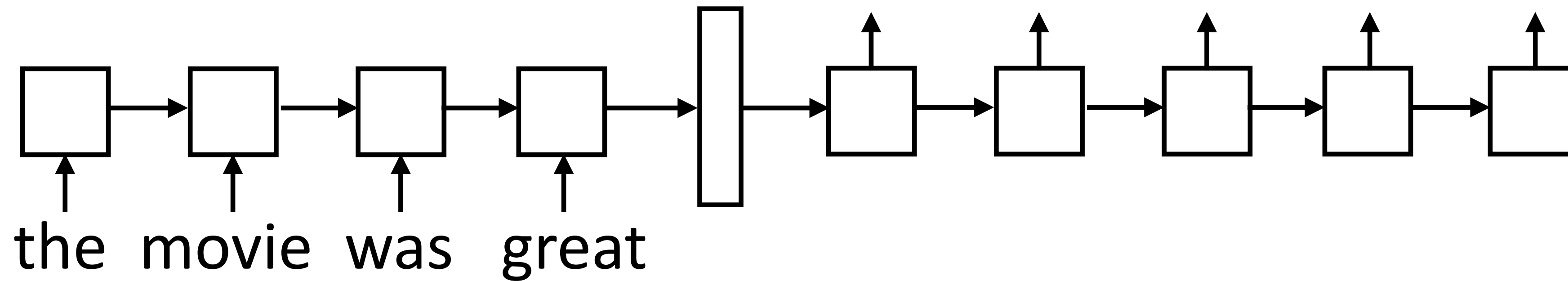
Training



- ▶ Objective: maximize $\sum_{(\mathbf{x}, \mathbf{y})} \sum_{i=1}^n \log P(y_i^* | \mathbf{x}, y_1^*, \dots, y_{i-1}^*)$
- ▶ One loss term for each target-sentence word, feed the correct word regardless of model's prediction (called "teacher forcing")

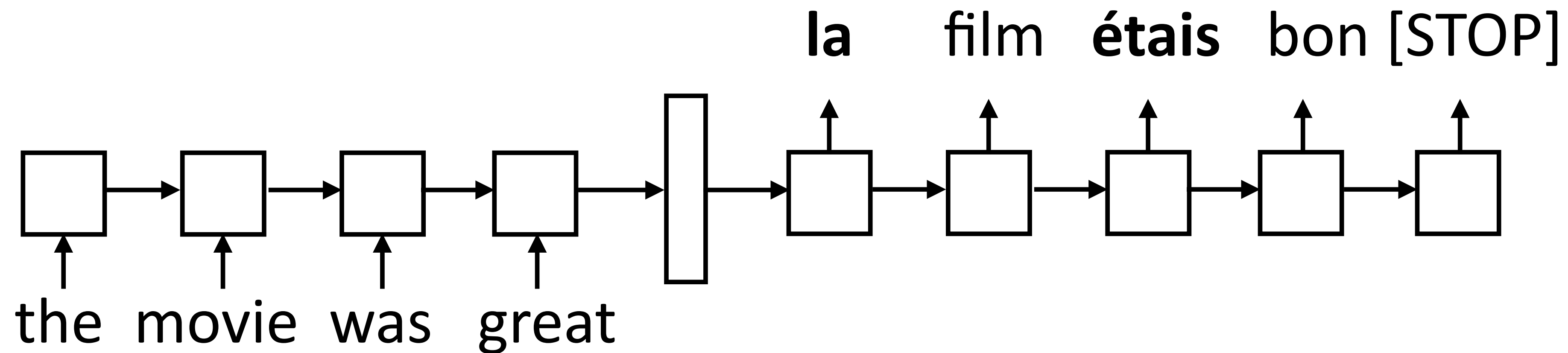
Training: Scheduled Sampling

- ▶ Model needs to do the right thing even with its own predictions



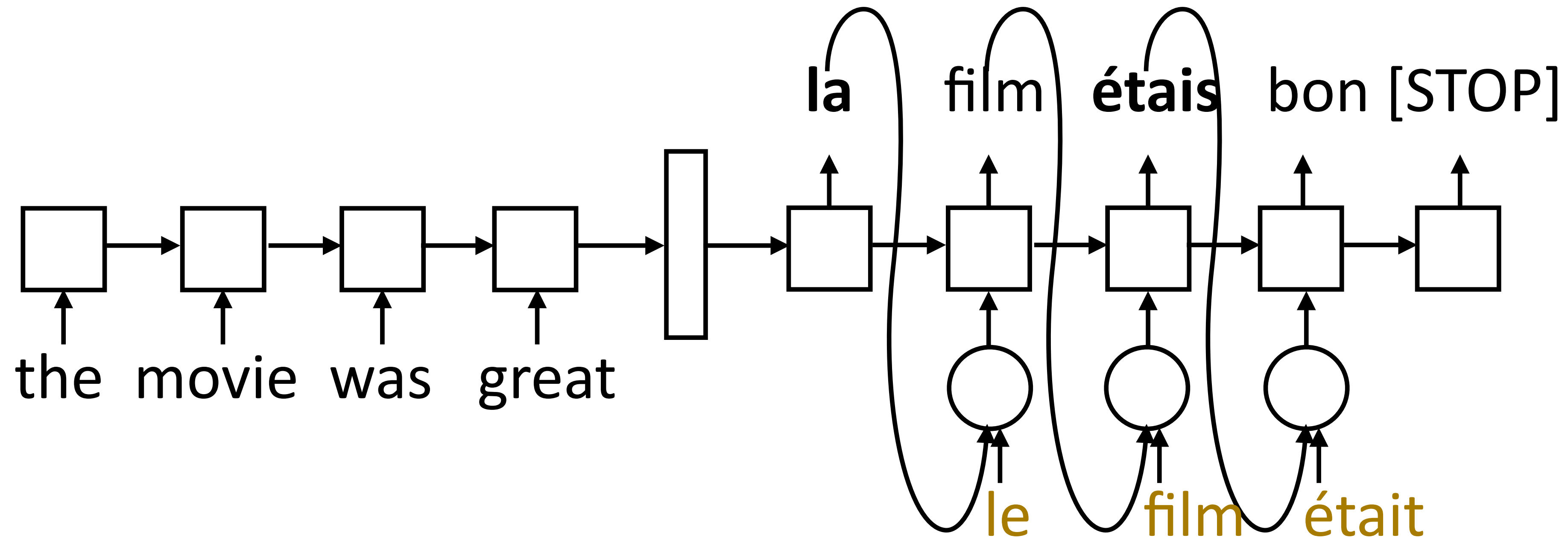
Training: Scheduled Sampling

- ▶ Model needs to do the right thing even with its own predictions



Training: Scheduled Sampling

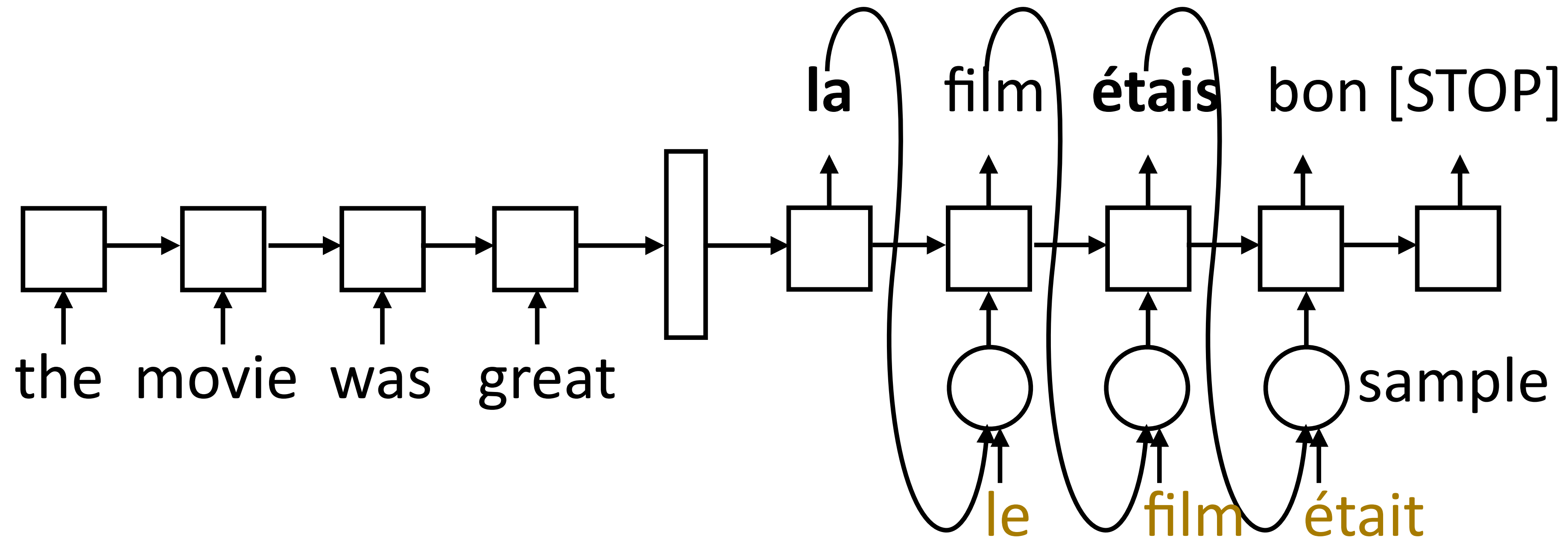
- ▶ Model needs to do the right thing even with its own predictions



- ▶ Scheduled sampling: with probability p , take the **gold (human) translation** as input, else take the model's prediction

Training: Scheduled Sampling

- ▶ Model needs to do the right thing even with its own predictions



- ▶ Scheduled sampling: with probability p , take the **gold (human) translation** as input, else take the model's prediction
- ▶ Starting with $p = 1$ and decaying it works best

Implementation Details

Implementation Details

- ▶ Sentence lengths vary for both encoder and decoder:

Implementation Details

- ▶ Sentence lengths vary for both encoder and decoder:
 - ▶ Typically pad everything to the right length

Implementation Details

- ▶ Sentence lengths vary for both encoder and decoder:
 - ▶ Typically pad everything to the right length
- ▶ Encoder: Can be a CNN/LSTM/Transformer...

Implementation Details

- ▶ Sentence lengths vary for both encoder and decoder:
 - ▶ Typically pad everything to the right length
- ▶ Encoder: Can be a CNN/LSTM/Transformer...
- ▶ Decoder: also flexible in terms of architecture (more later). Execute one step of computation at a time, so computation graph is formulated as taking one input + hidden state

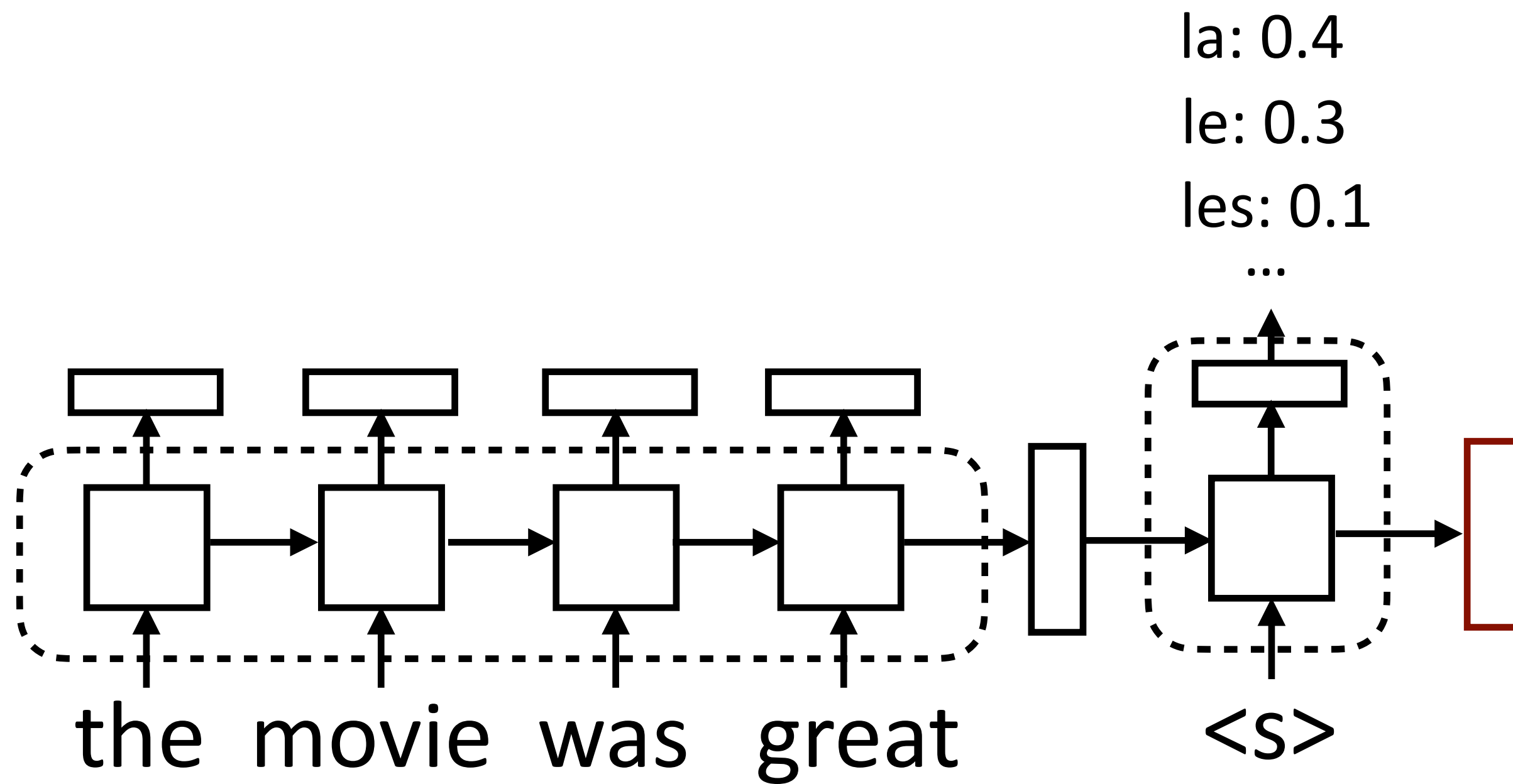
Implementation Details

- ▶ Sentence lengths vary for both encoder and decoder:
 - ▶ Typically pad everything to the right length
- ▶ Encoder: Can be a CNN/LSTM/Transformer...
- ▶ Decoder: also flexible in terms of architecture (more later). Execute one step of computation at a time, so computation graph is formulated as taking one input + hidden state
- ▶ Beam search: can help with lookahead. Finds the (approximate) highest scoring sequence:

$$\operatorname{argmax}_{\mathbf{y}} \prod_{i=1}^n P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$$

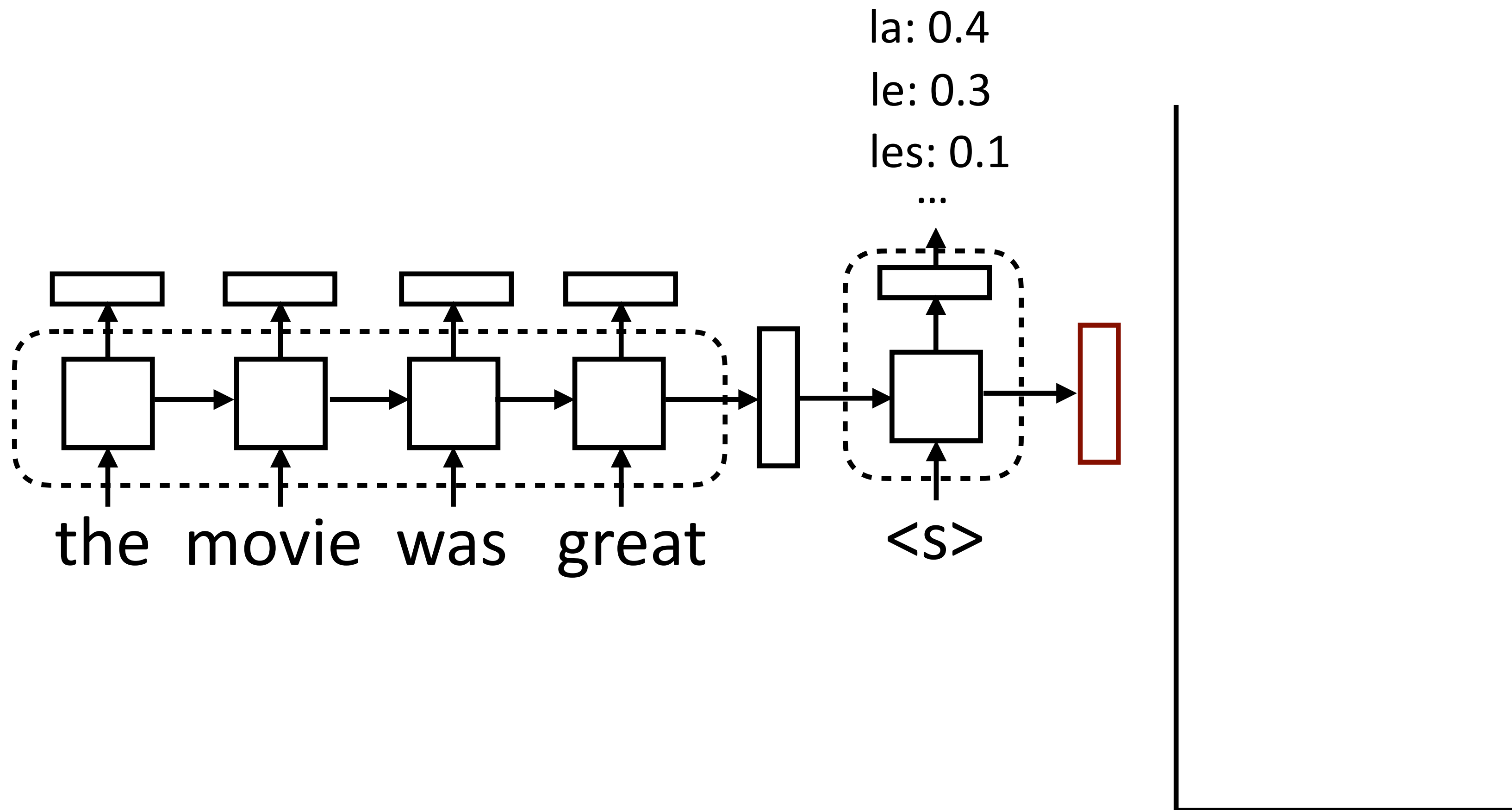
Beam Search

- ▶ Maintain decoder state, token history in beam



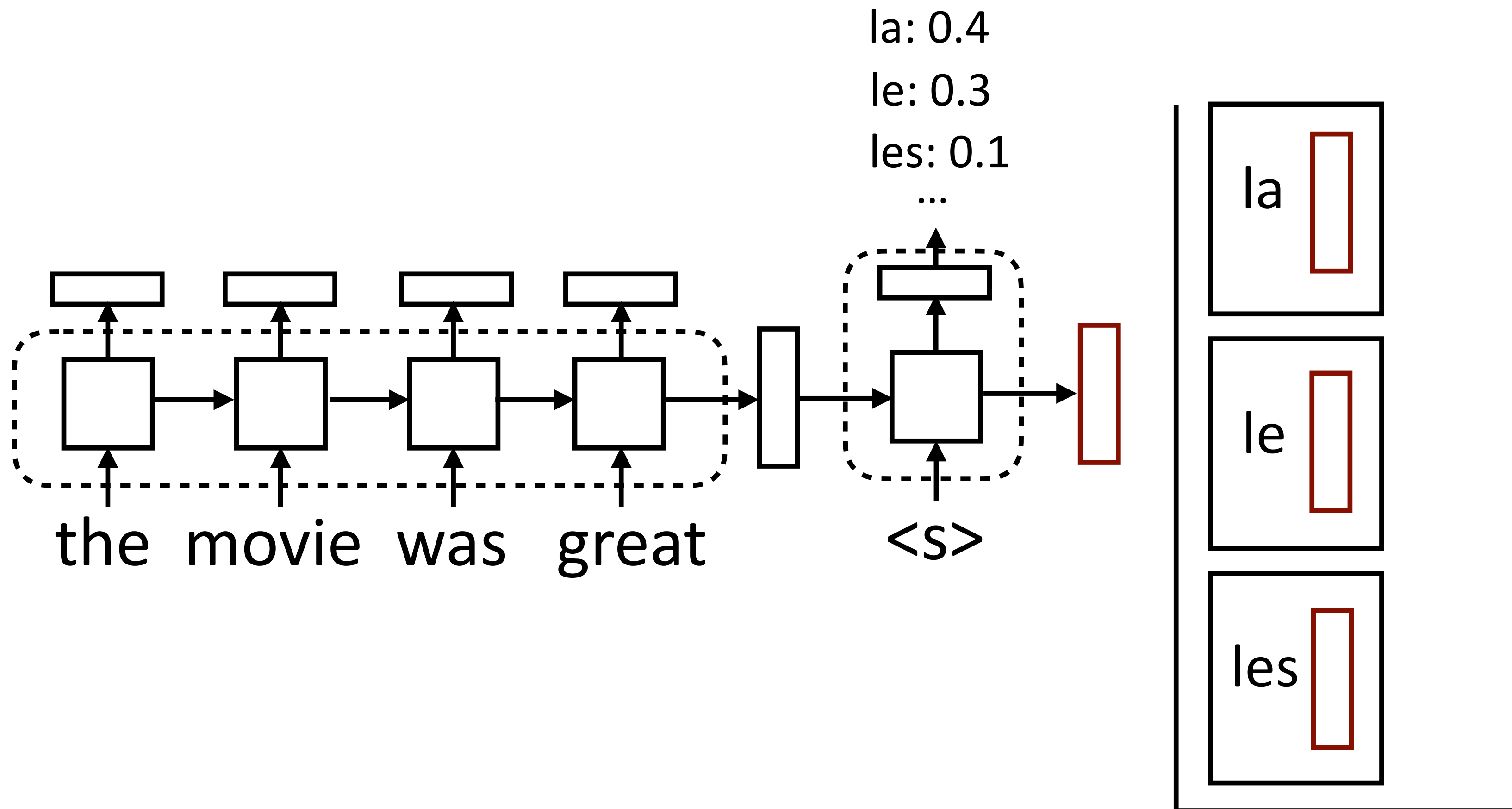
Beam Search

- ▶ Maintain decoder state, token history in beam



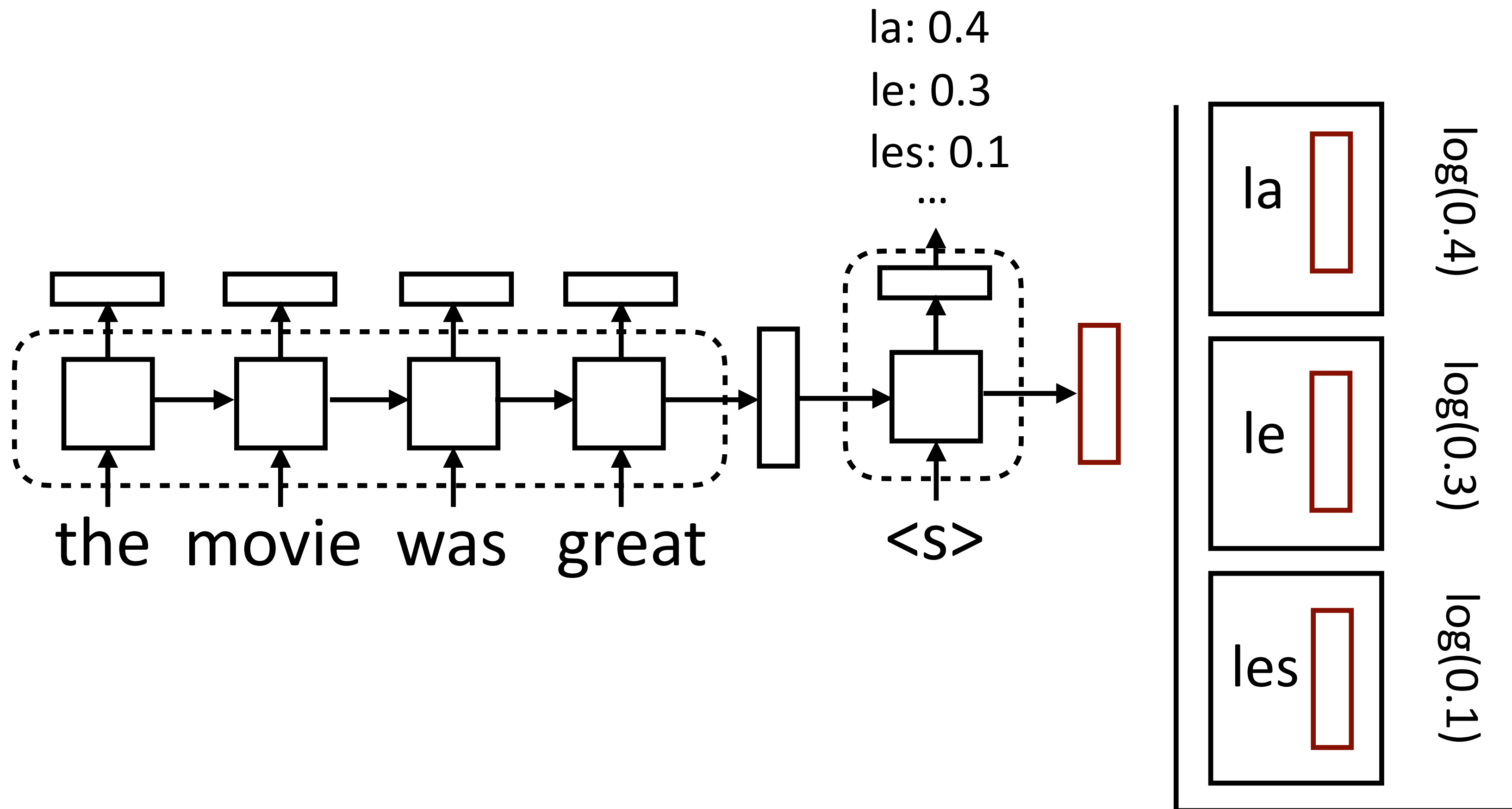
Beam Search

- ▶ Maintain decoder state, token history in beam



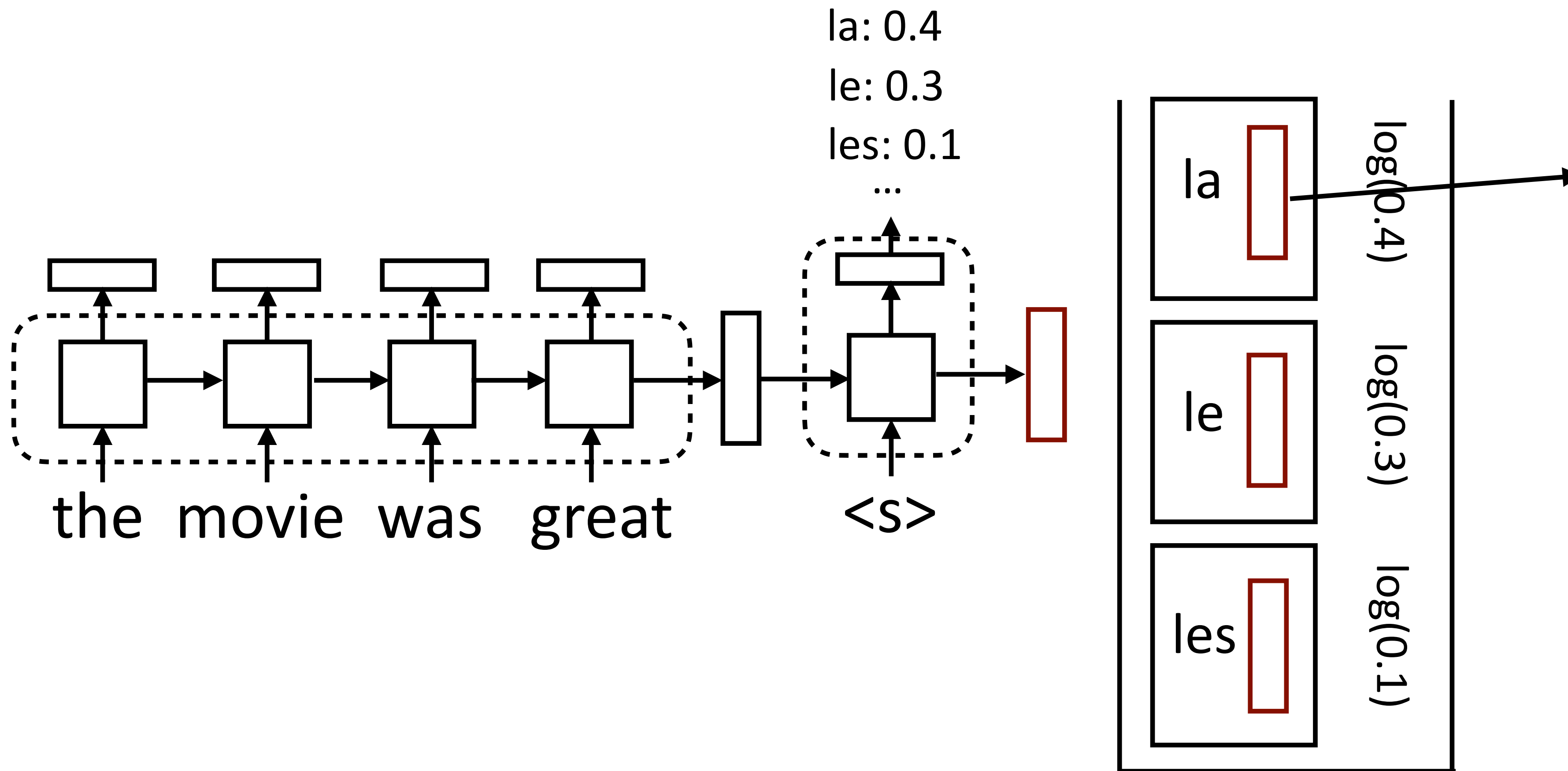
Beam Search

- ▶ Maintain decoder state, token history in beam



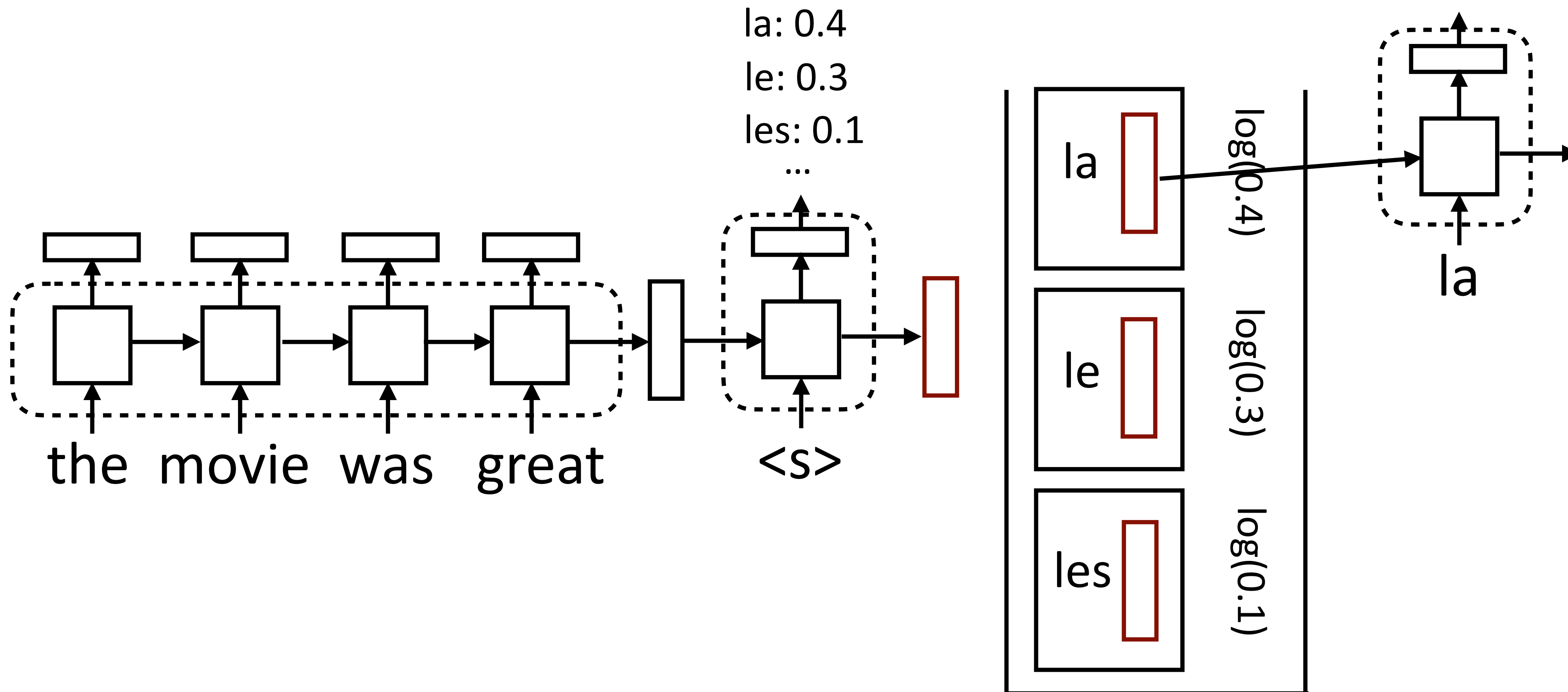
Beam Search

- ▶ Maintain decoder state, token history in beam



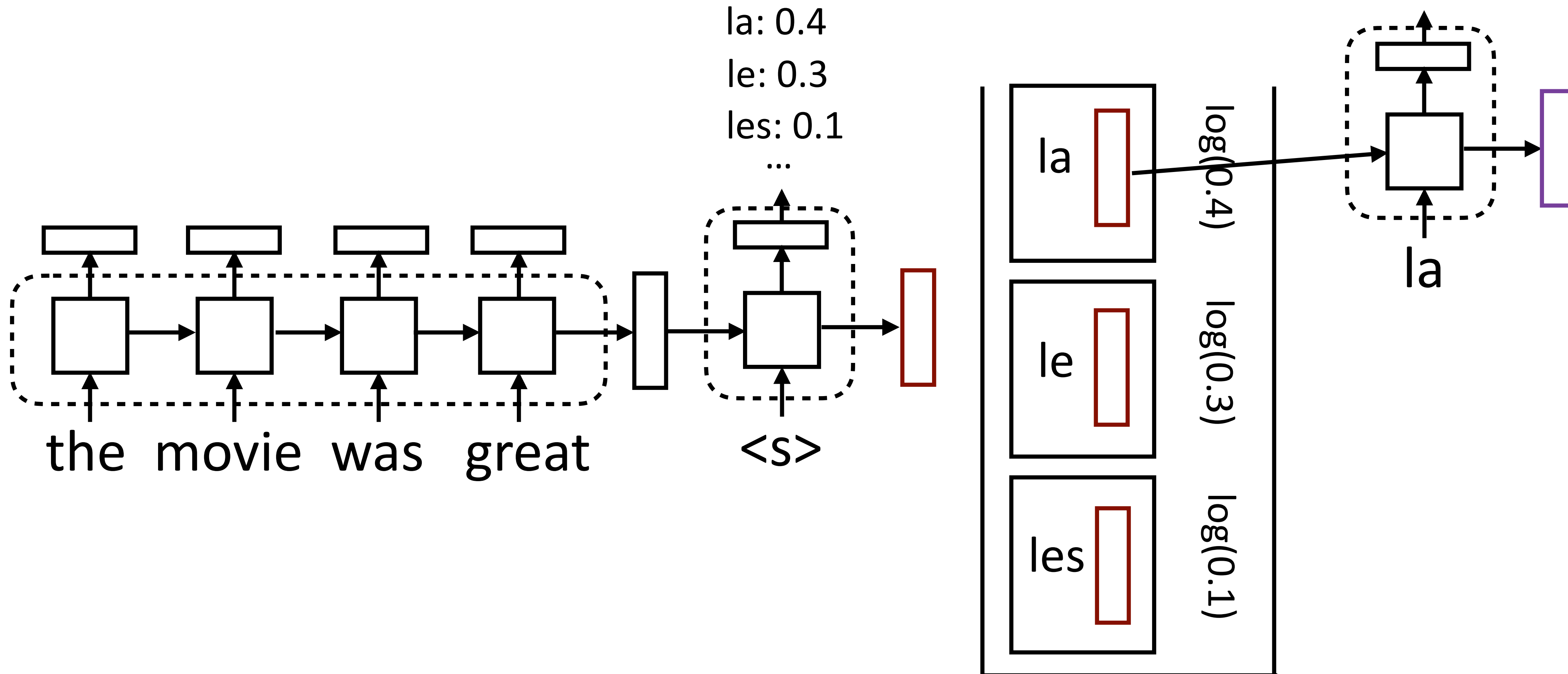
Beam Search

- ▶ Maintain decoder state, token history in beam



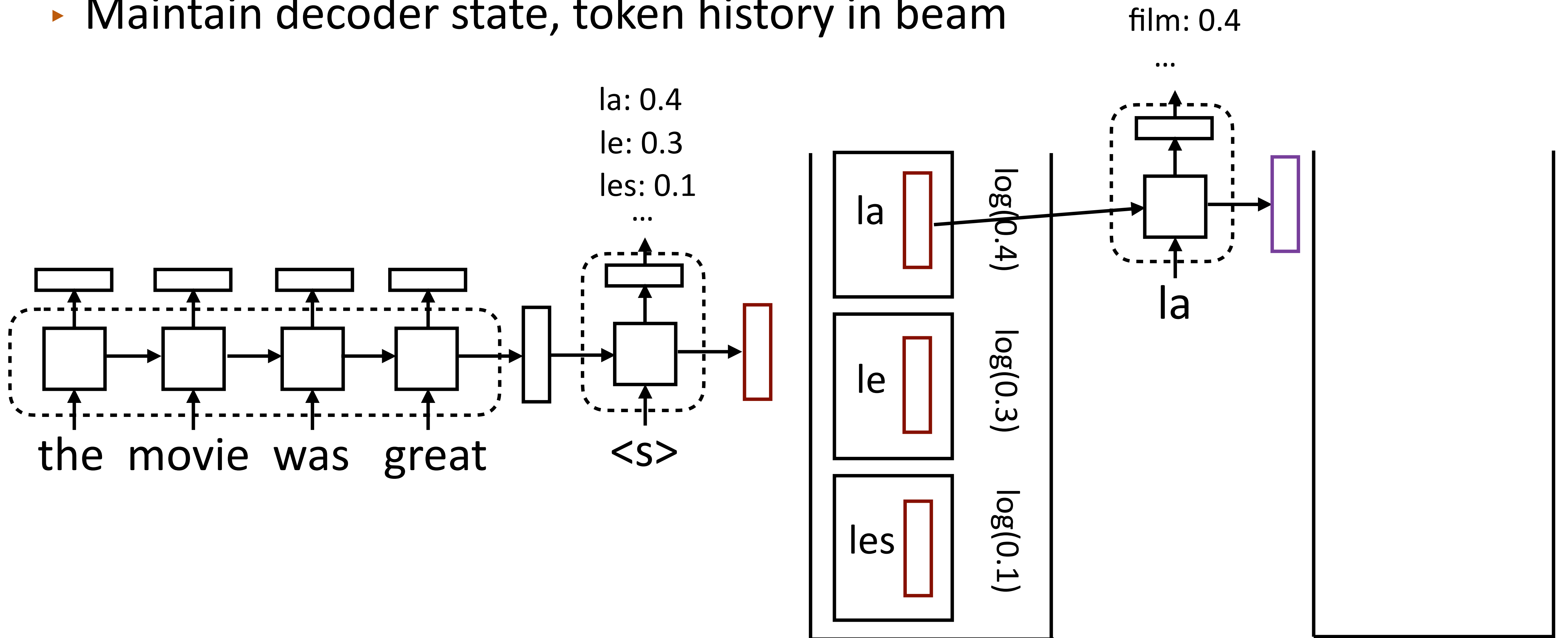
Beam Search

- ▶ Maintain decoder state, token history in beam



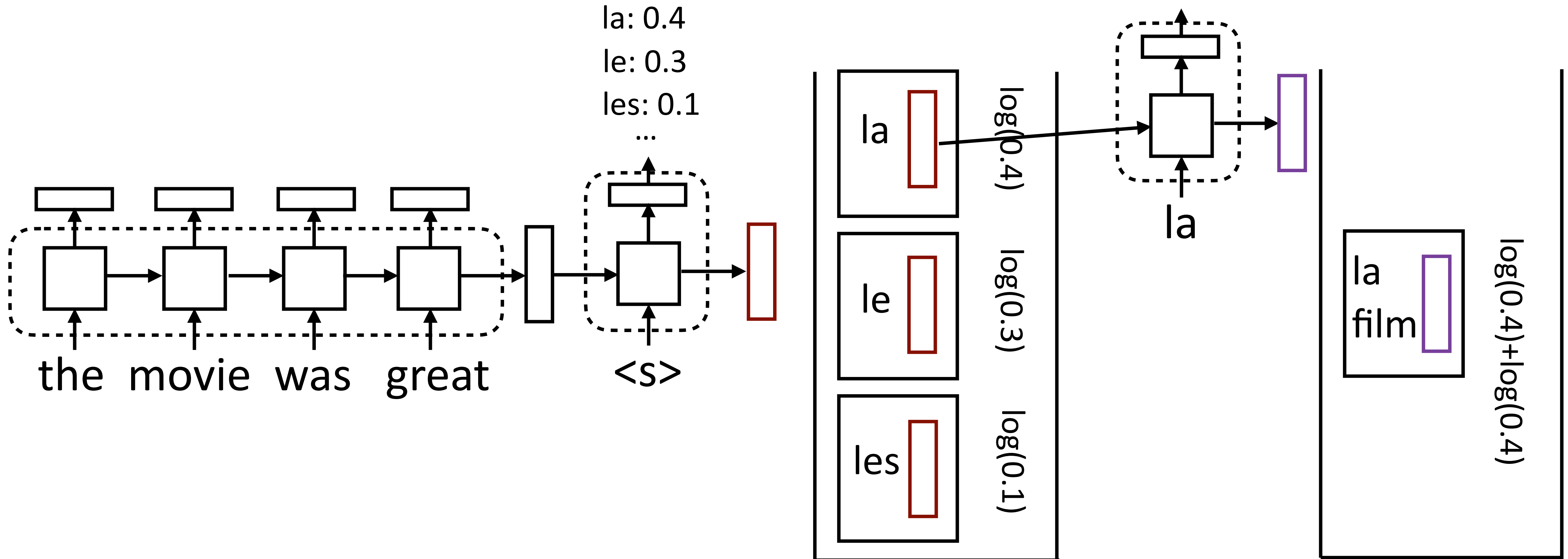
Beam Search

- Maintain decoder state, token history in beam



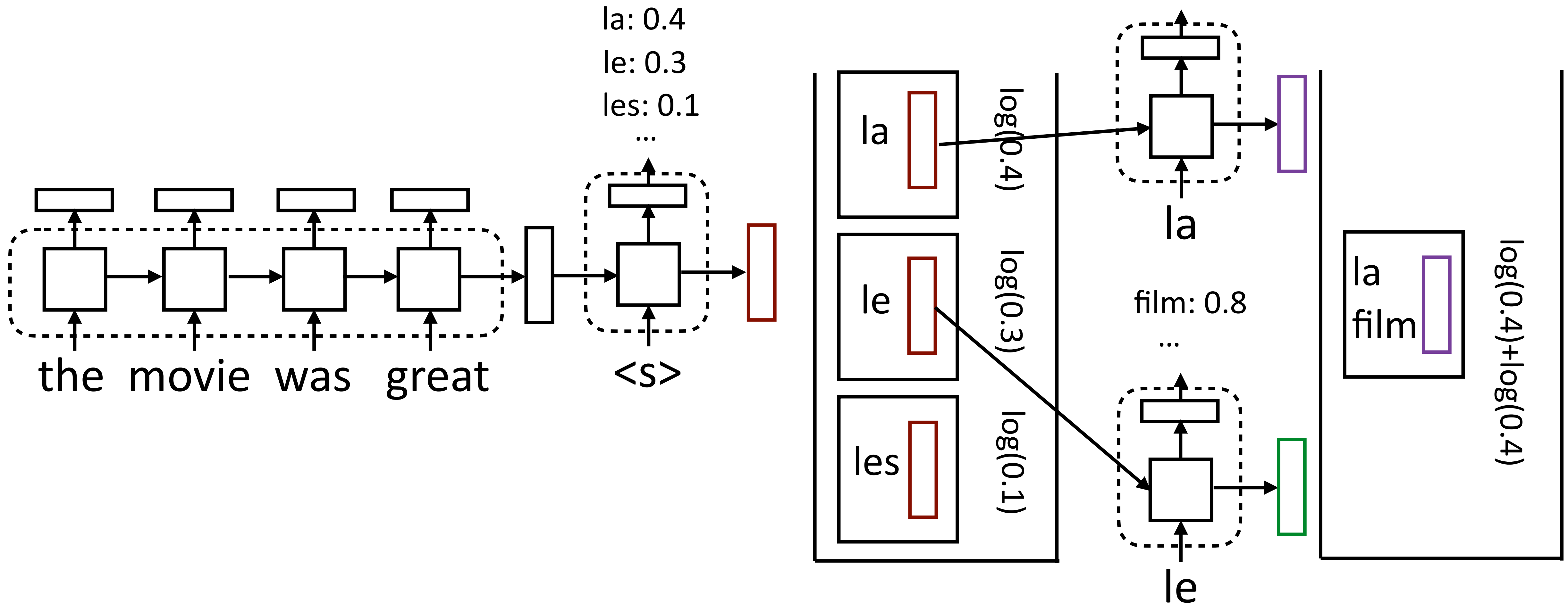
Beam Search

- Maintain decoder state, token history in beam



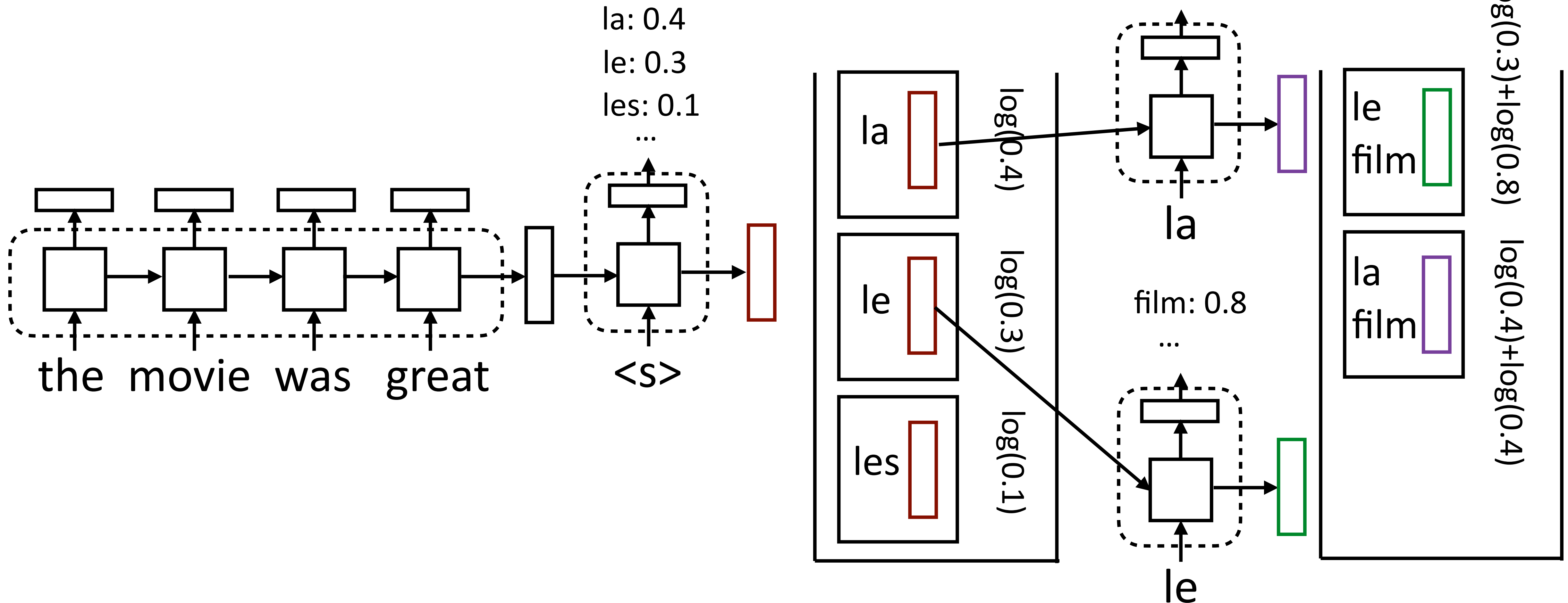
Beam Search

- Maintain decoder state, token history in beam



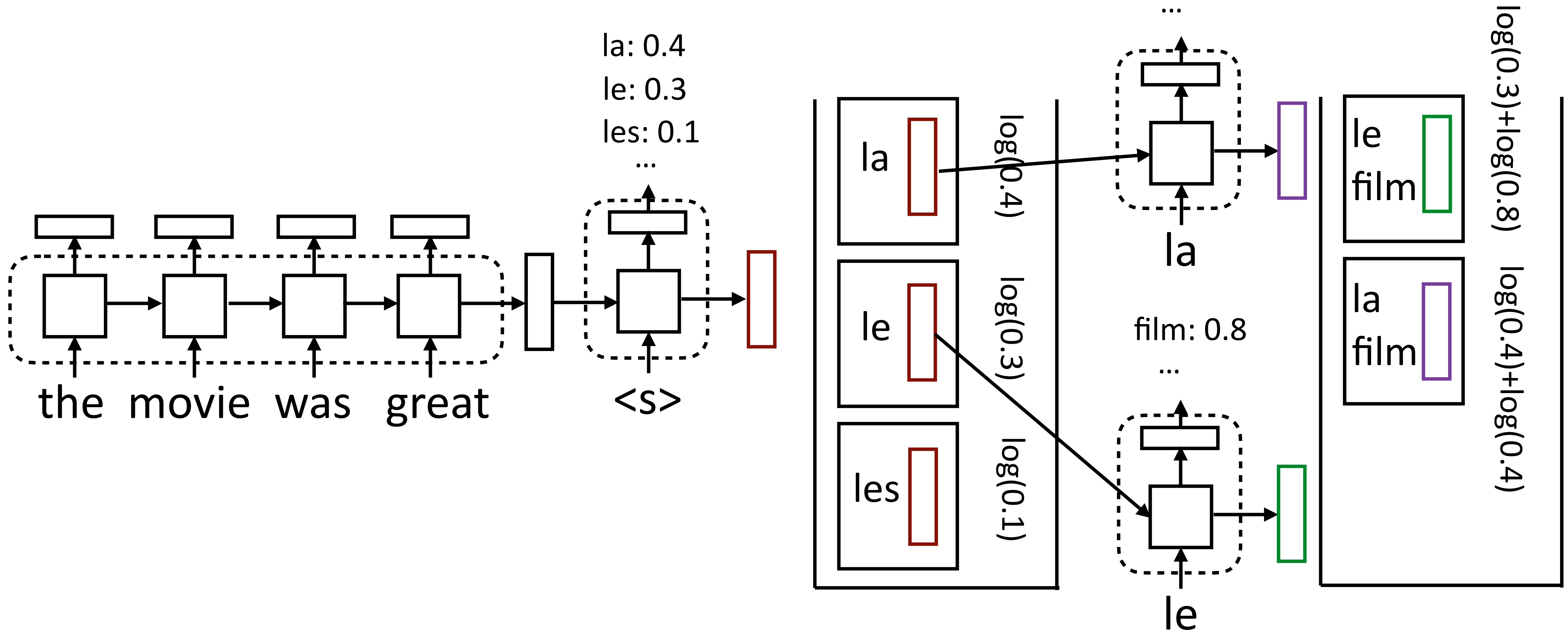
Beam Search

- Maintain decoder state, token history in beam



Beam Search

- Maintain decoder state, token history in beam



- Do **not** max over the two *film* states! Hidden state vectors are different

Attention

Problems with Seq2seq Models

- ▶ Encoder-decoder models like to repeat themselves:

Problems with Seq2seq Models

- ▶ Encoder-decoder models like to repeat themselves:

Un garçon joue dans la neige → A boy plays in the snow **boy plays boy plays**

Problems with Seq2seq Models

- ▶ Encoder-decoder models like to repeat themselves:

Un garçon joue dans la neige → A boy plays in the snow **boy plays boy plays**

- ▶ Often a byproduct of training these models poorly

Problems with Seq2seq Models

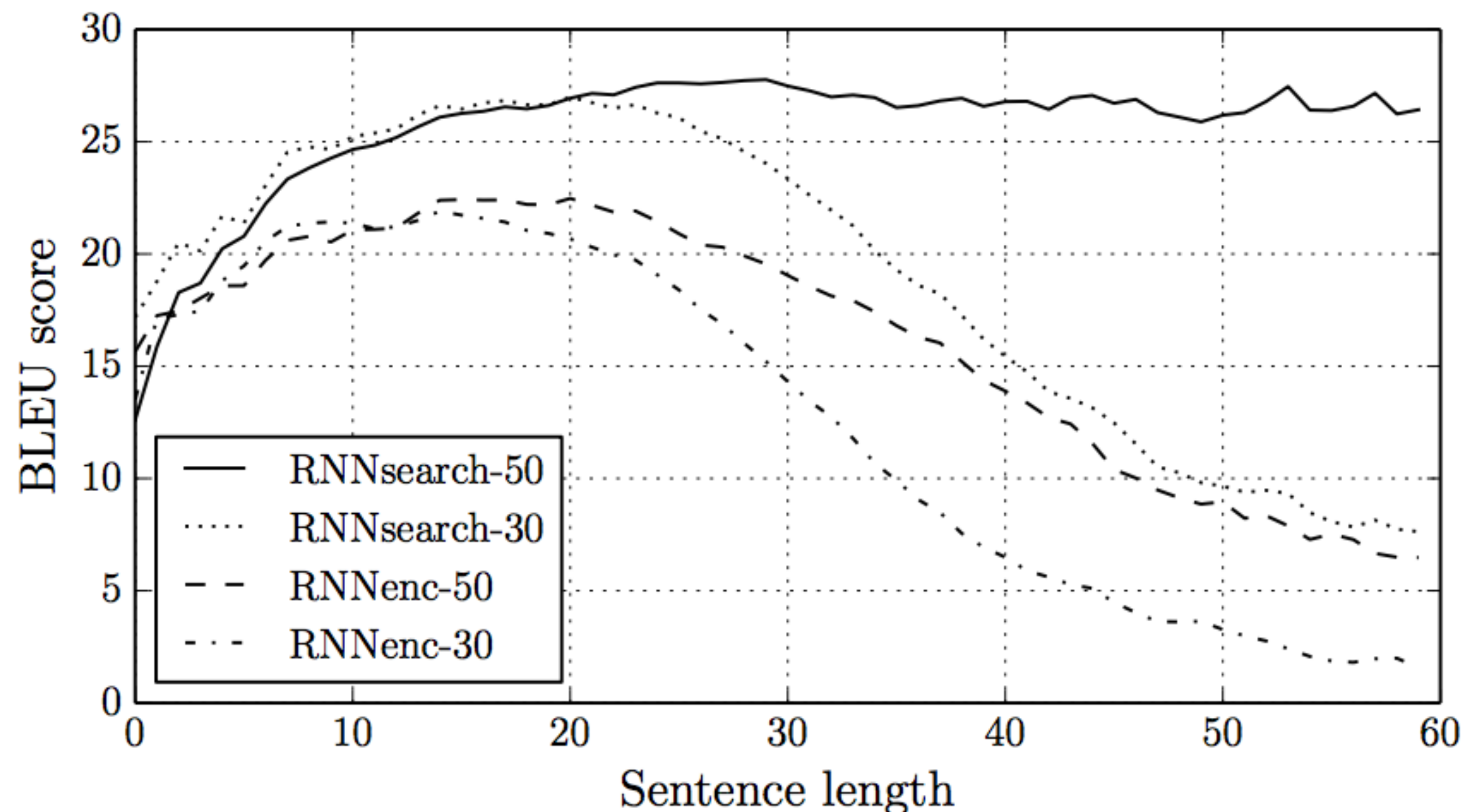
- ▶ Encoder-decoder models like to repeat themselves:

Un garçon joue dans la neige → A boy plays in the snow **boy plays boy plays**

- ▶ Often a byproduct of training these models poorly
- ▶ Need some notion of input coverage or what input words we've translated

Problems with Seq2seq Models

- ▶ Bad at long sentences: 1) a fixed-size hidden representation doesn't scale; 2) LSTMs still have a hard time remembering for really long sentences



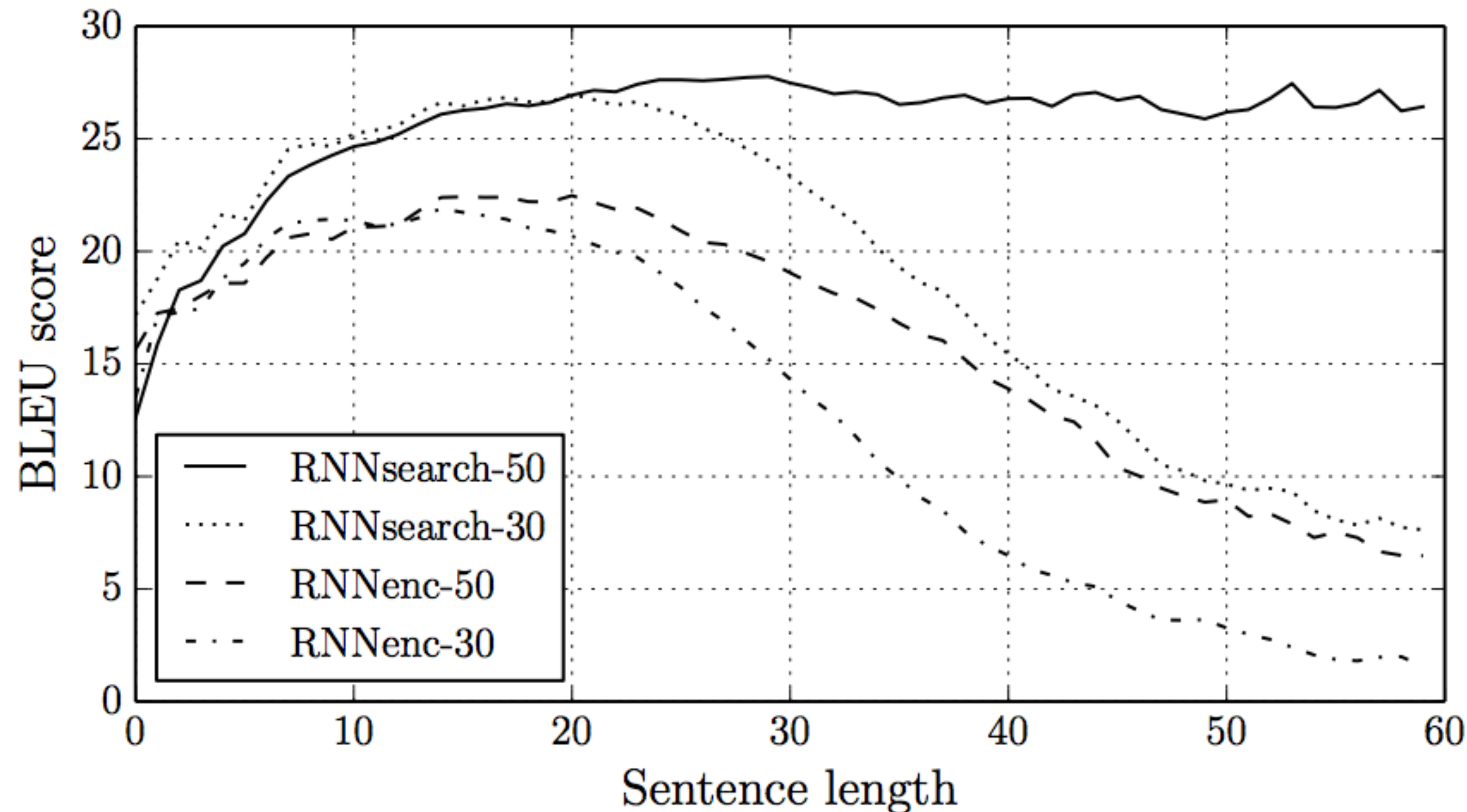
RNNenc: the model we've discussed so far

RNNsearch: uses attention

Bahdanau et al. (2014)

Problems with Seq2seq Models

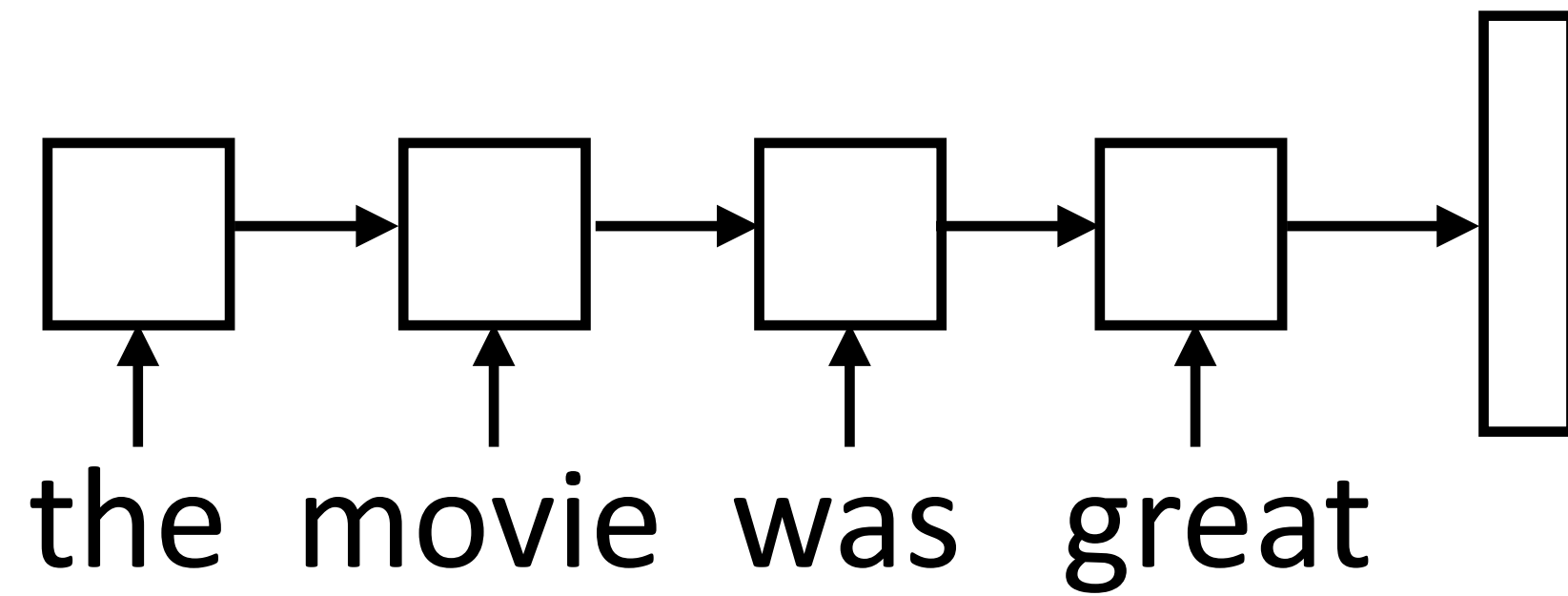
- ▶ Bad at long sentences: 1) a fixed-size representation doesn't scale; 2) LSTMs still have a hard time remembering for really long periods of time



RNNsearch: introduces attention mechanism to give “variable-sized” representation

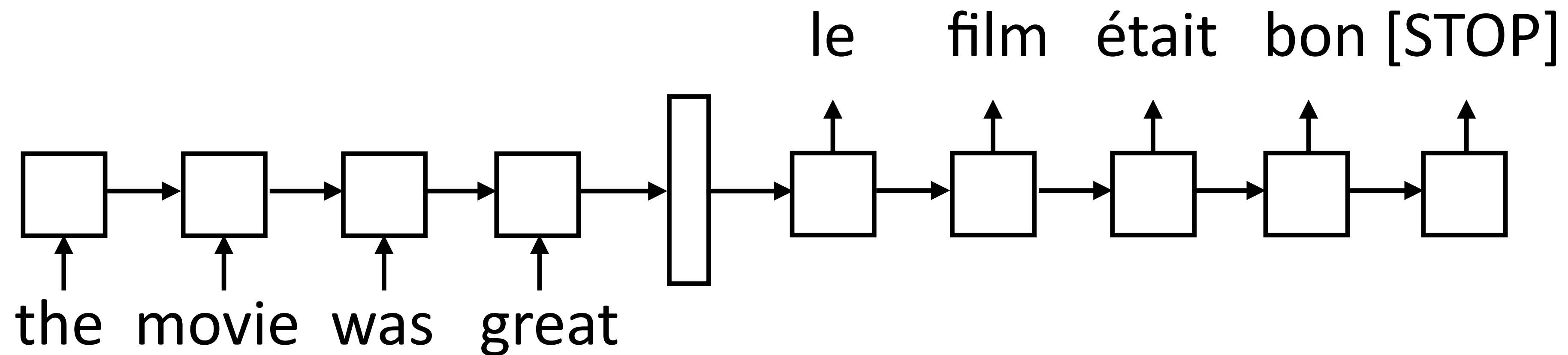
Encoder-Decoder

- ▶ Encode a sequence into a fixed-sized vector



Encoder-Decoder

- ▶ Encode a sequence into a fixed-sized vector



- ▶ Now use that vector to produce a series of tokens as output from a separate LSTM *decoder*

Aligned Inputs

- ▶ Suppose we knew the source and target would be word-by-word translated

Aligned Inputs

- ▶ Suppose we knew the source and target would be word-by-word translated

the movie was great
/ / / /
le film était bon

Aligned Inputs

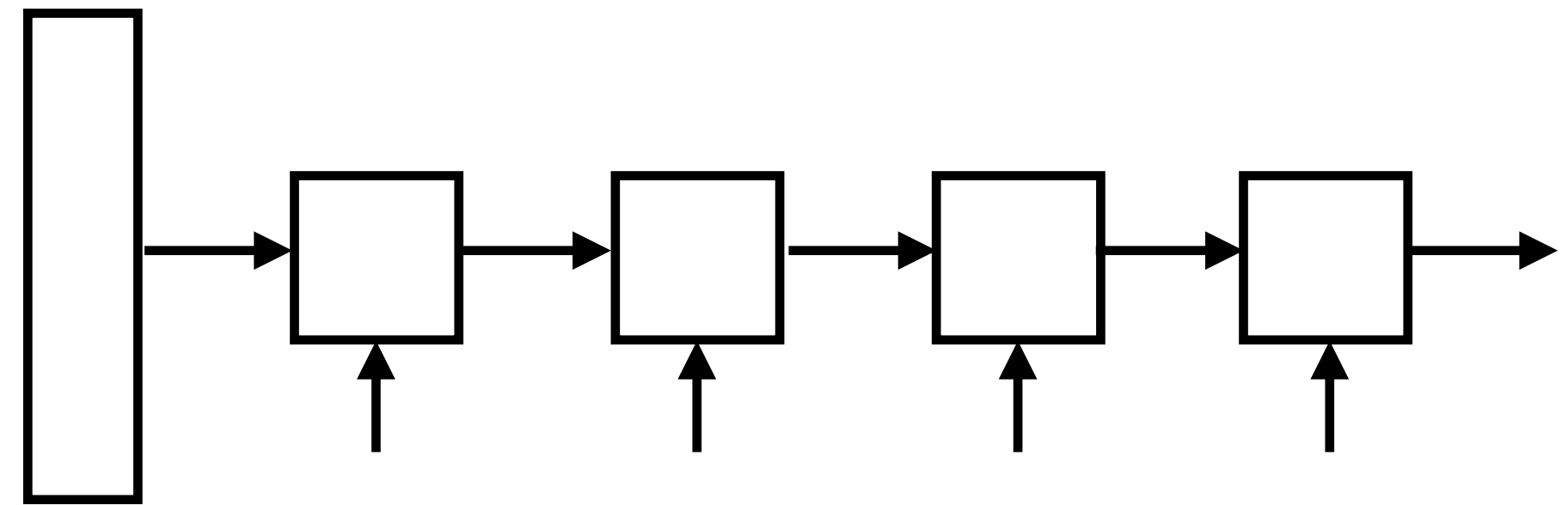
- ▶ Suppose we knew the source and target would be word-by-word translated
- ▶ Can look at the corresponding input word when translating — this could scale!

the movie was great
/ / / /
le film était bon

Aligned Inputs

- ▶ Suppose we knew the source and target would be word-by-word translated
- ▶ Can look at the corresponding input word when translating — this could scale!

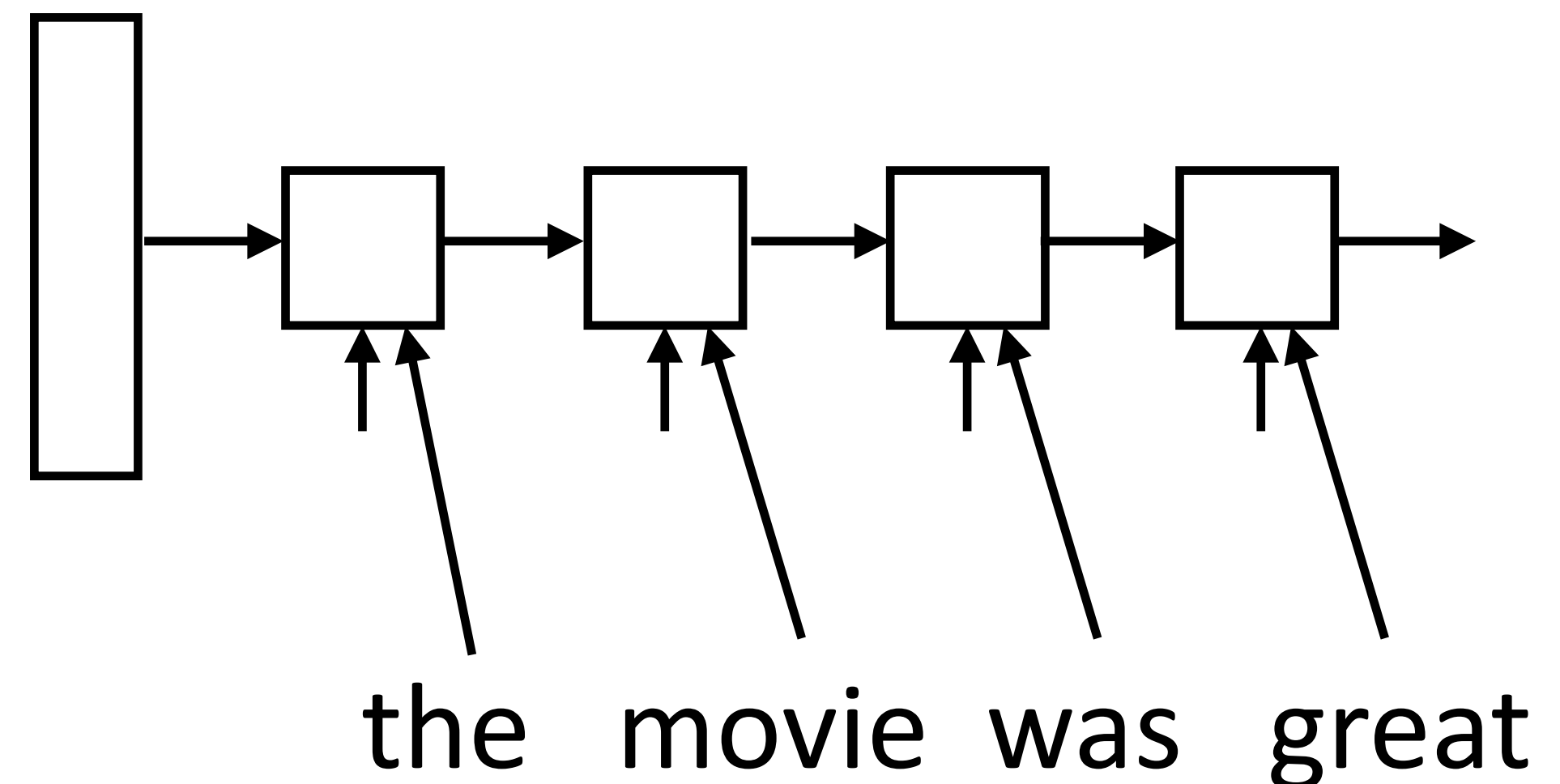
the movie was great
/ / / /
le film était bon



Aligned Inputs

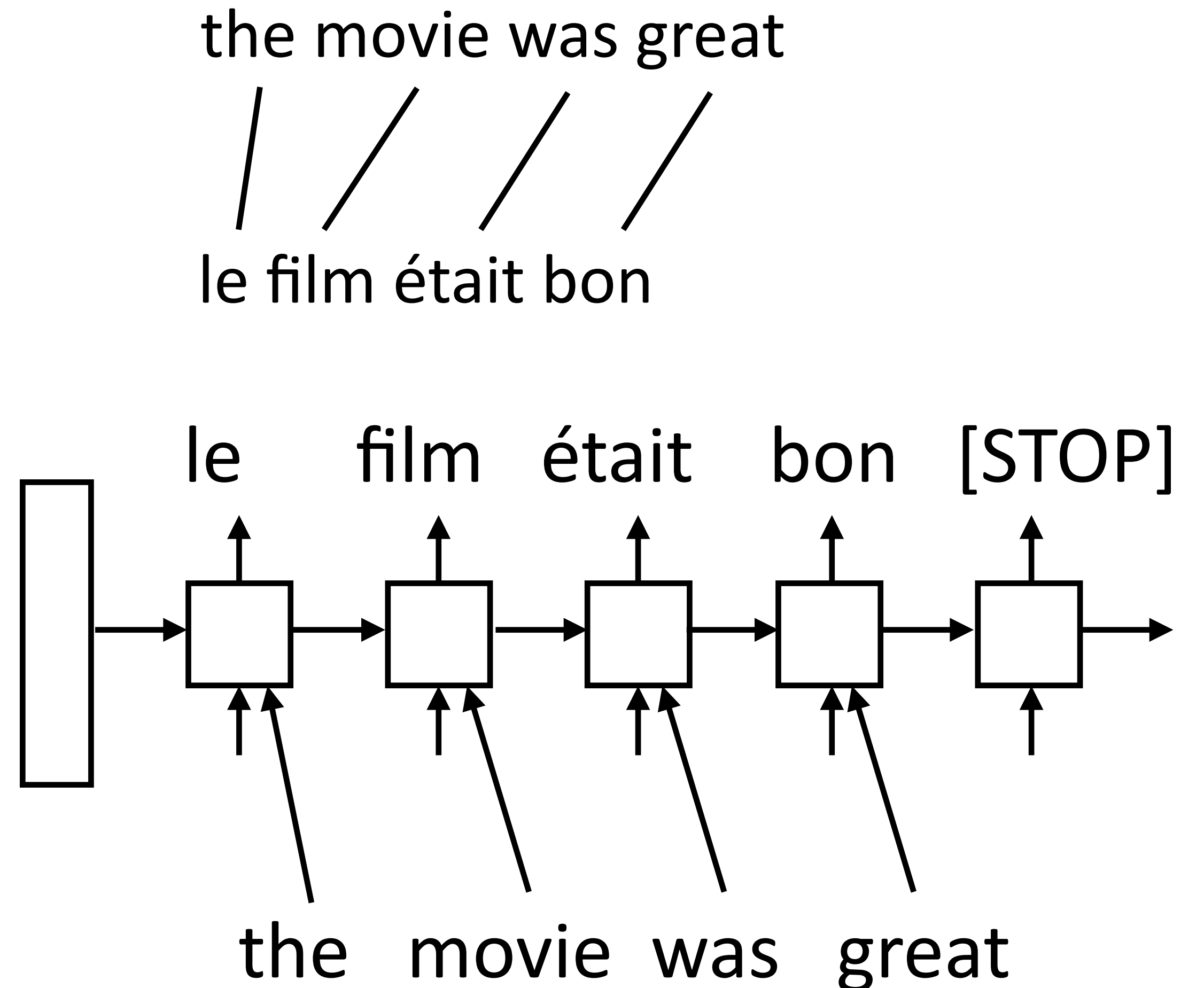
- ▶ Suppose we knew the source and target would be word-by-word translated
- ▶ Can look at the corresponding input word when translating — this could scale!

the movie was great
/ / / /
le film était bon



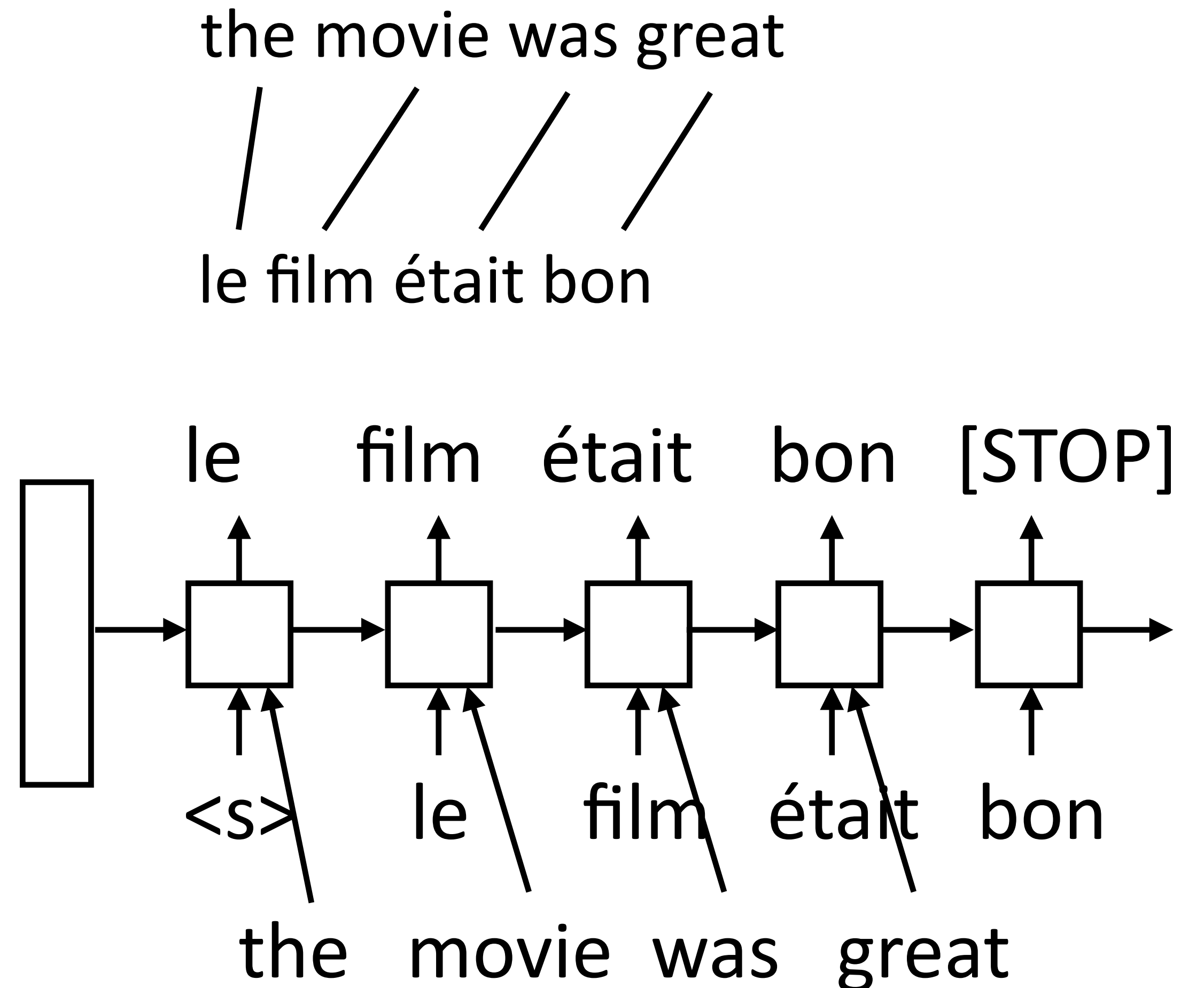
Aligned Inputs

- ▶ Suppose we knew the source and target would be word-by-word translated
- ▶ Can look at the corresponding input word when translating — this could scale!



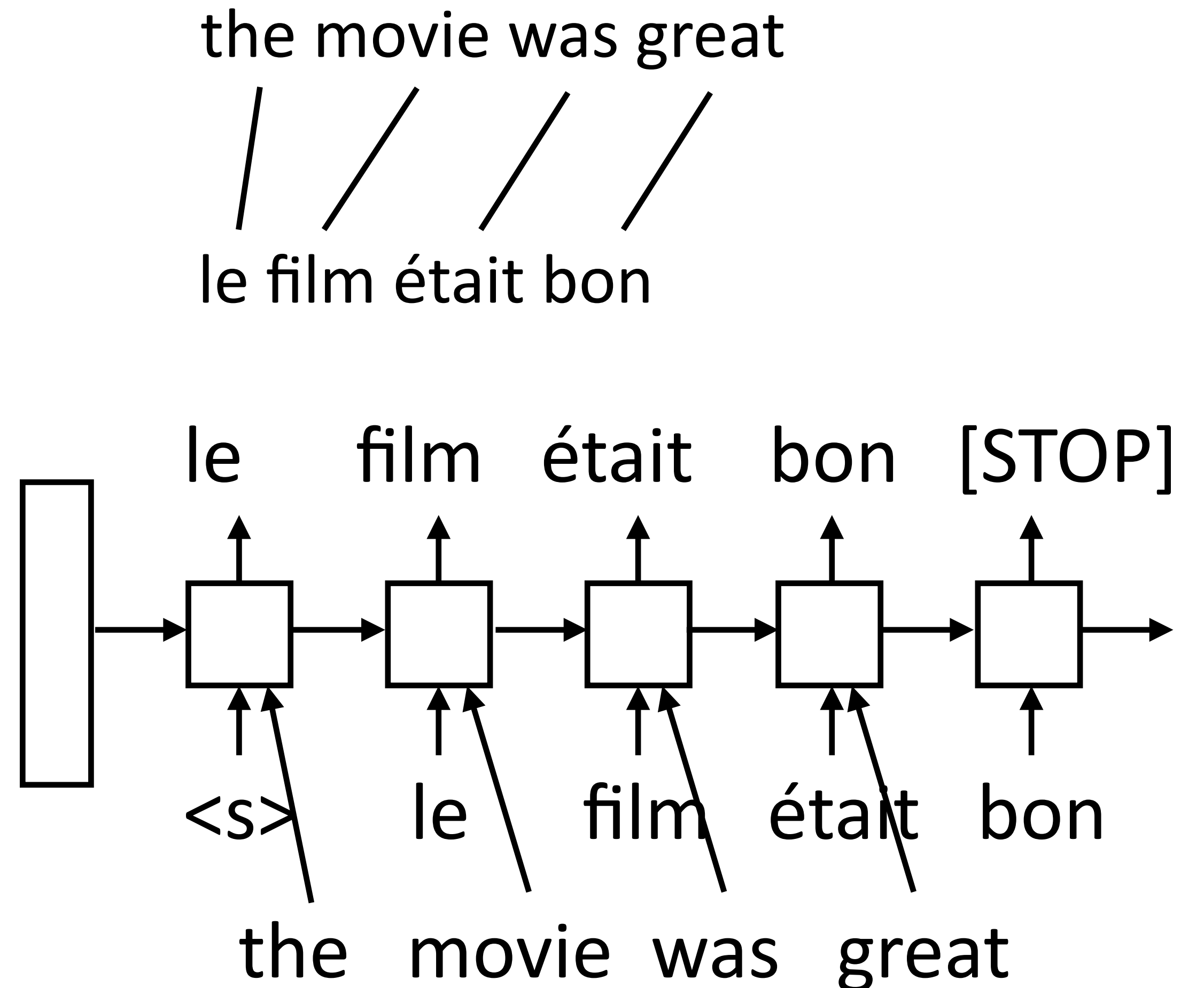
Aligned Inputs

- Suppose we knew the source and target would be word-by-word translated
- Can look at the corresponding input word when translating — this could scale!



Aligned Inputs

- Suppose we knew the source and target would be word-by-word translated
- Can look at the corresponding input word when translating — this could scale!
- Much less burden on the hidden state



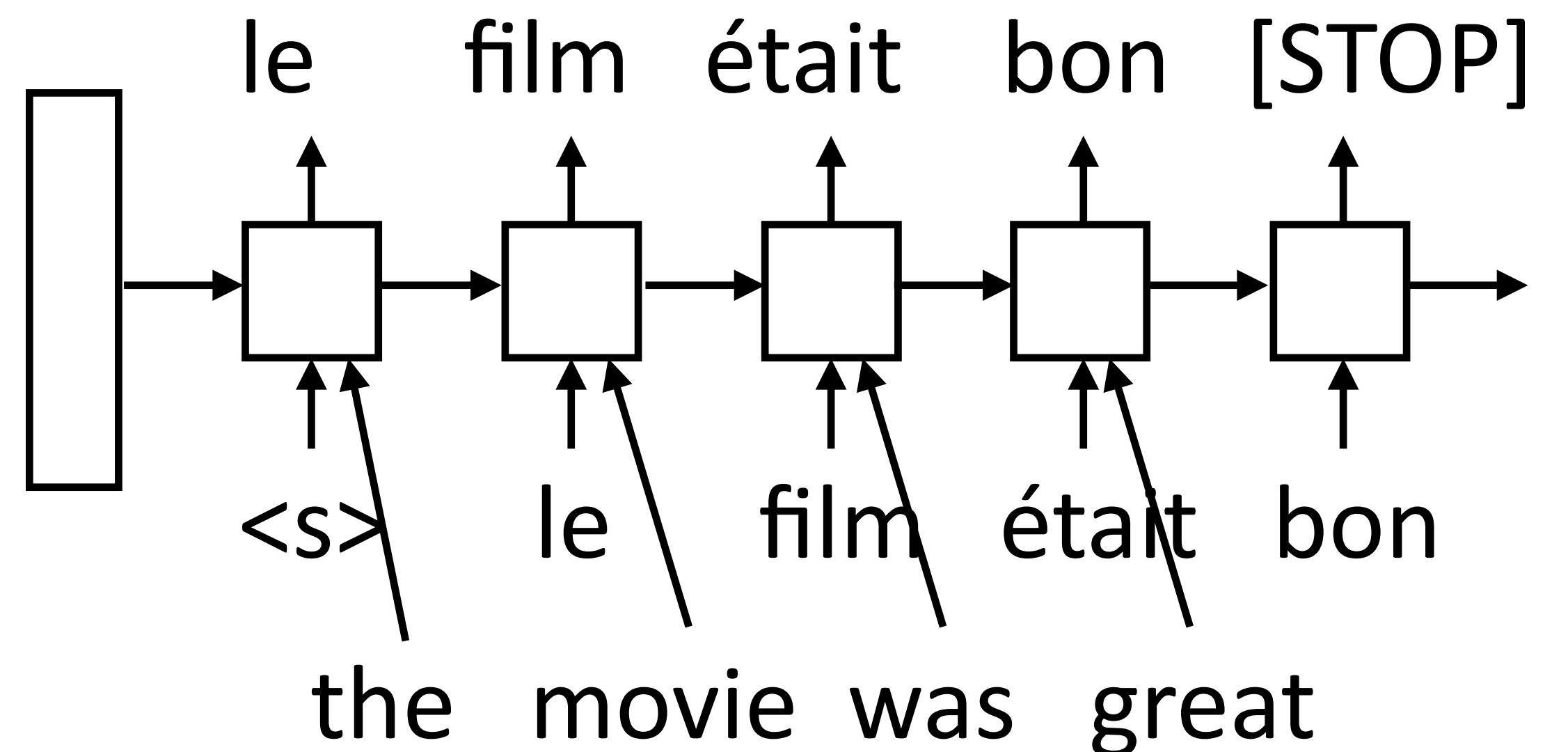
Aligned Inputs

- Suppose we knew the source and target would be word-by-word translated

the movie was great
/ / / /
le film était bon

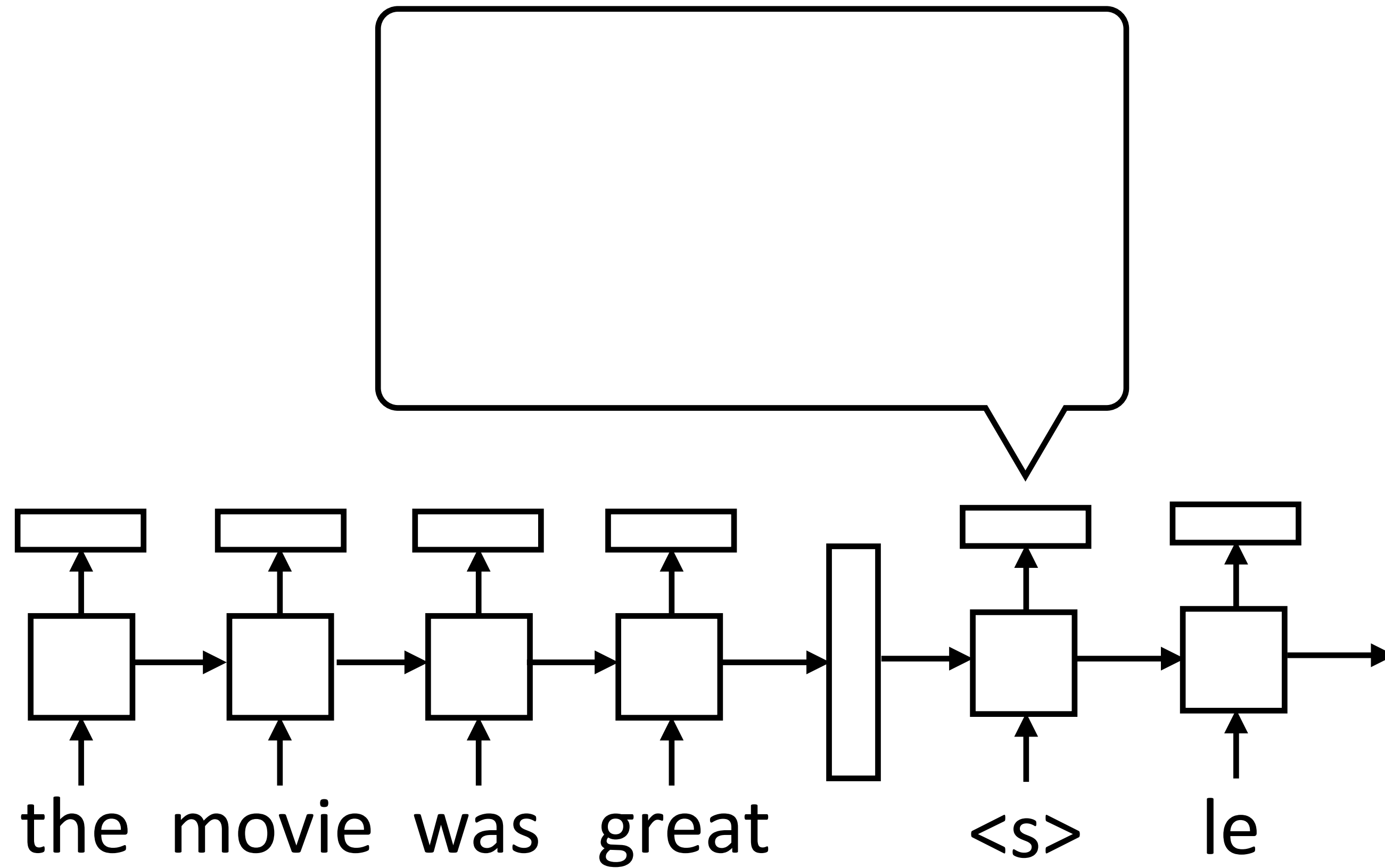
- Can look at the corresponding input word when translating — this could scale!

- Much less burden on the hidden state

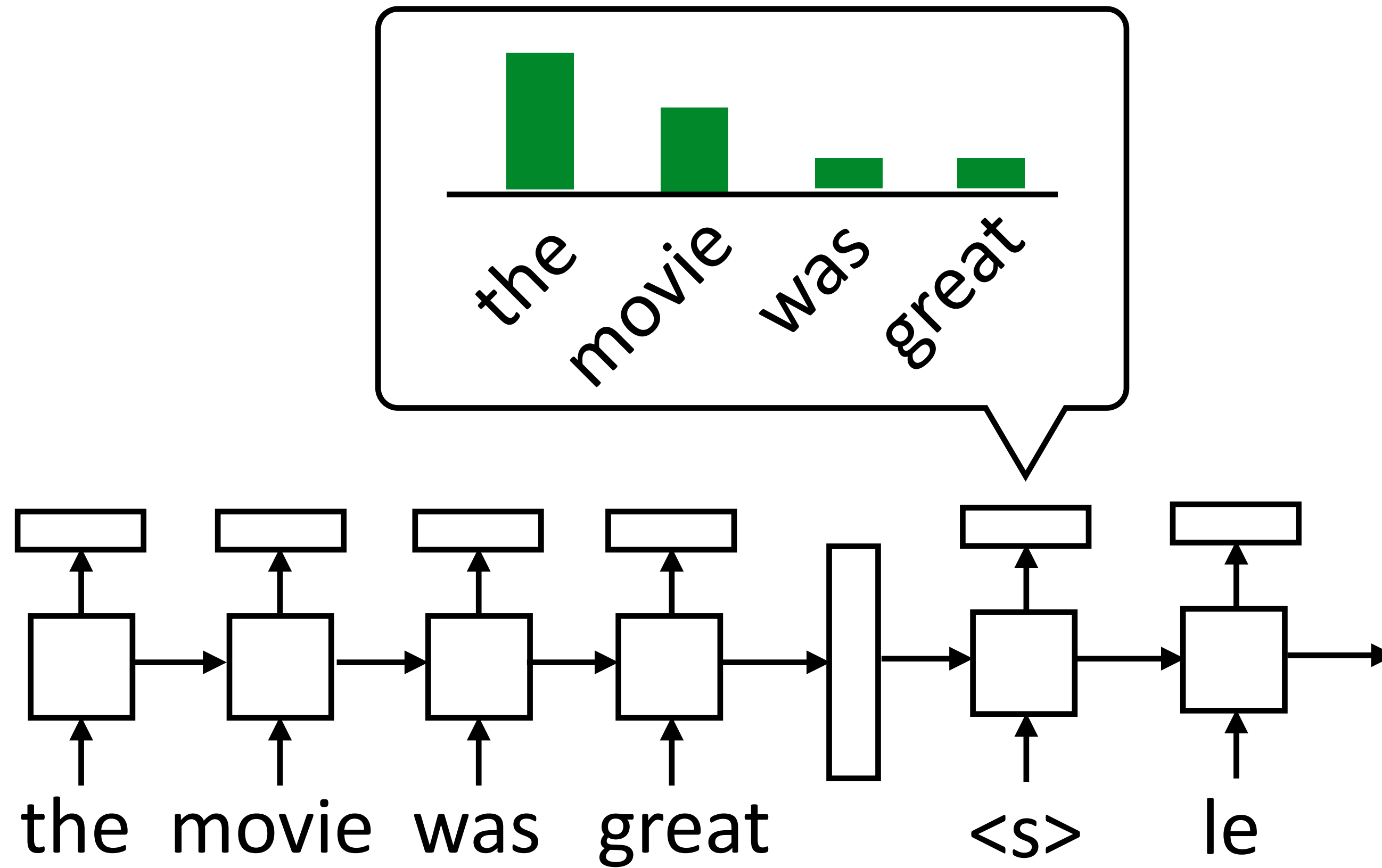


- How can we achieve this without hardcoding it?

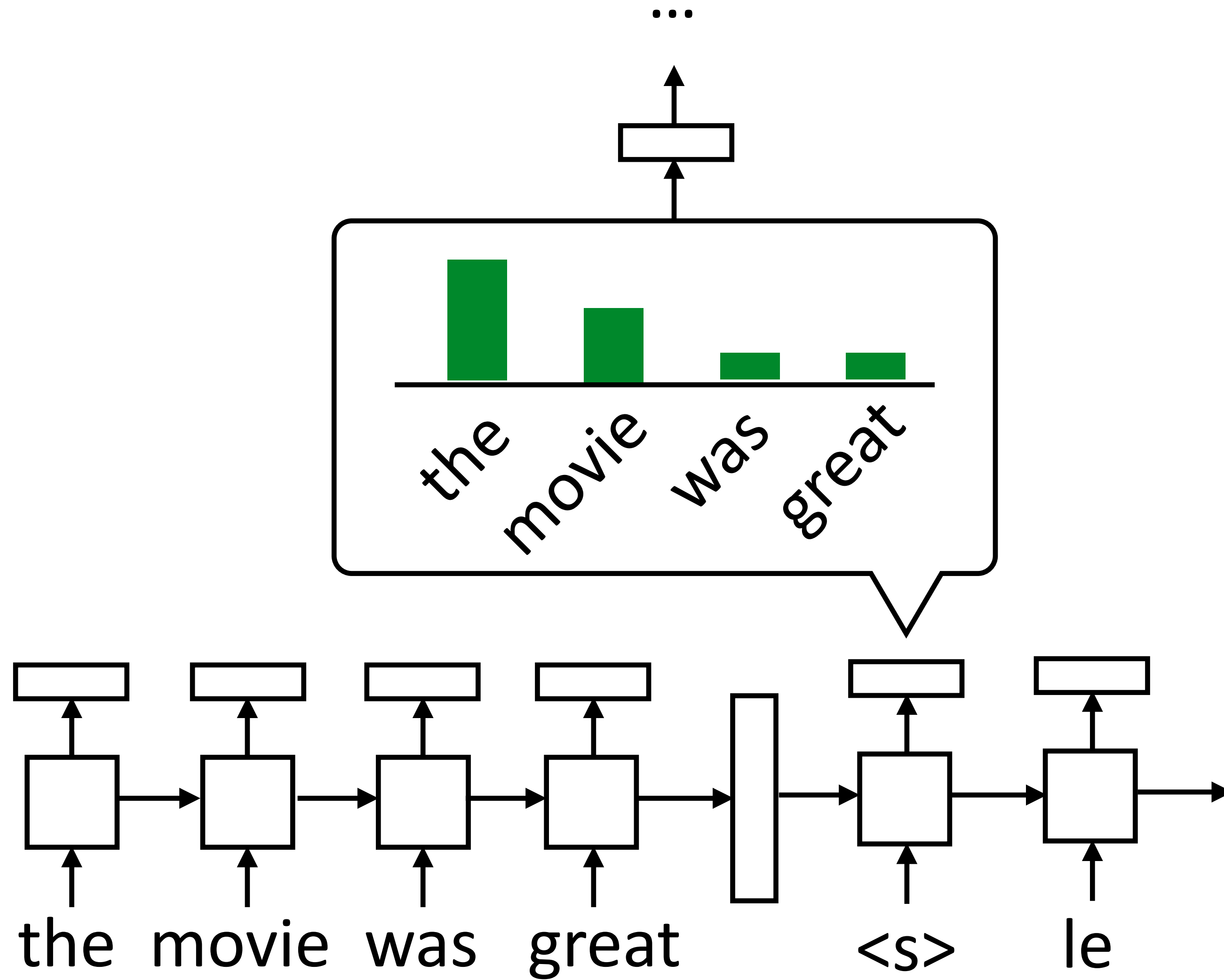
Attention



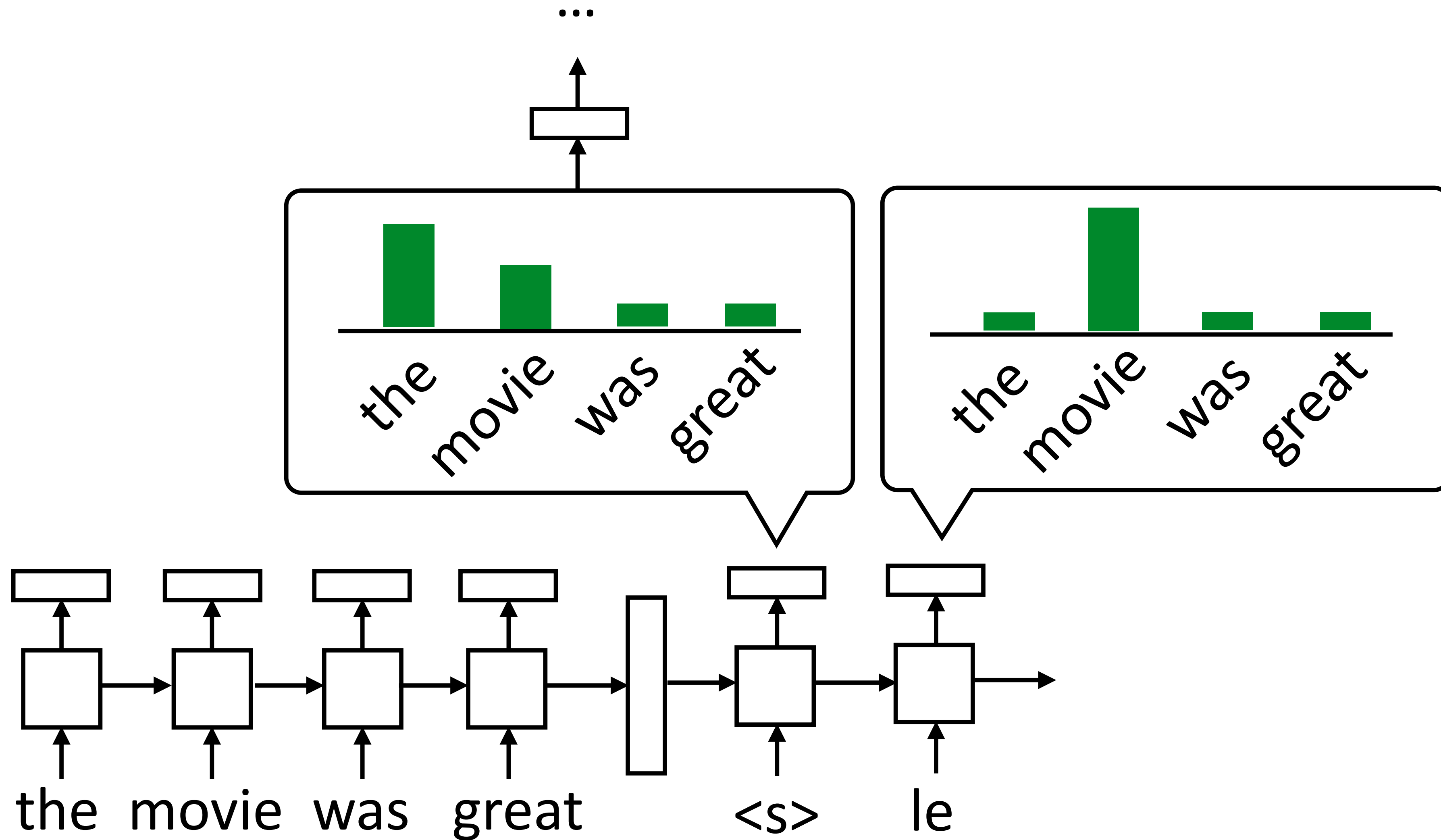
Attention



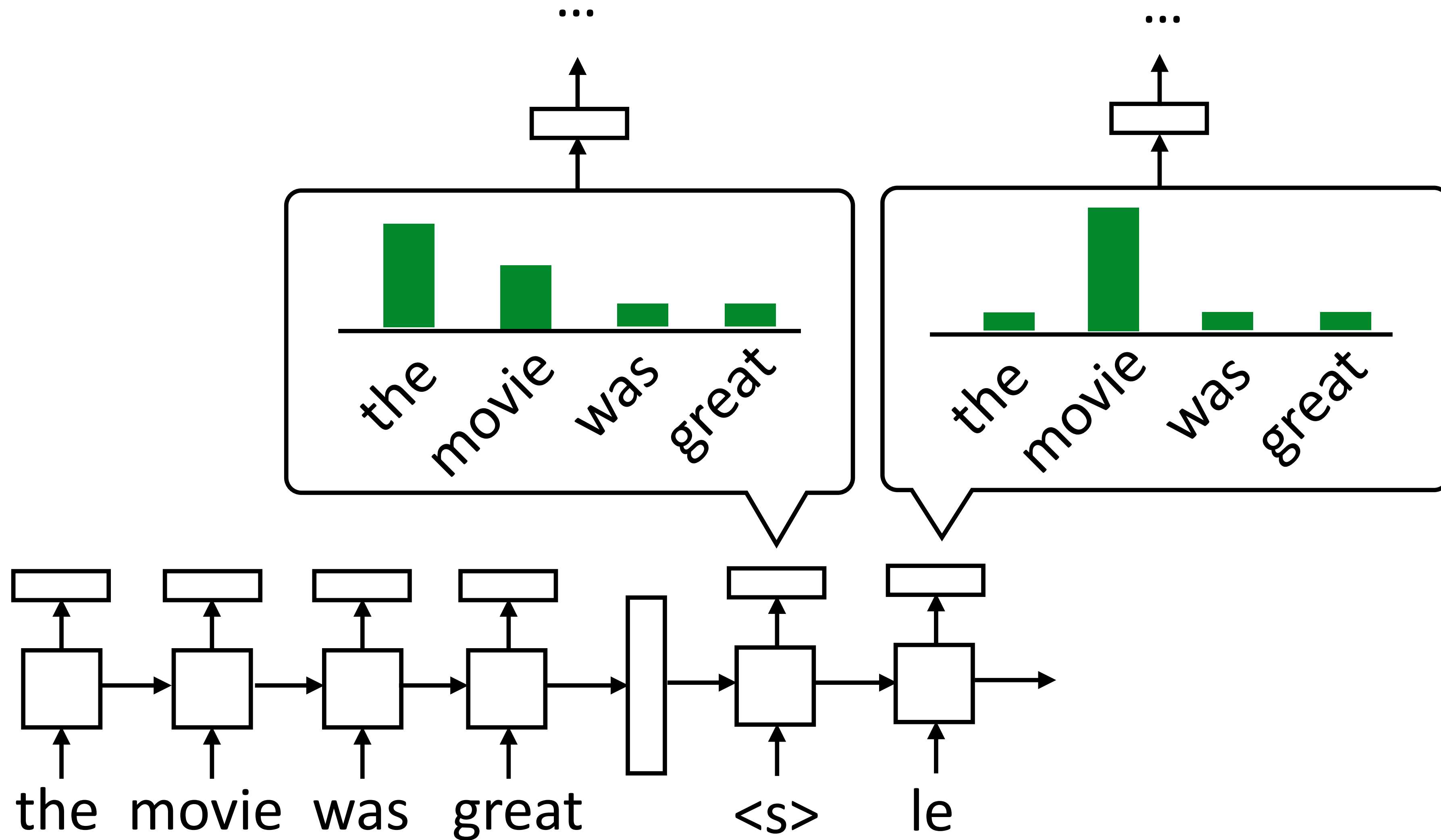
Attention



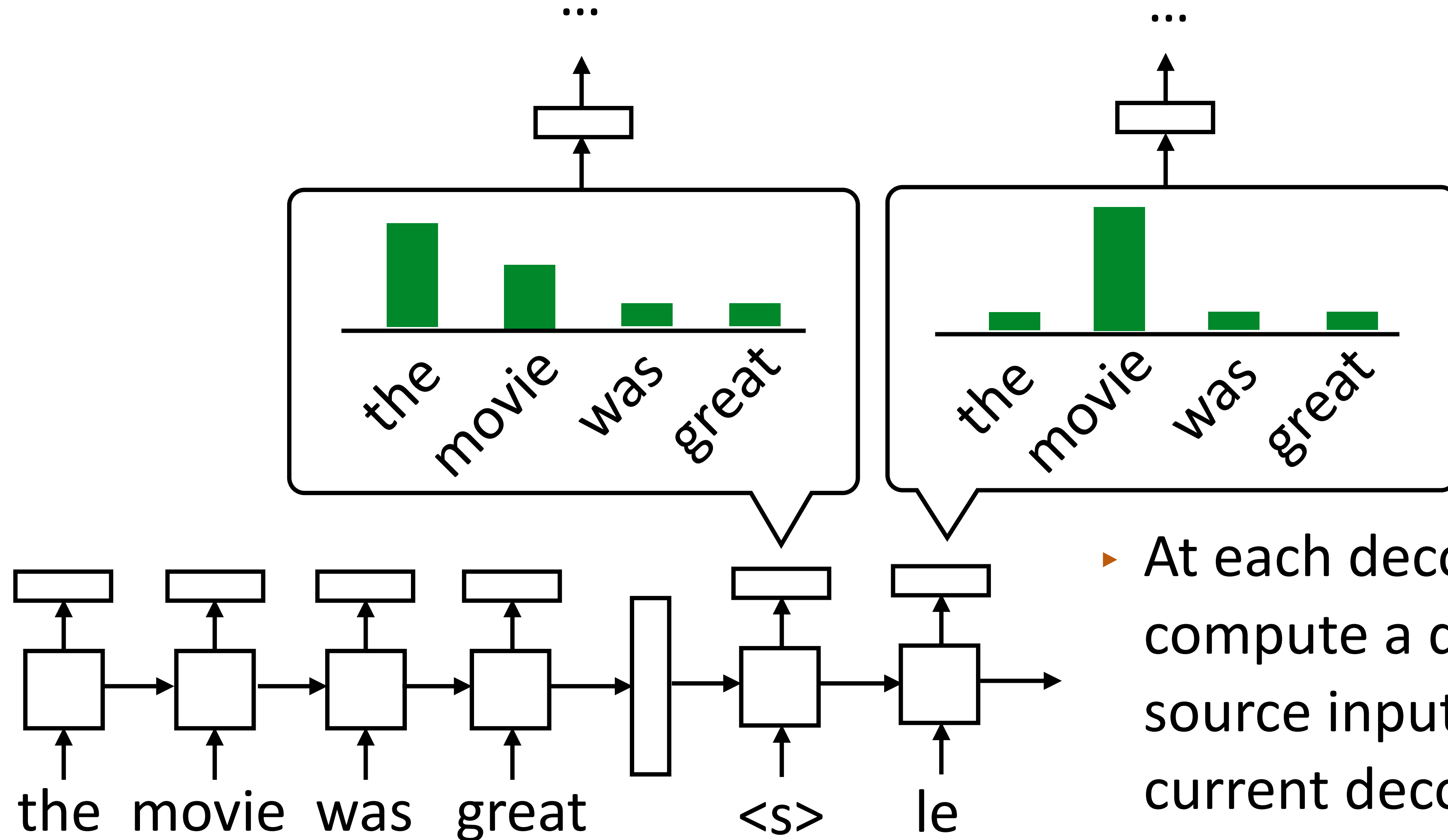
Attention



Attention

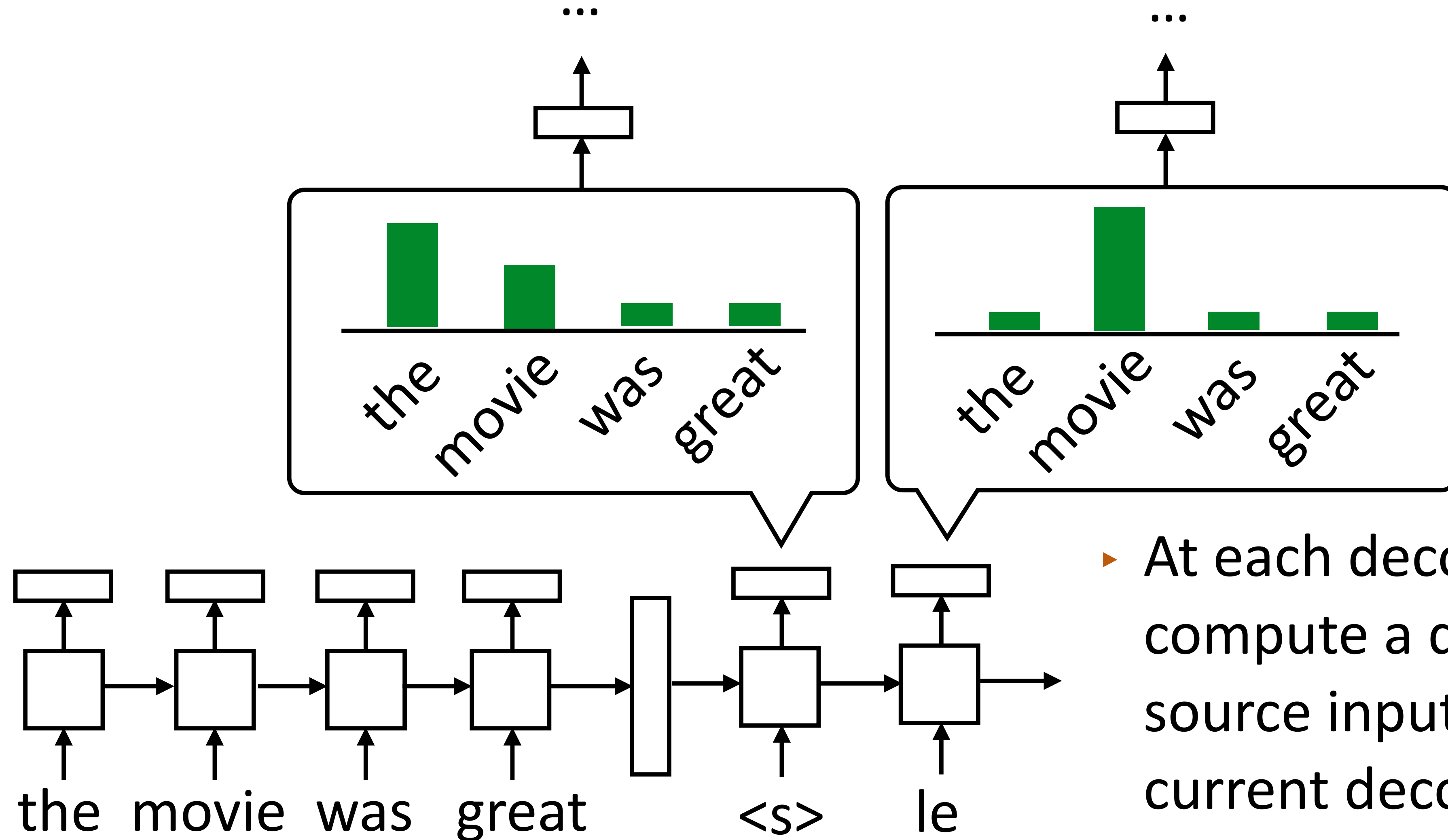


Attention



- At each decoder state, compute a distribution over source inputs based on current decoder state

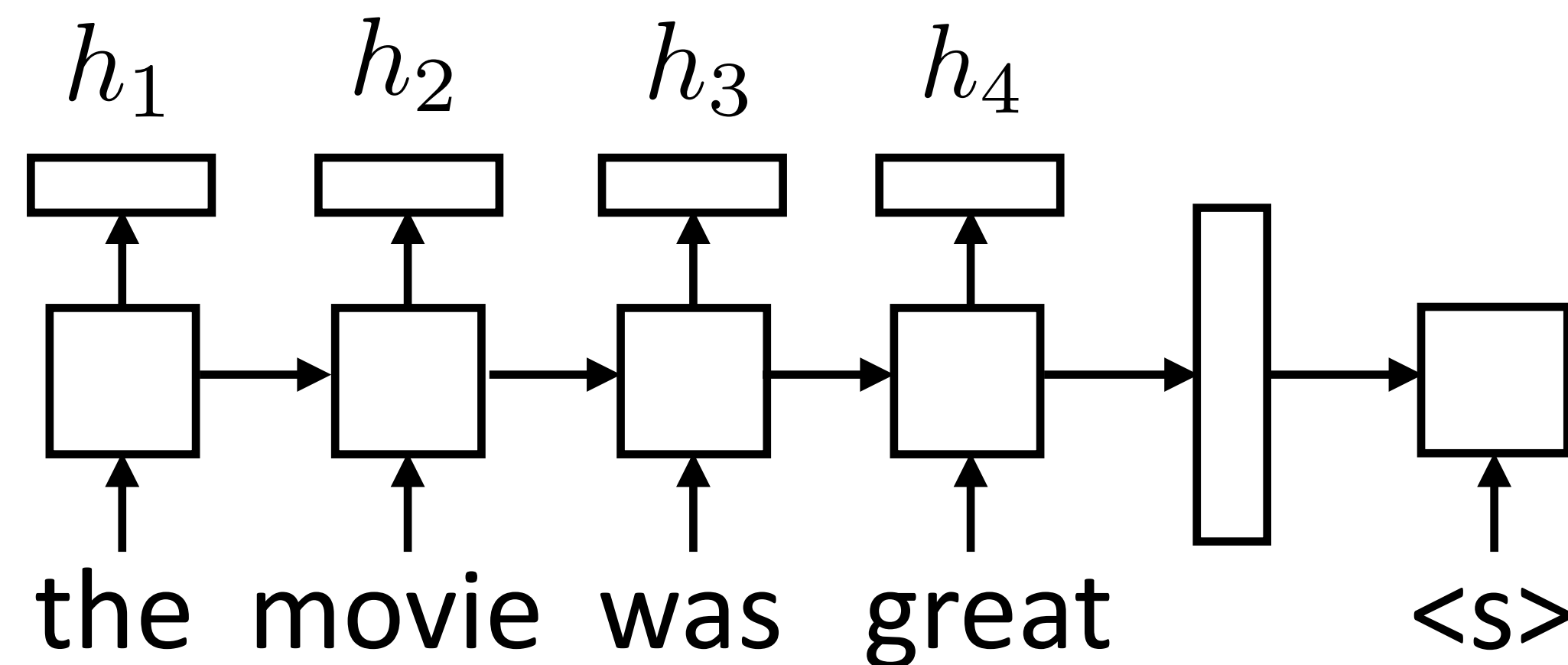
Attention



- ▶ At each decoder state, compute a distribution over source inputs based on current decoder state
- ▶ Use that in output layer

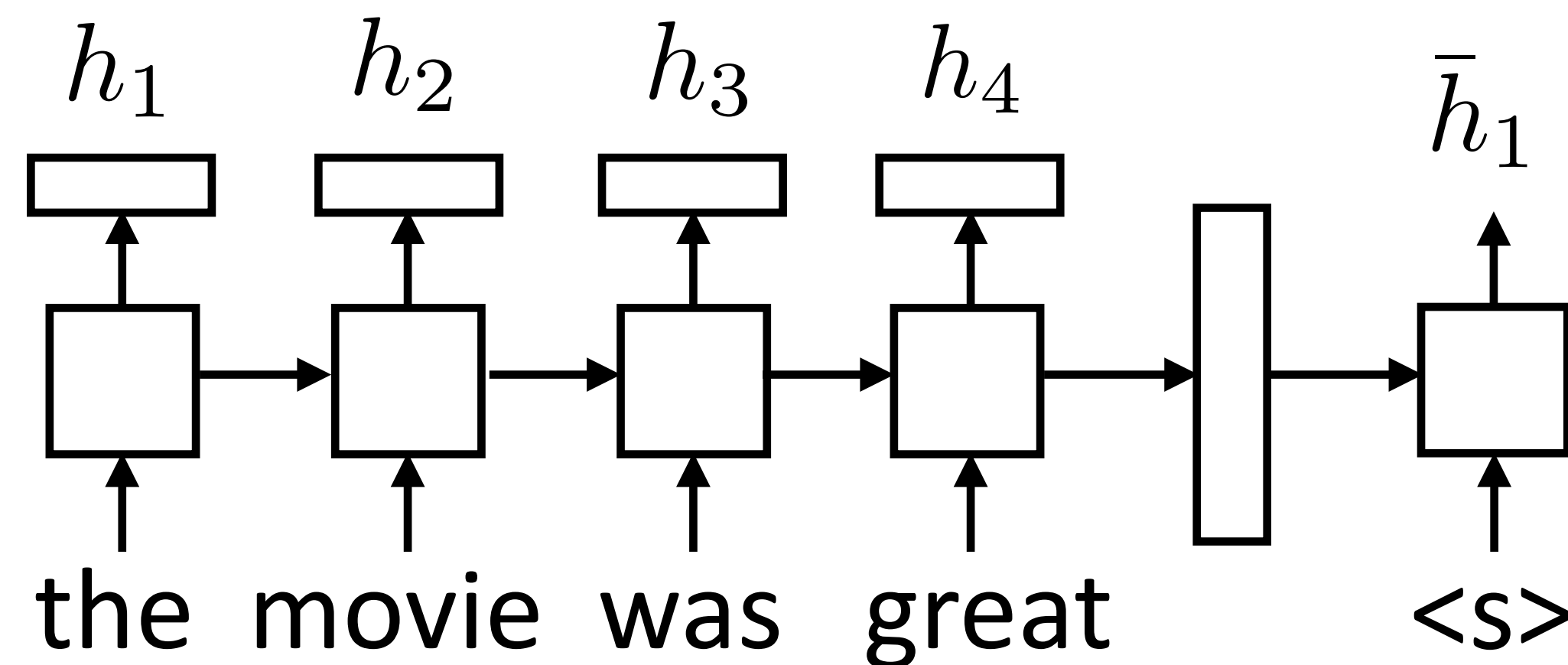
Attention

- ▶ For each decoder state, compute weighted sum of input states



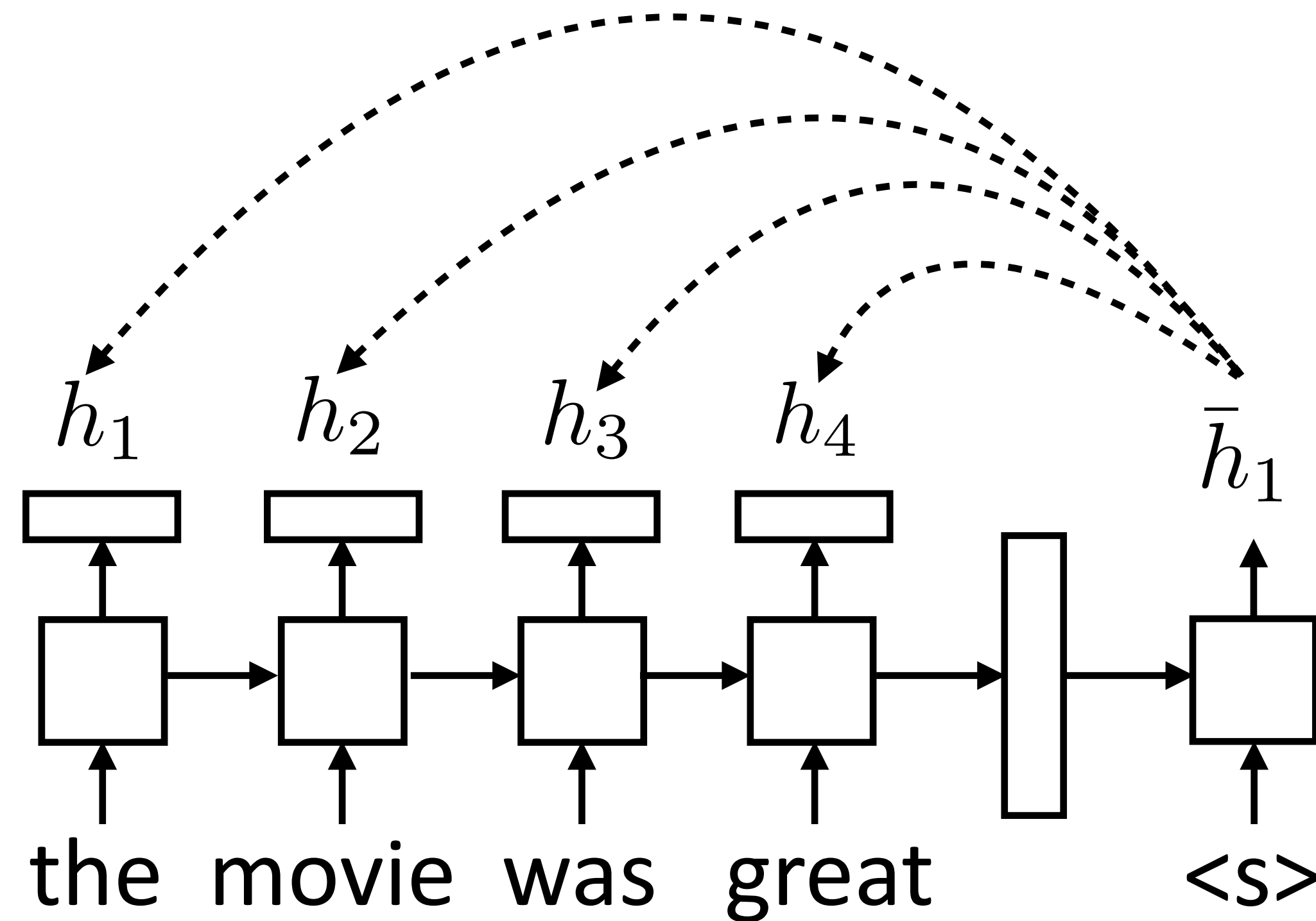
Attention

- For each decoder state, compute weighted sum of input states



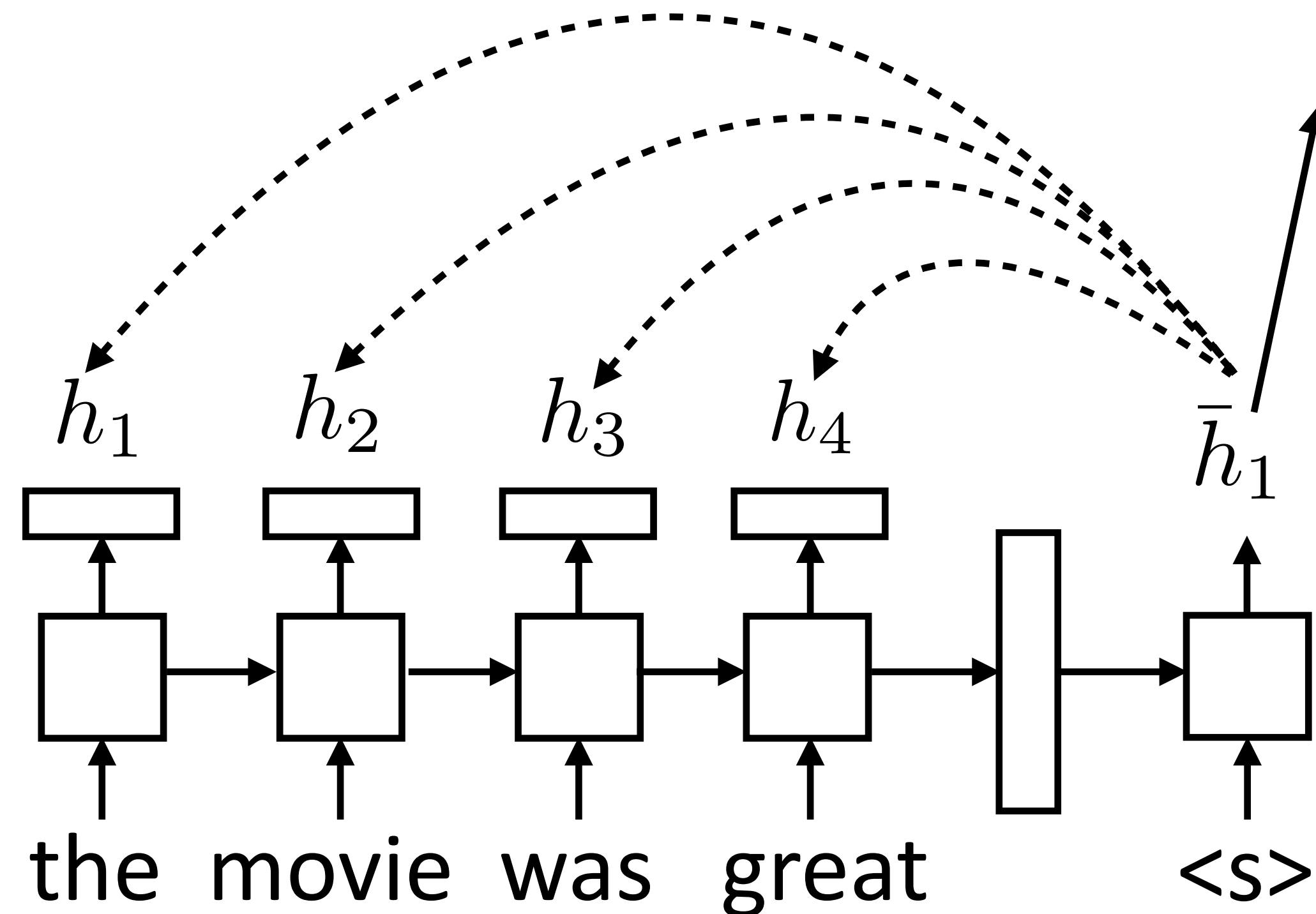
Attention

- For each decoder state, compute weighted sum of input states



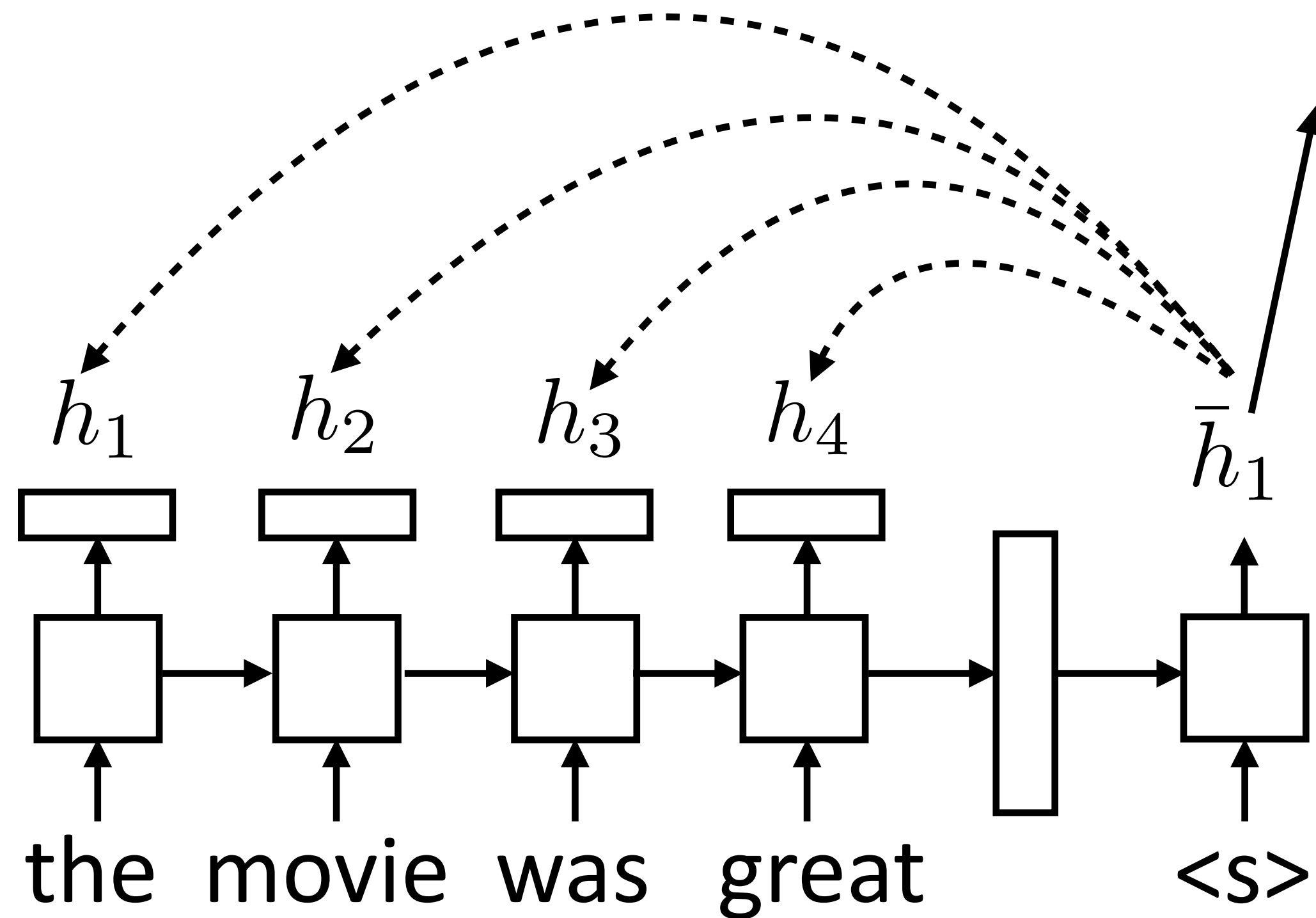
Attention

- ▶ For each decoder state, compute weighted sum of input states



Attention

- ▶ For each decoder state, compute weighted sum of input states

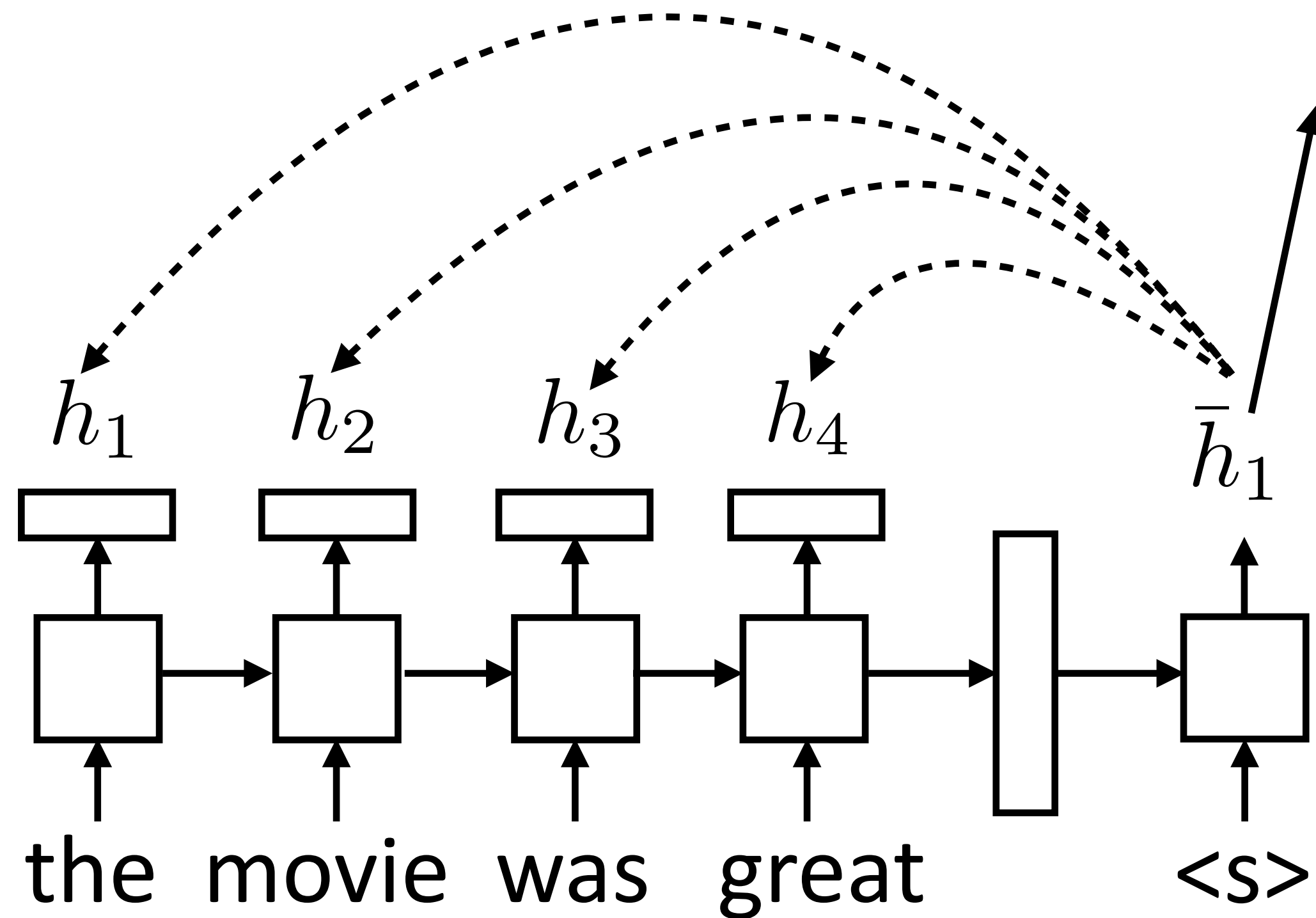


$$e_{ij} = f(\bar{h}_i, h_j)$$

- ▶ Unnormalized scalar weight

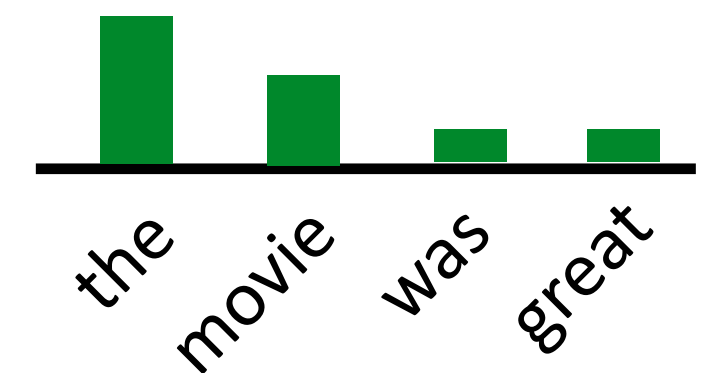
Attention

- For each decoder state, compute weighted sum of input states



$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

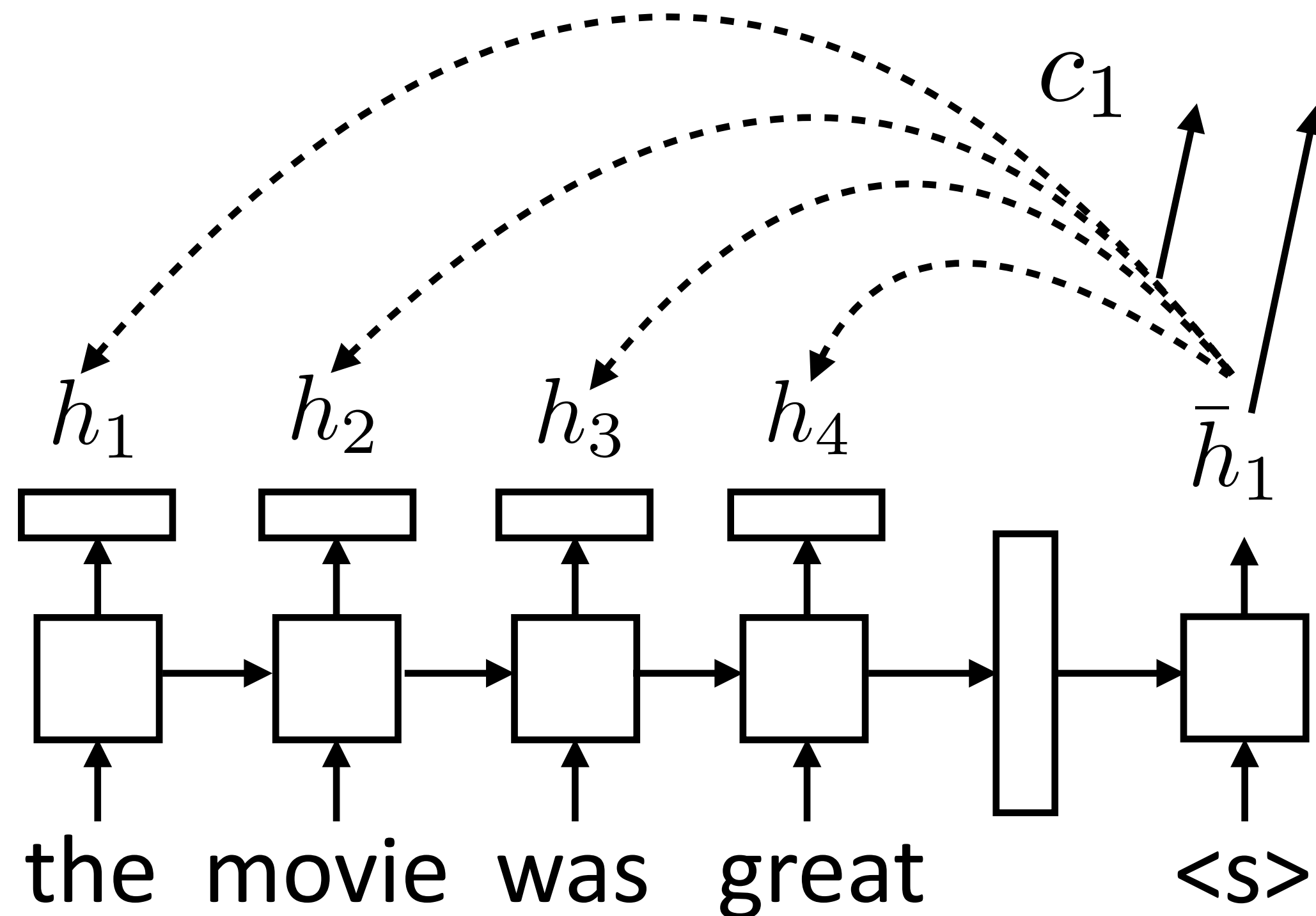
$$e_{ij} = f(\bar{h}_i, h_j)$$



- Unnormalized scalar weight

Attention

- ▶ For each decoder state, compute weighted sum of input states

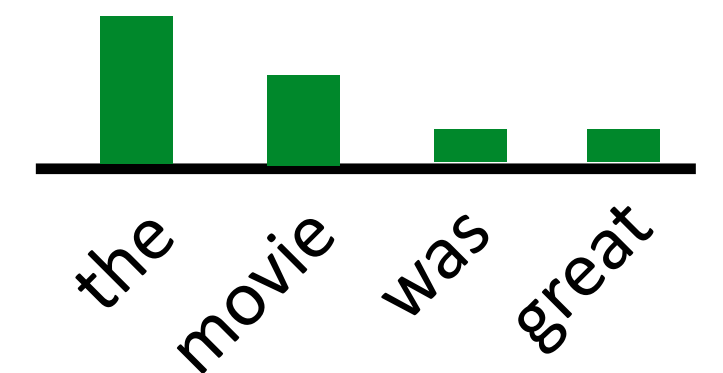


$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

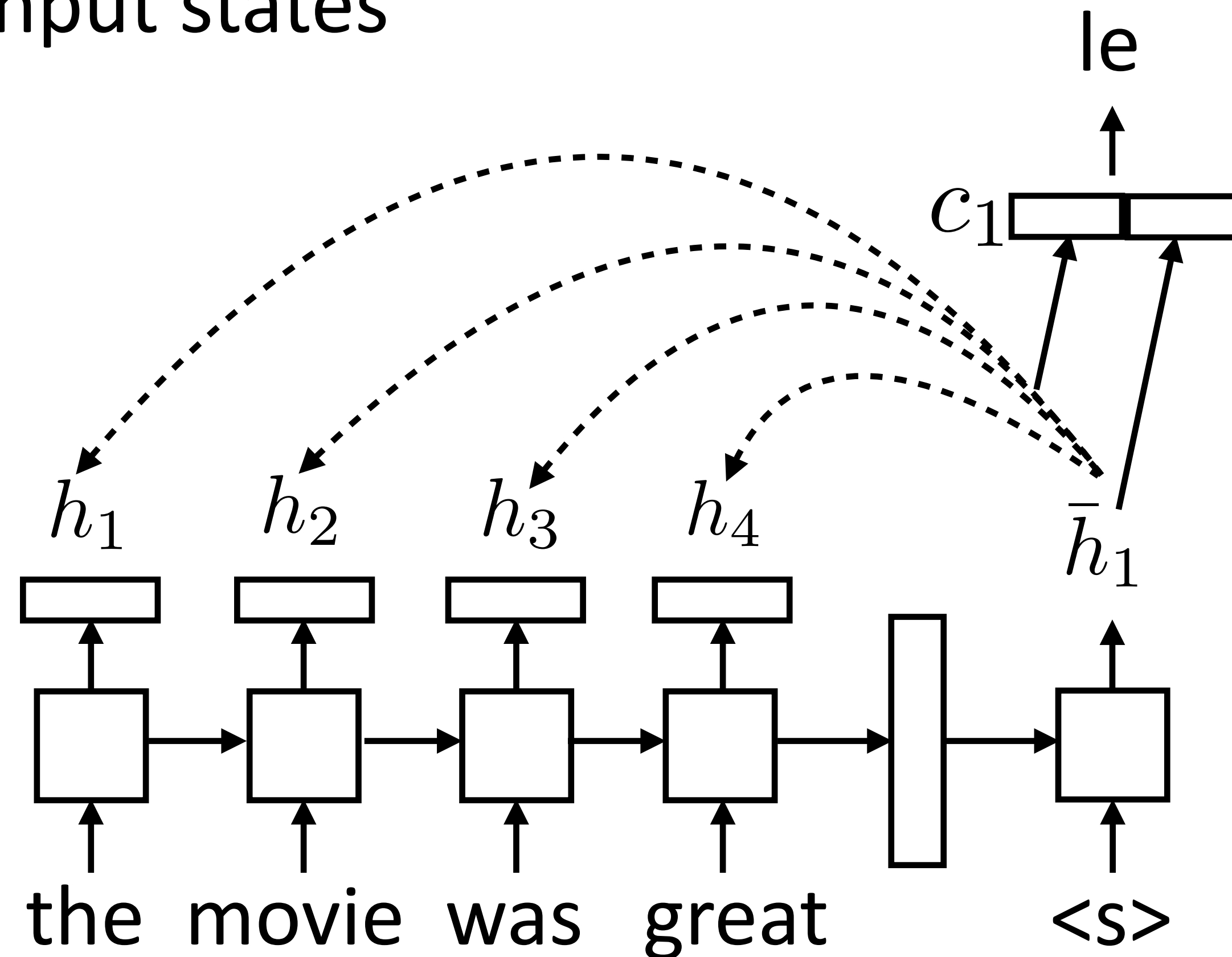
- ▶ Weighted sum of input hidden states (vector)



- ▶ Unnormalized scalar weight

Attention

- For each decoder state, compute weighted sum of input states

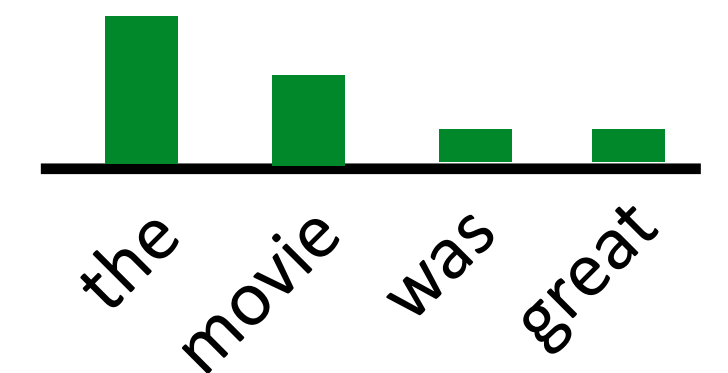


$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

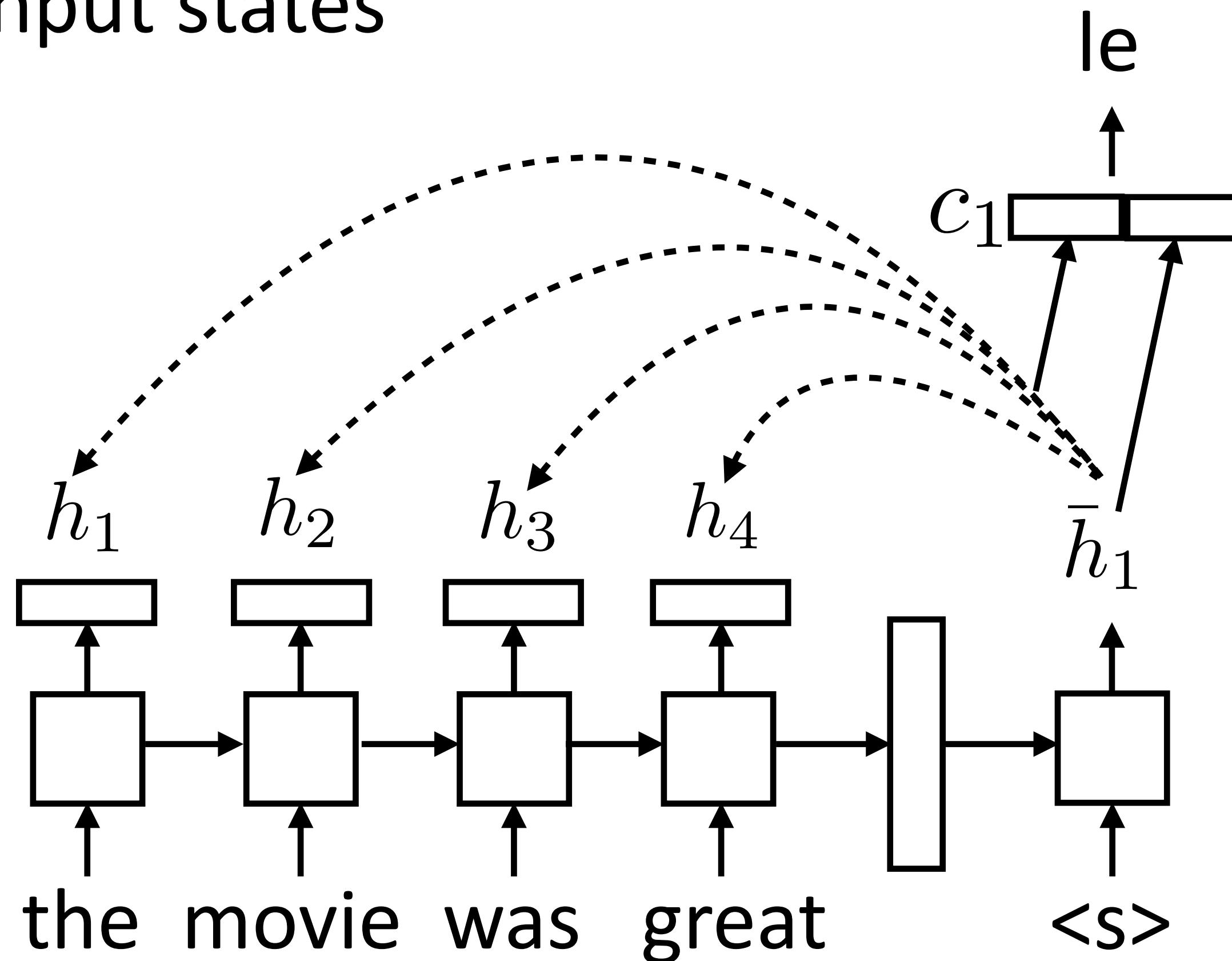
- Weighted sum of input hidden states (vector)



- Unnormalized scalar weight

Attention

- For each decoder state, compute weighted sum of input states



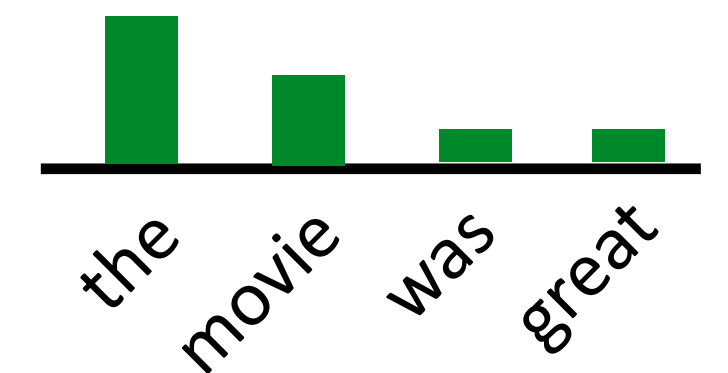
$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W[c_i; \bar{h}_i])$$

$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

- Weighted sum of input hidden states (vector)

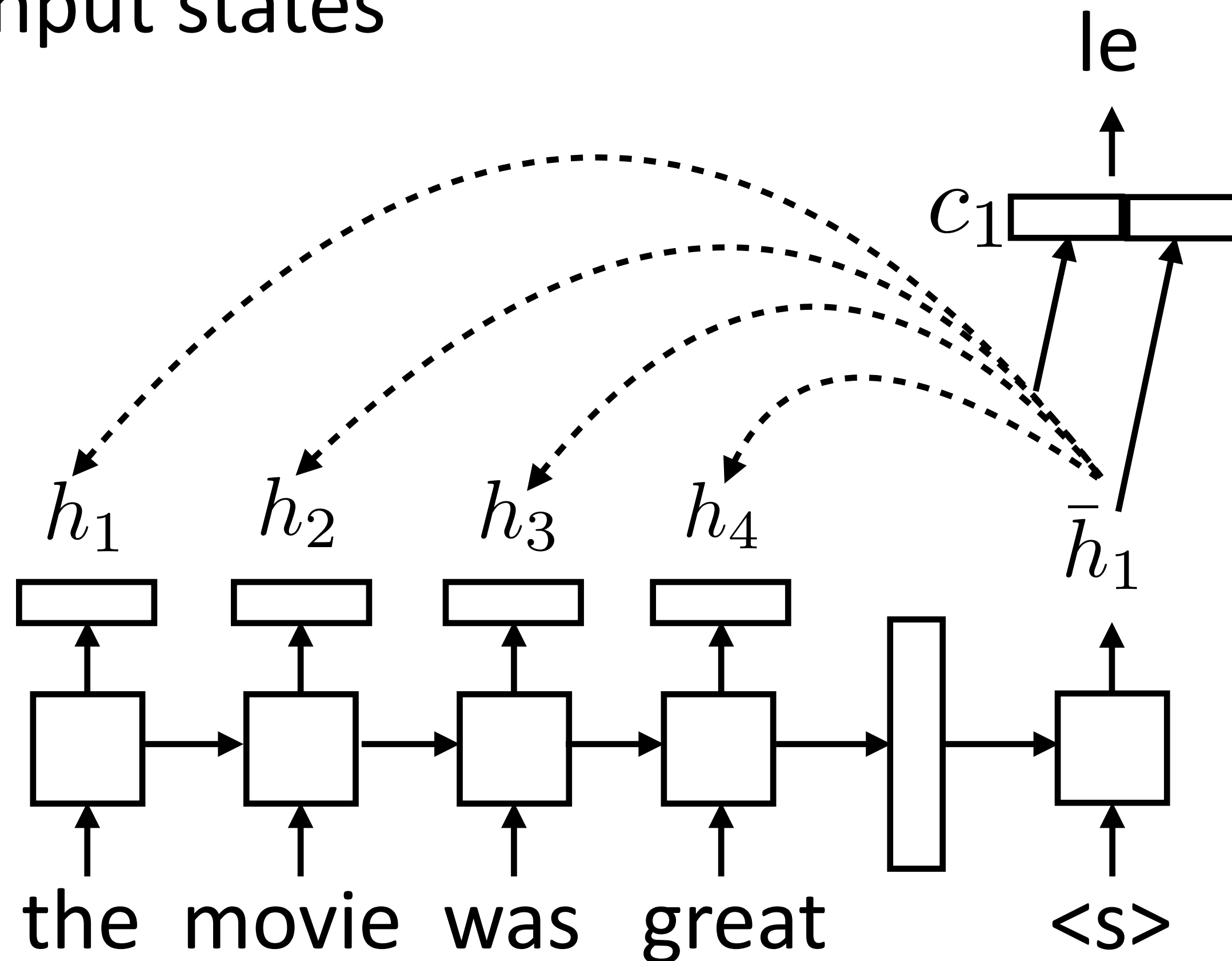


- Unnormalized scalar weight

Attention

- For each decoder state, compute weighted sum of input states

- No attn: $P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h}_i)$



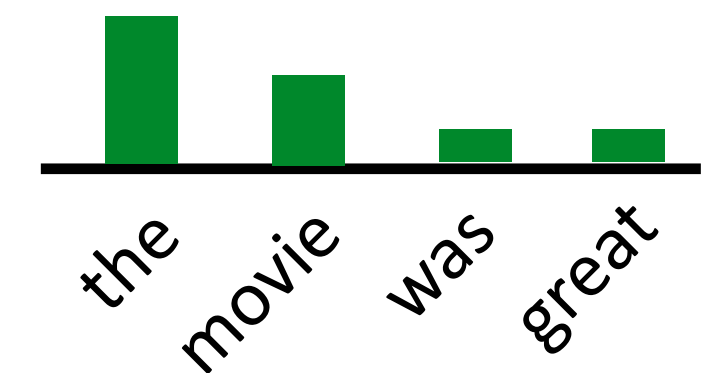
$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W[c_i; \bar{h}_i])$$

$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

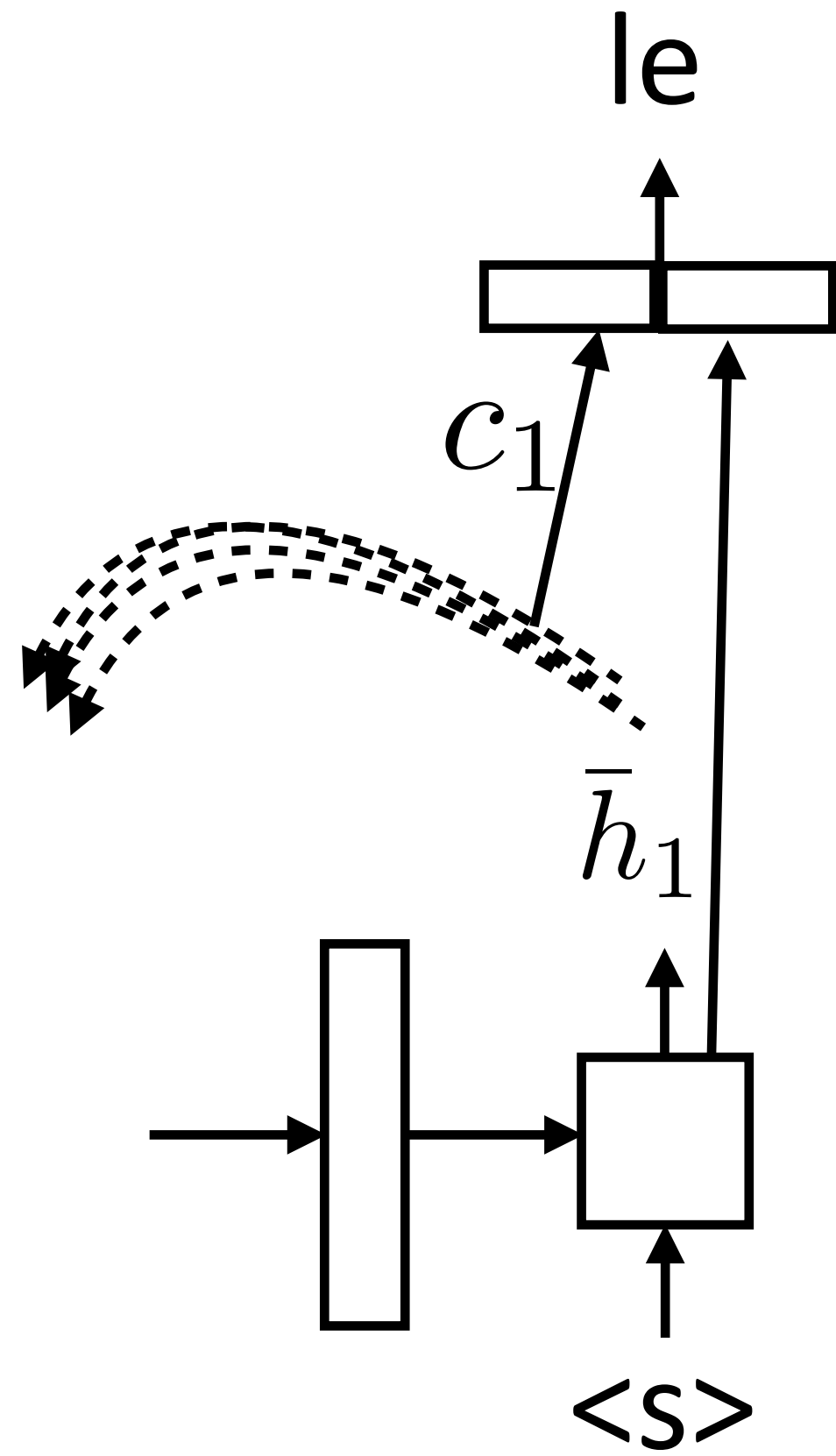
$$e_{ij} = f(\bar{h}_i, h_j)$$

- Weighted sum of input hidden states (vector)



- Unnormalized scalar weight

Attention

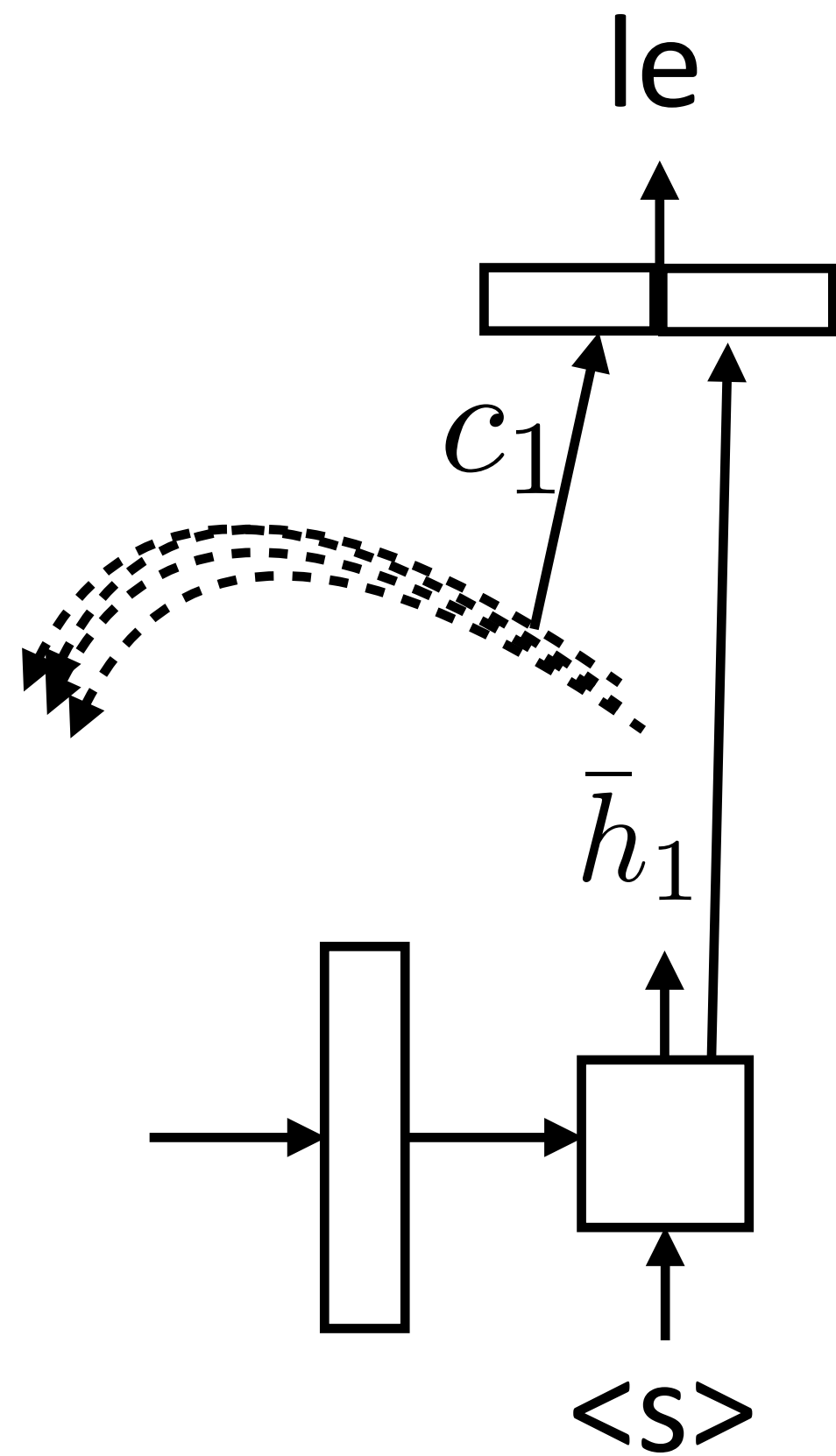


$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

Attention



$$c_i = \sum_j \alpha_{ij} h_j$$

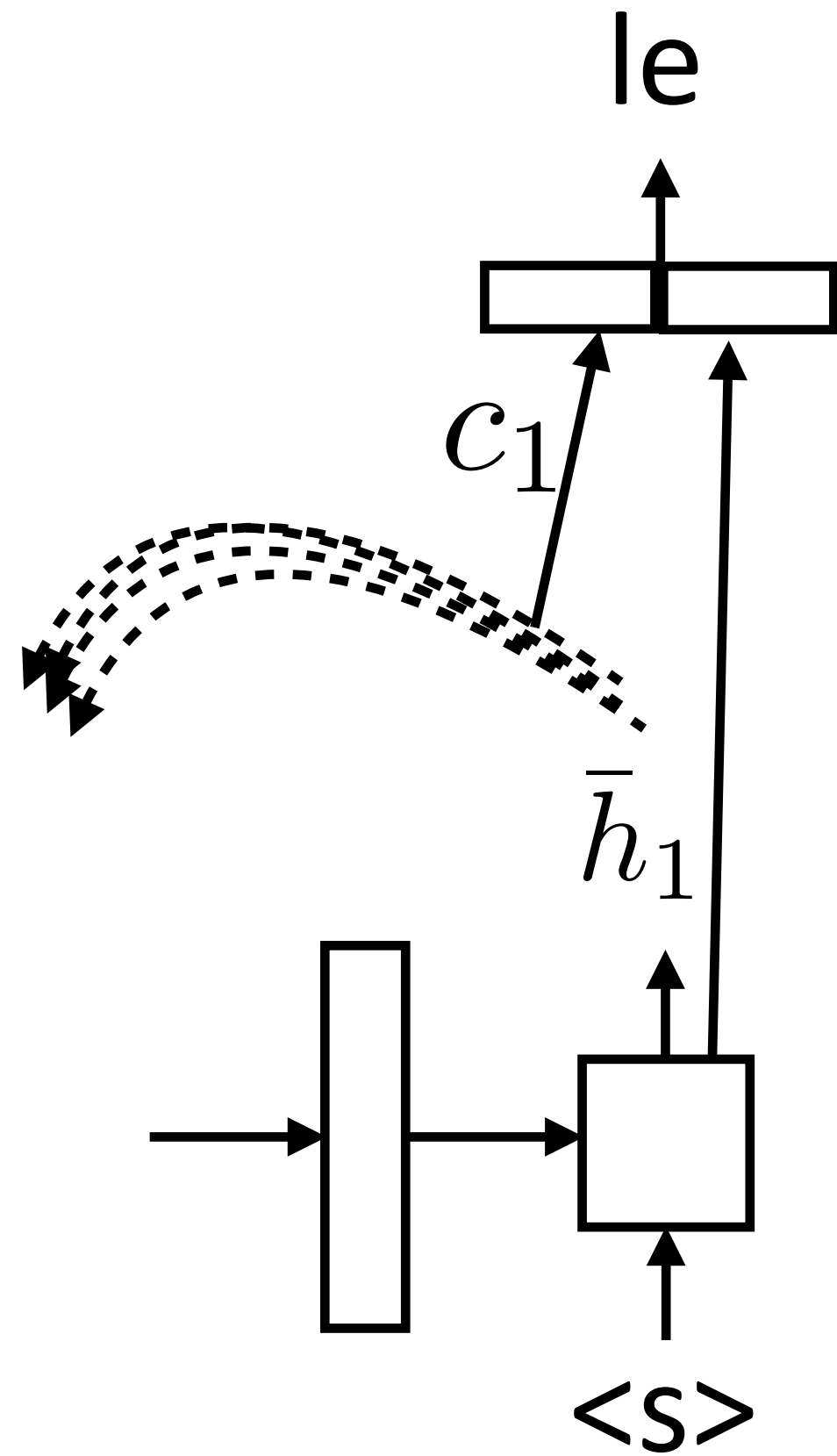
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

$$f(\bar{h}_i, h_j) = \tanh(W[\bar{h}_i, h_j])$$

► Bahdanau+ (2014): additive

Attention



$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

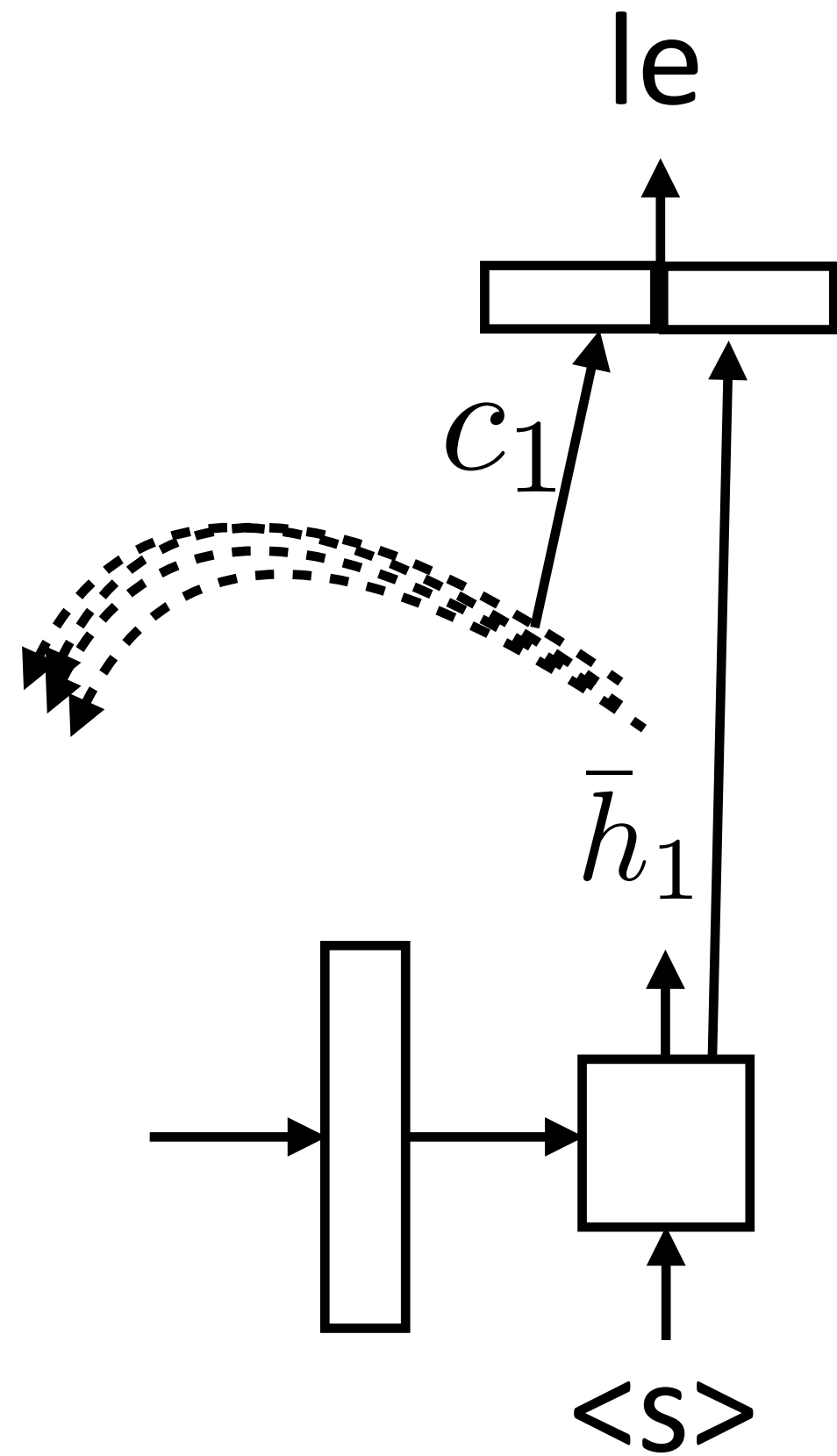
$$f(\bar{h}_i, h_j) = \tanh(W[\bar{h}_i, h_j])$$

► Bahdanau+ (2014): additive

$$f(\bar{h}_i, h_j) = \bar{h}_i \cdot h_j$$

► Luong+ (2015): dot product

Attention



$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

$$f(\bar{h}_i, h_j) = \tanh(W[\bar{h}_i, h_j])$$

► Bahdanau+ (2014): additive

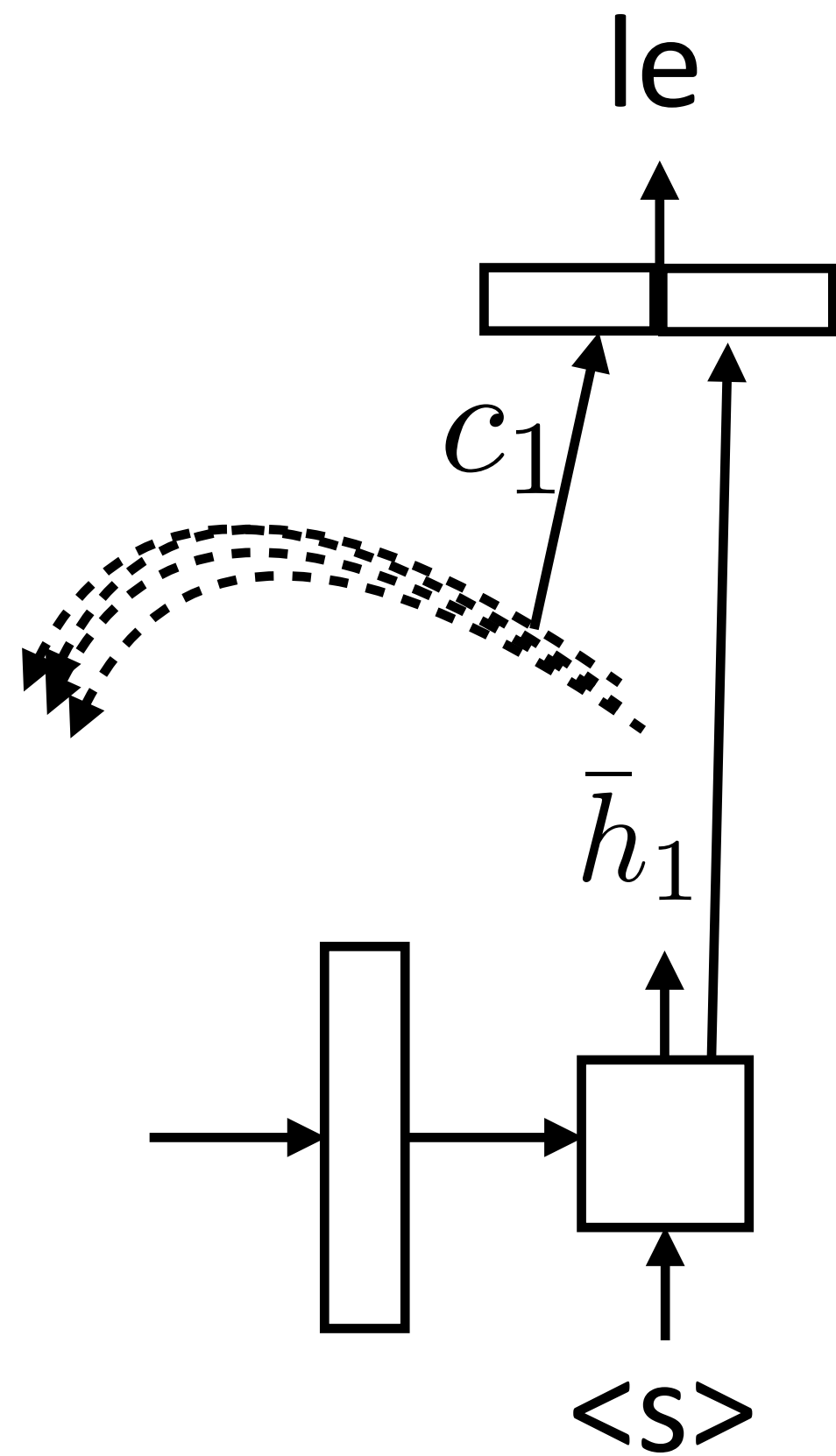
$$f(\bar{h}_i, h_j) = \bar{h}_i \cdot h_j$$

► Luong+ (2015): dot product

$$f(\bar{h}_i, h_j) = \bar{h}_i^\top W h_j$$

► Luong+ (2015): bilinear

Attention



$$c_i = \sum_j \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

$$e_{ij} = f(\bar{h}_i, h_j)$$

$$f(\bar{h}_i, h_j) = \tanh(W[\bar{h}_i, h_j])$$

► Bahdanau+ (2014): additive

$$f(\bar{h}_i, h_j) = \bar{h}_i \cdot h_j$$

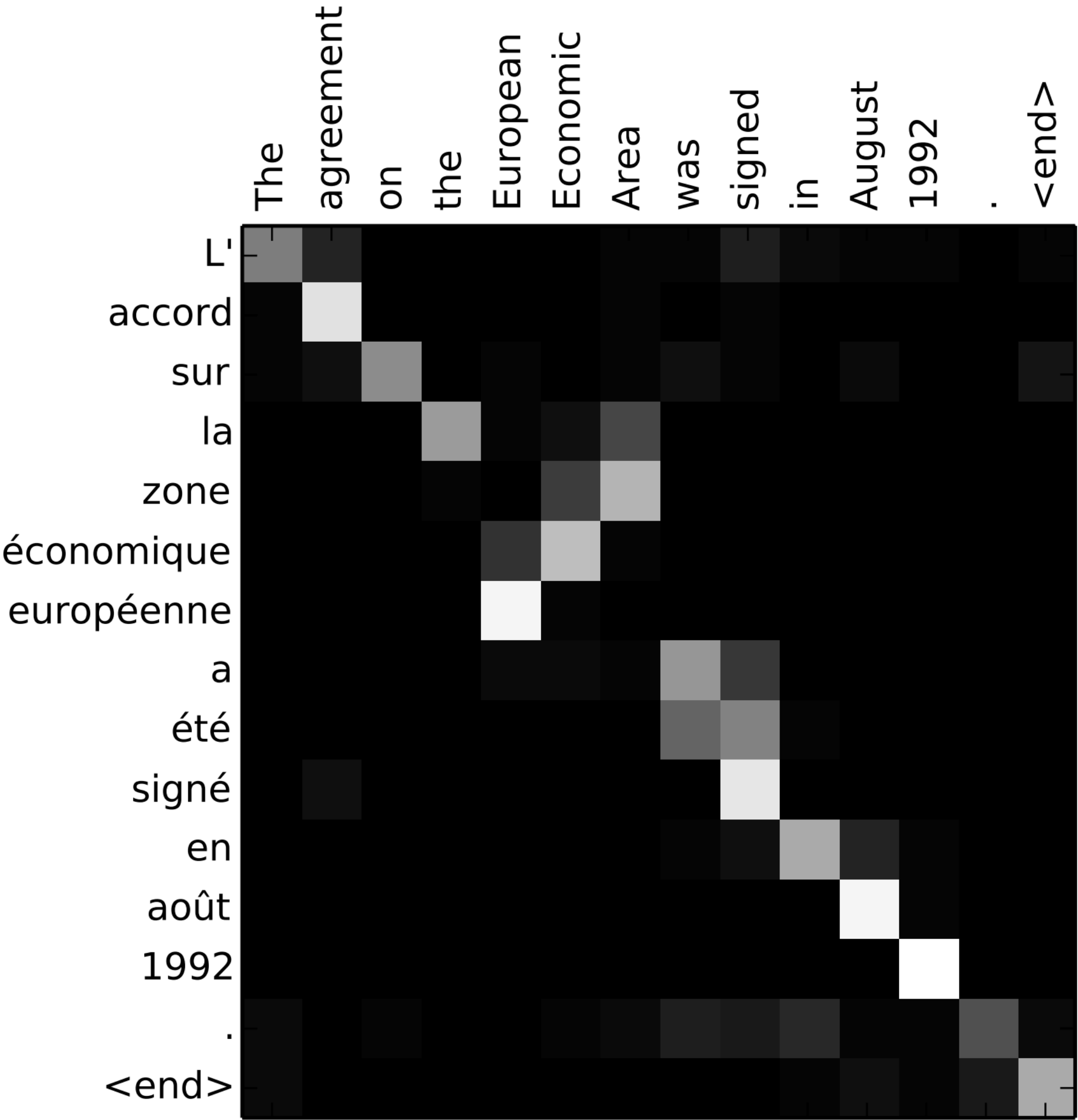
► Luong+ (2015): dot product

$$f(\bar{h}_i, h_j) = \bar{h}_i^\top W h_j$$

► Luong+ (2015): bilinear

- Note that this all uses outputs of hidden layers

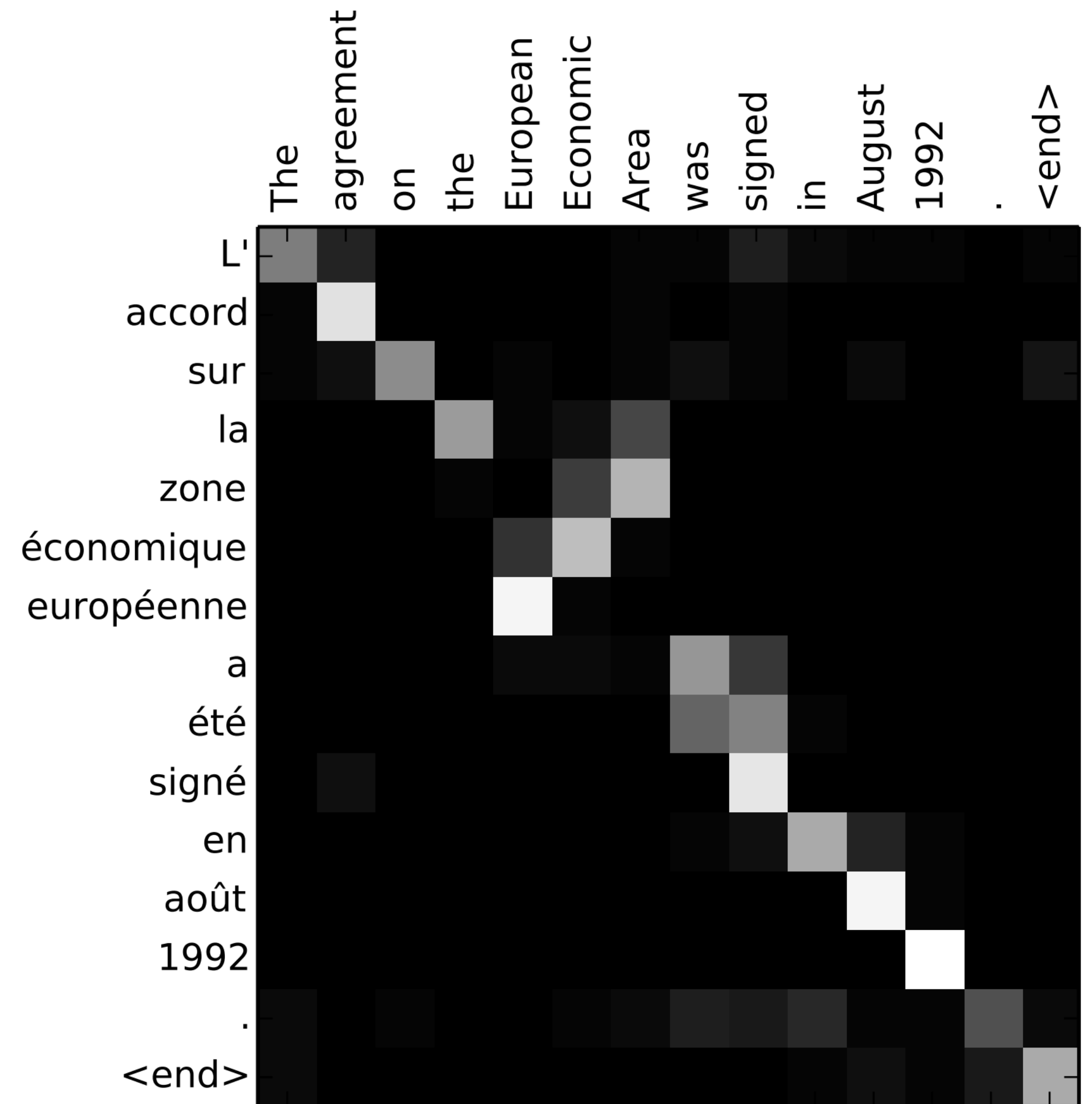
Attention



Bahdanau et al. (2014)

Attention

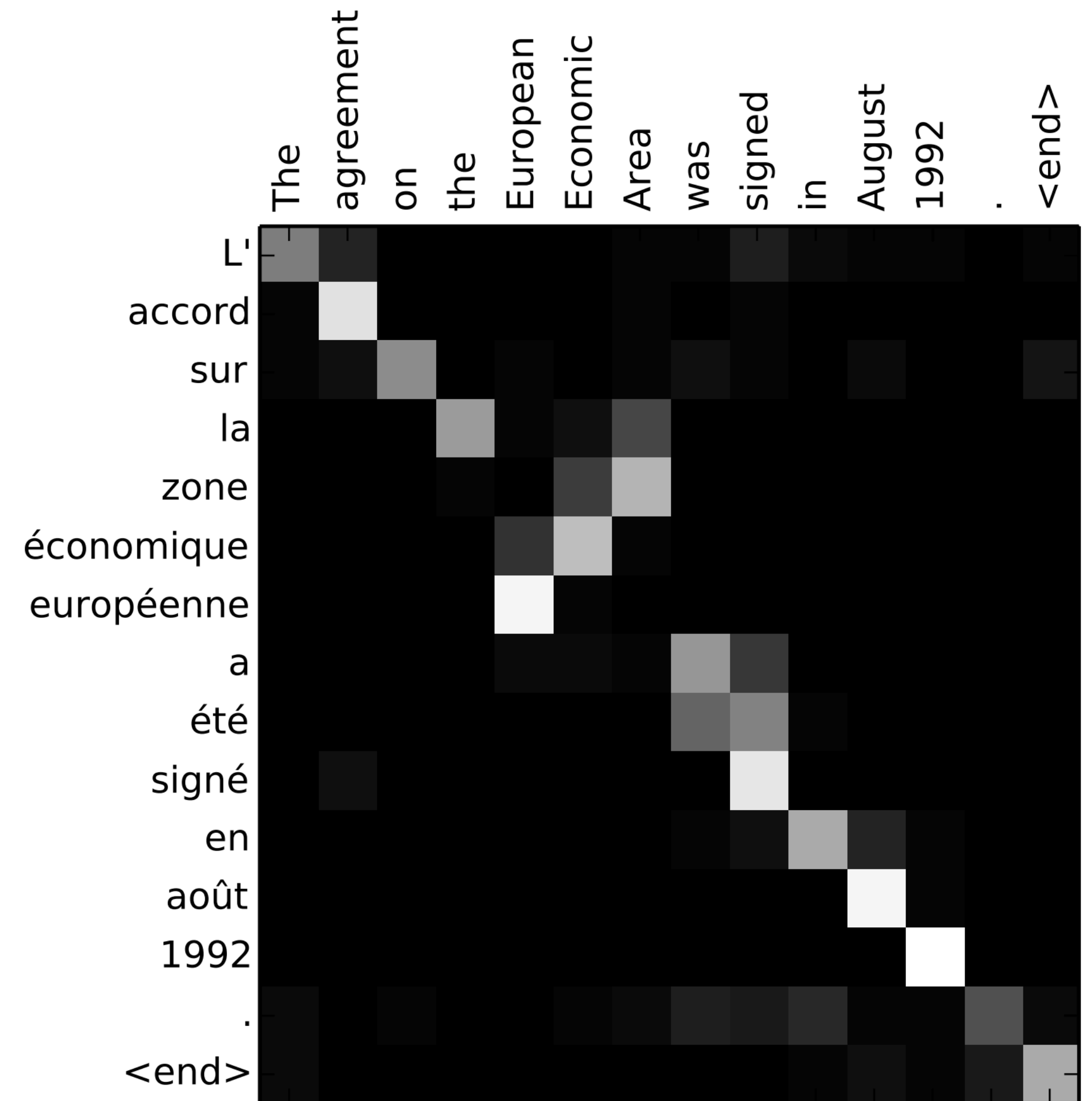
- ▶ Encoder hidden states capture contextual source word identity (“soft” word alignment)



Bahdanau et al. (2014)

Attention

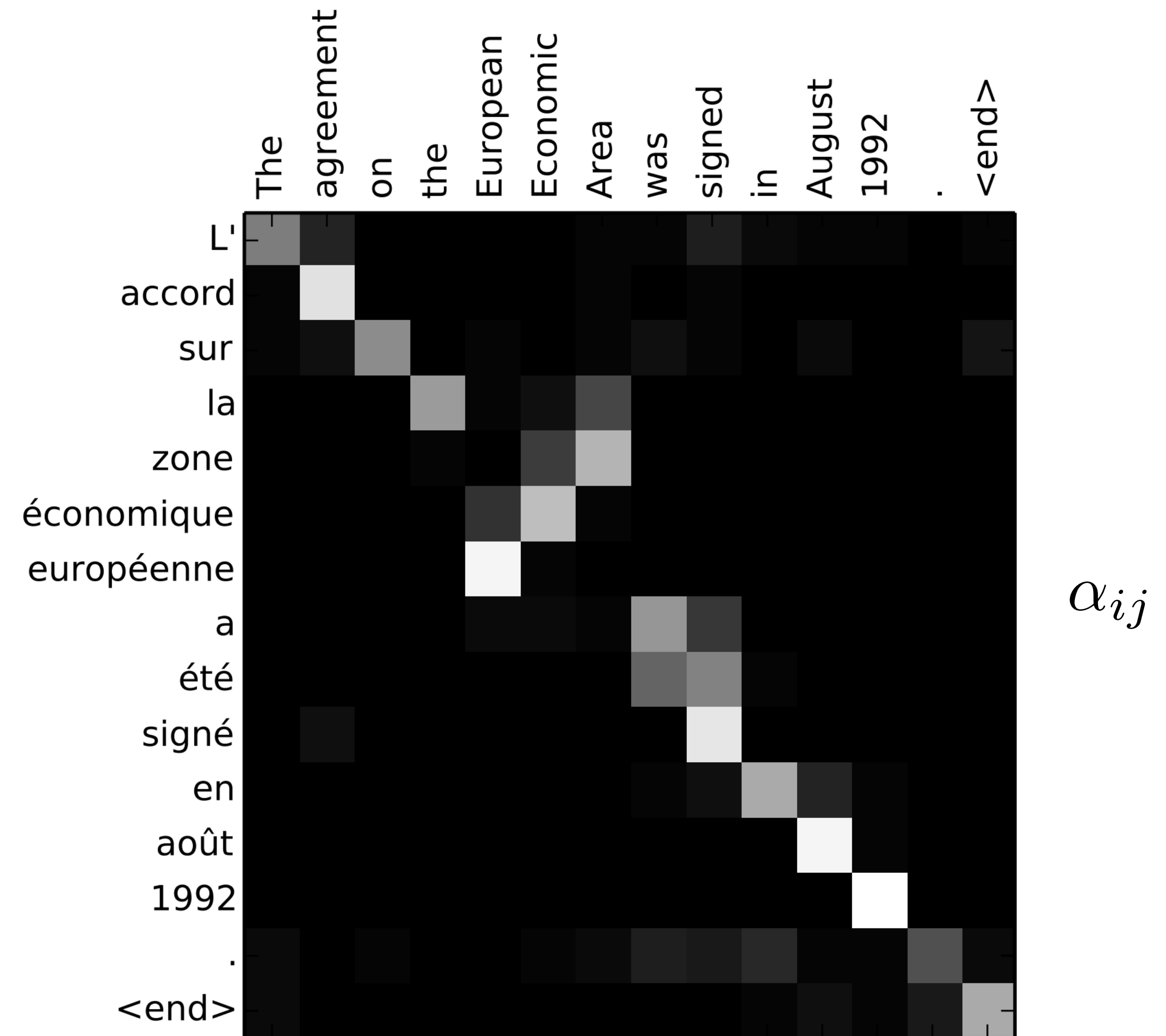
- ▶ Encoder hidden states capture contextual source word identity (“soft” word alignment)
- ▶ Decoder hidden states are now mostly responsible for selecting what to attend to



Bahdanau et al. (2014)

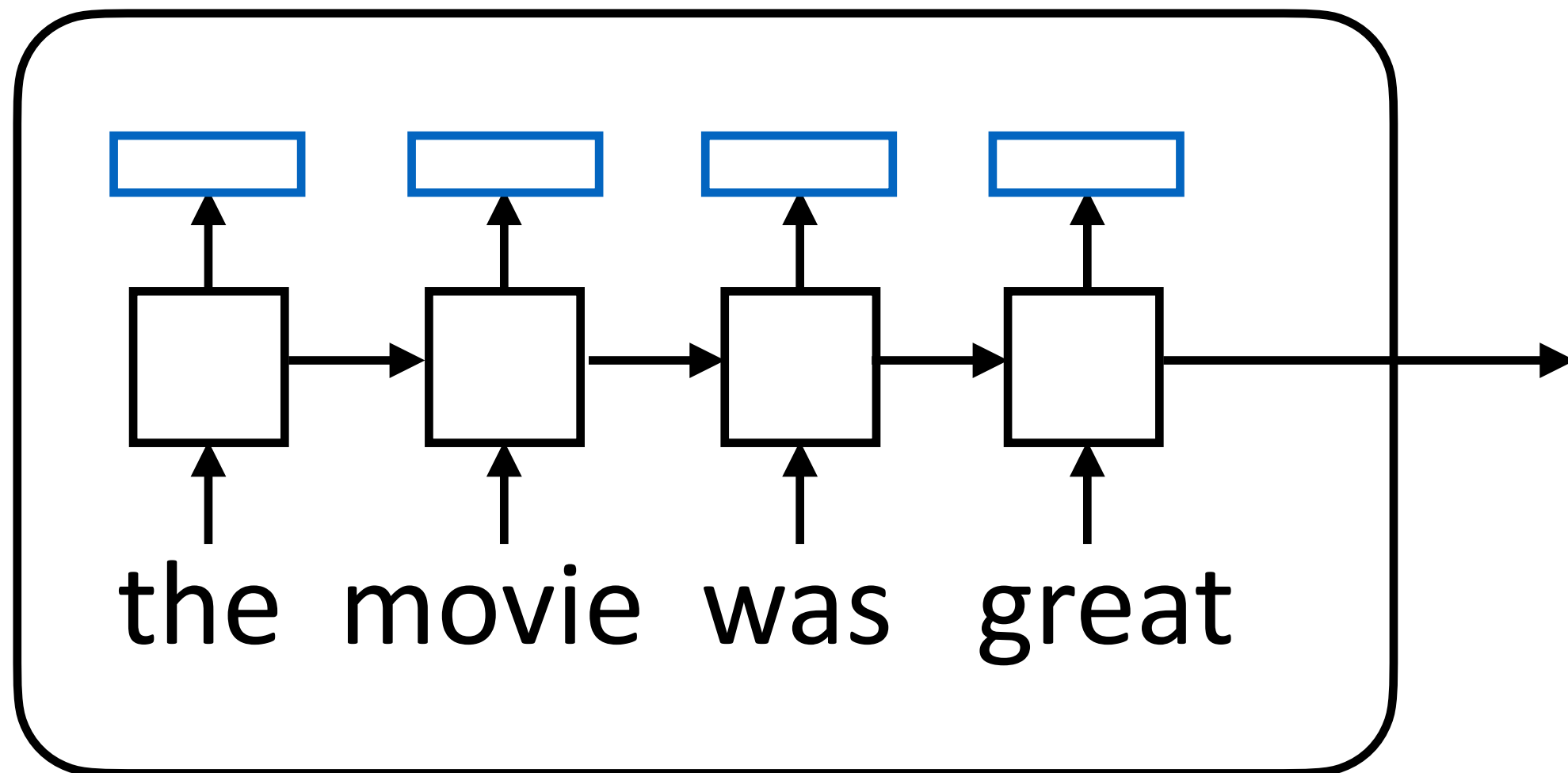
Attention

- ▶ Encoder hidden states capture contextual source word identity (“soft” word alignment)
- ▶ Decoder hidden states are now mostly responsible for selecting what to attend to
- ▶ Doesn’t take a complex hidden state to walk monotonically through a sentence and spit out word-by-word translations



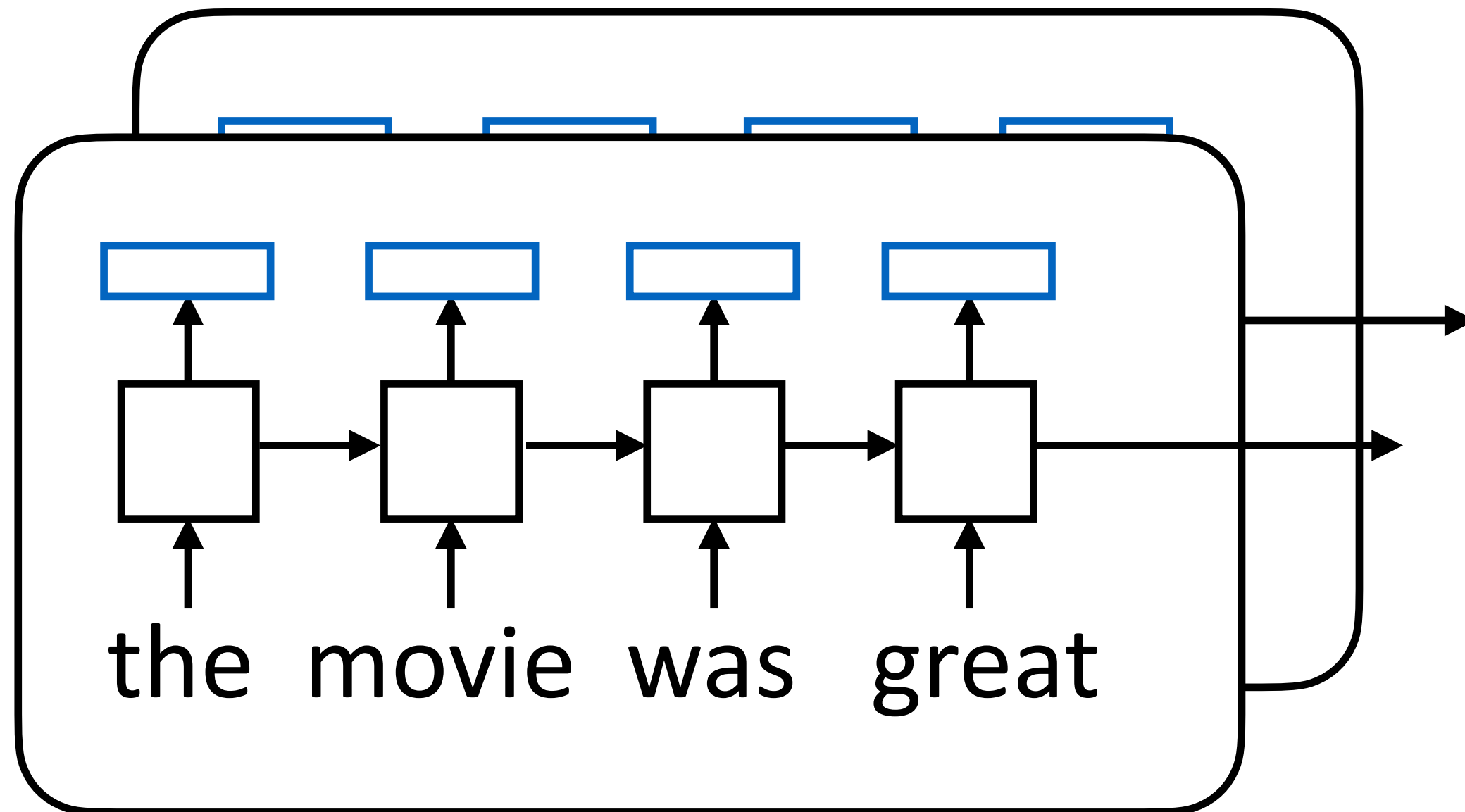
Bahdanau et al. (2014)

Batching Attention



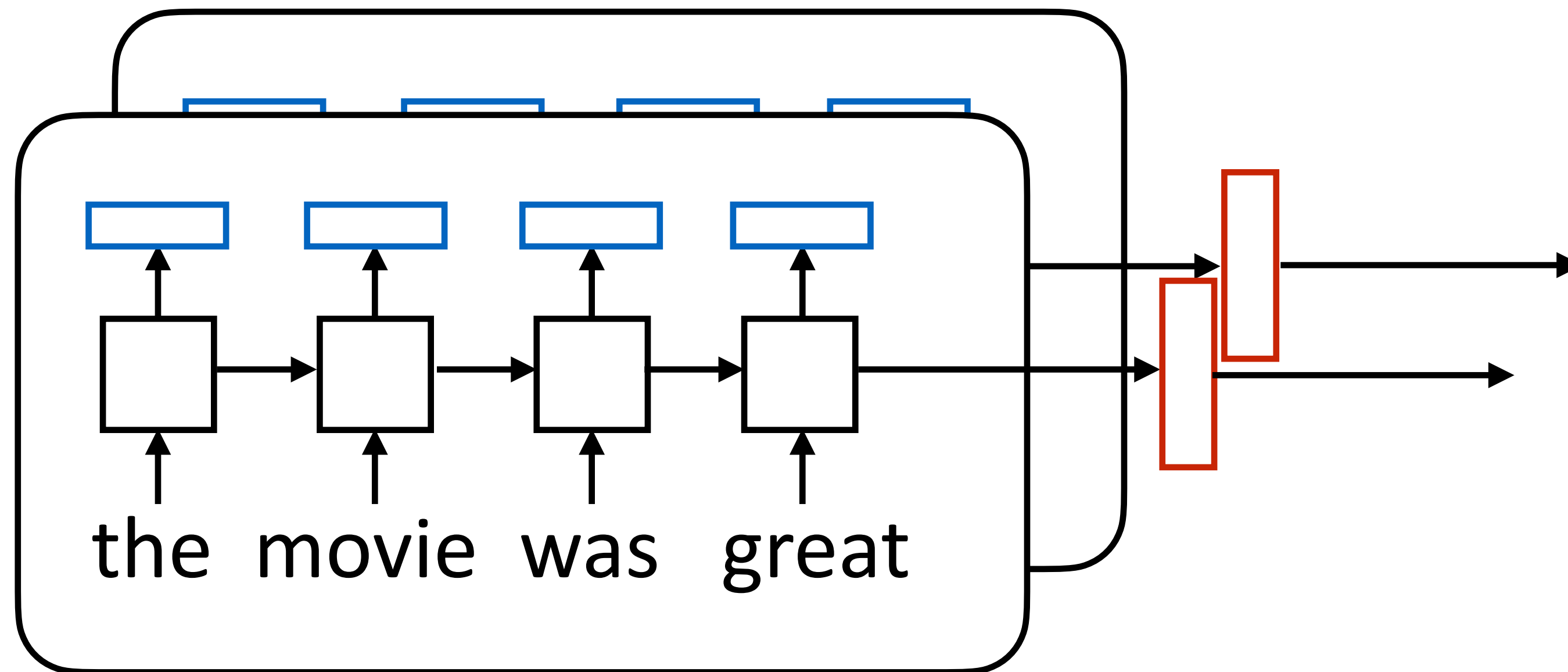
Batching Attention

token outputs: batch size x sentence length x hidden size



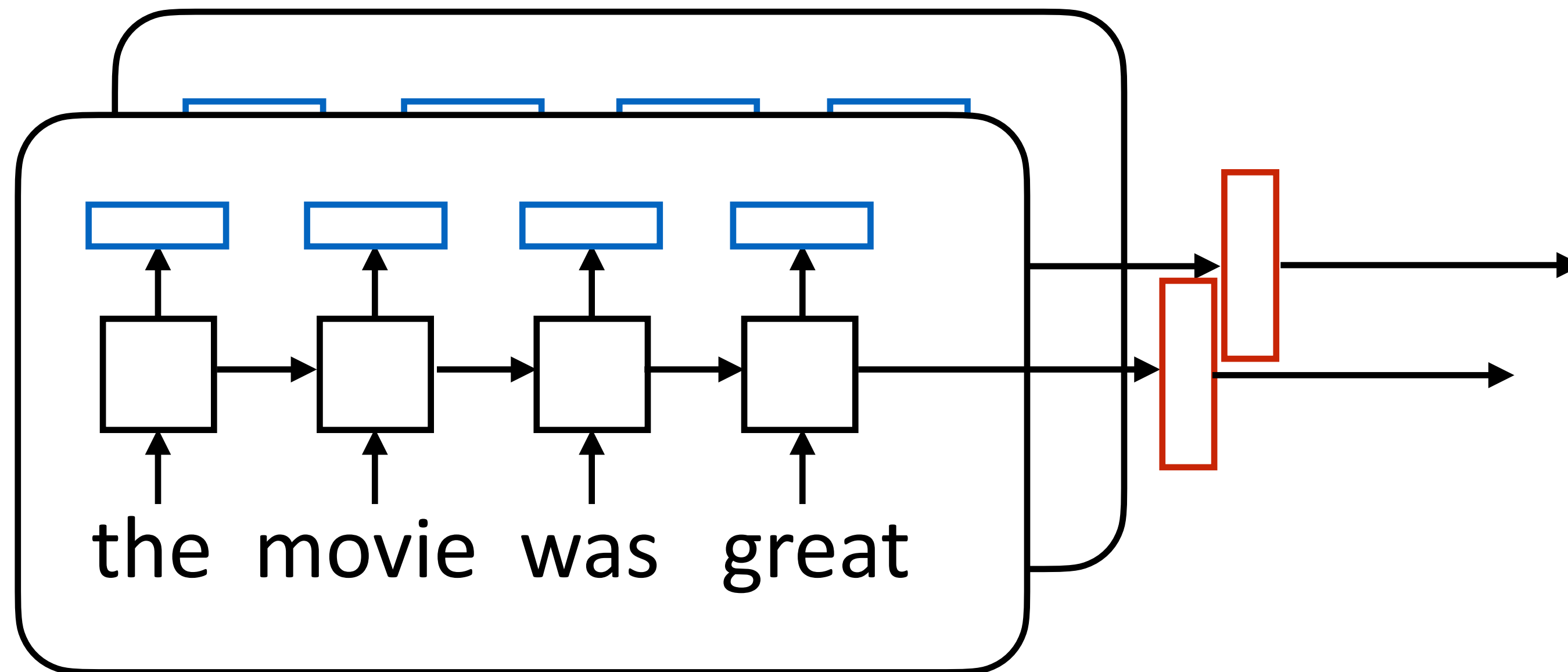
Batching Attention

token outputs: batch size x sentence length x hidden size



Batching Attention

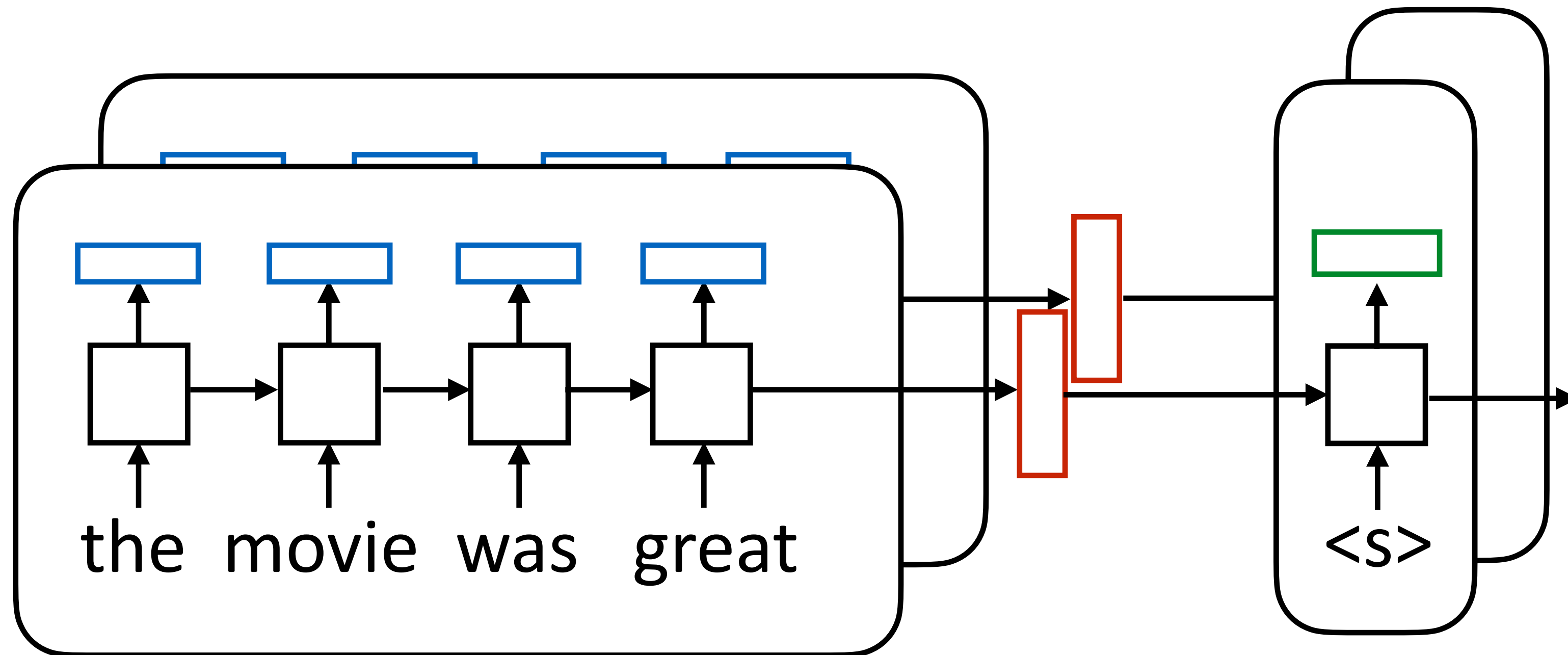
token outputs: batch size x sentence length x hidden size



sentence outputs:
batch size x hidden size

Batching Attention

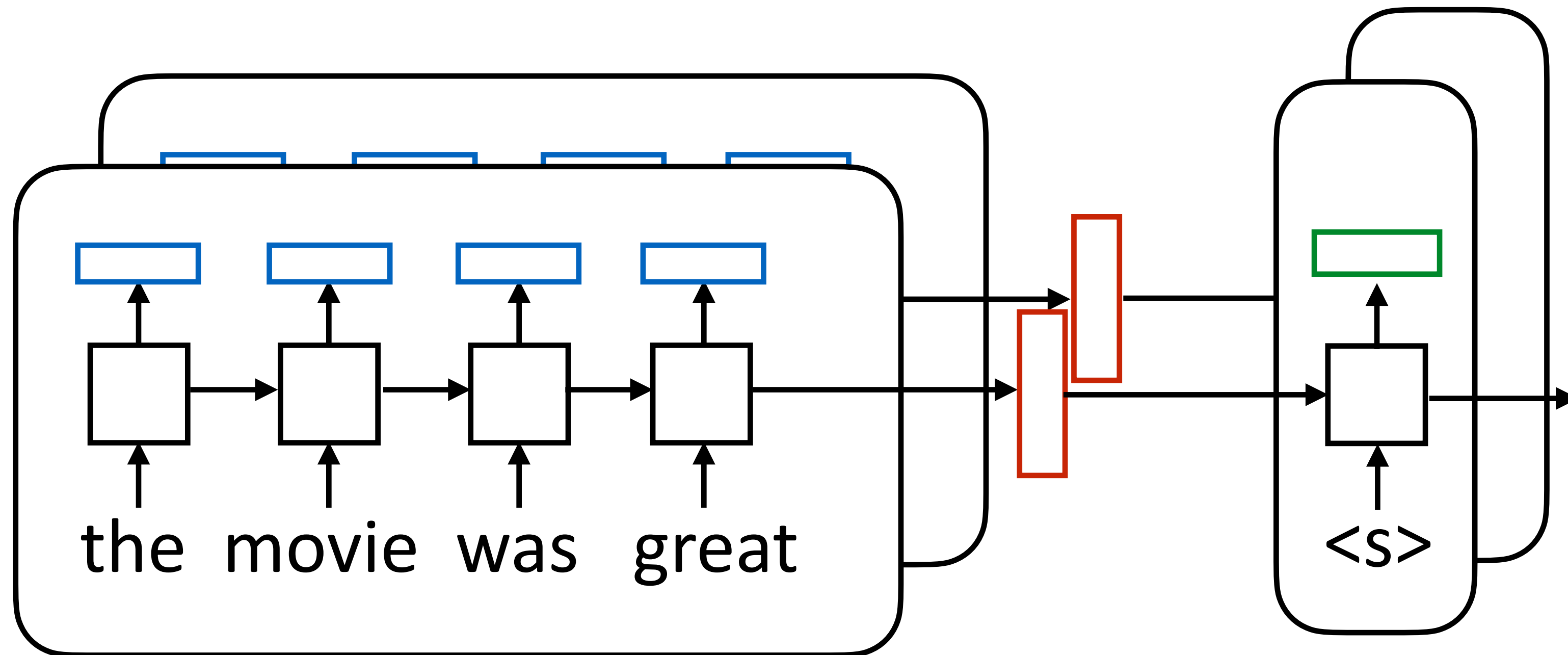
token outputs: batch size x sentence length x hidden size



sentence outputs:
batch size x hidden size

Batching Attention

token outputs: batch size x sentence length x hidden size

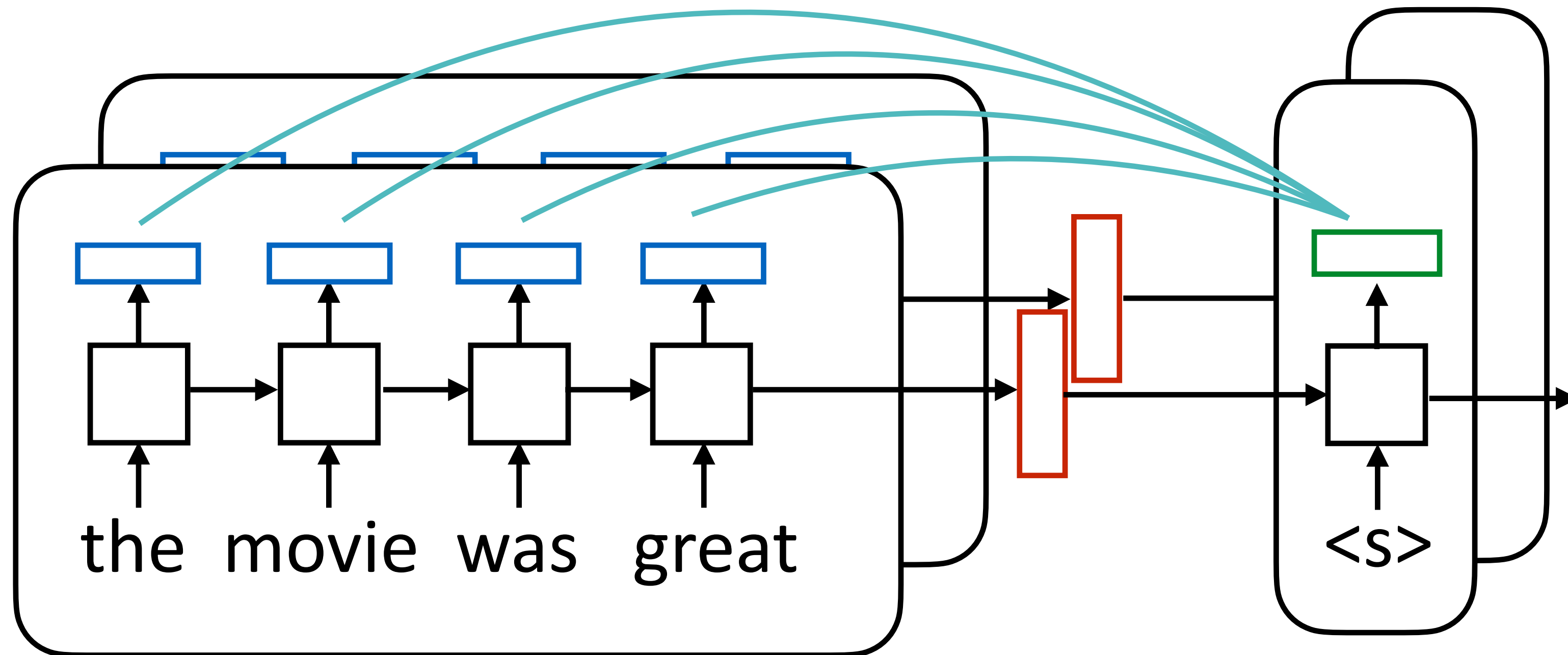


hidden state: batch size
x hidden size

sentence outputs:
batch size x hidden size

Batching Attention

token outputs: batch size x sentence length x hidden size

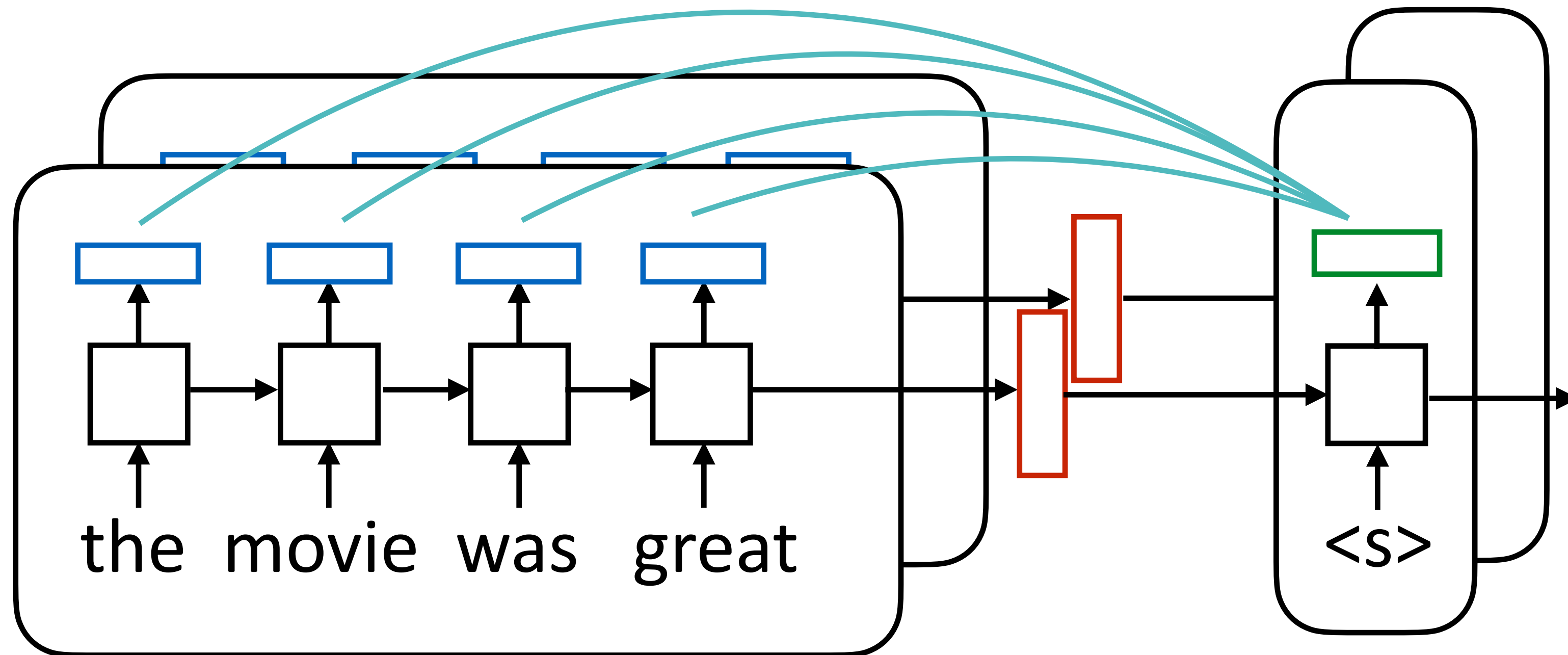


hidden state: batch size
x hidden size

sentence outputs:
batch size x hidden size

Batching Attention

token outputs: batch size x sentence length x hidden size



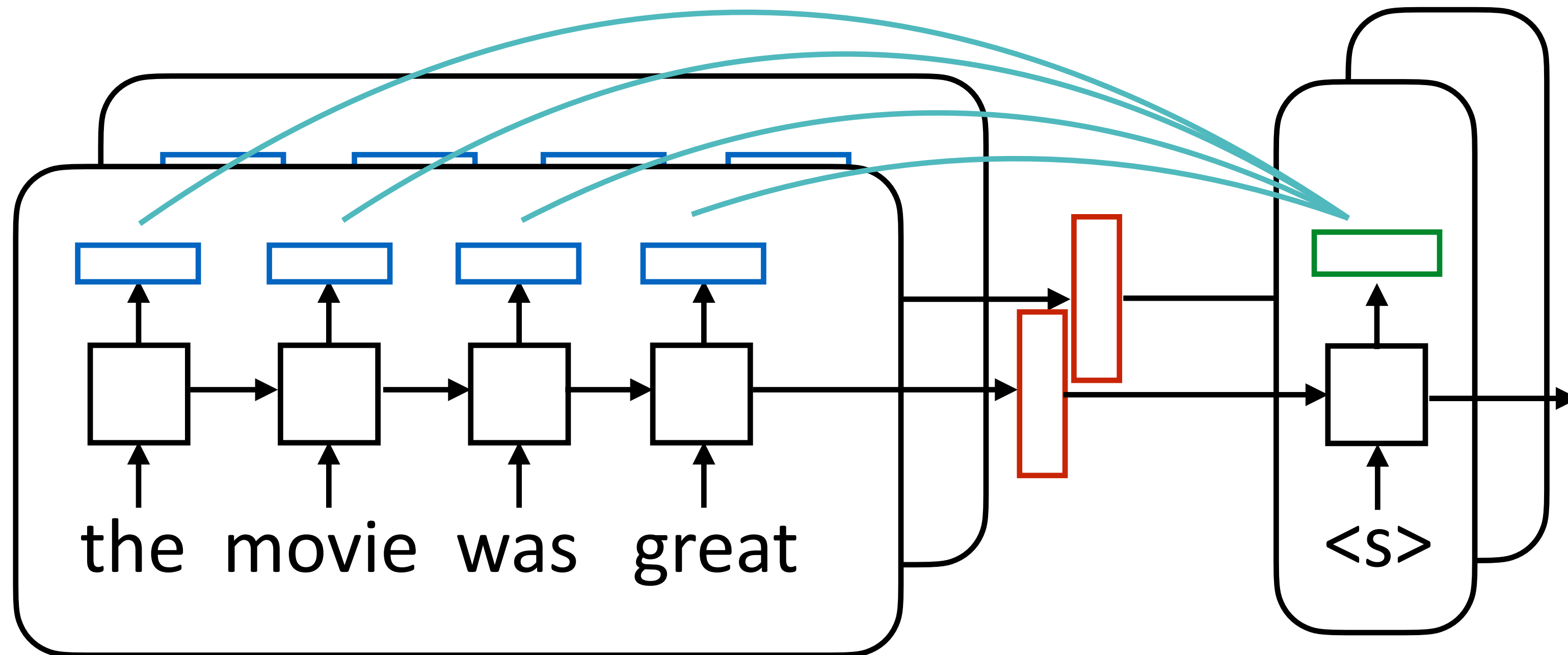
sentence outputs:
batch size x hidden size

hidden state: batch size
x hidden size

$$e_{ij} = f(\bar{h}_i, h_j)$$

Batching Attention

token outputs: batch size x sentence length x hidden size



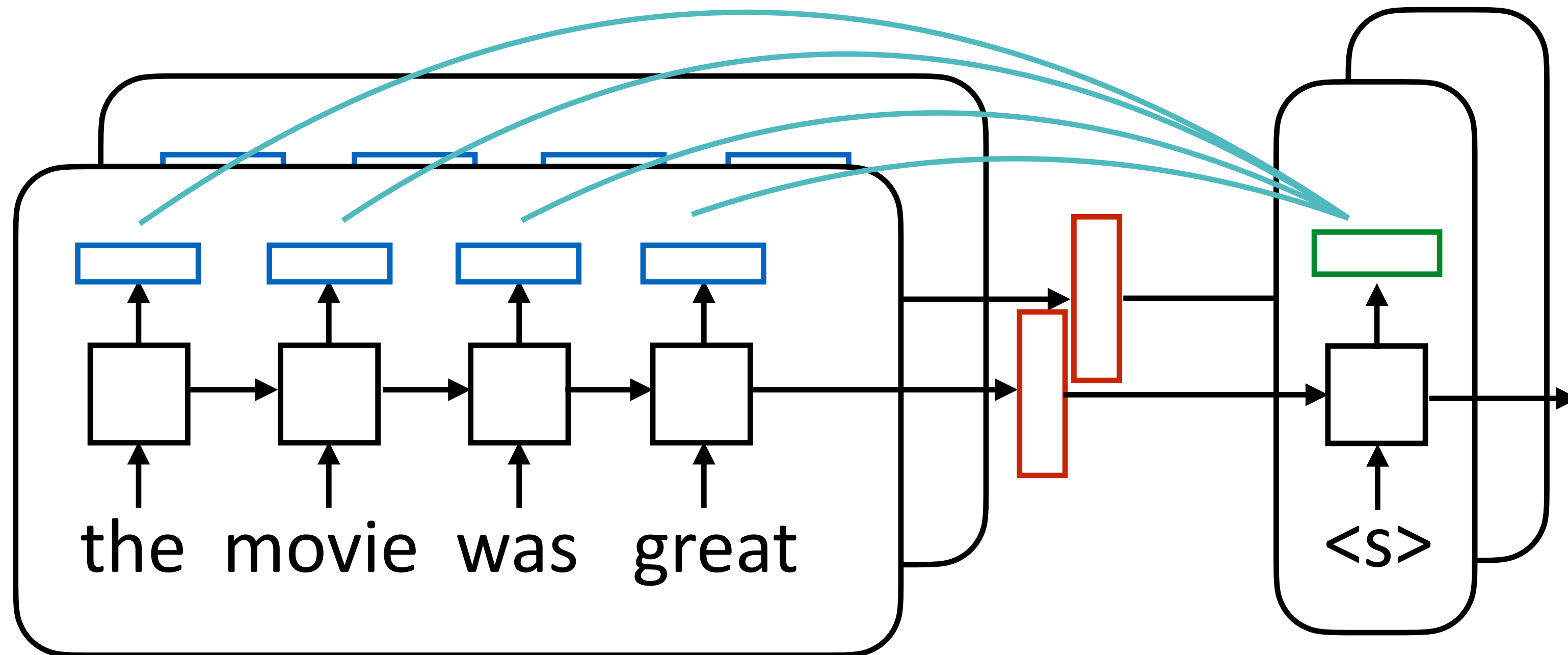
sentence outputs:
batch size x hidden size

hidden state: batch size
x hidden size

$$e_{ij} = f(\bar{h}_i, h_j)$$
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

Batching Attention

token outputs: batch size x sentence length x hidden size



sentence outputs:
batch size x hidden size

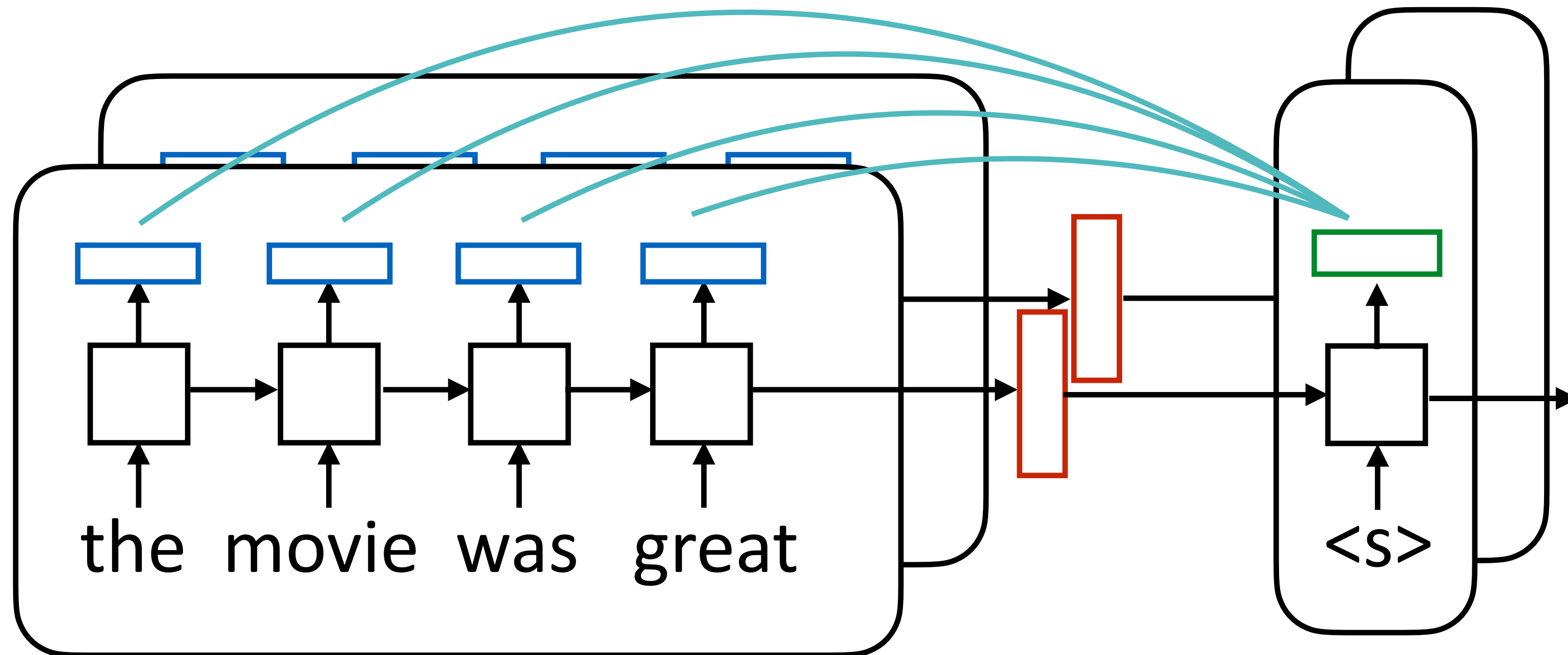
hidden state: batch size
x hidden size

$$e_{ij} = f(\bar{h}_i, h_j)$$
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

attention scores = batch size x sentence length

Batching Attention

token outputs: batch size x sentence length x hidden size



sentence outputs:
batch size x hidden size

hidden state: batch size
x hidden size

$$e_{ij} = f(\bar{h}_i, h_j)$$
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

attention scores = batch size x sentence length

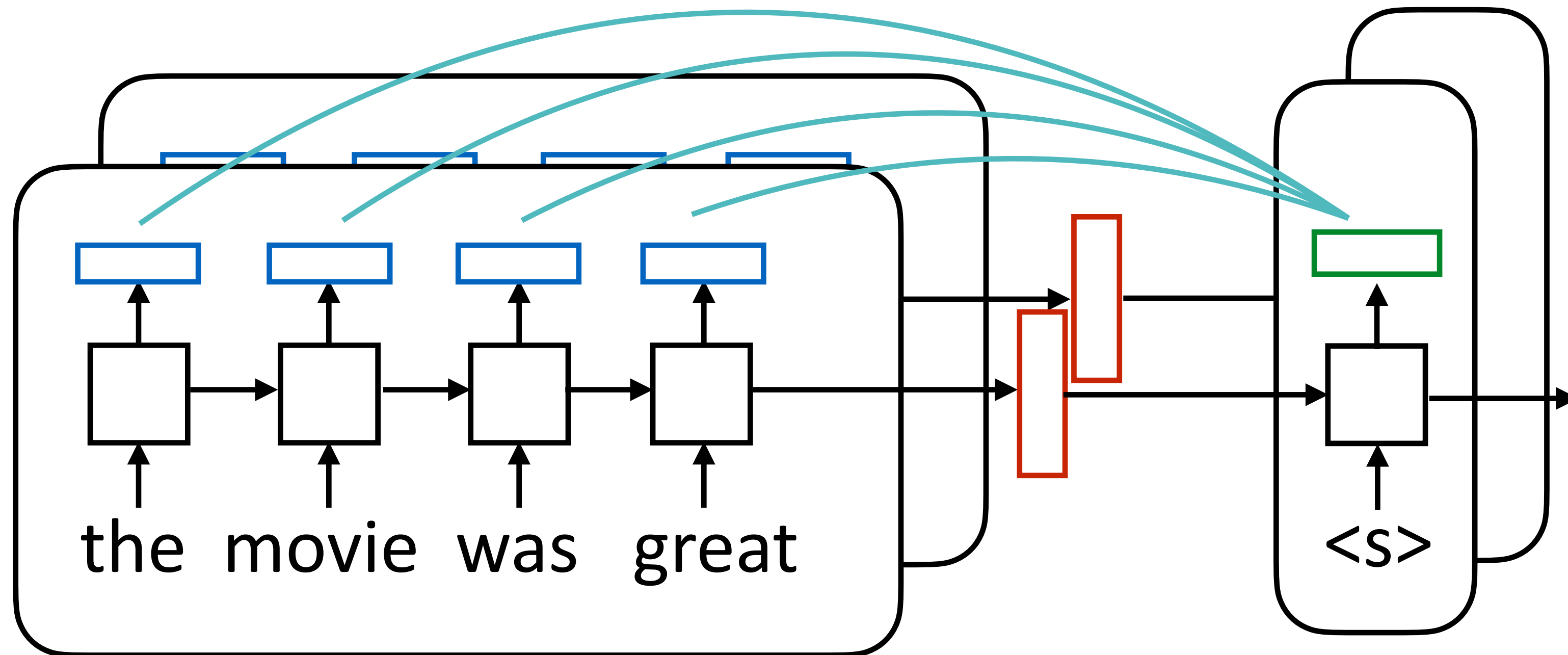
c = batch size x hidden size

$$c_i = \sum_j \alpha_{ij} h_j$$

Luong et al. (2015)

Batching Attention

token outputs: batch size x sentence length x hidden size



sentence outputs:
batch size x hidden size

hidden state: batch size
x hidden size

$$e_{ij} = f(\bar{h}_i, h_j)$$
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$$

attention scores = batch size x sentence length

c = batch size x hidden size

$$c_i = \sum_j \alpha_{ij} h_j$$

- Make sure tensors are the right size!

Luong et al. (2015)

Encoder-Decoder MT Results

- ▶ Sutskever seq2seq paper: first major application of LSTMs to NLP

Encoder-Decoder MT Results

- ▶ Sutskever seq2seq paper: first major application of LSTMs to NLP
- ▶ Basic encoder-decoder with beam search

Encoder-Decoder MT Results

- ▶ Sutskever seq2seq paper: first major application of LSTMs to NLP
- ▶ Basic encoder-decoder with beam search

Method	test BLEU score (ntst14)
Bahdanau et al. [2]	28.45
Baseline System [29]	33.30
Single forward LSTM, beam size 12	26.17
Single reversed LSTM, beam size 12	30.59
Ensemble of 5 reversed LSTMs, beam size 12	34.81

Encoder-Decoder MT Results

- ▶ Sutskever seq2seq paper: first major application of LSTMs to NLP
- ▶ Basic encoder-decoder with beam search

Method	test BLEU score (ntst14)
Bahdanau et al. [2]	28.45
Baseline System [29]	33.30
Single forward LSTM, beam size 12	26.17
Single reversed LSTM, beam size 12	30.59
Ensemble of 5 reversed LSTMs, beam size 12	34.81

- ▶ SOTA = 37.0 — not all that competitive...

Sutskever et al. (2014)

Results: WMT English-German

- ▶ 4.5M sentence pairs

Classic phrase-based system: **20.7** BLEU

Luong+ (2014) seq2seq: **14** BLEU

Luong+ (2015) seq2seq ensemble w/ attention, rare word handling: **23.0** BLEU

- ▶ Not nearly as good in absolute BLEU, but BLEU is not really comparable across languages

Results: WMT English-German

- ▶ 4.5M sentence pairs

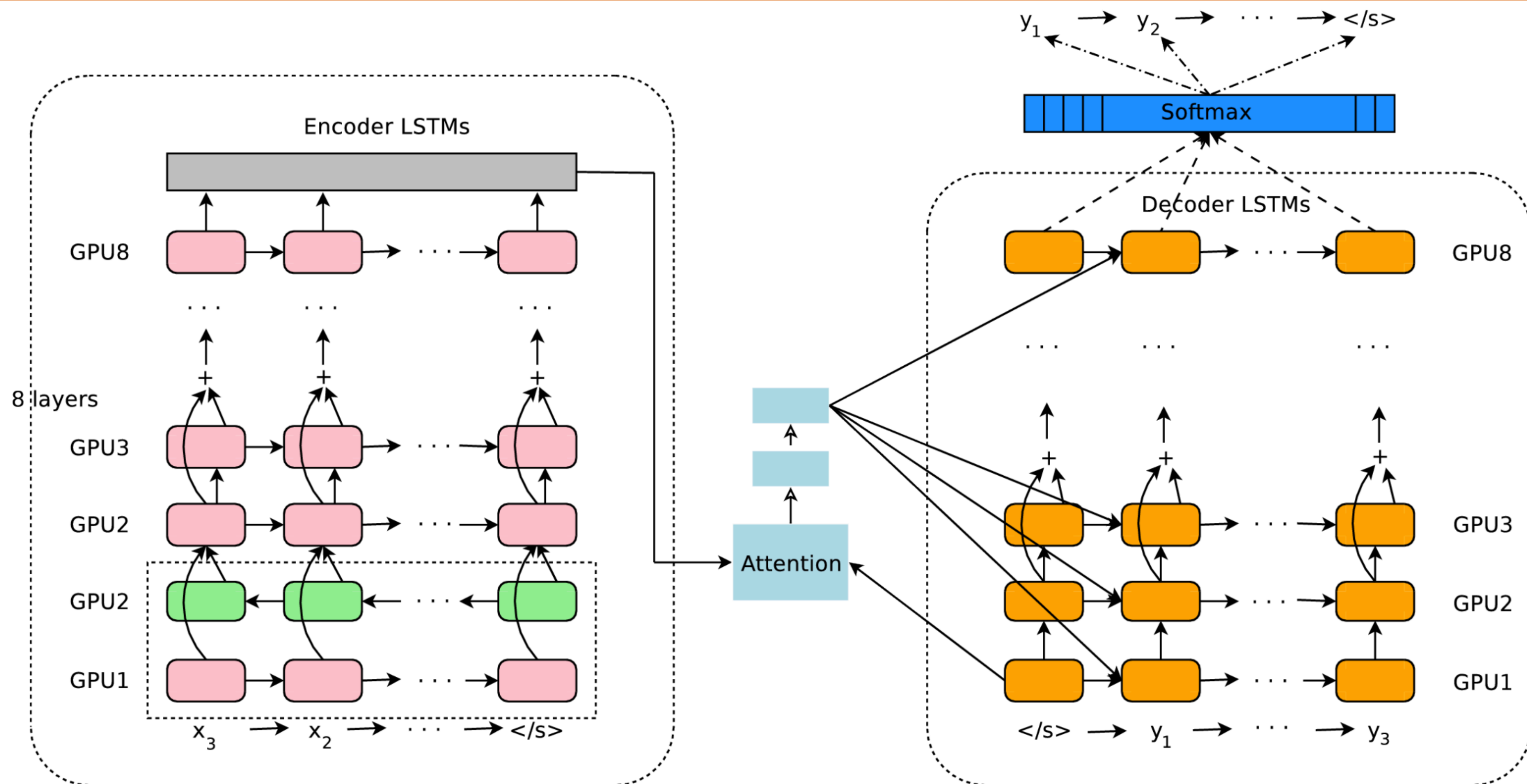
Classic phrase-based system: **20.7** BLEU

Luong+ (2014) seq2seq: **14** BLEU

Luong+ (2015) seq2seq ensemble w/ attention, rare word handling: **23.0** BLEU

- ▶ Not nearly as good in absolute BLEU, but BLEU is not really comparable across languages
- ▶ French, Spanish = easiest
German, Czech = harder
Japanese, Russian = hard (grammatically different, lots of morphology...)

Google's NMT System



- ▶ 8-layer LSTM encoder-decoder with attention
- ▶ word piece (token) vocabulary of 8k-32k

Generation Tasks

- ▶ There are a range of tasks that seq2seq models can be used for
- ▶ which often call for different decoding strategies

Less constrained

More constrained



Unconditioned sampling/
“story generation”

Dialogue

Translation

Text-to-code

Summarization

Data-to-text

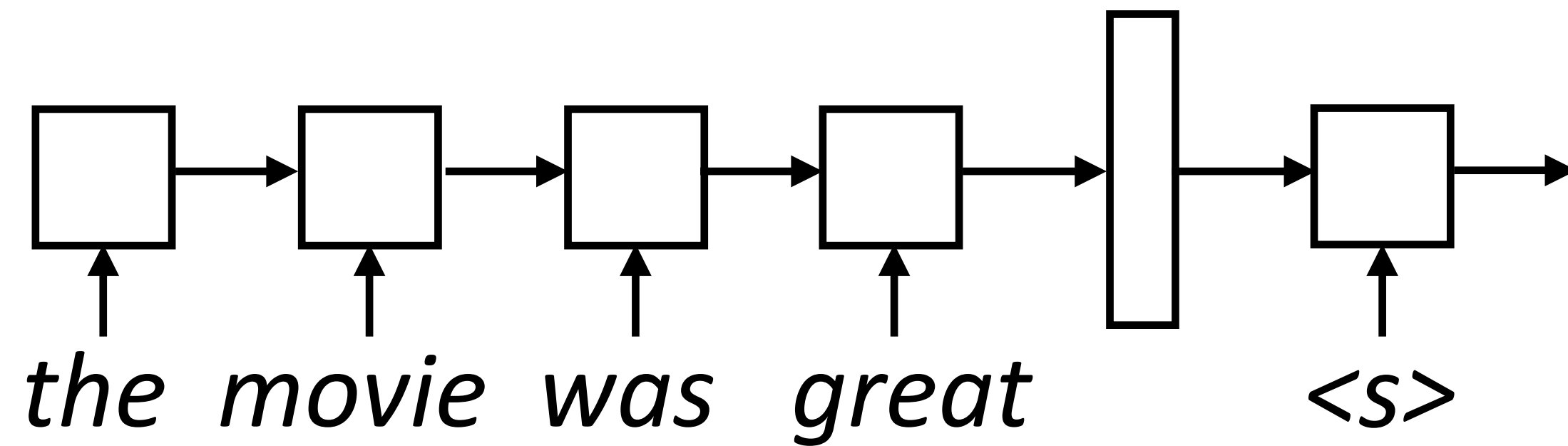
Decoding Strategies

Decoding Strategies

- ▶ LMs place a distribution $P(y_i | y_1, \dots, y_{i-1})$
- ▶ seq2seq models place a distribution $P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$
- ▶ Generation from both models looks similar; how do we do it?

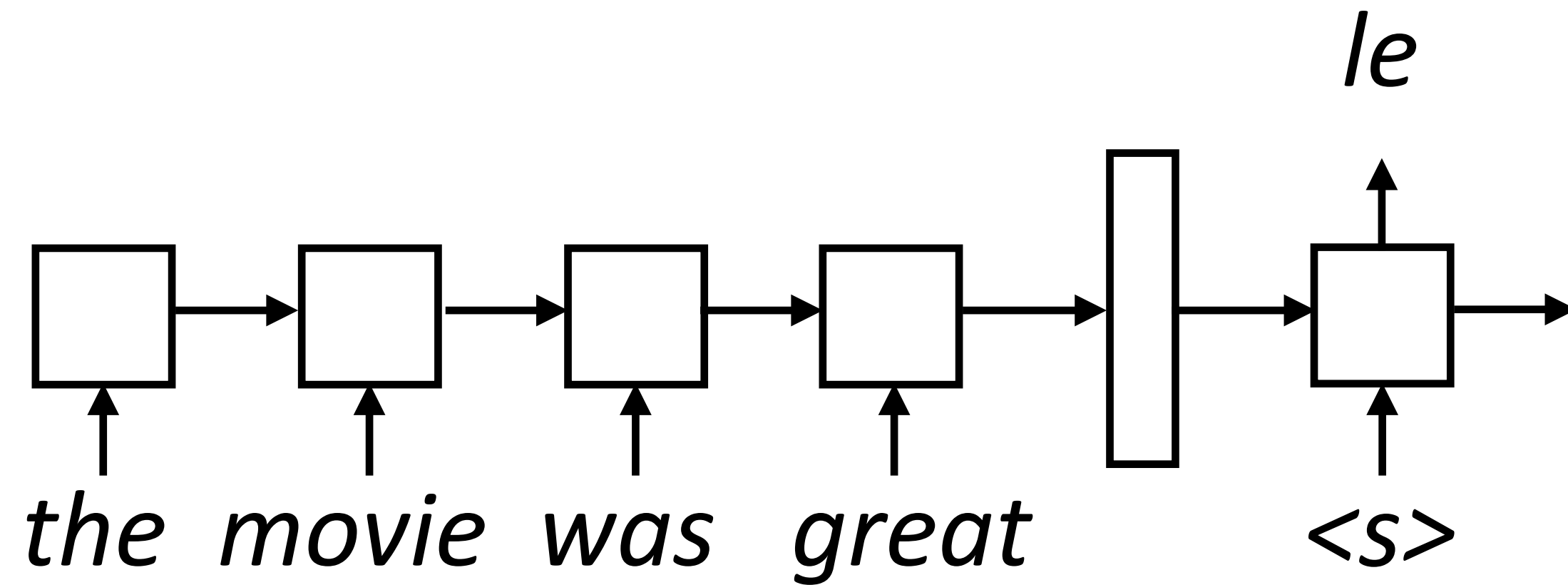
Greedy Decoding

- ▶ Generate next word conditioned on previous word as well as hidden state



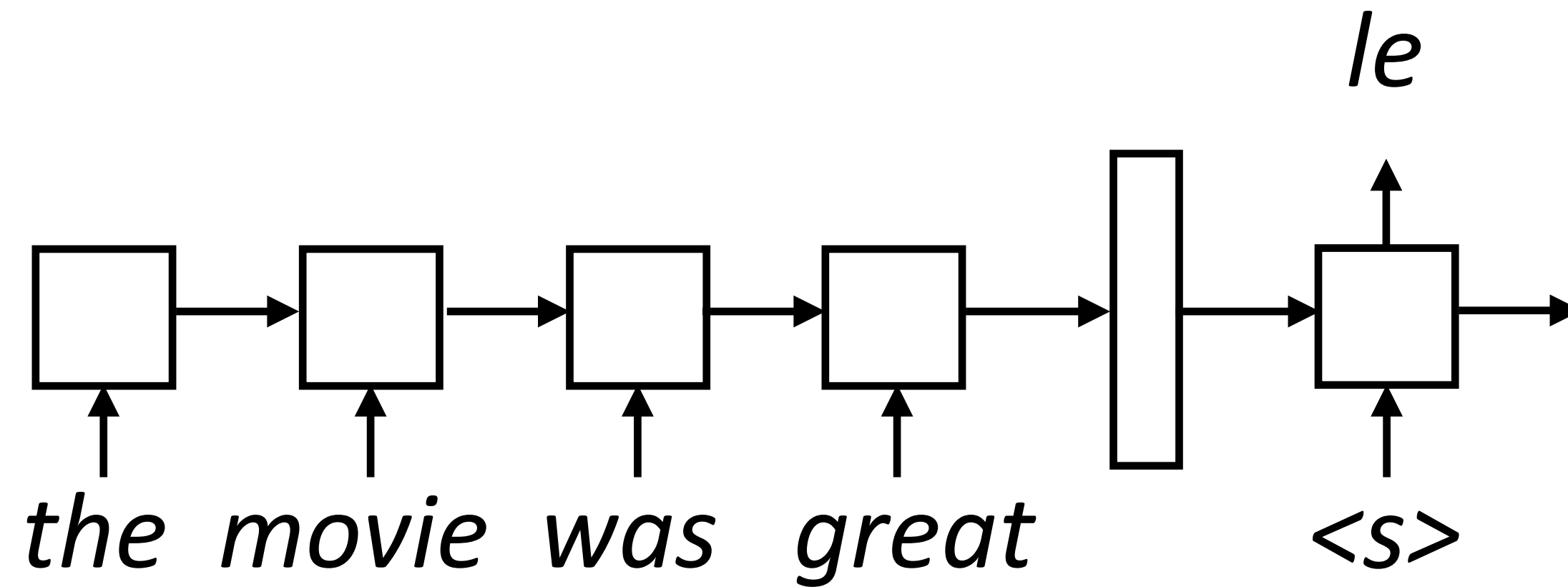
Greedy Decoding

- ▶ Generate next word conditioned on previous word as well as hidden state



Greedy Decoding

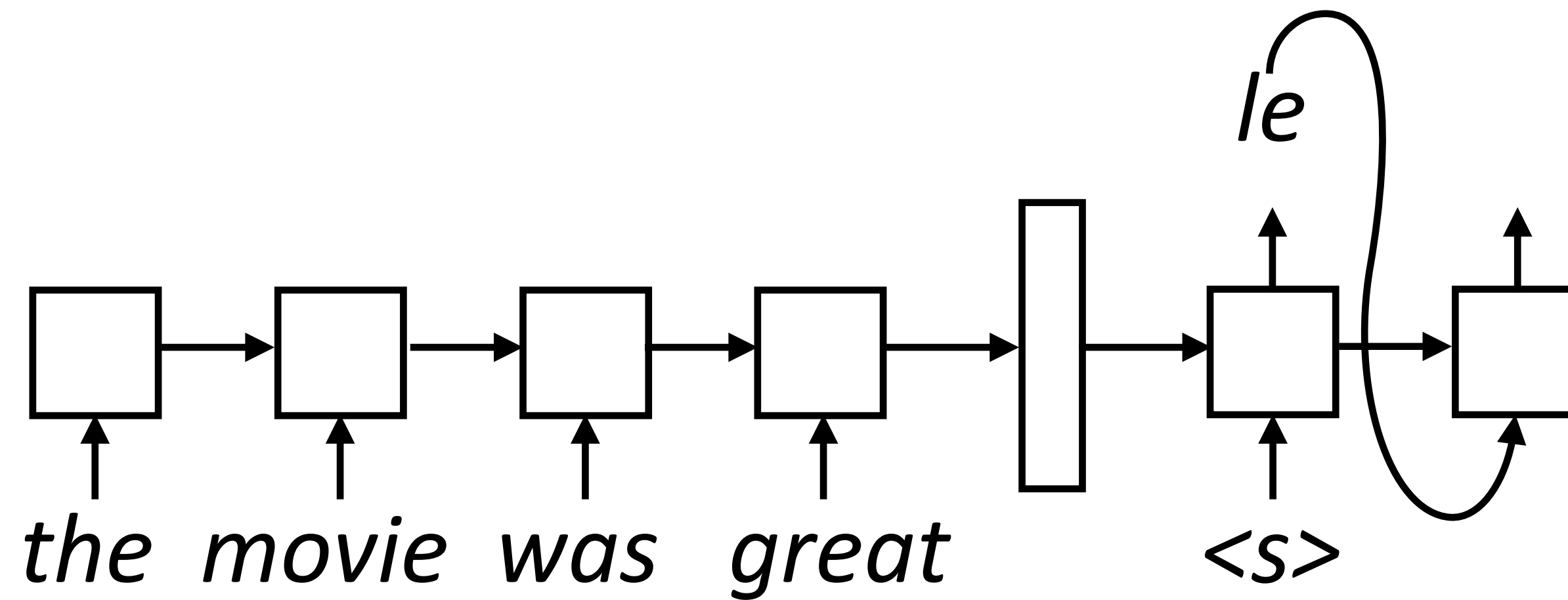
- ▶ Generate next word conditioned on previous word as well as hidden state



- ▶ During inference: need to compute the argmax over the word predictions and then feed that to the next RNN state. This is **greedy decoding**

Greedy Decoding

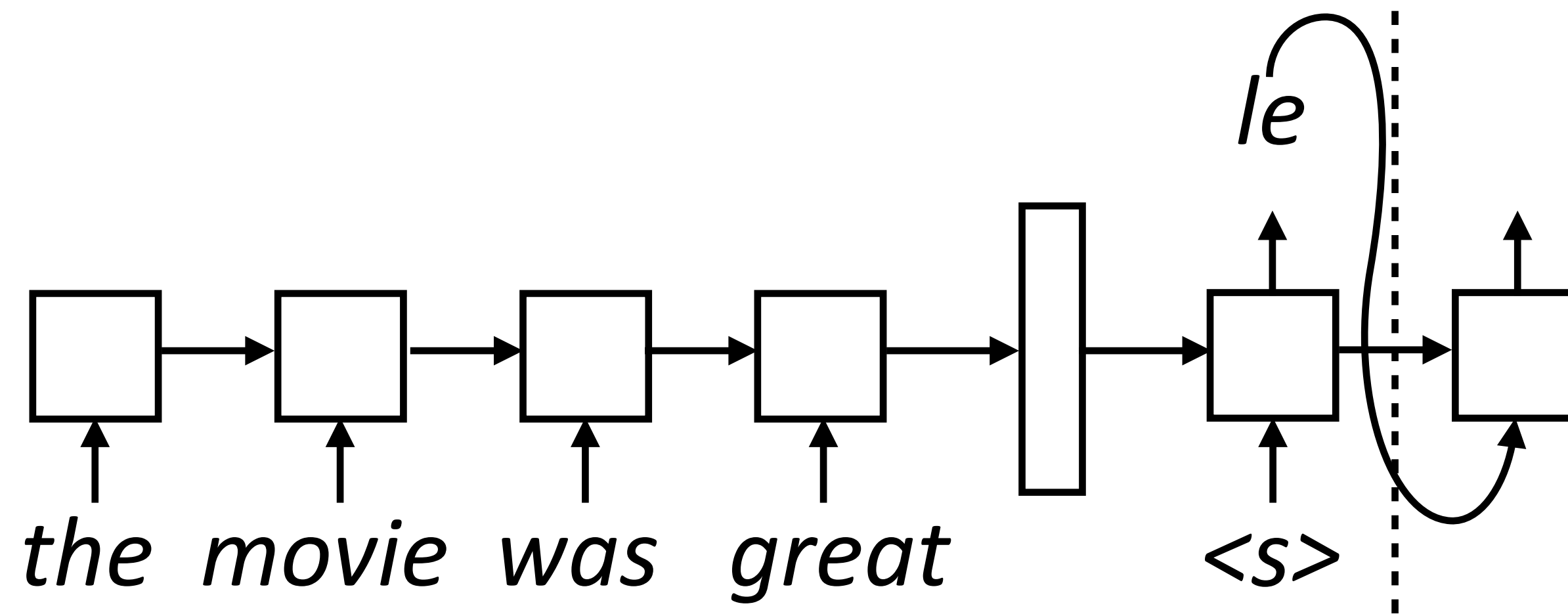
- ▶ Generate next word conditioned on previous word as well as hidden state



- ▶ During inference: need to compute the argmax over the word predictions and then feed that to the next RNN state. This is **greedy decoding**

Greedy Decoding

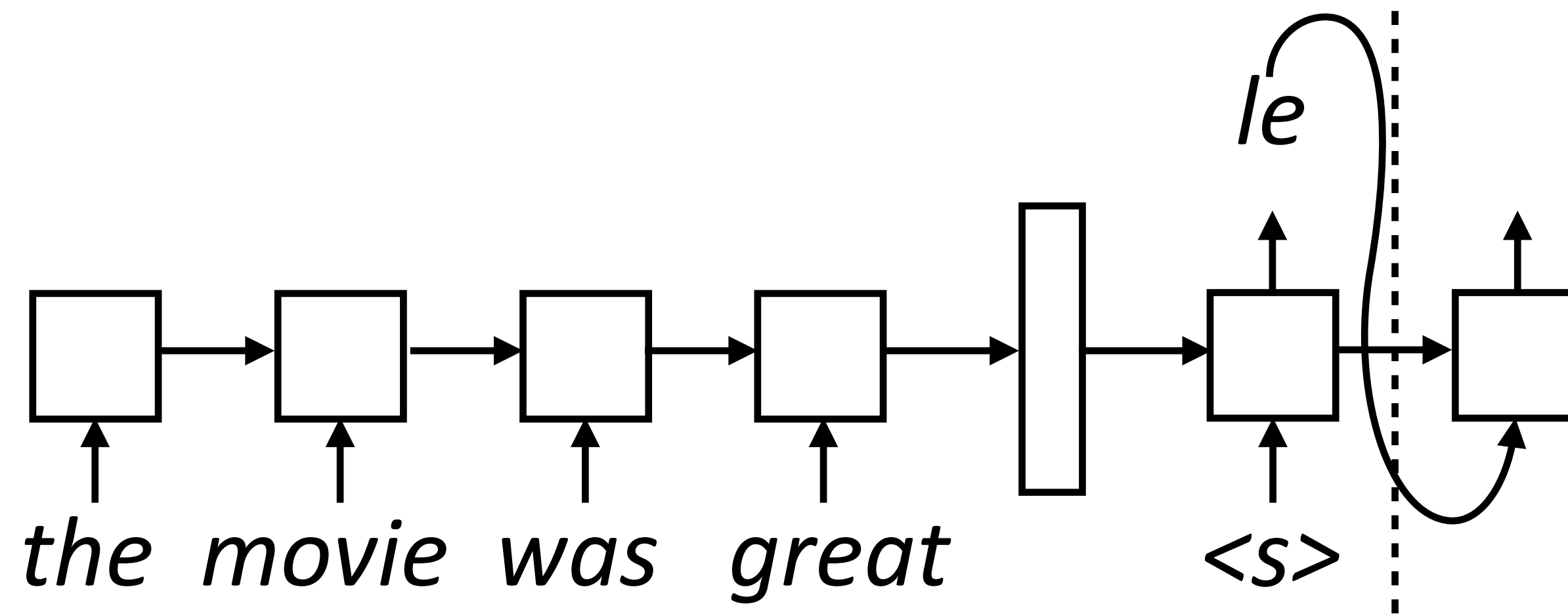
- ▶ Generate next word conditioned on previous word as well as hidden state



- ▶ During inference: need to compute the argmax over the word predictions and then feed that to the next RNN state. This is **greedy decoding**

Greedy Decoding

- ▶ Generate next word conditioned on previous word as well as hidden state



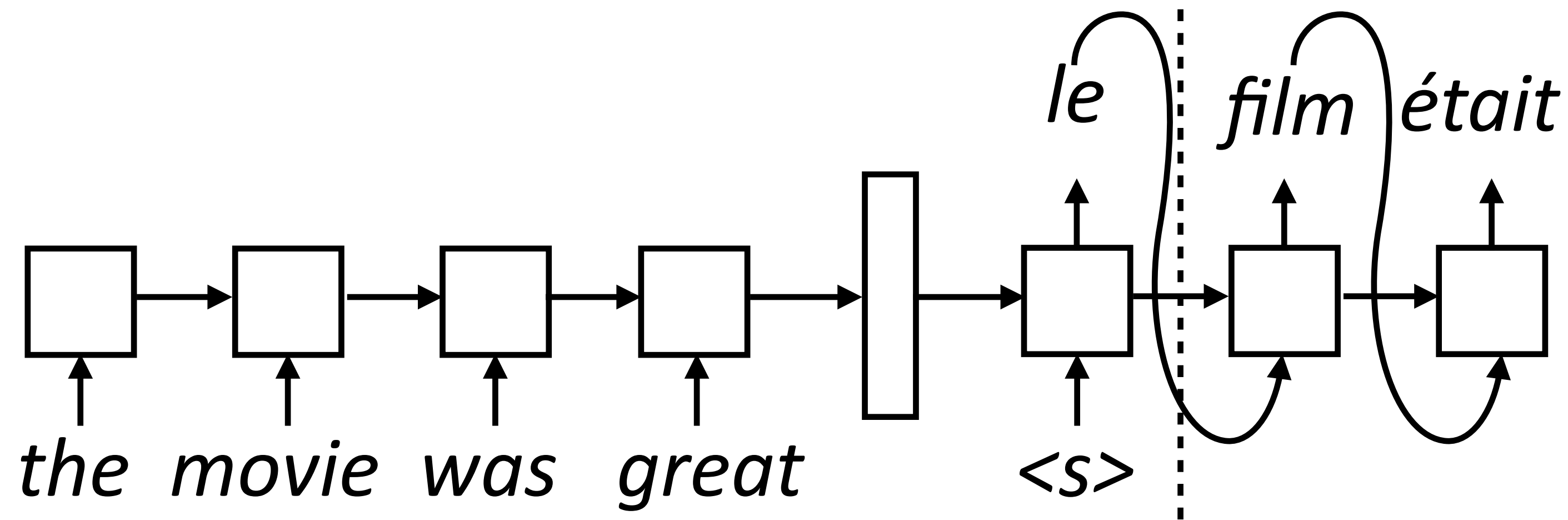
- ▶ During inference: need to compute the argmax over the word predictions and then feed that to the next RNN state. This is **greedy decoding**

$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h})$$

$$y_{\text{pred}} = \text{argmax}_y P(y | \mathbf{x}, y_1, \dots, y_{i-1})$$

Greedy Decoding

- ▶ Generate next word conditioned on previous word as well as hidden state



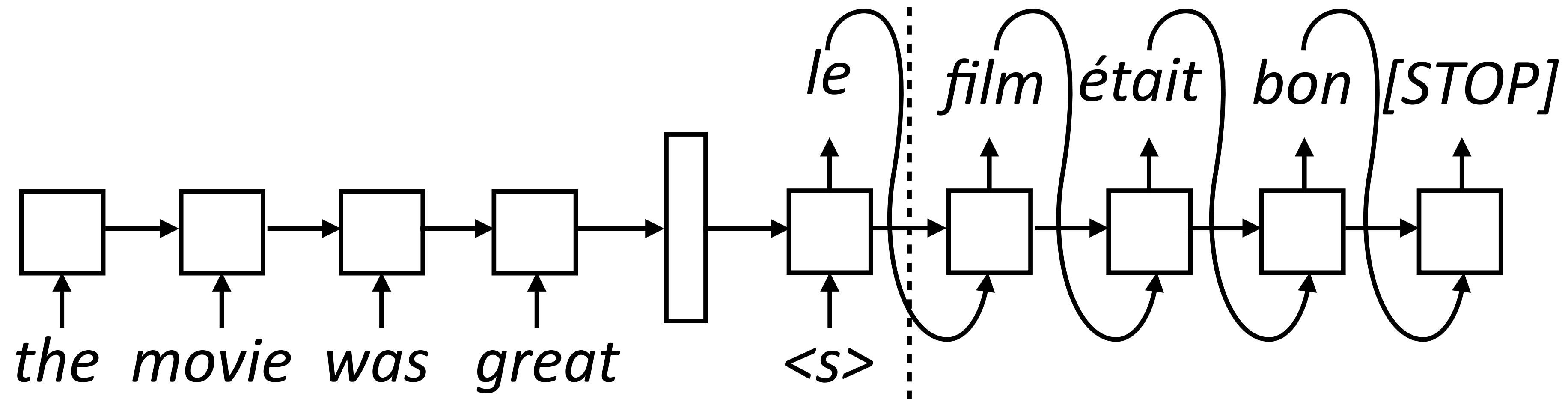
- ▶ During inference: need to compute the argmax over the word predictions and then feed that to the next RNN state. This is **greedy decoding**

$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h})$$

$$y_{\text{pred}} = \text{argmax}_y P(y | \mathbf{x}, y_1, \dots, y_{i-1})$$

Greedy Decoding

- ▶ Generate next word conditioned on previous word as well as hidden state



- ▶ During inference: need to compute the argmax over the word predictions and then feed that to the next RNN state. This is **greedy decoding**

$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W \bar{h})$$

$$y_{\text{pred}} = \text{argmax}_y P(y | \mathbf{x}, y_1, \dots, y_{i-1})$$

Problems with Greedy Decoding

- ▶ Only returns one solution, and it may not be optimal

Problems with Greedy Decoding

- ▶ Only returns one solution, and it may not be optimal
- ▶ Can address this with **beam search**, which usually works better...but even beam search may not find the correct answer! (max probability sequence)

Problems with Greedy Decoding

- ▶ Only returns one solution, and it may not be optimal
- ▶ Can address this with **beam search**, which usually works better...but even beam search may not find the correct answer! (max probability sequence)

Model	Beam-10	
	BLEU	#Search err.
LSTM*	28.6	58.4%
SliceNet*	28.8	46.0%
Transformer-Base	30.3	57.7%
Transformer-Big*	31.7	32.1%

Problems with Greedy Decoding

- ▶ Only returns one solution, and it may not be optimal
- ▶ Can address this with **beam search**, which usually works better...but even beam search may not find the correct answer! (max probability sequence)

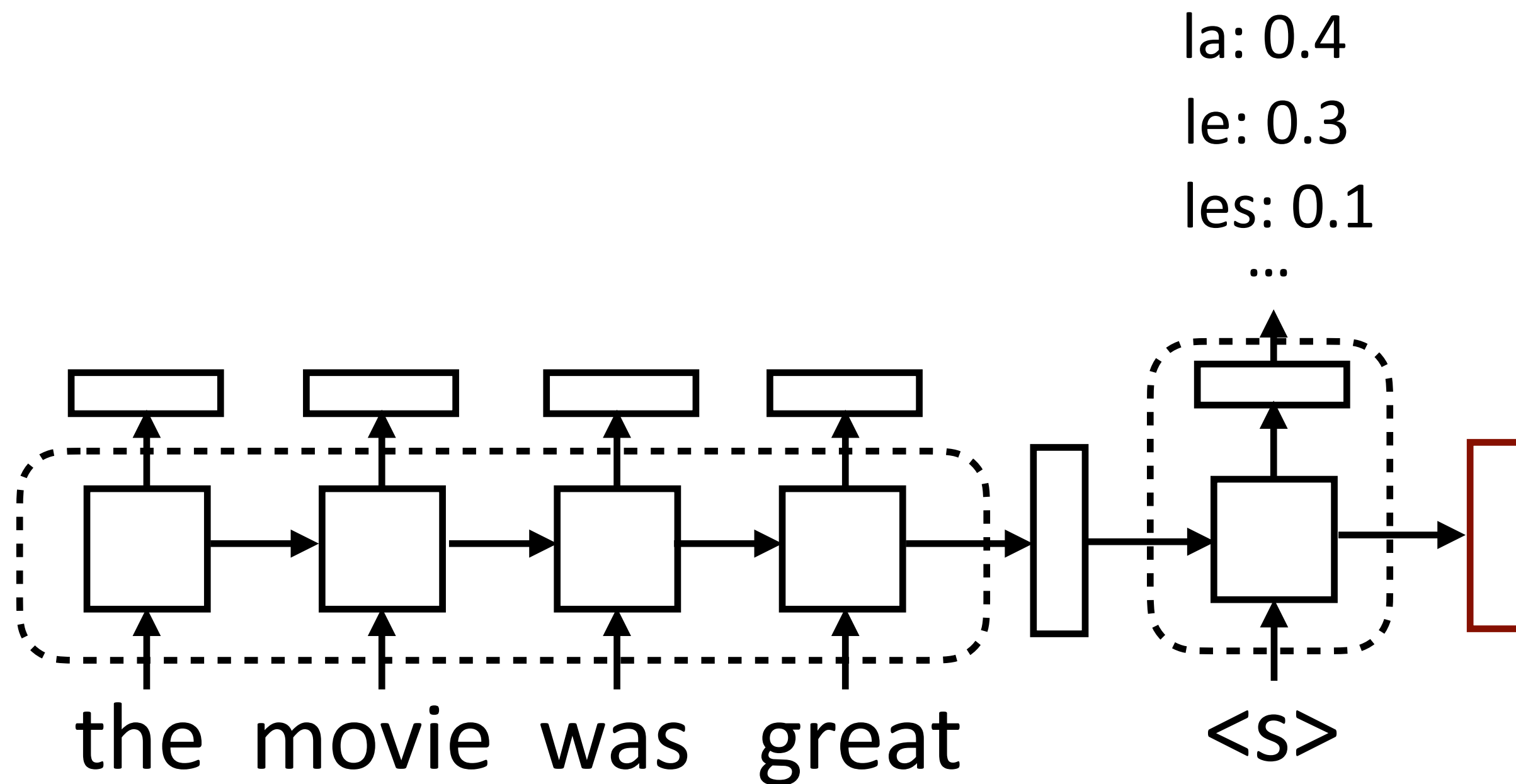
Model	Beam-10	
	BLEU	#Search err.
LSTM*	28.6	58.4%
SliceNet*	28.8	46.0%
Transformer-Base	30.3	57.7%
Transformer-Big*	31.7	32.1%

↖
A sentence is classified as search error if the decoder does not find the global best model score.

Stahlberg and Byrne (2019)

Beam Search

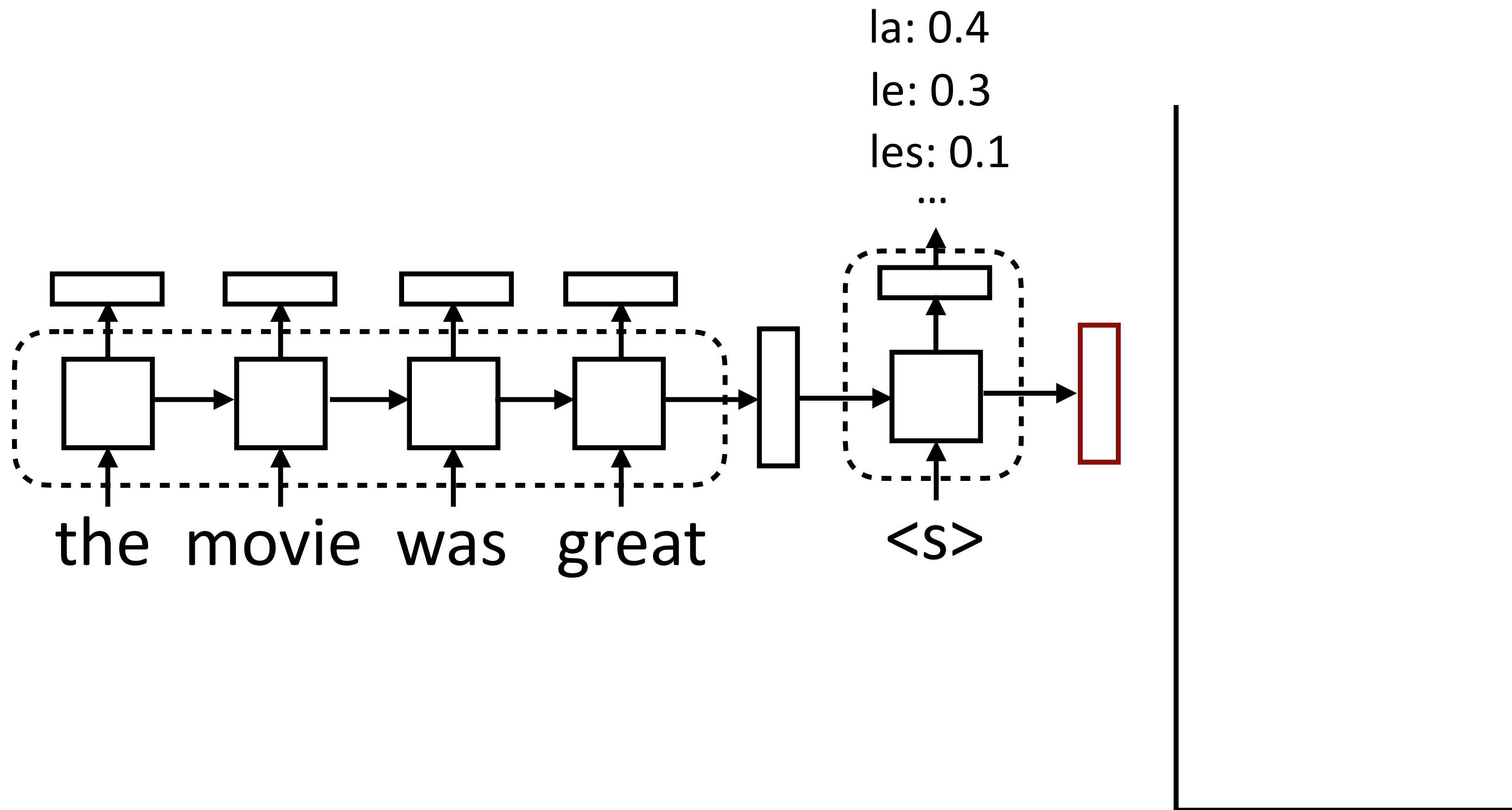
- ▶ Maintain decoder state, token history in beam



- ▶ NMT usually use beam ≤ 5

Beam Search

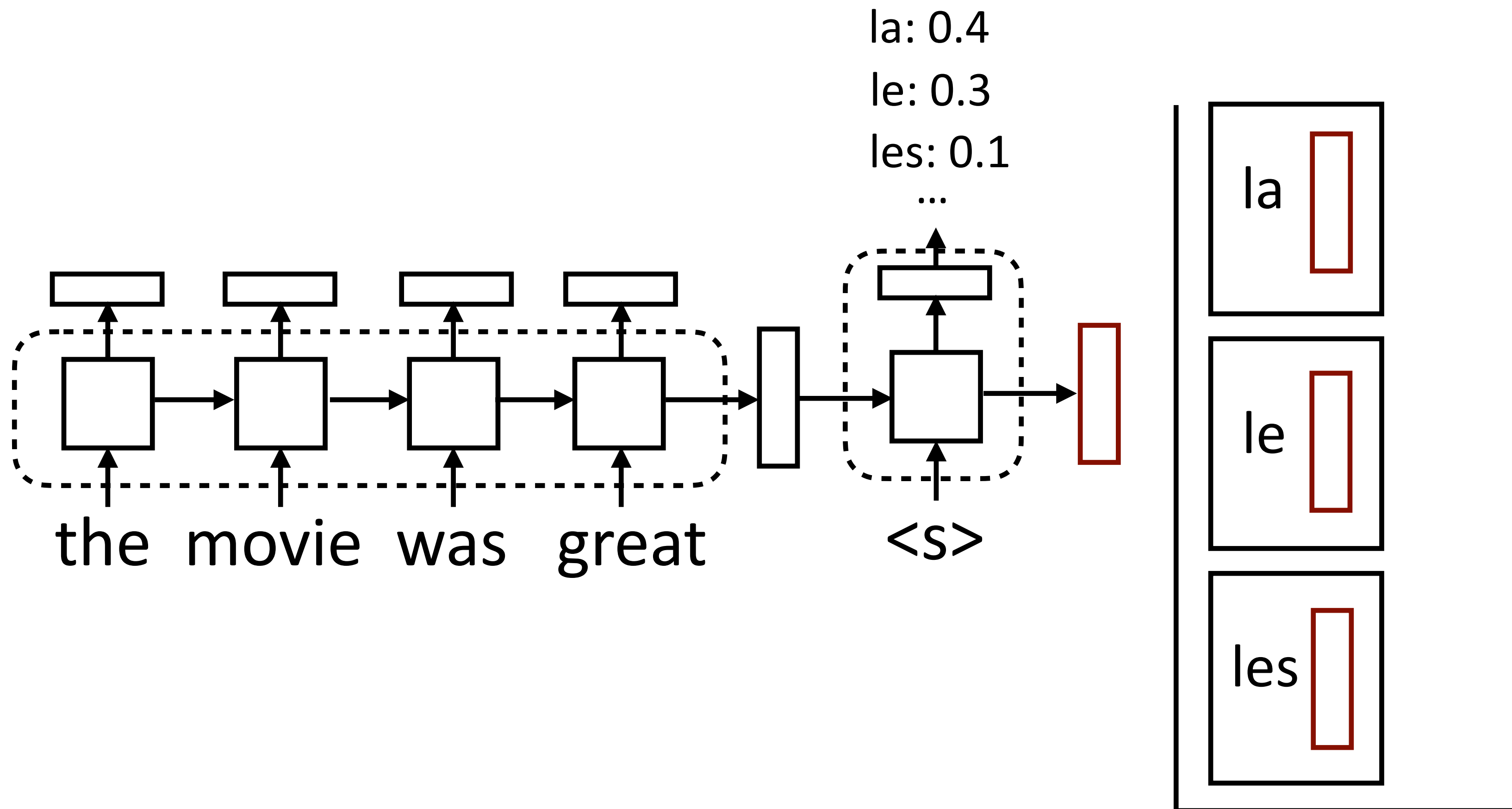
- ▶ Maintain decoder state, token history in beam



- ▶ NMT usually use beam ≤ 5

Beam Search

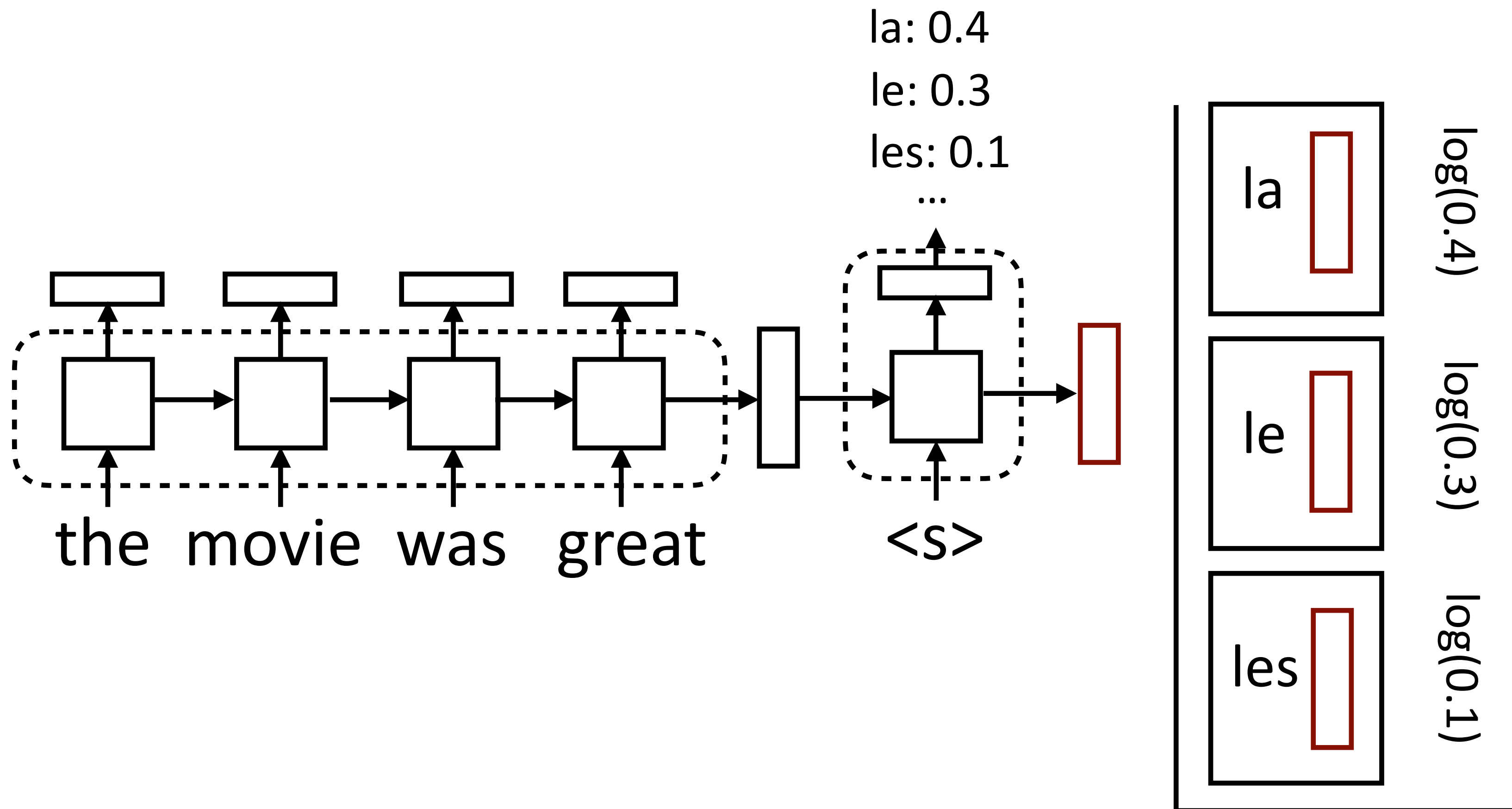
- ▶ Maintain decoder state, token history in beam



- ▶ NMT usually use beam ≤ 5

Beam Search

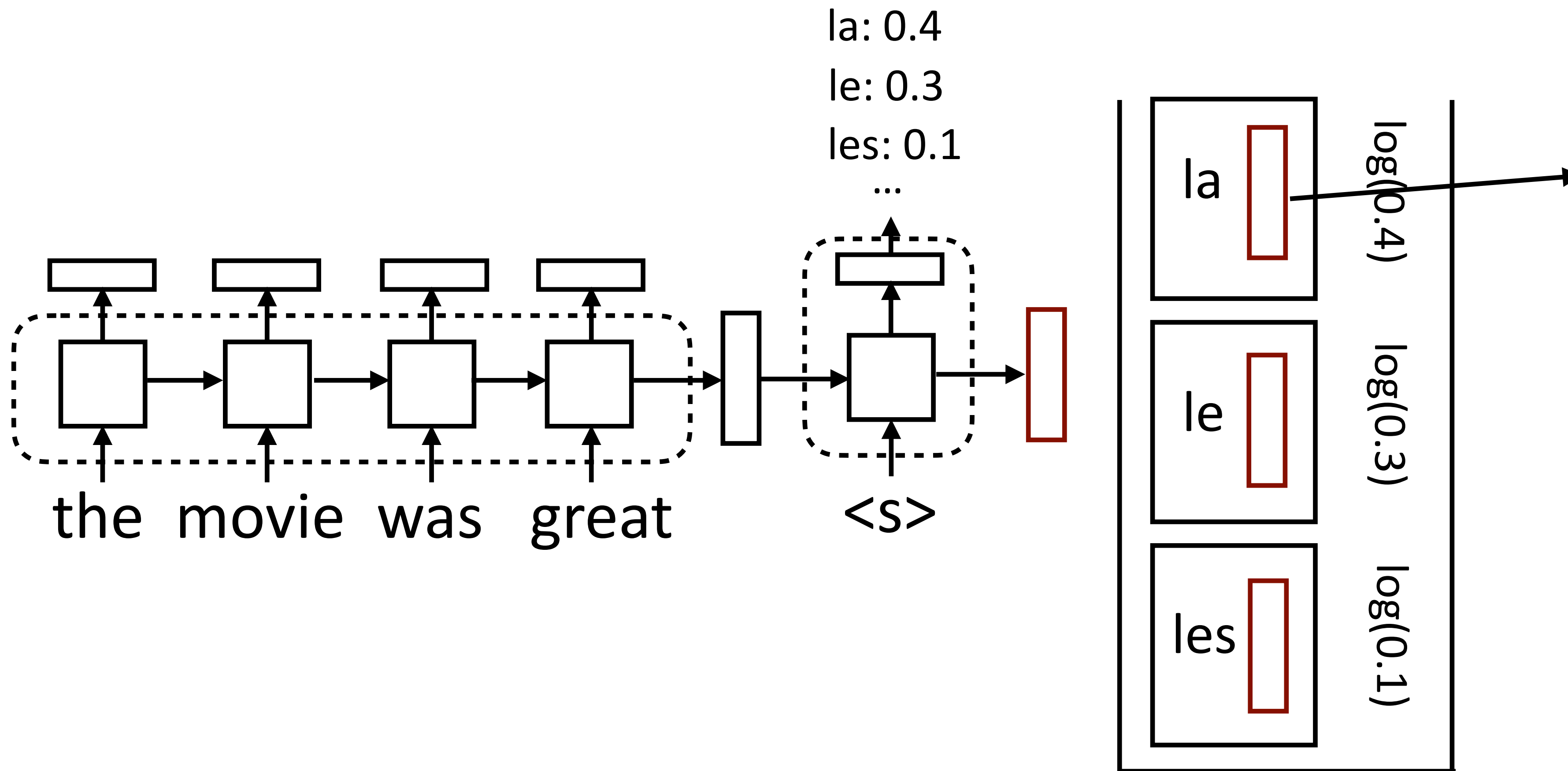
- ▶ Maintain decoder state, token history in beam



- ▶ NMT usually use beam ≤ 5

Beam Search

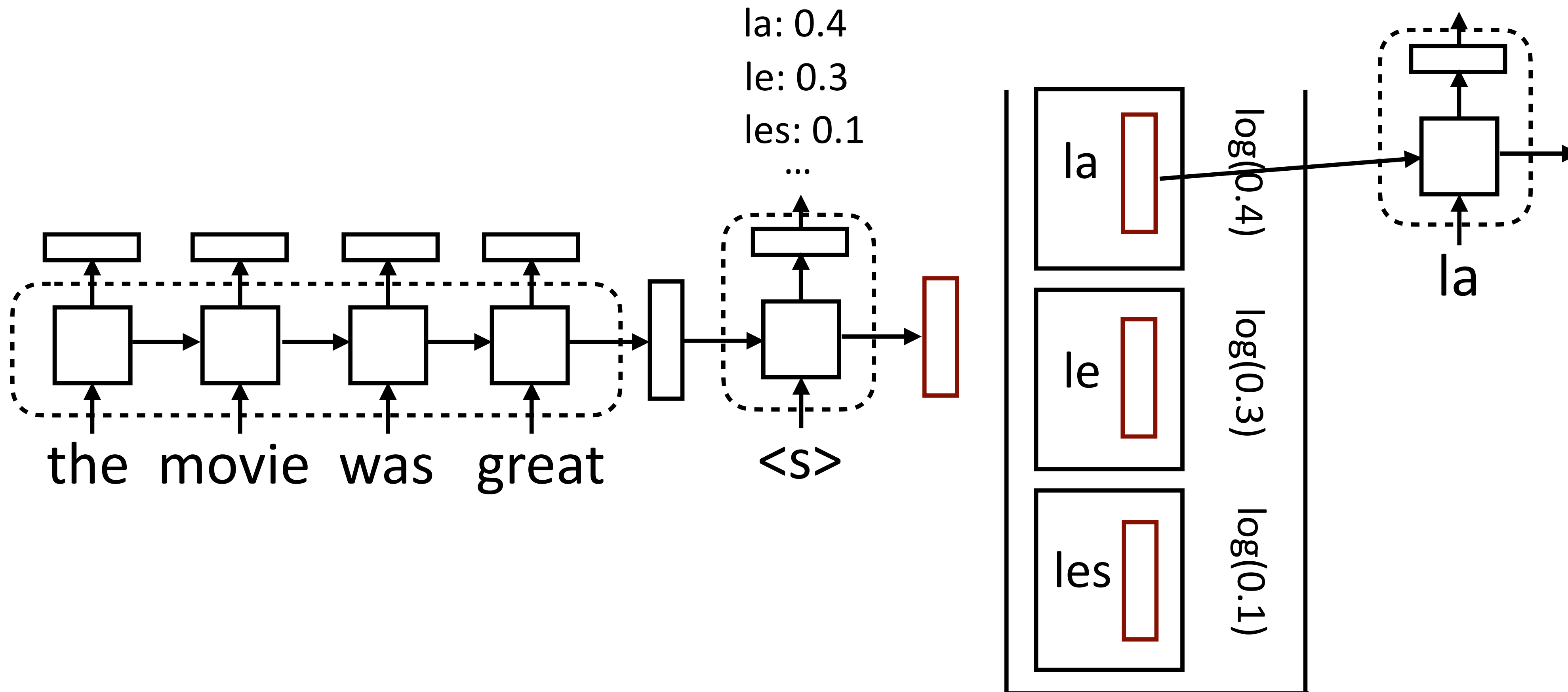
- ▶ Maintain decoder state, token history in beam



- ▶ NMT usually use beam ≤ 5

Beam Search

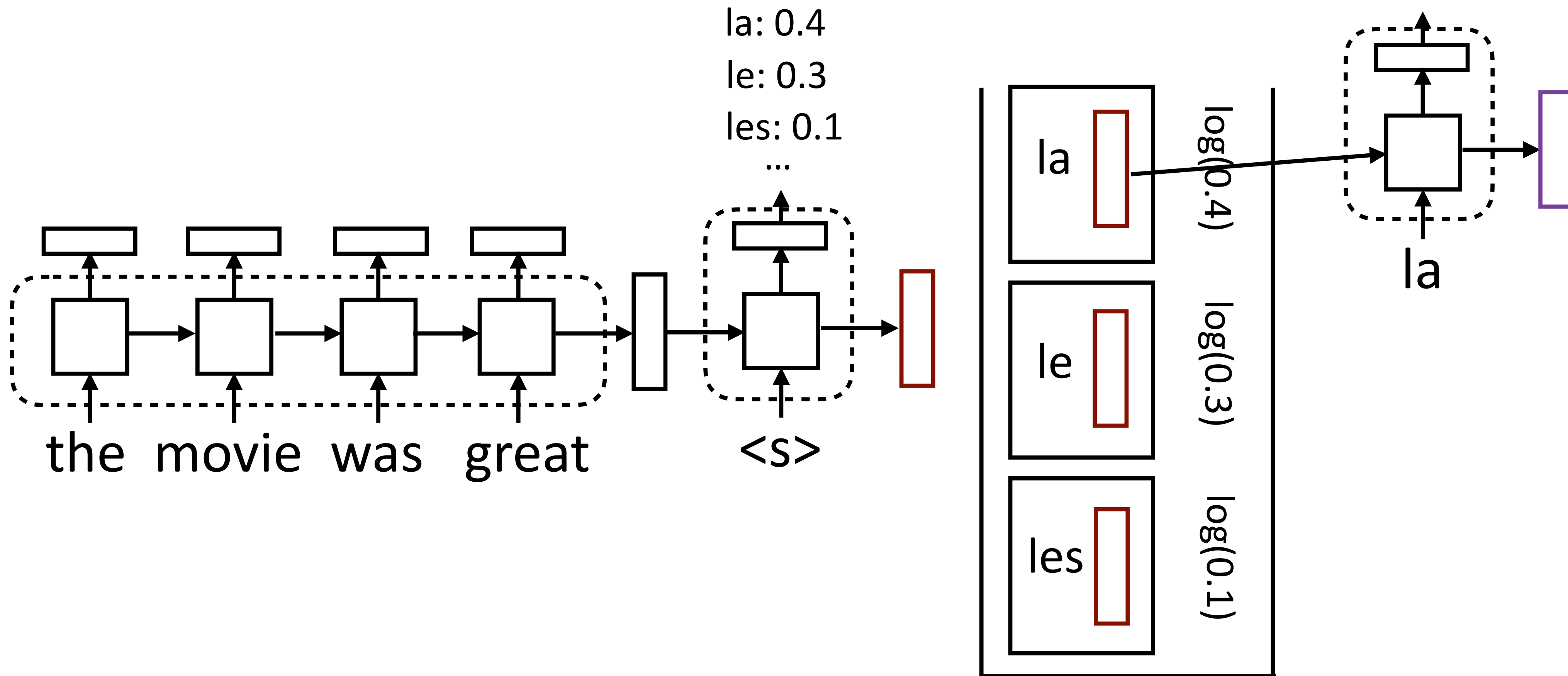
- ▶ Maintain decoder state, token history in beam



- ▶ NMT usually use beam ≤ 5

Beam Search

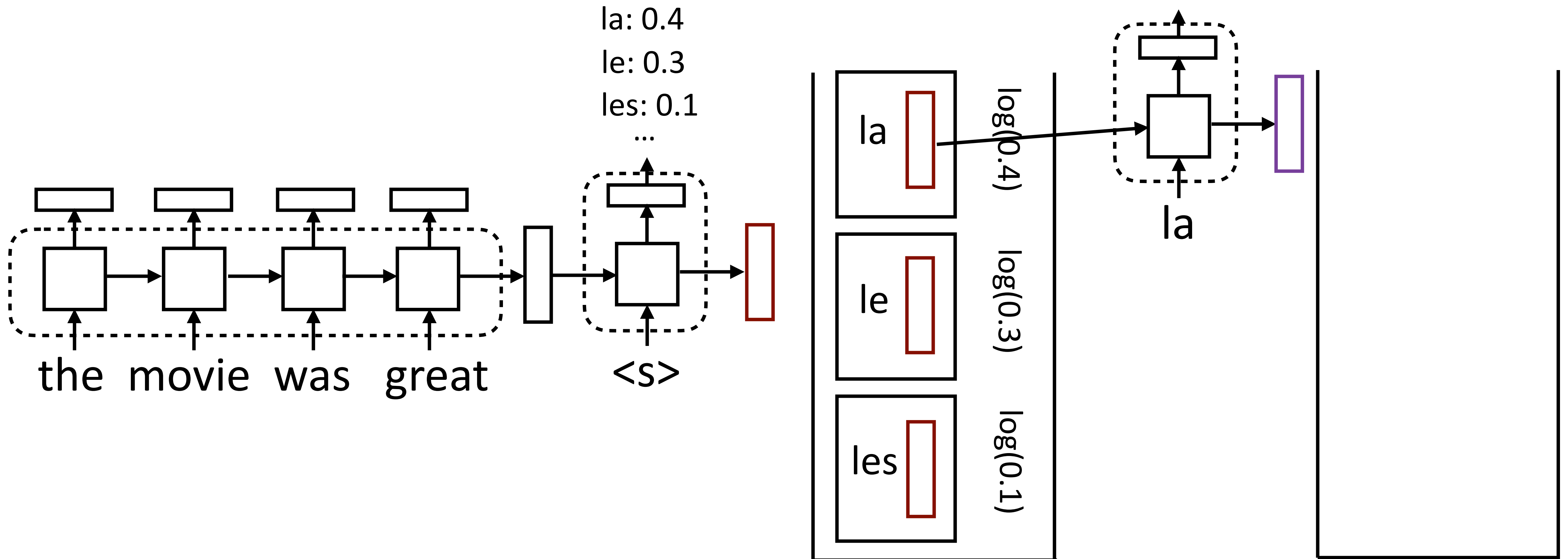
- ▶ Maintain decoder state, token history in beam



- ▶ NMT usually use beam ≤ 5

Beam Search

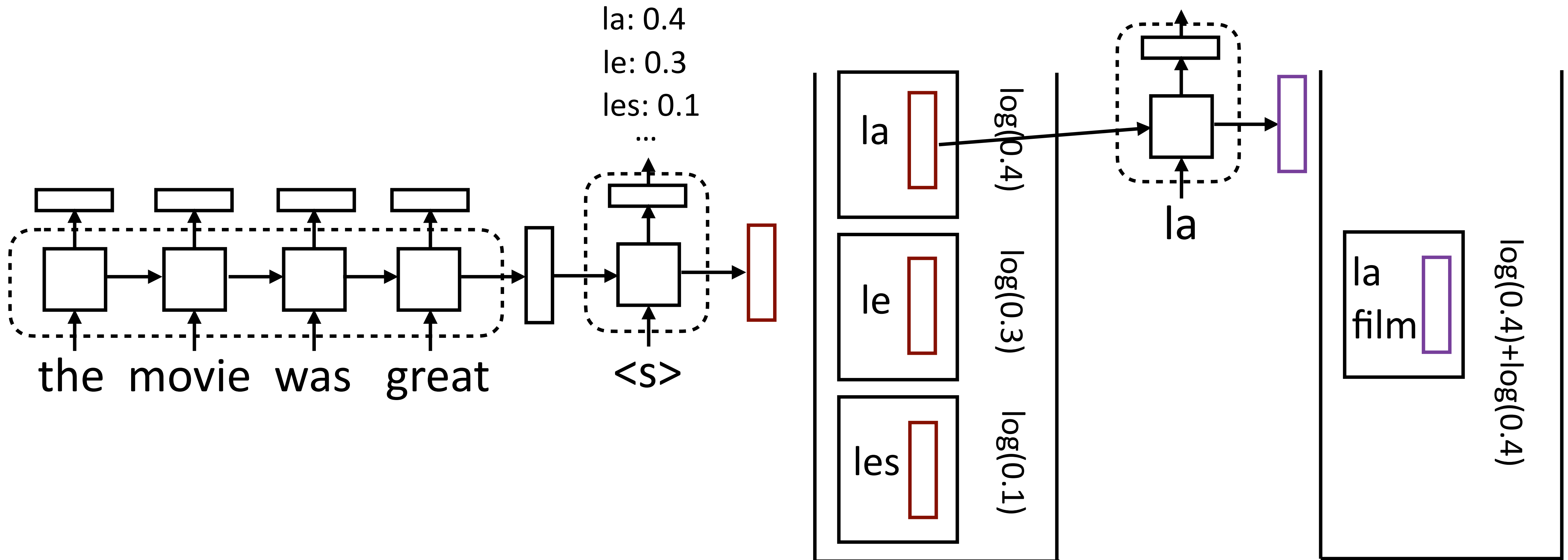
- ▶ Maintain decoder state, token history in beam



- ▶ NMT usually use beam ≤ 5

Beam Search

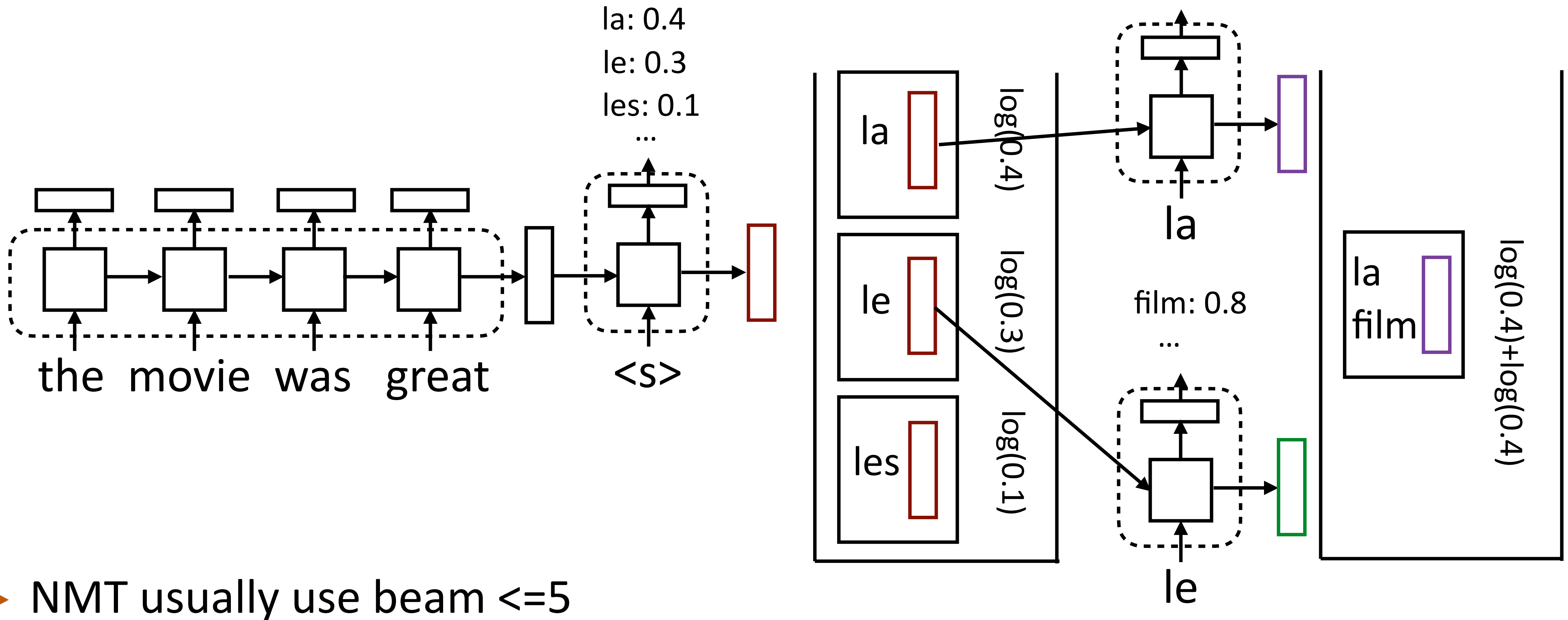
- ▶ Maintain decoder state, token history in beam



- ▶ NMT usually use beam ≤ 5

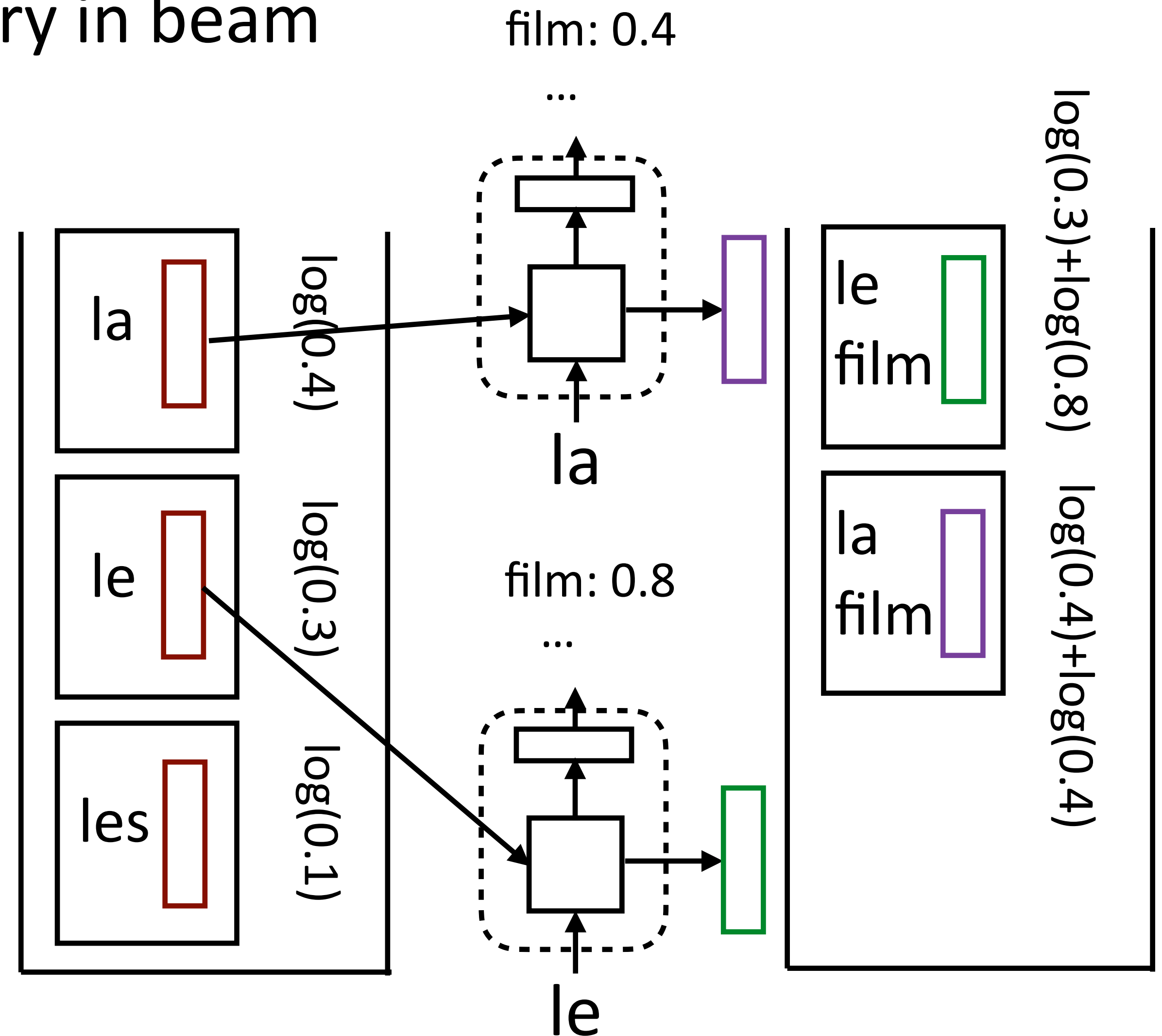
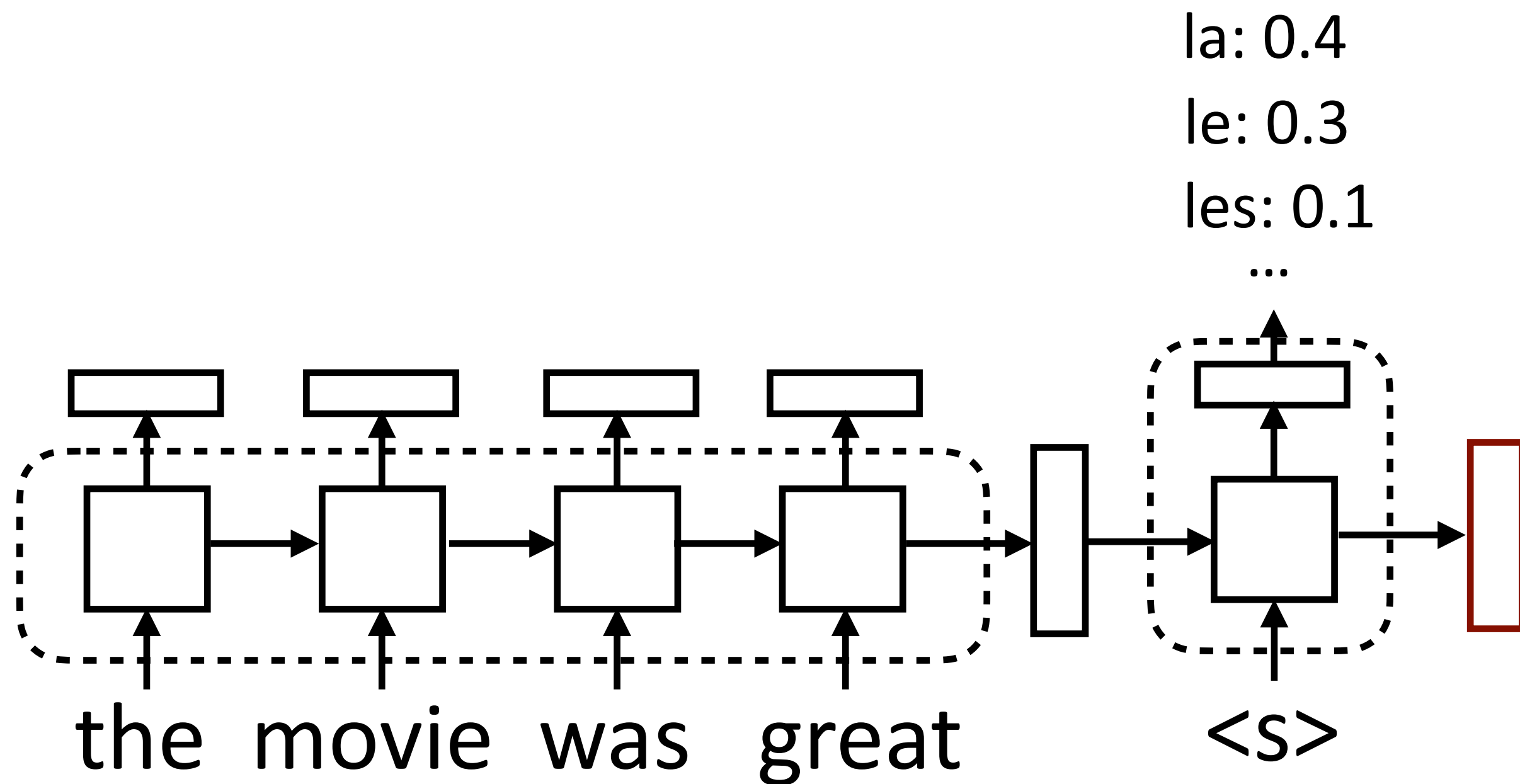
Beam Search

- ▶ Maintain decoder state, token history in beam



Beam Search

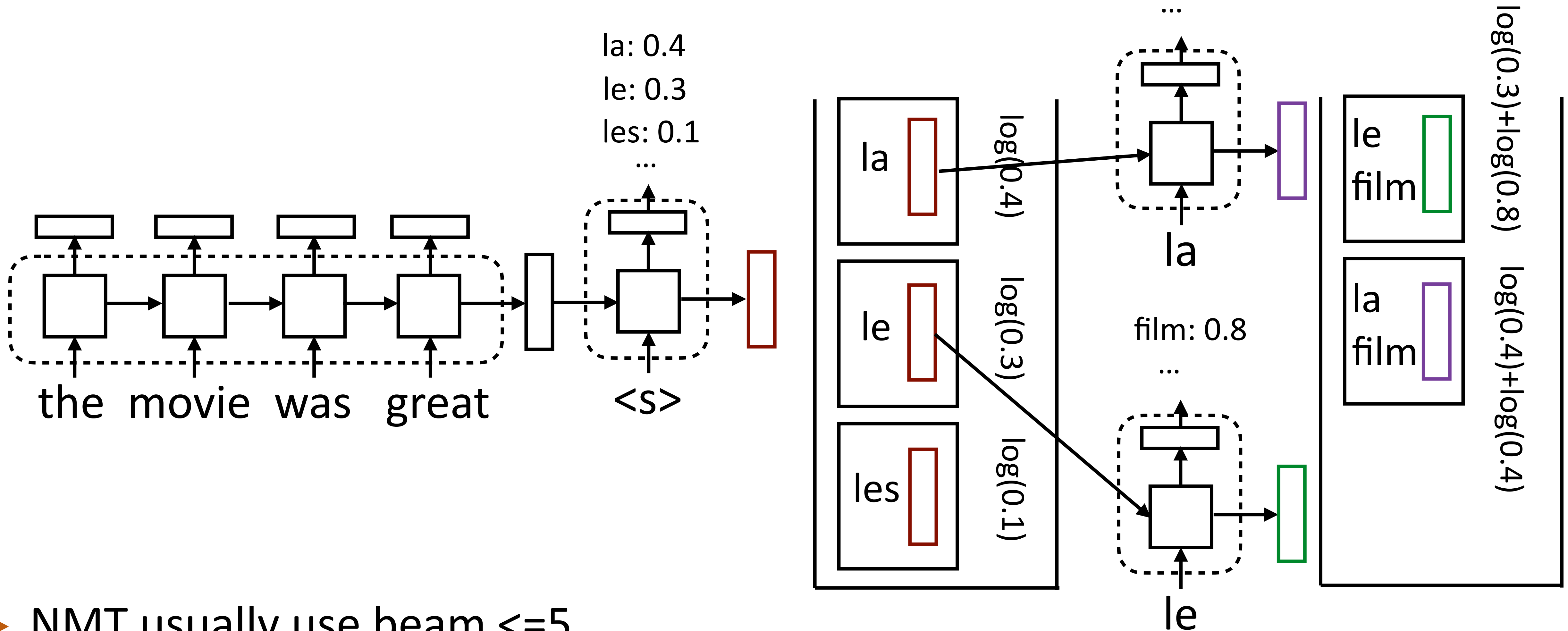
- ▶ Maintain decoder state, token history in beam



- ▶ NMT usually use beam ≤ 5

Beam Search

- ▶ Maintain decoder state, token history in beam



- ▶ NMT usually use $\text{beam} \leq 5$
- ▶ Keep **both** *film* states! Hidden state vectors are different

Sampling

- ▶ Beam search may give many similar sequences, and these actually may be *too close* to the optimal. Can sample instead:

$$P(y_i | \mathbf{x}, y_1, \dots, y_{i-1}) = \text{softmax}(W\bar{h})$$

$$y_{\text{sampled}} \sim P(y | \mathbf{x}, y_1, \dots, y_{i-1})$$

- ▶ Text *degeneration*: greedy solution can be uninteresting / vacuous for various reasons. Sampling can help.

Beam Search vs. Sampling

- ▶ These are samples from an unconditioned language model (not seq2seq model)

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Beam Search, $b=32$:

"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

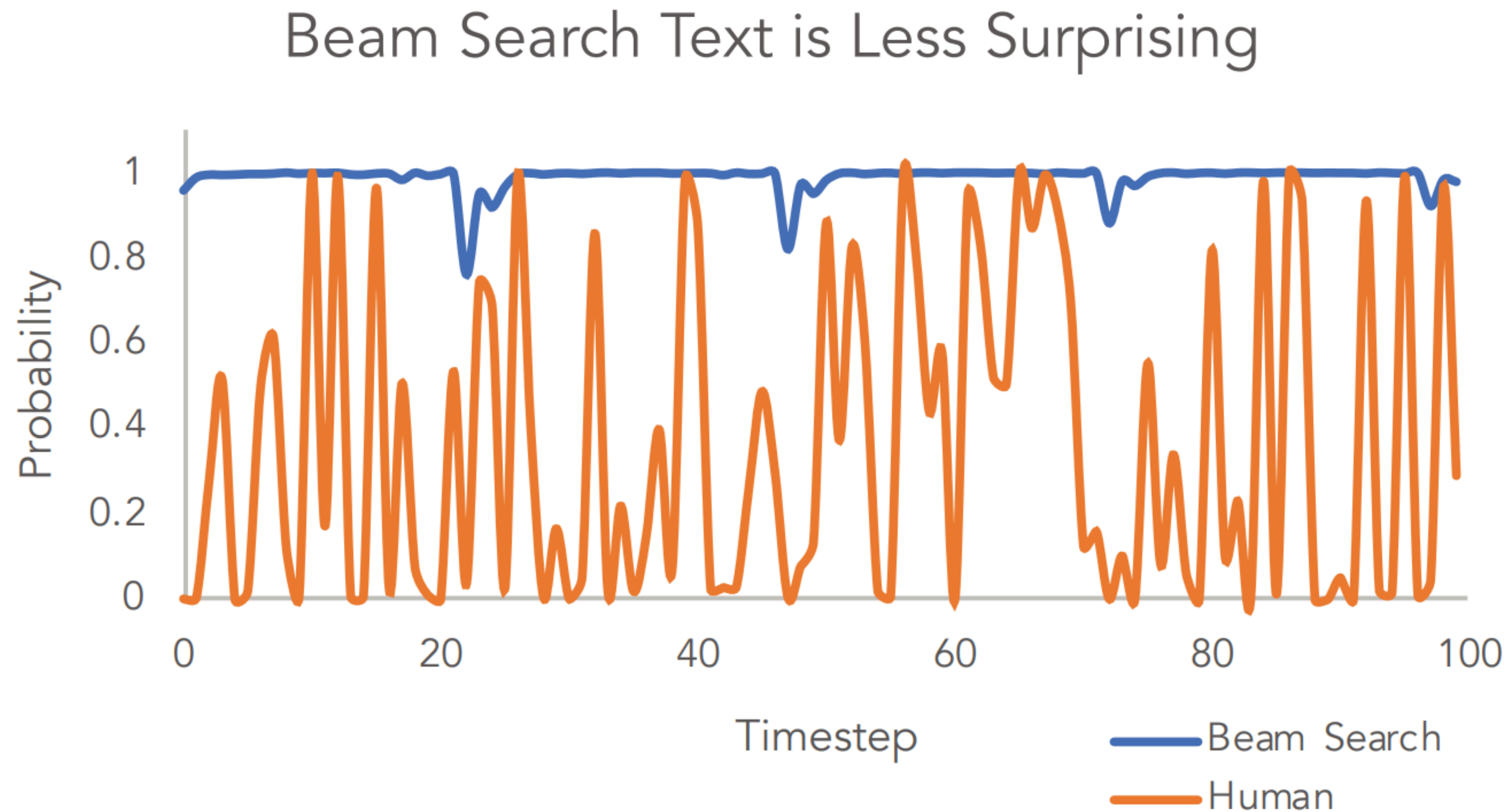
Pure Sampling:

They were cattle called **Bolivian Cavalleros**; they live in a remote desert **uninterrupted by town**, and they speak **huge, beautiful, paradisiacal Bolivian linguistic thing**. They say, **'Lunch, marge.'** They don't tell what the lunch is, "director Professor Chuperas Omwell told Sky News. **"They've only been talking to scientists, like we're being interviewed by TV reporters. We don't even stick around to be interviewed by TV reporters. Maybe that's how they figured out that they're cosplaying as the Bolivian Cavalleros."**

- ▶ Sampling is better but sometimes draws too far from the tail of the distribution

Holtzman et al. (2019)

Beam Search vs. Sampling



Decoding Strategies

- ▶ Greedy
- ▶ Beam search
- ▶ Sampling (e.g., top-k or Nucleus sampling)

Decoding Strategies

- ▶ Greedy
- ▶ Beam search
- ▶ Sampling (e.g., top-k or Nucleus sampling)
 - ▶ Top-k: take the top k most likely words ($k=5$), sample from those

Decoding Strategies

- ▶ Greedy
- ▶ Beam search
- ▶ Sampling (e.g., top-k or Nucleus sampling)
 - ▶ Top-k: take the top k most likely words ($k=5$), sample from those
 - ▶ Nucleus: take the top p% (95%) of the distribution, sample from within that

Generation Tasks



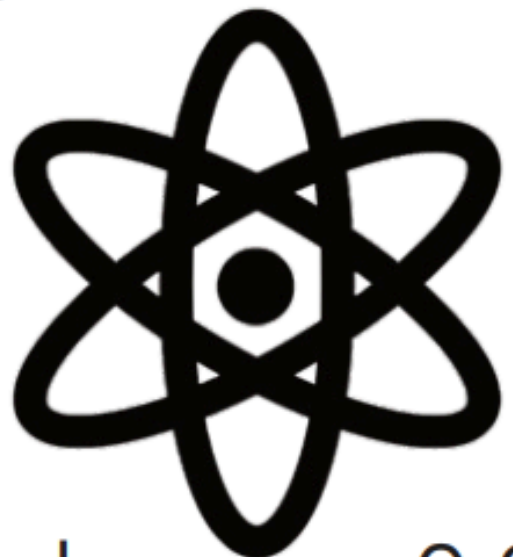
WebText



Beam Search, $b=16$



Pure Sampling



Nucleus, $p=0.95$

An unprecedented number of mostly young whales have become stranded on the West Australian coast since 2008.

The number of stranded whales has increased by more than 50 per cent in the past year, with the number of stranded whales on the West Australian coast increasing by more than 50 per cent in the past year. The number of whales stranded on the West Australian coast has increased by more than 50 per cent in the past year, with the number of stranded whales on the West Australian coast increasing by more than 50 per cent in the past year.

The Australian Food Safety Authority has warned Australia's beaches may be revitalised this year because healthy seabirds and seals have been on the move. More than 50,000 seabirds, sea mammals and seahorses have been swept into the sea by the Holden CS118 and Adelaide Airport CS300 from 2013. A major white-bat and umidauda migration across Australia is under way in Australia for the first time, with numbers reaching an estimated 50,000.

There has been an unprecedented number of calves caught in the nets of whaling stations that operate in WA. Pilot whales continue to migrate to feeding grounds to feed their calves. They are now vulnerable due to the decline of wild populations; they are restricted to one breeding site each year. Image copyright Yoon Bo Kim But, with sharp decline in wild populations the size of the Petrels are shrinking and dwindling population means there will only be room for a few new fowl.

Generation Tasks

- ▶ There are a range of seq2seq modeling tasks we will address
- ▶ For more constrained problems: greedy/beam decoding are usually best
- ▶ For less constrained problems: nucleus sampling introduces favorable variation in the output

Less constrained

More constrained



Unconditioned sampling/
“story generation”

Dialogue

Translation

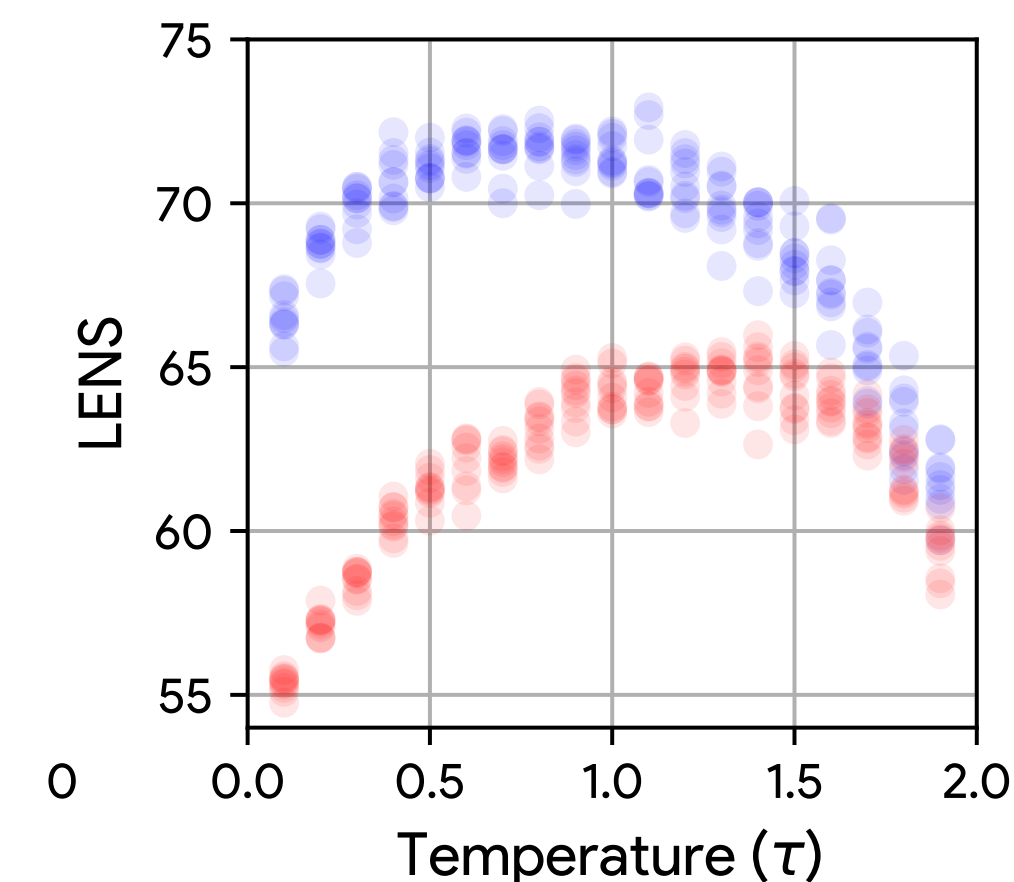
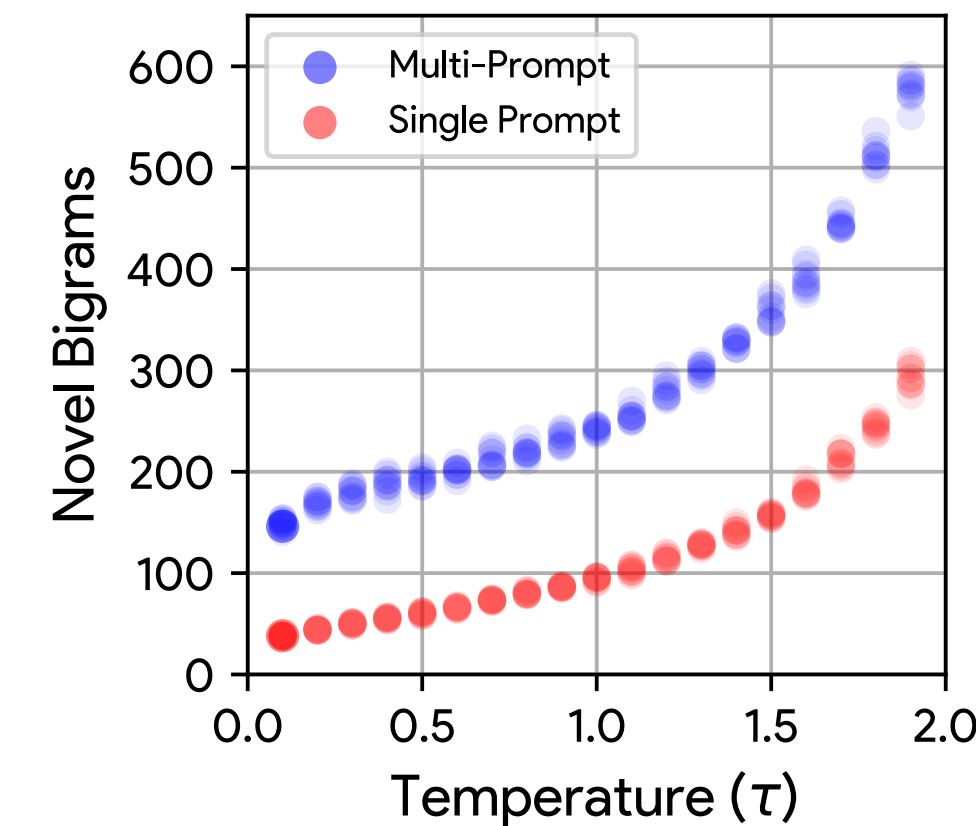
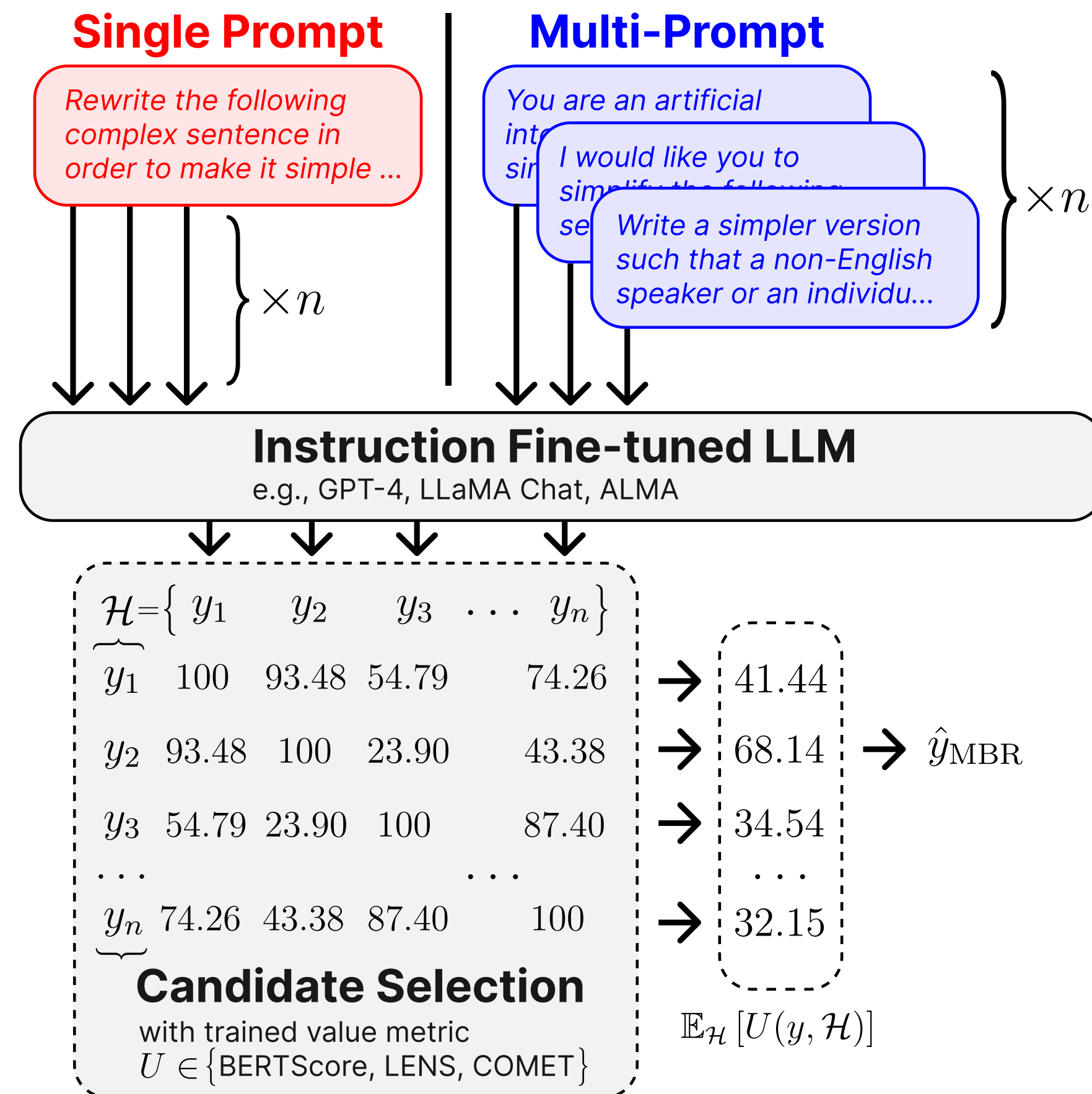
Text-to-code

Summarization

Data-to-text

Minimal Bayes Risk (MBR) Decoding

- MBR aims to find an output that maximizes the expected utility, i.e., metrics like COMET (for translation) or LENS (for simplification).



Decoding Strategies

- ▶ LMs place a distribution $P(y_i | y_1, \dots, y_{i-1})$
- ▶ seq2seq models place a distribution $P(y_i | \mathbf{x}, y_1, \dots, y_{i-1})$
- ▶ Generation from both models looks similar; how do we do it?
- ▶ Why the Viterbi algorithm is typically not a good fit for this?

Takeaways

- ▶ Can build MT systems with LSTM encoder-decoders (aka seq2seq) (or Transformer encoder-decoders — next class)
- ▶ Attention mechanism is very effective
- ▶ Decoding strategies (needed in both seq2seq & LMs)